

Data Structures and Algorithms

Antti Laaksonen

Data Structures and Algorithms
Antti Laaksonen
University of Helsinki, 2022
ISBN 978-951-51-8804-5

Introduction

Learning to program is a long process that can be divided into two phases. The first phase is learning the basic programming skills, such as how to use variables, conditions, loops, and arrays. Basic programming courses cover these topics. The second phase, which we focus on in this course, is learning to create *efficient* algorithms.

When we encounter a programming problem, it is a gateway to an adventure where many challenges may await. All methods are permitted, as long as we create an algorithm that solves the problem efficiently. This course teaches many techniques and ideas that are useful in solving algorithmic problems.

Algorithm design holds a central position in theoretical computer science research, but efficient algorithms are also important in many practical applications. Throughout the course, we will continuously observe the connection between theoretical results and how well algorithms work in practice.

Contents

Introduction	i
1 Introduction	1
1.1 What are algorithms?	1
1.2 The basic building blocks of programming	2
2 Efficiency	7
2.1 Time Complexity	7
2.1.1 Calculation Rules	8
2.1.2 Yleisiä aikavaativuuksia	10
2.1.3 Tehokkuuden arviointi	12
2.1.4 Esimerkki: Merkkijonot	13
2.2 Lisää algoritmien analysoinnista	15
2.2.1 Merkinnät O , Ω ja Θ	16
2.2.2 Tilavaativuus	17
2.2.3 Rajojen todistaminen	18
3 Järjestäminen	21
3.1 Järjestäminen ajassa $O(n^2)$	21
3.1.1 Lisäysjärjestäminen	21
3.1.2 Inversiot	23
3.2 Järjestäminen ajassa $O(n \log n)$	23
3.2.1 Lomitusjärjestäminen	24
3.2.2 Pikajärjestäminen	26
3.2.3 Algoritmien vertailua	28
3.3 Järjestämisen alaraja	29
3.3.1 Alarajatodistus	30
3.3.2 Laskemisjärjestäminen	30
3.4 Ohjelmointikielten toteutukset	31
3.4.1 Java	31
3.4.2 Python	33

3.5	Järjestämisen sovelluksia	34
3.5.1	Taulukkoalgoritmeja	34
3.5.2	Binäärihaku	35
4	Lista	37
4.1	Taulukkolista	37
4.1.1	Muutokset lopussa	37
4.1.2	Muutokset alussa ja lopussa	39
4.2	Linkitetty lista	40
4.2.1	Listan operaatiot	41
4.2.2	Listojen vertailua	42
4.3	Pino ja jono	43
4.4	Ohjelmointikielten toteutukset	44
4.4.1	Java	44
4.4.2	Python	46
4.5	Miten valita lista?	47
5	Hajautustaulu	49
5.1	Hajautustaulun toteutus	49
5.1.1	Ketjuhajautus	50
5.1.2	Avoim hajautus	51
5.1.3	Hajautusfunktion valinta	52
5.1.4	Miten hyvin hajautus toimii?	53
5.1.5	Hajautustaulu hakemistona	54
5.2	Ohjelmointikielten toteutukset	55
5.2.1	Java	55
5.2.2	Python	56
5.3	Hajautustaulu algoritmeissa	57
6	Binäärihakupuu	59
6.1	Taustaa binääripuista	59
6.1.1	Binääripuun käsittely	60
6.1.2	Läpikäyntijärjestykset	61
6.2	Binäärihakupuun toiminta	62
6.2.1	Operaatioiden toteutus	62
6.2.2	Operaatioiden tehokkuus	65
6.3	AVL-puu	65
6.3.1	Tasapainoehto	66
6.3.2	Kiertojen toteuttaminen	67
6.4	Ohjelmointikielten toteutukset	70
6.4.1	Java	70

6.4.2	Python	71
6.5	Tehokkuusvertailu	72
7	Keko	75
7.1	Binäärikeko	75
7.1.1	Keon tallentaminen	76
7.1.2	Operaatioiden toteutus	77
7.2	Lisää keosta	79
7.2.1	Taulukosta keoksi	79
7.2.2	Kekojärjestäminen	80
7.3	Ohjelmointikielten toteutukset	81
7.3.1	Java	81
7.3.2	Python	82
7.4	Tehokkuusvertailu	83
8	Peruuttava haku	85
8.1	Silmukoista rekursioon	85
8.1.1	Haun toteuttaminen	86
8.1.2	Osajoukkojen läpikäynti	87
8.1.3	Permutaatioiden läpikäynti	87
8.2	Esimerkkejä	88
8.2.1	Kuningatarongelma	88
8.2.2	Työtehtävien jakaminen	91
8.3	Pelin tekoäly	94
8.3.1	Minimax-algoritmi	94
8.3.2	Alfa-beeta-karsinta	95
9	Dynaaminen ohjelmointi	97
9.1	Perustekniikat	97
9.1.1	Rekursiivinen esitys	98
9.1.2	Tehokas toteutus	99
9.2	Esimerkkejä	100
9.2.1	Pisin nouseva alijono	100
9.2.2	Reitti ruudukossa	102
9.2.3	Repunpakkaus	103
9.2.4	Binomikertoimet	105
10	Verkkojen perusteet	107
10.1	Verkkojen käsitteitä	107
10.2	Verkot ohjelmoinnissa	110
10.2.1	Vieruslistaesitys	110

10.2.2	Kaarilistaesitys	111
10.2.3	Vierusmatriisiesitys	112
10.3	Verkon läpikäynti	112
10.3.1	Syvyyshaku	113
10.3.2	Leveyshaku	115
10.3.3	Esimerkki: Labyrintti	116
11	Lyhimmät polut	119
11.1	Lyhimmät polut lähtösolmusta	119
11.1.1	Bellmanin ja Fordin algoritmi	120
11.1.2	Dijkstran algoritmi	123
11.1.3	Esimerkki: Reittiopas	126
11.2	Kaikki lyhimmät polut	127
11.2.1	Floydin ja Warshallin algoritmi	127
11.2.2	Algoritmien vertailua	129
12	Suunnatut syklittömät verkot	131
12.1	Topologinen järjestys	132
12.1.1	Järjestyksen muodostaminen	132
12.1.2	Esimerkki: Kurssivalinnat	134
12.2	Dynaaminen ohjelmointi	135
12.2.1	Polkujen laskeminen	135
12.2.2	Ongelmat verkkoina	136
12.3	Vahvasti yhtenäisyys	137
12.3.1	Kosarajun algoritmi	137
12.3.2	Esimerkki: Luolapeli	139
13	Komponentit ja virittävät puut	141
13.1	Union-find-rakenne	141
13.1.1	Rakenteen toteutus	142
13.1.2	Esimerkki: Kaupungit	144
13.2	Pienin virittävä puu	145
13.2.1	Kruskalin algoritmi	145
13.2.2	Primin algoritmi	147
13.2.3	Miksi algoritmit toimivat?	148
14	Maksimivirtaus	151
14.1	Maksimivirtauksen laskeminen	151
14.1.1	Fordin ja Fulkersonin algoritmi	152
14.1.2	Yhteys minimileikkaukseen	153
14.1.3	Polkujen valitseminen	155

14.2	Maksimivirtauksen sovelluksia	156
14.2.1	Erilliset polut	156
14.2.2	Maksimiparitus	157
14.2.3	Pienin polkupeite	158
15	NP-ongelmat	161
15.1	Vaativuusluokat	161
15.1.1	Luokka P	161
15.1.2	Luokka NP	162
15.1.3	P vs. NP	162
15.1.4	Muita luokkia	163
15.2	NP-täydellisyys	164
15.2.1	SAT-ongelma	164
15.2.2	Ongelmien palautukset	165
15.2.3	Lisää ongelmia	168
15.2.4	Optimointiongelmat	169
15.3	Ongelmien ratkaiseminen	169
A	Matemaattinen tausta	173

Chapter 1

Introduction

Purpose of the course *Data structures and algorithms* is to teach methods by which we can solve *effectively* computational problems. Basic programming courses are focused for learning programming skills. Now it's time to go one step further and start pinning also pay attention to how fast the algorithms work.

The efficiency of algorithms is of great importance in practice. For example, an online route guide is useful because that it gives a suggestion of a route right after we have announced where we want to travel to. If you have to wait a minute or an hour for a suggested route, this would greatly limit the use of the service.

In order for the route guide to work effectively, it is based on well designed algorithm. In this course we will learn how we can create similar algorithms ourselves. In the course, we get to know both the theory of algorithm design and into practice – we want to deeply understand what algorithms are about, but also know how to implement them in practice.

1.1 What are algorithms?

Algorithm is a procedure that we can follow to solve some computational problem. The algorithm is given *input* (*input*), which describes the case of the problem to be solved, and the algorithm must produce *output*, which is a response to the input given to it.

As an example, consider the problem, where the input is a list containing n integers and the task is to calculate the sum of the numbers. For example, if the input is $[2, 4, 1, 8]$, the desired output is 15 because $2+4+1+8 = 15$. We can solve this problem with an algorithm, which loops through the numbers and counts them to the amount variable.

There are several possibilities for representing the operation of the algorithm. One way is to verbally explain how the algorithm works, as we just did. Another way is to give in some programming language written code that implements the algorithm. For example, we can make the sum of the numbers count this is how the algorithm works in Java and Python:

```
int sum = 0;
for (int i = 0; i < n; i++) {
    sum+= numbers[i];
}
System.out.println(sum);
```

```
sum= 0
for i in range(n):
    sum+= numbers[i]
print(sum )
```

We can also represent the algorithm using **pseudocode** instead of an actual programming language. This means we write code that closely resembles real programming languages but allows us to decide the exact syntax and take some liberties, making it easier to describe the algorithm. For example, we can present the recent algorithm in pseudocode as follows:

```
sum = 0
for i = 0 to n-1
    sum += numbers[i]
print(sum)
```

There are many ways to notate pseudocode, and we use quite a lot of Python-like pseudocode in this book. We present algorithms using pseudocode, because the course deals with general programming theory, which is not related to a specific programming language. In addition to this, we will get to know how certain algorithms and data structures are implemented in Java and Python in standard libraries.

1.2 The basic building blocks of programming

An intriguing aspect of programming is that even complex algorithms are built from simple components. Next, we will go through the fundamental building blocks of programming, which form the foundation for designing algorithms.

Varia

Variable

Variable (*muuttuja*) contains the data processed by the algorithm. In pseudocode, it is common practice to use variables directly without separate declarations:

```
a = 5
b = 7
c = a+b
```

Conditional Statement

Conditional statement (*ehtolause*) allows the program's execution to branch. In pseudocode, we use the following syntax:

```
if x%2 == 0
  print("even")
else
  print("odd")
```

Loop

Loop repeats the code inside it. In pseudocode, we denote a for-loop that iterates through the numbers from 1 to 100 as follows:

```
for i = 1 to 100
  print(i)
```

The following for-loop, in turn, iterates through the elements in a list:

```
for x in list
  print(x)
```

Another type of loop is the while-loop, which repeats the code as long as the condition given at the beginning of the loop holds. For example, the following loop continues as long as the value of x is at least 1:

```
while x >= 1
  print(x)
  x /= 2
```

Array

A fundamental data structure in programming is the *array* (*taulukko*), which contains a collection of sequential elements. We can refer to array elements using the `[]` syntax:

```
numbers[0] = 4
numbers[3] = 2
```

We follow the common convention in this book where the elements of an array are indexed as $0, 1, \dots, n - 1$ when the array contains n elements.

An array has the properties that it contains a fixed number of elements and allows efficient access to any element based on its index. An array corresponds to the way data is stored in a computer's memory, and we can use arrays to implement any other data structure.

The table has had a strong position in traditional programming languages, but the situation is changing. An array is a basic data structure in Java, but in Python instead of a table, the basic data structure is *list*. However, we can use a Python list like a table, because the only essential difference is that its size can change. The internal implementation of the list is based on a table, and in Chapter 4 we will see how this happens in practice.

Subroutines

In this book, we use the term *procedure* (*proseduuri*) for a subroutine that does not return a value, and the term *function* (*funktio*) for a subroutine that returns a value.

For example, the following procedure prints the numbers $1 \dots n$:

```
procedure print(n)
  for i = 1 to n
    print(i)
```

The following function, on the other hand, calculates $1 \dots n$ of numbers amount:

```
function sum(n)
  s = 0
  for i = 1 to n
    s += i
  return s
```

The terminology related to subroutines varies depending on the program-

ming language: for example, in Java, the term *method* is used in both cases, while in Python, the terms *function* or *method* are used depending on the context.

Recursion

A common programming technique in algorithm design is *recursion* (*rekursion*), which means that a subroutine calls itself.

Here is an example of using recursion:

```
procedure test(n)
  if n == 0
    return
  else
    print("hello")
    test(n-1)
```

This procedure prints the line "hello" n times. For example, the call `test(3)` produces the following output:

```
hello
hello
hello
```

The idea is that if $n = 0$, the procedure does nothing, because there is nothing to print. Otherwise, the procedure prints the line "hello" and then calls itself with the parameter $n - 1$.

We use recursion often during the course, and little by little we get to know its possibilities in more detail.

We have now gone through the ingredients, which allow us to implement *any* algorithm. It is reassuring to know that even such a small number of techniques is sufficient for designing algorithms. Now it's all about how we know how to *apply* these techniques in different situations.

Chapter 2

Efficiency

The goal in algorithm design is to create algorithms that work *efficiently*. We want to develop algorithms that can handle large datasets without taking too long to execute. We consider an algorithm to be *good* if it provides a quick response even when the amount of data is large.

In this chapter, we will explore tools that allow us to assess the efficiency of algorithms. A key concept is *time complexity*, which concisely describes the time usage of an algorithm. With time complexity, we can estimate an algorithm's efficiency based on its structure without having to implement and test the algorithm just to determine how fast it is.

2.1 Time Complexity

The efficiency of an algorithm depends on how many steps it performs. Our goal is to estimate the number of steps the algorithm takes in relation to the input size n . For example, if the input is an array, n is the size of the array, and if the input is a string, n is the length of the string.

Let's consider an example algorithm that counts how many times the element x appears in an array containing n numbers.

```
1   counter = 0
2   for i = 0 to n-1
3       if numbers[i] == x
4           counter += 1
```

We can evaluate the efficiency of the algorithm by analyzing how many times each line is executed. Line 1 is executed only once at the start of the algorithm. Then, the loop begins, where lines 2 and 3 are each executed n times, and line 4 is executed between 0 and n times, depending on how often

the number x appears in the array.

Thus, the algorithm performs at least $2n + 1$ and at most $3n + 1$ steps.

A detailed analysis like this is usually not necessary; it is sufficient to determine a rough *upper bound* for time usage. An algorithm runs in time $O(f(n))$, meaning its *time complexity* is $O(f(n))$, if it executes at most $cf(n)$ steps whenever $n \geq n_0$, where c and n_0 are constants. For example, the previous algorithm runs in time $O(n)$ because it executes at most $4n$ steps for all values of n , meaning we can choose $c = 4$ and $n_0 = 1$.

2.1.1 Calculation Rules

One of the advantages of time complexity is that we can usually determine it easily from the structure of the algorithm. Next, we will explore calculation rules that make this possible.

Single Commands

If the code does not contain loops but only individual commands, its time complexity is $O(1)$. This is the case, for example, in the following code:

```
c = a+b
if c >= 0
print(c)
```

Loops

We use $\dots$ to represent code with time complexity $O(1)$. If the code contains a single loop that executes n steps, its time complexity is $O(n)$:

```
for i = 1 to n
...
```

If there are two nested loops of this kind, the time complexity is $O(n^2)$:

```
for i = 1 to n
  for j = 1 to n
    ...
```

More generally, if the code contains k nested loops in a similar manner, its time complexity is $O(n^k)$.

Note that constant factors and lower-order terms do not affect time complexity. For example, in the following code snippets, the loops execute $2n$ and $n - 1$ steps, but the time complexity of both codes is still $O(n)$.

```
for i = 1 to 2*n  
...
```

```
for i = 1 to n-1  
...
```

Sequential Sections

If the code has sequential sections, its time complexity is the largest time complexity of a single section. For example, the following code has a time complexity of $O(n^2)$, because its sections have time complexities of $O(n)$, $O(n^2)$, and $O(n)$.

```
for i = 1 to n  
...  
for i = 1 to n  
for j = 1 to n  
...  
for i = 1 to n  
...
```

Multiple Variables

Sometimes, the time complexity depends on multiple factors, which means the formula has multiple variables. For example, the following code has a time complexity of $O(nm)$:

```
for i = 1 to n  
for j = 1 to m  
...
```

Recursive Algorithms

In a recursive algorithm, we calculate how many recursive calls are made and how long a single call takes. Let's consider the following procedure, which is called with the parameter n :

```

procedure f(n)
  if n == 1
    return
  f(n-1)

```

The procedure is called a total of n times, and each call takes $O(1)$ time. We determine the time complexity of the procedure by multiplying these values together, so the procedure takes $O(n)$ time.

Now, let's consider the following procedure:

```

procedure g(n)
  if n == 1
    return
  g(n-1)
  g(n-1)

```

Tässä tapauksessa jokainen proseduurin kutsu tuottaa kaksi uutta kutsua, joten proseduuria kutsutaan kaikkiaan

$$1 + 2 + 4 + \dots + 2^{n-1} = 2^n - 1$$

kertaa. Jokainen kutsu vie aikaa $O(1)$, joten aikavaativuus on $O(2^n)$.

2.1.2 Yleisiä aikavaativuuksia

Tietyt aikavaativuudet esiintyvät usein algoritmeissa. Käymme seuraavaksi läpi joukon tällaisia aikavaativuuksia.

$O(1)$ (vakioaikainen)

Vakioaikainen (*constant time*) algoritmi suorittaa kiinteän määrän komentoja, eikä syötteen suuruus vaikuta algoritmin nopeuteen. Esimerkiksi seuraava algoritmi laskee summan $1 + 2 + \dots + n$ vakioajassa summakaavalla:

```

summa = n*(n+1)/2

```

$O(\log n)$ (logaritminen)

Logaritminen (*logarithmic*) algoritmi puolittaa usein syötteen koon joka askeleella. Esimerkiksi seuraavan algoritmin aikavaativuus on $O(\log n)$:

```
laskuri = 0
while n >= 1
  laskuri += 1
  n /= 2
```

Tärkeä seikka logaritmeihin liittyen on, että $\log n$ on *pieni* luku, kun n on mikä tahansa tyypillinen algoritmeissa esiintyvä luku. Esimerkiksi $\log 10^6 \approx 20$ ja $\log 10^9 \approx 30$, kun logaritmin kantaluku on 2. Niinpä jos algoritmi tekee jotain logaritmisessa ajassa, siinä ei kulu kauan aikaa.

$O(n)$ (lineaarinen)

Lineaarinen (*linear*) algoritmi voi käydä läpi syötteen kiinteän määrän kertoja. Esimerkiksi seuraava $O(n)$ -algoritmi laskee taulukon lukujen summan:

```
summa = 0
for i = 0 to n-1
  summa += taulu[i]
```

Kun algoritmin syötteenä on aineisto, jossa on n alkiota, lineaarinen aikavaativuus on yleensä paras mahdollinen, minkä voimme saavuttaa. Tämä johtuu siitä, että algoritmin täytyy käydä syöte ainakin kerran läpi, ennen kuin se voi ilmoittaa vastauksen.

$O(n \log n)$ (järjestäminen)

Aikavaativuus $O(n \log n)$ viittaa usein siihen, että algoritmin osana on *järjestämistä*, koska tehokkaat järjestämisalgoritmit toimivat ajassa $O(n \log n)$. Esimerkiksi seuraava $O(n \log n)$ -aikainen algoritmi tarkastaa, onko taulukossa kahta samaa alkiota:

```
sort(taulu) // järjestäminen
samat = false
for i = 1 to n-1
  if taulu[i] == taulu[i-1]
    samat = true
```

Algoritmi järjestää ensin taulukon, minkä jälkeen yhtä suuret alkiot ovat vierekkäin ja ne on helppoa löytää. Järjestäminen vie aikaa $O(n \log n)$ ja silmukka vie aikaa $O(n)$, joten algoritmi vie yhteensä aikaa $O(n \log n)$.

Tutustumme tarkemmin järjestämiseen ja sitä käytäviin algoritmeihin seuraavassa luvussa 3.

$O(n^2)$ (neliöllinen)

Neliöllinen (quadratic) algoritmi voi käydä läpi kaikki tavat valita kaksi alkia syötteestä. Esimerkiksi seuraava $O(n^2)$ -algoritmi tutkii, onko taulukossa kahta lukua, joiden summa on x .

```
ok = false
for i = 0 to n-1
  for j = i+1 to n-1
    if taulu[i]+taulu[j] == x
      ok = true
```

 $O(n^3)$ (kuutiollinen)

Kuutiollinen (cubic) algoritmi voi käydä läpi kaikki tavat valita kolme alkia syötteestä. Esimerkiksi seuraava $O(n^3)$ -algoritmi tutkii, onko taulukossa kolmea lukua, joiden summa on x .

```
ok = false
for i = 0 to n-1
  for j = i+1 to n-1
    for k = j+1 to n-1
      if taulu[i]+taulu[j]+taulu[k] == x
        ok = true
```

 $O(2^n)$ (osajoukot)

Aikavaativuus $O(2^n)$ viittaa usein siihen, että algoritmi käy läpi syötteen alkien osajoukot.

 $O(n!)$ (permutaatiot)

Aikavaativuus $O(n!)$ viittaa usein siihen, että algoritmi käy läpi syötteen alkien permutaatiot.

2.1.3 Tehokkuuden arviointi

Mitä hyötyä on määrittää algoritmin aikavaativuus? Hyötynä on, että aikavaativuus antaa arvion siitä, kuinka *hyvä* algoritmi on eli miten suuria syötteitä sillä voi käsitellä tehokkaasti. Aikavaativuuden avulla algoritmin tehokkuudesta pystyy saamaan hyvän käsityksen ennen algoritmin toteuttamista ja testaamista käytännössä.

syötteen kokoluokka n	tarvittava aikavaativuus
10	$O(n!)$
20	$O(2^n)$
500	$O(n^3)$
5000	$O(n^2)$
10^6	$O(n)$ tai $O(n \log n)$
suuri	$O(1)$ tai $O(\log n)$

Table 2.1: Kuinka suuren syötteen algoritmi voi käsitellä nopeasti?

Aikavaativuutta voi ajatella samalla tavalla kuin hotellin tähtiluokitusta: se kertoo tiiviissä muodossa, mistä asiassa on kysymys, eikä tarvitse ottaa selvää yksityiskohdista. Jos majoitus on neljän tähden hotellissa, tämä antaa heti jonkin käsityksen huoneen tasosta, vaikka tiedossa ei olisi tarkkaa listausta huoneen varustelusta. Vastaavasti jos jonkin algoritmin aikavaativuus on $O(n \log n)$, tämän perusteella voi heti arvioida karkeasti, miten suuria syötteitä algoritmi pystyy käsittelemään, vaikka algoritmin kaikki yksityiskohdat eivät olisi tiedossa.

Yksi kiinnostava näkökulma algoritmin tehokkuuteen on, miten suuren syötteen algoritmi voi käsitellä *nopeasti* (noin sekunnissa). Tämä on hyvä vaatimus, kun haluamme käyttää algoritmia jossakin käytännön sovelluksessa. Taulukossa 2.1 on joitakin hyödyllisiä arvioita, kun algoritmi suoritetaan nykyaikaisella tietokoneella. Esimerkiksi jos meillä on $O(n^2)$ -algoritmi, voimme käsitellä sillä nopeasti syötteen, jossa on luokkaa 5000 alkiota. Jos haluamme käsitellä tehokkaasti suurempia syötteitä, meidän tulisi löytää $O(n)$ -tai $O(n \log n)$ -aikainen algoritmi.

Kannattaa silti pitää mielessä, että nämä luvut ovat vain arvioita ja algoritmin todelliseen ajankäyttöön vaikuttavat monet asiat. Saman algoritmin hyvä toteutus saattaa olla kymmeniä kertoja nopeampi kuin huono toteutus, ja suuri merkitys on myös ohjelmointikielellä, jolla algoritmi on toteutettu. Tässä kirjassa analysoimme algoritmeja sekä aikavaativuuksien avulla että mittaamalla todellisia suoritusajoja.

2.1.4 Esimerkki: Merkkijonot

Tehtävän ratkaisemiseen on usein monenlaisia algoritmeja. Seuraavaksi ratkaisemme saman tehtävän kahdella algoritmilla, joista ensimmäinen on suoraviivainen raa'an voiman algoritmi, joka toimii ajassa $O(n^2)$. Toinen algoritmi on puolestaan tehokas algoritmi, joka vie aikaa vain $O(n)$.

Tehtävämme on seuraava: Annettuna on merkkijono, jonka pituus on n ja jokainen merkki on 0 tai 1. Haluamme laskea, monellako tavalla voimme

valita kaksi kohtaa niin, että vasen merkki on 0 ja oikea merkki on 1. Esimerkiksi merkkijonossa 01001 tapoja on neljä: 01001, 01001, 01001 ja 01001.

$O(n^2)$ -algoritmi

Voimme ratkaista tehtävän raa'alla voimalla käymällä läpi kaikki mahdolliset tavat valita vasen ja oikea kohta. Tällöin voimme laskea yksi kerrallaan, monessako tavassa vasen merkki on 0 ja oikea merkki on 1. Seuraava koodi toteuttaa algoritmin:

```
laskuri = 0
for i = 0 to n-1
  for j = i+1 to n-1
    if merkit[i] == 0 and merkit[j] == 1
      laskuri += 1
print(laskuri)
```

Algoritmin aikavaativuus on $O(n^2)$, koska siinä on kaksi sisäkkäistä silmukkaa, jotka käyvät läpi syötteen.

$O(n)$ -algoritmi

Kuinka voisimme ratkaista tehtävän tehokkaammin? Meidän tulisi keksiä tapa, jolla saisimme pois toisen silmukan koodista.

Tässä auttaa lähestyä ongelmaa hieman toisesta näkökulmasta: kun olemme tiettyssä kohdassa merkkijonoa, monellako tavalla voimme muodostaa parin, jonka oikea merkki on nykyisessä kohdassamme? Jos olemme merkin 0 kohdalla, pareja ei ole yhtään, mutta jos merkinä on 1, voimme valita *minkä tahansa* vasemmalla puolella olevan merkin 0 pariin.

Tämän havainnon ansiosta riittää käydä läpi merkkijono kerran vasemmalta oikealle ja pitää kirjaa, montako merkkiä 0 olemme nähneet. Sitten jokaisen merkin 1 kohdalla kasvatamme vastausta tämänhetkisellä merkkien 0 määrällä. Seuraava koodi toteuttaa algoritmin:

```
laskuri = 0
nollat = 0
for i = 0 to n-1
  if merkit[i] == 0
    nollat += 1
  else
    laskuri += nollat
print(laskuri)
```


syötteen koko n	$O(n^2)$ -algoritmi	$O(n)$ -algoritmi
10	0.00 s	0.00 s
10^2	0.00 s	0.00 s
10^3	0.00 s	0.00 s
10^4	0.14 s	0.00 s
10^5	16.22 s	0.00 s
10^6	- -	0.01 s

Table 2.2: Algoritmien suoritusaikojen vertailu.

Algoritmissa on vain yksi silmukka, joka käy syötteen läpi, joten sen aikavaativuus on $O(n)$.

Algoritmien vertailua

Meillä on nyt siis kaksi algoritmia, joiden aikavaativuudet ovat $O(n^2)$ ja $O(n)$, mutta mitä tämä tarkoittaa käytännössä? Saamme tämän selville toteuttamalla algoritmit jollakin oikealla ohjelmointikielellä ja mittaamalla niiden suoritusaikoja erikokoisilla syötteillä.

Taulukko 2.2 näyttää vertailun tulokset, kun algoritmit on toteutettu Javalla ja syöteinä on satunnaisia merkkijonoja. Pienillä n :n arvoilla molemmat algoritmit toimivat hyvin tehokkaasti, mutta suuremmilla syötteillä on nähtävissä huomattavia eroja. Raakaan voimaan perustuva $O(n^2)$ -algoritmi alkaa hidastua selvästi testistä $n = 10^4$ alkaen, ja testissä $n = 10^6$ emme jaksakaan enää odottaa algoritmin valmistumista. Tehokas $O(n)$ -algoritmi taas selvittää suuretkin testit salamannopeasti.

Tämän kurssin jatkuvana teemana on luoda algoritmeja, jotka toimivat tehokkaasti myös silloin, kun niille annetaan suuria syötteitä. Tämä tarkoittaa käytännössä sitä, että algoritmin aikavaativuuden tulisi olla $O(n)$ tai $O(n \log n)$. Jos algoritmin aikavaativuus on esimerkiksi $O(n^2)$, se on auttamatta liian hidas suurien syötteiden käsittelyyn.

2.2 Lisää algoritmien analysoinnista

Aikavaativuuksissa esiintyvä O -merkintä on yksi monista merkinnöistä, joiden avulla voimme arvioida funktioiden kasvunopeutta. Tutustumme seuraavaksi tarkemmin näihin merkintöihin.

2.2.1 Merkinnät O , Ω ja Θ

Algoritmien analysoinnissa usein esiintyviä merkintöjä ovat:

- *Yläraja*: Funktio $g(n)$ on luokkaa $O(f(n))$, jos on olemassa vakiot c ja n_0 niin, että $g(n) \leq cf(n)$ aina kun $n \geq n_0$.
- *Alaraja*: Funktio $g(n)$ on luokkaa $\Omega(f(n))$, jos on olemassa vakiot c ja n_0 niin, että $g(n) \geq cf(n)$ aina kun $n \geq n_0$.
- *Tarkka arvio*: Funktio $g(n)$ on luokkaa $\Theta(f(n))$, jos se on sekä luokkaa $O(f(n))$ että luokkaa $\Omega(f(n))$.

Vakion c tarkoituksena on, että saamme arvion kasvunopeuden suuruusluokalle välittämättä vakiokertoimista. Vakion n_0 ansiosta meidän riittää tarkastella kasvunopeutta suurilla n :n arvoilla. Voimme myös kirjoittaa $g(n) = O(f(n))$, kun haluamme ilmaista, että funktio $g(n)$ on luokkaa $O(f(n))$, ja vastaavasti Ω - ja Θ -merkinnöissä.

Kun sanomme, että algoritmi toimii ajassa $O(f(n))$, tarkoitamme, että se suorittaa *pahimmassa tapauksessa* $O(f(n))$ askelta. Tämä on yleensä hyvä tapa ilmoittaa algoritmin tehokkuus, koska silloin annamme takuun siitä, että algoritmin ajankäytöllä on tietty yläraja, vaikka syöte olisi valittu mahdollisimman ikävästi algoritmin kannalta.

Tarkastellaan esimerkkinä seuraavaa algoritmia, joka laskee taulukon lukujen summan:

```
summa = 0
for i = 0 to n-1
  summa += taulu[i]
```

Tämä algoritmi toimii samalla tavalla riippumatta taulukon sisällöstä, koska se käy aina läpi koko taulukon. Niinpä yläraja ajankäytölle on $O(n)$ ja alaraja ajankäytölle on samoin $\Omega(n)$, joten voimme sanoa, että algoritmi vie aikaa $\Theta(n)$ kaikissa tapauksissa.

Tarkastellaan sitten seuraavaa algoritmia, joka selvittää, onko taulukossa lukua x :

```
ok = false
for i = 0 to n-1
  if taulu[i] == x
    ok = true
  break
```

Tässä algoritmin pahin ja paras tapaus eroavat. Ajankäytön yläraja on $O(n)$, koska algoritmi joutuu käymään läpi kaikki taulukon alkioit silloin, kun luku x ei esiinny taulukossa. Toisaalta ajankäytön alaraja on $\Omega(1)$, koska jos luku x on taulukon ensimmäinen alkio, algoritmi pysähtyy heti taulukon alussa. Voimme myös sanoa, että algoritmi suorittaa pahimmassa tapauksessa $\Theta(n)$ askelta ja parhaassa tapauksessa $\Theta(1)$ askelta.

Huomaa, että O -merkinnän antama yläraja voi olla *mikä tahansa* yläraja, ei välttämättä tarkka yläraja. On siis oikein sanoa esimerkiksi, että algoritmi vie aikaa $O(n^2)$, vaikka on olemassa parempi yläraja $O(n)$. Miksi sitten käytämme O -merkintää, vaikka voisimme usein myös ilmaista tarkan ajankäytön Θ -merkinnällä? Tämä on vakiintunut ja käytännössä toimiva tapa. Olisi hyvin harhaanjohtavaa antaa algoritmille yläraja $O(n^2)$, jos näemme suoraan, että aikaa kuluu vain $O(n)$.

Asiaa voi ajatella niin, että O -merkintää käytetään algoritmin *markkinoinnissa*. Jos annamme liian suuren ylärajan, algoritmista tulee väärä käsitys yleisölle. Vertauksena jos myymme urheiluautoa, jonka huippunopeus on 250 km/h, on sinänsä paikkansa pitävä väite, että autolla pystyy ajamaan 100 km/h. Meidän ei kuitenkaan kannata antaa tällaista vähättelevää tietoa, vaan kertoa, että autolla pystyy ajamaan 250 km/h.

Merkintöjä O , Ω ja Θ voi käyttää kaikenlaisissa yhteyksissä, ei vain algoritmin ajankäytön arvioinnissa. Esimerkiksi voimme sanoa, että algoritmi suorittaa silmukkaa $O(\log n)$ kierrosta tai että listassa on $O(n)$ lukua.

2.2.2 Tilavaativuus

Aikavaativuuden lisäksi kiinnostava tieto algoritmista voi olla sen *tilavaativuus* (*space complexity*). Tämä kuvaa sitä, miten paljon algoritmi käyttää muistia syötteen *lisäksi*. Jos tilavaativuus on $O(1)$, algoritmi tarvitsee muistia vain yksittäisille muuttujille. Jos tilavaativuus on $O(n)$, algoritmi voi varata esimerkiksi aputaulukon, jonka koko vastaa syötteen kokoa.

Tarkastellaan esimerkkinä tehtävää, jossa taulukossa on luvut $1, 2, \dots, n$ yhtä lukuun ottamatta, ja tehtävämme on selvittää puuttuva luku. Yksi tapa ratkaista tehtävä $O(n)$ -ajassa on luoda aputaulukko, joka pitää kirjaa mukana olevista luvuista. Tällaisen ratkaisun tilavaativuus on $O(n)$, koska aputaulukko vie $O(n)$ muistia.

```
for i = 0 to n-2
  mukana[taulu[i]] = true
for i = 1 to n
  if not mukana[i]
    puuttuva = i
```

Tehtävään on kuitenkin olemassa myös toinen algoritmi, jossa aikavaativuus on edelleen $O(n)$ mutta tilavaativuus on vain $O(1)$. Tällainen algoritmi laskee ensin lukujen $1, 2, \dots, n$ summan ja vähentää sitten taulukossa esiintyvät luvut siitä. Jäljelle jäävä luku on puuttuva luku.

```
summa = 0
for i = 1 to n
  summa += i
for i = 0 to n-2
  summa -= taulu[i]
puuttuva = summa
```

Huomaa, että muuttujien ja taulukoiden lisäksi myös *rekursio* voi viedä tilaa, koska rekursiivisen aliohjelman kutsuihin liittyvät tiedot ovat muistissa rekursiopinossa. Esimerkiksi seuraavan proseduurin tilavaativuus on $O(n)$, koska muistissa on korkeimmillaan n kerrosta rekursiivisia kutsuja.

```
procedure f(n)
  if n == 1
    return
  f(n-1)
```

Käytännössä tilavaativuus on yleensä sivuroolissa algoritmeissa, koska jos algoritmi vie vain vähän aikaa, se ei *ehdi* käyttää kovin paljon muistia. Erityisesti tilavaativuus ei voi olla suurempi kuin aikavaativuus. Niinpä riittää tavallisesti keskittyä suunnittelemaan algoritmeja, jotka toimivat nopeasti, ja vertailla algoritmien aikavaativuuksia.

2.2.3 Rajojen todistaminen

Jos haluamme todistaa täsmällisesti, että jokin raja pätee, meidän täytyy löytää vakiot c ja n_0 , jotka osoittavat asian. Jos taas haluamme todistaa, että raja ei päde, meidän täytyy näyttää, että mikään vakioiden c ja n_0 valinta ei ole kelvollinen.

Jos haluamme todistaa rajan pätemisen, tämä onnistuu yleensä helposti valitsemalla vakio c tarpeeksi suureksi ja arvioimalla summan osia ylöspäin

tarvittaessa. Esimerkiksi jos haluamme todistaa, että $3n+5 = O(n)$, meidän tulee löytää vakiot c ja n_0 , joille pätee, että $3n+5 \leq cn$ aina kun $n \geq n_0$. Tässä tapauksessa voimme valita esimerkiksi $c = 8$ ja $n_0 = 1$, jolloin voimme arvioida $3n+5 \leq 3n+5n = 8n$, kun $n \geq 1$.

Jos haluamme todistaa, että raja ei päde, tilanne on hankalampi, koska meidän täytyy näyttää, että ei ole olemassa *mitään* kelvollista tapaa valita vakioita c ja n_0 . Tässä auttaa tyypillisesti vastaoletuksen tekeminen: olettamme, että raja pätee ja voimme valita vakiot, ja näytämme sitten, että tämä oletus johtaa ristiriitaan.

Todistetaan esimerkkinä, että $n^2 \neq O(n)$. Jos pätesi $n^2 = O(n)$, niin olisi olemassa vakiot c ja n_0 , joille $n^2 \leq cn$ aina kun $n \geq n_0$. Voimme kuitenkin osoittaa, että tämä aiheuttaa ristiriidan. Jos $n^2 \leq cn$, niin voimme jakaa epäyhtälön molemmat puolet n :llä ja saamme $n \leq c$. Tämä tarkoittaa, että n on aina enintään yhtä suuri kuin vakio c . Tämä ei ole kuitenkaan mahdollista, koska n voi olla miten suuri tahansa, joten ei voi päteä $n^2 = O(n)$.

Määritelmistä lähtevä todistaminen on sinänsä mukavaa ajanvietettä, mutta sille on äärimmäisen harvoin tarvetta käytännössä, kun haluamme tutkia algoritmien tehokkuutta. Voimme koko kurssin ajan huoletta päätellä algoritmin aikavaativuuden katsomalla, mikä sen rakenne on, kuten olemme tehneet tämän luvun alkuosassa.

Chapter 3

Järjestäminen

Järjestäminen (*sorting*) on keskeinen algoritmiikan ongelma, jossa tehtävänä on järjestää n alkioita sisältävä taulukko suuruusjärjestykseen. Esimerkiksi jos taulukko on $[5, 2, 4, 2, 6, 1]$ ja sen alkiot järjestetään pienimmästä suurimpaan, tuloksena on taulukko $[1, 2, 2, 4, 5, 6]$.

Tavoitteena on toteuttaa järjestäminen *tehokkaasti*. On helppoa järjestää taulukko ajassa $O(n^2)$, mutta tämä on liian hidasta suurella taulukolla. Tässä luvussa opimme kaksi tehokasta järjestämisalgoritmia, jotka vievät aikaa vain $O(n \log n)$. Toisaalta osoittautuu, että ei ole olemassa yleistä järjestämisalgoritmia, joka toimisi nopeammin kuin $O(n \log n)$.

Voimme käyttää järjestämistä monella tavalla algoritmien suunnittelussa, koska voimme usein helpottaa ongelman ratkaisemista järjestämällä ensin aineiston. Luvun lopussa näemme esimerkkejä ongelmista, jotka saamme ratkaistua tehokkaasti järjestämisen avulla.

3.1 Järjestäminen ajassa $O(n^2)$

Tutustumme aluksi yksinkertaiseen järjestämisalgoritmiin, joka järjestää n -alkioisen taulukon ajassa $O(n^2)$ kahden silmukan avulla. Vaikka algoritmi ei ole nopea, se on tutustumisen arvoinen ja antaa hyvän lähtökohdan tehokkaampien algoritmien suunnittelemiselle.

3.1.1 Lisäysjärjestäminen

Lisäysjärjestäminen (*insertion sort*) käy läpi taulukon vasemmalta oikealle. Kun algoritmi tulee tiettyyn taulukon kohtaan, se siirtää kyseisessä kohdassa olevan alkion oikeaan paikkaan taulukon alkuosassa niin, että taulukon alkuosa on tämän jälkeen järjestyksessä. Niinpä kun algoritmi pääsee taulukon

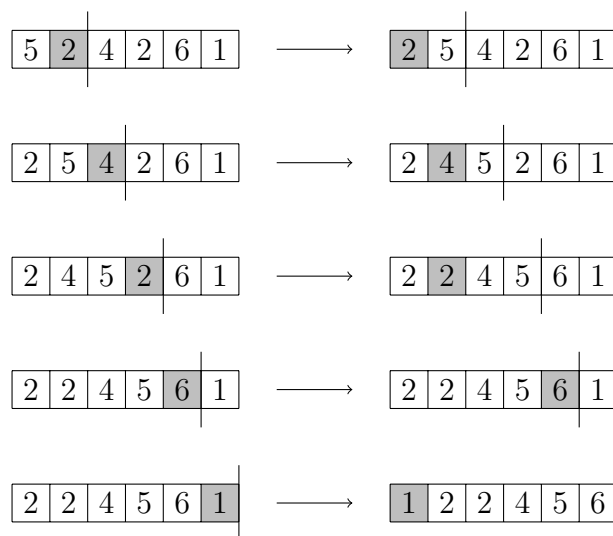


Figure 3.1: Lisäysjärjestäminen taulukolle $[5, 2, 4, 2, 6, 1]$.

loppuun, koko taulukko on järjestyksessä.

Kuva 3.1 näyttää esimerkin lisäysjärjestämisen toiminnasta, kun järjestettävänä on taulukko $[5, 2, 4, 2, 6, 1]$. Jokaisella rivillä algoritmi siirtää harmaataustaisen alkion sen oikealle paikalle taulukon alkuosassa. Pystyviiva ilmaisee kohdan, johon asti taulukko on järjestyksessä siirron jälkeen. Algoritmin päätteeksi tuloksena on järjestetty taulukko $[1, 2, 2, 4, 5, 6]$.

Seuraava koodi toteuttaa lisäysjärjestämisen:

```

for i = 1 to n-1
  j = i-1
  while j >= 0 and taulu[j] > taulu[j+1]
    swap(taulu[j], taulu[j+1])
    j -= 1

```

Koodi käy läpi taulukon kohdat $1 \dots n-1$ ja siirtää aina kohdassa i olevan alkion oikeaan paikkaan. Tämä tapahtuu sisäsilmutalla, jonka jokainen askel vaihtaa keskenään alkion ja sen vasemmalla puolella olevan alkion niin kauan, kuin vasen alkio on suurempi. Huomaa pseudokoodin komento **swap**, joka ilmaisee, että kaksi alkioita vaihdetaan keskenään.

Lisäysjärjestämisen tehokkuus riippuu siitä, mikä on järjestettävän taulukon sisältö. Algoritmi toimii sitä paremmin, mitä lähempänä järjestystä taulukko on valmiiksi. Jos taulukko on järjestyksessä, aikaa kuluu vain $O(n)$, koska ei tarvitse siirtää mitään alkioita. Pahin tapaus algoritmille on kuitenkin, että taulukko on *käänteisessä* järjestyksessä, jolloin jokainen alkio täytyy

siirtää taulukon alkuun ja aikaa kuluu $O(n^2)$.

3.1.2 Inversiot

Hyödyllinen käsite järjestämisalgoritmien analysoinnissa on *inversio*: kaksi taulukossa olevaa alkioita, jotka ovat väärässä järjestyksessä. Esimerkiksi taulukossa $[3, 1, 4, 2]$ on kolme inversiota: $(3, 1)$, $(3, 2)$ ja $(4, 2)$. Inversioiden määrä kertoo taulukon järjestyksestä: mitä vähemmän inversioita taulukossa on, sitä lähempänä se on järjestystä. Erityisesti taulukko on järjestyksessä tarkalleen silloin, kun siinä ei ole yhtään inversiota.

Kun järjestämisalgoritmi järjestää taulukon, se *poistaa* siitä inversioita. Esimerkiksi aina kun lisäysjärjestäminen vaihtaa vierekkäiset alkiot keskenään, se poistaa taulukosta yhden inversion. Niinpä lisäysjärjestämisen työmäärä on yhtä suuri kuin järjestettävän taulukon inversioiden määrä.

Olemme jo todenneet, että pahin mahdollinen syöte lisäysjärjestämiselle on käänteisessä järjestyksessä oleva taulukko. Tällaisessa taulukossa jokainen alkio pari muodostaa inversion, joten inversioiden määrä on

$$\frac{n(n-1)}{2} = O(n^2).$$

Entä kuinka hyvin lisäysjärjestäminen toimii *keskimäärin*? Jos oletamme, että taulukossa on n eri alkioita satunnaisessa järjestyksessä, jokainen taulukossa oleva alkio pari muodostaa inversion todennäköisyydellä $1/2$. Niinpä inversioiden määrän *odotusarvo* on

$$\frac{n(n-1)}{4} = O(n^2),$$

eli aikaa kuluu neliöllinen määrä myös keskimääräisessä tapauksessa.

Syy lisäysjärjestämisen hitauteen on, että se ei poista taulukosta inversioita riittävän tehokkaasti. Jos haluamme kehittää paremman järjestämisalgoritmin, se täytyy suunnitella niin, että se voi poistaa useita inversioita *yhtä aikaa*. Käytännössä algoritmin täytyy pystyä siirtämään väärässä paikassa oleva alkio tehokkaasti taulukon toiselle puolelle.

3.2 Järjestäminen ajassa $O(n \log n)$

Seuraavaksi tutustumme kahteen tehokkaaseen järjestämisalgoritmiin, jotka perustuvat rekursioon. Molemmissa algoritmeissa on ideana järjestää taulukko jakamalla se kahteen pienempään osataulukoon ja järjestämällä ne rekursiivisesti. Tämän jälkeen järjestetyt osataulukot yhdistetään kokonaiseksi järjestetyksi taulukoksi.

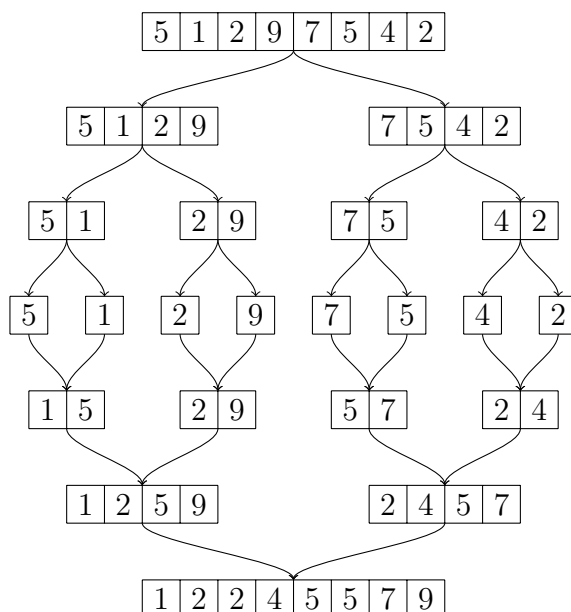


Figure 3.2: Lomitusjärjestäminen taulukolle $[5, 1, 2, 9, 7, 5, 4, 2]$.

3.2.1 Lomitusjärjestäminen

Lomitusjärjestäminen (*merge sort*) on rekursiivinen järjestämisalgoritmi, joka perustuu taulukon puolituksiin. Kun järjestettävänä on n -alkioinen taulukko, se jaetaan keskeltä kahdeksi osataulukoksi, joissa molemmissa on noin $n/2$ alkioita. Tämän jälkeen osataulukot järjestetään erikseen rekursiivisesti ja järjestetyt osataulukot *lomitetaan* niin, että niistä muodostuu kokonainen järjestetty taulukko. Rekursio päättyy tapaukseen $n = 1$, jolloin taulukko on valmiiksi järjestyksessä eikä tarvitse tehdä mitään.

Seuraava koodi esittää tarkemmin lomitusjärjestämisen toiminnan:

```

procedure jarjesta(a,b)
  if a == b
    return
  k = (a+b)/2
  jarjesta(a,k)
  jarjesta(k+1,b)
  lomita(a,k,k+1,b)

```

Proseduuri *jarjesta* järjestää taulukon välin $a \dots b$ (osataulukon kohdasta a kohtaan b), eli kun haluamme järjestää koko taulukon, kutsumme proseduuria parametreilla $a = 0$ ja $b = n - 1$. Proseduuri tarkastaa ensin, onko osataulukossa vain yksi alkio, ja jos näin on, proseduuri päättyy

heti. Muuten se laskee muuttujaan k järjestettävän välin keskikohdan ja järjestää vasemman ja oikean puoliskon rekursiivisesti. Lopuksi se kutsuu proseduuria `lomita`, joka yhdistää järjestetyt puoliskot. Seuraava koodi näyttää, kuinka voimme toteuttaa tämän proseduurin:

```
procedure lomita(a1, b1, a2, b2)
  a = a1, b = b2
  for i = a to b
    if a2 > b2 or (a1 <= b1 and taulu[a1] <= taulu[a2])
      apu[i] = taulu[a1]
      a1 += 1
    else
      apu[i] = taulu[a2]
      a2 += 1
  for i = a to b
    taulu[i] = apu[i]
```

Parametreina annetaan välit $a_1 \dots b_1$ ja $a_2 \dots b_2$, missä $b_1 + 1 = a_2$. Proseduri olettaa, että näillä väleillä olevat taulukon alkiot on järjestetty, ja se lomittaa alkiot niin, että taulukon koko väli $a_1 \dots b_2$ on järjestetty. Proseduurin perustana on silmukka, joka käy läpi välejä $a_1 \dots b_1$ ja $a_2 \dots b_2$ rinnakkain ja valitsee aina seuraavaksi pienimmän alkion lopulliseen järjestykseen. Jotta lomitusta ei sotke taulukkoa, proseduri käyttää globaalia aputaulukkoa, johon se ensin muodostaa järjestetyn osataulukon, ja kopioi sitten alkiot aputaulukosta varsinaiseen taulukkoon.

Kuva 3.2 näyttää, miten lomituserjestäminen toimii, kun sille annetaan taulukko $[5, 1, 2, 9, 7, 5, 4, 2]$. Algoritmi puolittaa ensin taulukon kahdeksi osataulukoksi $[5, 1, 2, 9]$ ja $[7, 5, 4, 2]$ ja järjestää molemmat osataulukot kutsumalla itseään. Kun algoritmi saa sitten järjestettäväksi taulukon $[5, 1, 2, 9]$, se jakaa taulukon edelleen osataulukoiksi $[5, 1]$ ja $[2, 9]$, jne. Lopulta jäljellä on vain yhden alkion kokoisia osataulukoita, jotka ovat valmiiksi järjestyksessä. Tällöin rekursiivinen jakautuminen päättyy ja algoritmi alkaa koota järjestetyjä osataulukoita pienimmästä suurimpaan.

Kuinka tehokas lomituserjestäminen on? Koska jokainen proseduurin `jarjesta` kutsu puolittaa taulukon koon, rekursiosta muodostuu $O(\log n)$ tasoa (kuva 3.2). Ylimmällä tasolla on taulukko, jossa on n alkia, seuraavalla tasolla on kaksi taulukkoa, joissa on $n/2$ alkia, seuraavalla tasolla on neljä taulukkoa, joissa on $n/4$ alkia, jne. Proseduri `lomita` toimii lineaarisessa ajassa, joten kullakin tasolla taulukoiden lomittamiset vievät yhteensä aikaa $O(n)$. Niinpä algoritmin kokonaisaikaavaativuus on $O(n \log n)$.

3.2.2 Pikajärjestäminen

Pikajärjestäminen (*quick sort*) tarjoaa toisenlaisen rekursiivisen lähestymistavan taulukon järjestämiseen. Kun saamme järjestettäväksi taulukon, valitsemme ensin jonkin sen alkioista *jakoalkioksi* (*pivot*). Tämän jälkeen siirrämme alkioita niin, että jakoalkiota pienemmät alkiot ovat sen vasemmalla puolella, suuremmat alkiot ovat sen oikealla puolella ja yhtä suuret alkiot voivat olla kummalla tahansa puolella. Lopuksi järjestämme rekursiivisesti osataulukot, jotka muodostuvat jakoalkion vasemmalle ja oikealle puolelle.

Seuraava koodi esittää pikajärjestämisen toiminnan:

```
procedure jarjesta(a, b)
  if a >= b
    return
  k = jako(a,b)
  jarjesta(a,k-1)
  jarjesta(k+1,b)
```

Proseduuri `jarjesta` järjestää taulukon välillä $a \dots b$ olevat alkiot. Jos väli on tyhjä tai siinä on vain yksi alkio, proseduuri ei tee mitään. Muuten se kutsuu funktiota `jako`, joka valitsee jakoalkion, siirtää taulukon alkioita sen mukaisesti ja palauttaa sitten kohdan k , jossa jakoalkio on siirtojen jälkeen. Tämän jälkeen taulukon vasen osa (väli $a \dots k-1$) ja oikea osa (väli $k+1 \dots b$) järjestetään rekursiivisesti.

Funktion `jako` voi toteuttaa monella tavalla, koska voimme valita minkä tahansa alkion jakoalkioksi ja lisäksi on monia tapoja siirtää alkioita. Käytämme tässä esimerkkinä seuraavaa toteutusta:

```
function jako(a,b):
  k = a
  for i = a+1 to b
    if taulu[i] < taulu[a]
      k += 1
      swap(taulu[i],taulu[k])
  swap(taulu[a],taulu[k])
  return k
```

Tässä jakoalkio on aina kohdassa a oleva välin ensimmäinen alkio. Funktio käy läpi välin alkiot ja siirtää jakoalkiota pienempiä alkioita taulukon alkuosaan. Muuttuja k määrittää kohdan, johon seuraava pienempi alkio siirretään. Lopuksi jakoalkio itse siirretään keskelle kohtaan k , jonka funktio myös palauttaa.

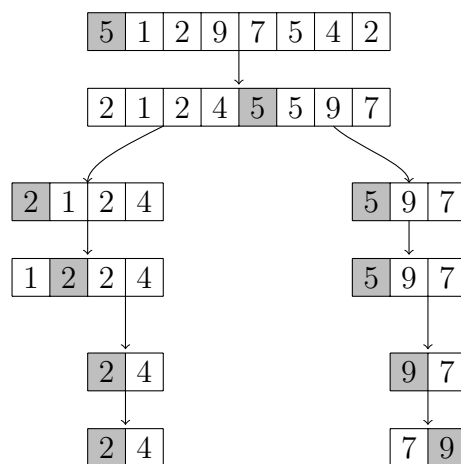


Figure 3.3: Pikajärjestäminen taulukolle $[5, 1, 2, 9, 7, 5, 4, 2]$.

Kuva 3.3 näyttää, miten pikajärjestäminen toimii, kun sille annetaan taulukko $[5, 1, 2, 9, 7, 5, 4, 2]$. Jokaisessa vaiheessa harmaa tausta osoittaa jakoalkion sijainnin. Aluksi koko taulukon jakoalkio on 5 ja algoritmi siirtää alkioita niin, että jakoalkion vasemmalla puolella ovat alkio $[2, 1, 2, 4]$ ja oikealla puolella alkio $[5, 9, 7]$. Tämän jälkeen vasen ja oikea osataulukko järjestetään vastaavasti rekursiivisesti.

Pikajärjestämisen tehokkuuteen vaikuttaa, miten alkio jakautuvat jakoalkion eri puolille. Jos hyvin käy, jakoalkion kummallekin puolelle siirretään suunnilleen yhtä monta alkioita. Tällöin taulukon koko puolittuu jokaisen jaon jälkeen ja pikajärjestäminen toimii tehokkaasti. Koska funktio `jako` toimii lineaarisessa ajassa, pikajärjestäminen vie tässä tapauksessa aikaa $O(n \log n)$ samaan tapaan kuin lomitusjärjestäminen. Uhkana on kuitenkin, että jakoalkio jakaa taulukon osiin *epätasaisesti*. Kuva 3.4 näyttää tilanteen, jossa jokaisessa jaossa kaikki alkioit jäävät jakoalkion oikealle puolelle. Tällöin pikajärjestäminen viekin aikaa $O(n^2)$, koska rekursiivisia tasoja on $O(n)$. Selvästikään *ei* ole kaikissa tilanteissa hyvä tapa valita taulukon ensimmäinen alkio jakoalkioksi. Esimerkiksi valmiiksi järjestyksessä olevan taulukon järjestäminen vie silloin aikaa $O(n^2)$.

Oma lukunsa on tilanne, jossa taulukossa on paljon samoja alkioita. Ääritapauksena *jokainen* alkio on sama, mikä on käytännössä hyvin mahdollinen syöte. Tällöin tässä esitetty funktio `jako` tuottaa aina huonon jaon, jossa kaikki alkioit menevät jakoalkion oikealle puolelle. Ei siis riitä, että jakoalkio on valittu hyvin, vaan myös tapa, jolla alkioita siirretään taulukossa, vaikuttaa algoritmin tehokkuuteen.

Miten meidän tulisi sitten toteuttaa pikajärjestäminen? Algoritmista on kehitetty vuosien aikana suuri määrä muunnelmia, jotka pyrkivät paranta-

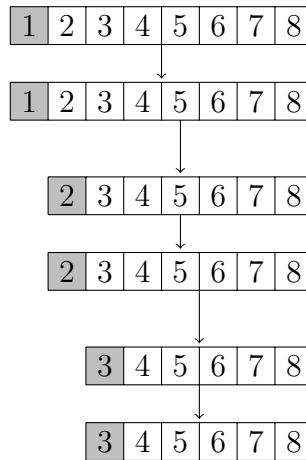


Figure 3.4: Pikajärjestämisen pahin tapaus: jokaisessa jaossa kaikki alkiot jäävät jakoalkion toiselle puolelle.

algoritmi	tehokkuus	lisätila	vakaa
lisäysjärjestäminen	$O(n^2)$	$O(1)$	kyllä
lomitusjärjestäminen	$O(n \log n)$	$O(n)$	kyllä
pikajärjestäminen (hyvä tapaus)	$O(n \log n)$	$O(\log n)$	ei
pikajärjestäminen (huono tapaus)	$O(n^2)$	$O(n)$	ei

Table 3.1: Järjestämisalgoritmien vertailua.

maan sen toimintaa eri tilanteissa. Yksi kehittyneempi tapa valita jakoalkio on ottaa tarkasteluun taulukon ensimmäinen, keskimäinen ja viimeinen alkio ja valita jakoalkioksi järjestyksessä keskimäinen näistä kolmesta alkioista. Tällainen valinta toimii käytännössä hyvin monissa tilanteissa. Parempi tapa toteuttaa alkioden siirtäminen on puolestaan käydä läpi rinnakkain alkioita alusta ja lopusta ja pitää huoli siitä, että jakokohta jää keskelle, jos kaikki alkiot ovat yhtä suuria. Pikajärjestämisen toteuttaminen hyvin ei ole helppo tehtävä, vaan vaatii paljon huolellisuutta.

3.2.3 Algoritmien vertailua

Taulukossa 3.1 on yhteenveto tähän mennessä käsitellyistä järjestämisalgoritmeista. Algoritmin lisätila tarkoittaa sen tilavaativuutta, eli paljonko tilaa algoritmi tarvitsee järjestettävän taulukon lisäksi. Lisäysjärjestäminen käyttää vain yksittäisiä muuttujia, joten sen lisätila on $O(1)$. Lomitusjärjestämisessä lomittaminen käyttää aputaulukkoa, mistä aiheutuu lisätila $O(n)$. Pikajärjestämisessä puolestaan rekursiivinen kutsupino vie tilaa muistissa riippuen rekursion syvyydestä. Al-

goritmi on *vakaa* (*stable*), jos se säilyttää yhtä suuret alkiot alkuperäisessä järjestyksessä. Lisäysjärjestäminen ja lomituserjestäminen ovat vakaita, koska ne eivät vaihda koskaan yhtä suurten alkioden järjestykstä, mutta pikajärjestäminen saattaa tehdä näin jakoalkion valinnan jälkeen. Algoritmin vakauudesta voi olla hyötyä sovelluksissa.

Meillä on nyt siis kaksi rekursiivista järjestämisalgoritmia: lomituserjestäminen toimii *aina* ajassa $O(n \log n)$, kun taas pikajärjestäminen toimii *ehkä* ajassa $O(n \log n)$, mutta saattaa viedä aikaa $O(n^2)$. Vaatii monenlaista virittelyä, ennen kuin pikajärjestämisen saa toimimaan tehokkaasti edes tapauksissa, joissa taulukko on valmiiksi järjestyksessä tai kaikki alkiot ovat samoja. Miksi haluaisimme koskaan käyttää epävarmaa pikajärjestämistä, kun voimme käyttää myös varmasti tehokasta lomituserjestämistä?

Syynä on, että pikajärjestämisen *vakiokertoimet* ovat pienet. Kokemus on osoittanut, että kun toteutamme lomituserjestämisen ja pikajärjestämisen ja mittaamme algoritmien todellisia suoritusajoja, pikajärjestäminen toimii usein nopeammin. Näin tapahtuu siitä huolimatta, että pikajärjestämisen pahimman tapauksen aikavaativuus on $O(n^2)$. Käytännössä pahin tapaus on kuitenkin harvinainen, jos jakoalkion valinta ja alkioden siirtäminen on toteutettu huolellisesti.

Jos taulukko on pieni, $O(n^2)$ -aikainen lisäysjärjestäminen toimii käytännössä nopeammin kuin rekursiiviset algoritmit, koska sen vakiokertoimet ovat hyvin pienet. Yksi mahdollisuus onkin toteuttaa *hybridialgoritmi* (*hybrid algorithm*), jossa suuret taulukot järjestetään rekursiivisesti ja pienet taulukot järjestetään lisäysjärjestämisellä. Tällöin eri algoritmien hyvät puolet pääsevät osaksi kokonaisalgoritmia. Käytännössä hybridialgoritmin voi tehdä niin, että valitaan jokin sopiva raja k ja taulukko järjestetään rekursiivisesti, jos siinä on ainakin k alkioita, ja muuten lisäysjärjestämisellä.

3.3 Järjestämisen alaraja

Olisiko mahdollista luoda järjestämisalgoritmi, joka toimisi nopeammin kuin $O(n \log n)$? Osoittautuu, että tämä *ei* ole mahdollista, jos oletamme, että algoritmin tulee perustua taulukon alkioden vertailuihin. Vertailuihin perustuva järjestämisalgoritmi järjestää taulukon tekemällä joukon vertailuja muotoa ”onko alkio x suurempi kuin alkio y ?”.

Vertailuihin perustuva järjestämisalgoritmi on *yleiskäyttöinen*: se pystyy järjestämään mitä tahansa alkioita, kunhan meillä on keino saada selville kahden alkion suuruusjärjestys. Tämä on ominaisuus, jota yleensä ottaen toivomme järjestämisalgoritmilta, joten vertailuihin perustuminen on luonteva rajoitus. Kaikki tähän mennessä käsittelemämme järjestämisalgoritmit

ovat olleet vertailuihin perustuvia.

3.3.1 Alarajatodistus

Voimme ajatella vertailuihin perustuvaa järjestämistä *prosessina*, jossa jokainen vertailu antaa tietoa taulukosta ja auttaa viemään taulukkoa lähemmäs järjestystä. Oletamme, että taulukon alkiot ovat $1, 2, \dots, n$, jolloin on $n!$ vaihtoehtoa, mikä on taulukon alkuperäinen järjestys. Jotta järjestämisalgoritmi voisi toimia oikein, sen täytyy käsitellä jokainen järjestys eri tavalla.

Esimerkiksi jos $n = 3$, taulukon mahdolliset järjestykset alussa ovat $[1, 2, 3]$, $[1, 3, 2]$, $[2, 1, 3]$, $[2, 3, 1]$, $[3, 1, 2]$ ja $[3, 2, 1]$. Algoritmi voi vertailla ensin vaikkapa ensimmäistä ja toista alkioita. Jos ensimmäinen alkio on pienempi, voimme päätellä, että mahdolliset taulukot ovat $[1, 2, 3]$, $[1, 3, 2]$ ja $[2, 3, 1]$. Jos taas ensimmäinen alkio on suurempi, mahdolliset taulukot ovat $[2, 1, 3]$, $[3, 1, 2]$ ja $[3, 2, 1]$. Tämän jälkeen voimme jatkaa vertailuja samaan tapaan ja saada lisää tietoa taulukosta. Algoritmi voi päättyä vasta silloin, kun jäljellä on vain yksi mahdollinen taulukko, jotta voimme olla varmoja, että olemme järjestäneet taulukon oikein.

Tärkeä seikka on, että jokaisessa vertailussa ainakin puolet jäljellä olevista taulukoista voi täsmätä vertailun tulokseen. Niinpä jos algoritmilla käy huono tuuri, se voi enintään puolittaa taulukoiden määrän joka askeleella. Tämä tarkoittaa, että algoritmi joutuu tekemään pahimmassa tapauksessa ainakin $\log(n!)$ vertailua. Logaritmien laskusääntöjen perusteella

$$\log(n!) = \log(1) + \log(2) + \dots + \log(n).$$

Saamme tälle summalle alarajan ottamalla huomioon vain $n/2$ viimeistä termiä ja arvioimalla niitä alaspäin niin, että jokaisen termin suuruus on vain $\log(n/2)$. Tuloksena on alaraja

$$\log(n!) \geq (n/2) \log(n/2),$$

mikä tarkoittaa, että algoritmi joutuu tekemään pahimmassa tapauksessa $\Omega(n \log n)$ vertailua.

3.3.2 Laskemisjärjestäminen

Millainen olisi sitten järjestämisalgoritmi, joka ei perustu vertailuihin ja toimii tehokkaammin kuin $O(n \log n)$? *Laskemisjärjestäminen* (*counting sort*) on $O(n)$ -aikainen järjestämisalgoritmi, jonka toiminta perustuu oletukseen, että taulukon alkiot ovat sopivan pieniä kokonaislukuja. Algoritmi olettaa, että jokainen alkio on kokonaisluku välillä $0 \dots k$, missä $k = O(n)$.

Algoritmi luo *kirjanpidon*, joka kertoo, montako kertaa mikäkin mahdollinen luku välillä $0 \dots k$ esiintyy taulukossa. Seuraavassa koodissa kirjanpito tallennetaan taulukkoon *laskuri* niin, että *laskuri*[*x*] ilmaisee, montako kertaa luku *x* esiintyy taulukossa. Tämän kirjanpidon avulla voimme luoda suoraan lopullisen järjestetyn taulukon.

```
for i = 0 to n-1
    laskuri[taulu[i]] += 1
i = 0
for x = 0 to k
    for j = 1 to laskuri[x]
        taulu[i] = x
        i += 1
```

Algoritmin molemmat vaiheet vievät aikaa $O(n)$, joten se toimii ajassa $O(n)$ ja on käytännössä hyvin tehokas. Algoritmi ei ole kuitenkaan yleinen järjestämisalgoritmi, koska sitä voi käyttää vain silloin, kun taulukon kaikki alkiot ovat sopivan pieniä kokonaislukuja.

3.4 Ohjelmointikielten toteutukset

Vaikka on hyödyllistä tuntea järjestämisen teoriaa, käytännössä ei ole hyvä idea toteuttaa itse järjestämisalgoritmia, koska nykypäivän ohjelmointikielissä on valmiit työkalut järjestämiseen. Valmiin algoritmin käyttämisessä on etuna, että se on varmasti hyvin toteutettu ja tehokas. Lisäksi ohjelmoijan aikaa säästyy, kun algoritmia ei joudu toteuttamaan itse.

3.4.1 Java

Javan standardikirjaston metodi `Arrays.sort` järjestää sille annetun taulukon seuraavaan tapaan:

```
int[] taulu = {4,2,5,8,2,1,5,6};
Arrays.sort(taulu);
// tulos: [1,2,2,4,5,5,6,8]
```

Metodin sisäinen toiminta riippuu siitä, minkä *tyyppistä* tietoa taulukossa on. Jos taulukon alkiot ovat alkeistyyppisiä (esimerkiksi `int`), Java käyttää pikajärjestämisen muunnelmaa, jossa on kaksi jakoalkiota. Jos taas alkiot ovat oliotyyppisiä (esimerkiksi `String`), algoritmina on Timsort, joka on optimoitu lomitusjärjestäminen.

Kun `Arrays.sort` järjestää olioita sisältävän taulukon, se kutsuu metodia `compareTo` aina, kun se haluaa selvittää kahden alkion suuruusjärjestyksen. Metodin tulee palauttaa negatiivinen arvo, nolla tai positiivinen arvo sen mukaan, onko olio itse pienempi, yhtä suuri vai suurempi kuin parametrina annettu olio. Javan omissa luokissa tällainen metodi on olemassa valmiina. Esimerkiksi `"apina".compareTo("banaani")` palauttaa negatiivisen arvon, koska `apina` on ennen `banaania` aakkosjärjestyksessä.

Tämän ansiosta voimme järjestää suoraan taulukoita, joissa on esimerkiksi merkkijonoja. Jos haluamme, että Java pystyy järjestämään omien luokkien olioita, meidän täytyy toteuttaa itse luokkaan metodi `compareTo` ja merkitä, että luokka toteuttaa rajapinnan `Comparable`.

Esimerkiksi seuraava koodi toteuttaa luokan `Piste`, johon voidaan tallentaa pisteen x- ja y-koordinaatit. Luokassa on metodi `compareTo`, joka määrittelee, että pisteet järjestetään ensisijaisesti x-koordinaatin ja toissijaisesti y-koordinaatin mukaan.

```
public class Piste implements Comparable<Piste> {
    private int x, y;

    public Piste(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public int compareTo(Piste p) {
        if (this.x != p.x) {
            return this.x - p.x;
        } else {
            return this.y - p.y;
        }
    }
}
```

Tässä metodi `compareTo` palauttaa x-koordinaattien erotuksen, jos x-koordinaatit eivät ole samat, ja muuten y-koordinaattien erotuksen. Metodi olettaa, että koordinaattien erotus voidaan esittää `int`-tyyppisenä arvona.

Voimme nyt järjestää taulukollisen pisteitä näin:

```
Piste[] pisteet = {new Piste(1,3),  
                    new Piste(2,3),  
                    new Piste(1,2)};  
Arrays.sort(pisteet);  
// tulos: [(1,2),(1,3),(2,3)]
```

3.4.2 Python

Pythonissa metodi `sort` järjestää listan sisällön:

```
lista = [4,2,5,8,2,1,5,6]  
lista.sort()  
# tulos: [1,2,2,4,5,5,6,8]
```

Toinen tapa on käyttää funktiota `sorted`, joka palauttaa järjestetyn kopion parametrina annetusta listasta:

```
kopio = sorted(lista)
```

Python käyttää järjestämiseen Timsort-algoritmia, joka on optimoitu lomitusjärjestäminen. Sama algoritmi on käytössä Javassa silloin, kun taulukossa on olioita, ja algoritmi on kulkeutunut Javaan Pythonista.

Järjestäminen toimii automaattisesti myös silloin, kun listan alkiot muodostuvat useammasta osasta:

```
lista = [(1,3),(2,3),(1,2)]  
lista.sort()  
# tulos: [(1,2),(1,3),(2,3)]
```

Oletuksena alkiot järjestetään pienimmästä suurimpaan, mutta voimme muuttaa suunnan parametrilla `reverse`:

```
lista.sort(reverse=True)
```

Lisäksi voimme antaa parametrilla `key` funktion, joka muuntaa alkion avaimeksi, jonka mukaan järjestäminen tapahtuu. Esimerkiksi seuraava koodi järjestää luvut niiden *itseisarvon* mukaan:

```
lista = [4,-2,-7,5,1]  
lista.sort(key=abs)  
# tulos: [1,-2,4,5,-7]
```

3.5 Järjestämisen sovelluksia

Järjestämisen merkitys algoritmikassa on, että voimme ratkaista monia ongelmia tehokkaasti, kunhan aineisto on järjestyksessä. Niinpä yleinen tapa luoda tehokas algoritmi on järjestää ensin syöte ajassa $O(n \log n)$ ja hyödyntää sitten tavalla tai toisella järjestystä algoritmin loppuosassa.

3.5.1 Taulukkoalgoritmeja

Järjestämisen avulla voimme ratkaista ajassa $O(n \log n)$ monia taulukoihin liittyviä tehtäviä. Käymme seuraavaksi läpi kaksi tällaista tehtävää.

Ensimmäinen tehtävämme on laskea, montako *eri* alkia annettussa taulukossa on. Esimerkiksi taulukko $[2, 1, 4, 2, 4, 2]$ sisältää kolme eri alkia: 1, 2 ja 4. Voimme ratkaista tehtävän järjestämällä ensin taulukon, minkä jälkeen yhtä suuret alkut ovat vierekkäin. Tämän jälkeen saamme laskettua vastauksen helposti, koska riittää tutkia, monessako taulukon kohdassa on *vierekkäin* kaksi eri alkia. Voimme toteuttaa algoritmin näin:

```
sort(taulu)
laskuri = 1
for i = 1 to n-1
    if taulu[i-1] != taulu[i]
        laskuri += 1
print(laskuri)
```

Algoritmi järjestää ensin taulukon ajassa $O(n \log n)$, minkä jälkeen se käy läpi taulukon sisällön for-silmukalla ajassa $O(n)$. Tämän ansiosta algoritmi vie aikaa yhteensä $O(n \log n)$.

Entä jos haluammekin selvittää, mikä on taulukon *yleisin* alkio? Esimerkiksi taulukon $[2, 1, 4, 2, 4, 2]$ yleisin alkio on 2, joka esiintyy kolme kertaa taulukossa. Tämäkin tehtävä ratkeaa järjestämisen avulla, koska järjestämisen jälkeen yhtä suuret alkut ovat peräkkäin ja meidän riittää etsiä pisin samaa alkia toistava osuus. Voimme toteuttaa algoritmin seuraavasti:

```
sort(taulu)
maara = 1
suurin = 1
yleisin = taulu[0]
for i = 1 to n-1
    if taulu[i-1] != taulu[i]
        maara = 0
    maara += 1
    if maara > suurin
        suurin = maara
        yleisin = taulu[i]
print(yleisin)
```

Tässäkin algoritmissa järjestäminen vie aikaa $O(n \log n)$ ja for-silmukka vie aikaa $O(n)$, joten algoritmin kokonaisaikavaativuus on $O(n \log n)$.

3.5.2 Binäärihaku

Binäärihaku (*binary search*) on menetelmä, jonka avulla voimme löytää alkion järjestetystä taulukosta ajassa $O(\log n)$. Ideana on pitää yllä hakuväliä, jossa etsittävä alkio voi olla, ja puolittaa väli joka askeleella tutkimalla välin keskimmäisenä olevaa alkia. Koska taulukko on järjestyksessä, voimme aina päätellä, kumpaan suuntaan hakua tulee jatkaa.

Seuraava koodi etsii binäärihaulla taulukosta alkia x :

```
a = 0
b = n-1
while a <= b
    k = (a+b)/2
    if taulu[k] == x
        // alkio löytyi
        break
    if taulu[k] < x
        a = k+1
    if taulu[k] > x
        b = k-1
```

Algoritmi pitää yllä hakuväliä $[a, b]$, joka on aluksi $[0, n - 1]$, koska alkio x saattaa olla missä tahansa kohdassa taulukossa. Joka askeleella algoritmi tarkastaa välin keskellä kohdassa $k = \lfloor (a+b)/2 \rfloor$ olevan alkion. Jos kyseinen alkio on x , haku päättyy. Jos taas alkio on pienempi kuin x , haku jatkuu välin oikeaan puoliskoon, ja jos alkio on suurempi kuin x , haku jatkuu välin

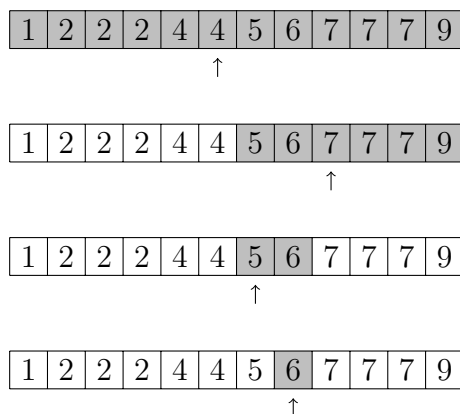


Figure 3.5: Luvun 6 etsiminen järjestetystä taulukosta binäärihaun avulla. Harmaa alue vastaa väliä $[a, b]$ ja nuoli osoittaa kohdan k .

vasempaan puoliskoon. Koska välin koko puolittuu joka askeleella, binäärihaun aikavaativuus on $O(\log n)$.

Kuva 3.5 näyttää esimerkin binäärihaun toiminnasta, kun etsimme lukua 6 järjestetystä taulukosta. Aluksi hakuvälinä on koko taulukko ja puolivälin kohdalla on luku 4. Niinpä voimme päätellä, että jos taulukossa on luku 6, sen täytyy esiintyä taulukon loppuosassa. Tämän jälkeen hakuvälinä on taulukon loppuosa ja puolivälin kohdalla on luku 7. Nyt voimme taas päätellä, että luvun 6 täytyy olla tämän kohdan vasemmalla puolella. Hakuväli puolittuu joka askeleella, ja kun jatkamme vastaavasti, kahden askeleen kuluttua hakuvälillä on vain yksi luku, joka on juuri haettu luku 6. Olemme löytäneet halutun luvun ja algoritmi päättyy.

Voimme selvittää binäärihaun avulla esimerkiksi tehokkaasti, onko taulukossa kahta alkia a ja b niin, että $a + b = x$, missä x on annettu arvo. Ideana on järjestää ensin taulukko ja käydä sitten läpi kaikki taulukon luvut. Jokaisen luvun kohdalla tutkimme, voisiko kyseinen luku olla a . Tällöin taulukossa pitäisi olla toinen luku b niin, että $a + b = x$, eli taulukossa pitäisi olla luku $x - a$. Pystymme tarkastamaan tämän binäärihaulla ajassa $O(\log n)$. Tuloksena on algoritmi, joka vie aikaa $O(n \log n)$, koska sekä järjestäminen että binäärihakua käyttävä läpikäynti vievät aikaa $O(n \log n)$.

Chapter 4

Lista

Lista (*list*) on taulukkoa muistuttava muuttuvan kokoinen tietorakenne, joka muodostuu peräkkäin olevista alkioista. Esimerkiksi $[3, 7, 2, 5]$ on lista, joka sisältää neljä alkioita. Haluamme toteuttaa listan niin, että pääsemme käsiksi listalla olevaan alkioon sen kohdan perusteella ja lisäksi pystymme lisäämään ja poistamaan alkioita.

Tässä luvussa tutustumme kahteen lähestymistapaan listan toteuttamiseen. Ensin toteutamme taulukkolistan, jossa listan alkiot tallennetaan taulukkoon. Tämän jälkeen toteutamme linkitetyn listan, joka muodostuu toisiinsa viittaavista solmuista. Kuten tulemme huomaamaan, molemmissa listan toteutuksissa on omat hyvät ja huonot puolensa.

4.1 Taulukkolistat

Taulukkolistat (*array list*) on lista, joka on tallennettu taulukkona. Koska taulukon alkiot ovat peräkkäin muistissa, pääsemme käsiksi mihin tahansa listan alkioon ajassa $O(1)$. Haasteena toteutuksessa on kuitenkin, että taulukon koko on *kiinteä* ja listan koon muuttamiseksi täytyy varata uusi taulukko ja kopioida sinne vanhan taulukon sisältö.

4.1.1 Muutokset lopussa

Toteutamme ensin taulukkolistan, jossa alkioden lisäykset ja poistot tapahtuvat listan lopussa. Tallennamme listan taulukkona niin, että tietty määrä alkioita taulukon alussa on listan käytössä ja loput tyhjät kohdat on varattu tuleville alkioille. Tämän ansiosta pystymme lisäämään uuden alkion listalle ajassa $O(1)$, jos taulukossa on tilaa, koska riittää ottaa käyttöön seuraava vapaana oleva kohta taulukosta.

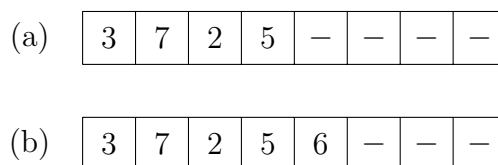


Figure 4.1: (a) Lista $[3, 7, 2, 5]$ tallennettuna taulukkoon. (b) Listan loppuun lisätään alkio 6.

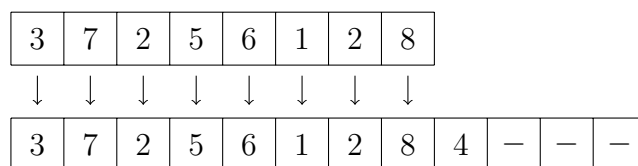


Figure 4.2: Taulukkoon ei mahdu enää uutta alkioita. Meidän täytyy varata uusi suurempi taulukko ja kopioida vanhan taulukon sisältö sinne.

Kuva 4.1 näyttää esimerkin, jossa taulukossa on tilaa yhteensä kahdeksalle alkioille ja siihen on tallennettu lista $[3, 7, 2, 5]$. Taulukon neljä ensimmäistä kohtaa ovat siis listan käytössä ja muut ovat varalla tulevia alkioita varten. Kun lisäämme listan loppuun uuden alkion 6, otamme käyttöön taulukosta uuden kohdan, johon alkio sijoitetaan.

Mitä tapahtuu sitten, kun jossain vaiheessa koko taulukko on täynnä eikä uusi listalle lisättävä alkio mahdu enää taulukkoon? Tällöin täytyy ensin varata uusi suurempi taulukko ja kopioida kaikki vanhan taulukon alkiot siihen, minkä jälkeen uusi alkio voidaan lisätä listalle. Tämä vie aikaa $O(n)$, koska kaikki listan alkiot täytyy kopioida uuteen paikkaan muistissa. Esimerkiksi kuvassa 4.2 uusi alkio 4 ei mahdu taulukkoon, joten joudumme varaamaan uuden taulukon ja kopioimaan alkiot.

Olemme saaneet siis aikaan listan, jossa lisääminen vie aikaa *joko* $O(1)$ tai $O(n)$ riippuen siitä, mahtuuko alkio nykyiseen taulukkoon vai täytyykö varata uusi taulukko. Jotta lista olisi käyttökelpoinen, hidas $O(n)$ -operaatio ei saisi esiintyä liian usein. Osoittautuu, että tämä tavoite toteutuu, kunhan varaamme uuden taulukon aina reilusti aiempaa suuremmaksi. Tyypillinen ratkaisu on *kaksinkertaistaa* taulukon koko aina, kun varaamme uuden taulukon. Kun toimimme näin, jokaisen alkion lisääminen listalle vie *keskimäärin* vain $O(1)$ aikaa.

Miksi aikaa kuluu keskimäärin vain $O(1)$? Tarkastellaan tilannetta, jossa listassa on n alkioita ja taulukko on tullut täyteen, joten alkiot täytyy kopioida uuteen taulukkoon. Tiedämme, että alkioita kopioitiin viimeksi silloin, kun listassa oli $n/2$ alkioita, joten listaan on lisätty välissä $n/2$ alkioita tehokkaasti. Niinpä n alkion kopioimisen kustannus voidaan jakaa $n/2$ aiem-

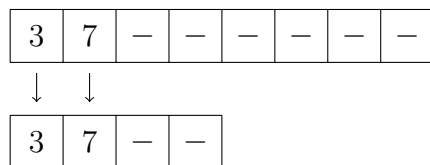


Figure 4.3: Poistojen jälkeen taulukon koko on käynyt tarpeettoman suureksi, ja puolitamme taulukon koon.

min lisätylle alkioille, ja jokaiseen alkioon kohdistuva lisäkustannus on vakio. Taulukon kasvattamisen vaikutus kokonaisuuteen on siis pieni.

Voimme poistaa alkion listan lopusta aina $O(1)$ -ajassa, koska taulukon kokoa ei tarvitse koskaan suurentaa. Tässä voi kuitenkin tulla ongelmaksi, että monien poistojen jälkeen taulukossa on turhan paljon tyhjää tilaa lopussa. Voimme soveltaa tässä käänteisesti samaa ideaa kuin lisäämisessä: jos poistamisen jälkeen vain *neljännes* taulukosta on käytössä, puolitamme taulukon koon. Kuva 4.3 näyttää esimerkin tällaisesta tilanteesta. Tällä tavalla myös poistamiset vievät keskimäärin aikaa $O(1)$.

Miksi emme voisi varata heti aluksi niin suurta taulukkoa, että lopullinen lista mahtuisi siihen varmasti? Tässä olisi huonona puolena, että listan toteutus tuhlaisi muistia. Algoritmissa saattaa olla samaan aikaan käytössä monia listoja, ja haluamme, että listalle varattu taulukko on samaa kokoluokkaa kuin listan todellinen sisältö.

4.1.2 Muutokset alussa ja lopussa

Melko samaan tapaan voimme myös luoda taulukkolistan, joka sallii tehokkaat alkiodien lisäykset ja poistot sekä listan alussa että lopussa. Jotta tämä onnistuisi, muutamme listan tallennustapaa niin, että lista voi alkaa ja päättyä missä tahansa taulukon kohdassa ja listan sisältö voi tarvittaessa jatkua taulukon lopusta alkuun.

Kuva 4.4 näyttää esimerkin listan $[3, 7, 2, 5]$ uudesta tallennustavasta. Merkki $\rightarrow$ osoittaa kohdan, josta lista alkaa, ja merkki $\leftarrow$ osoittaa kohdan, johon lista päättyy. Kun haluamme lisätä alkion listan alkuun, siirrymme vasemmalle kohdasta $\rightarrow$, ja kun haluamme lisätä alkion listan loppuun, siirrymme oikealle kohdasta $\leftarrow$. Kun haluamme poistaa alkioita listasta, menetelmämme käänteisesti.

Jos kohdat $\leftarrow$ ja $\rightarrow$ ovat vierekkäin tässä järjestyksessä, tämä voi tarkoittaa kahta asiaa: joko lista on tyhjä tai sitten kaikki taulukon kohdat ovat käytössä. Kuva 4.5 näyttää esimerkin näistä tilanteista. Kun pidämme muistissa alkiodien määrää, voimme päätellä siitä, kumpi tilanne on ky-

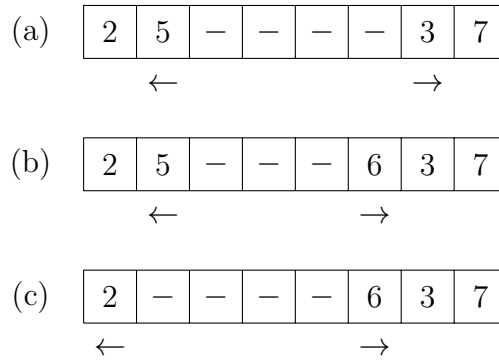


Figure 4.4: (a) Lista $[3, 7, 2, 5]$ tallennettuna taulukkoon. (b) Listan alkuun lisätään alkio 6. (c) Listan lopusta poistetaan alkio 5.

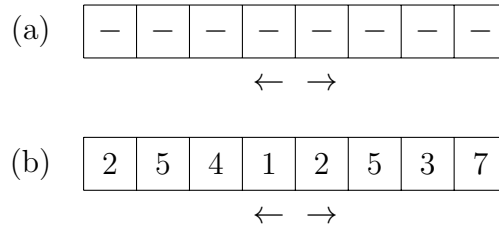


Figure 4.5: Kaksi samantapaista tilannetta: (a) Listassa ei ole yhtään alkioita. (b) Listan alkiot täyttävät koko taulukon.

seessä. Jos taulukko on täynnä ja haluamme lisätä uuden alkion, meidän täytyy varata uusi suurempi taulukko, johon listan sisältö siirretään. Voimme menetellä samalla tavalla kuin aiemmin ja esimerkiksi kaksinkertaistaa taulukon koon joka vaiheessa, jolloin operaatiot vievät keskimäärin aikaa $O(1)$.

4.2 Linkitetty lista

Linkitetty lista (*linked list*) muodostuu solmuista, joista jokainen sisältää yhden listan alkion. Linkitetty lista voi olla yhteen tai kahteen suuntaan linkitetty. Yhteen suuntaan linkitettyssä listassa jokaisesta solmusta on viittaus seuraavaan solmuun, ja kahteen suuntaan linkitettyssä listassa jokaisesta solmusta on viittaus sekä seuraavaan että edelliseen solmuun.

Kuva 4.6 näyttää esimerkkinä listan $[3, 7, 2, 5]$ yhteen ja kahteen suuntaan linkitettyinä. Molemmissa listoissa tiedossa on viittaukset listan alkuun ja loppuun. Yhteen suuntaan linkitettyssä listassa voimme käydä läpi listan alkiot alusta loppuun, kun taas kahteen suuntaan linkitettyssä listassa voimme kulkea sekä alusta loppuun että lopusta alkuun.

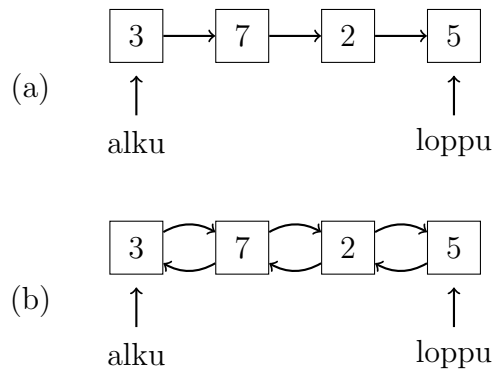


Figure 4.6: Lista $[3, 7, 2, 5]$ linkitettyinä listana. (a) Yhteen suuntaan linkitetty lista. (b) Kahteen suuntaan linkitetty lista.

Tavallinen tapa toteuttaa linkitetty rakenne ohjelmointikielissä on luoda luokka, jonka oliot vastaavat rakenteen solmuja. Esimerkiksi voisimme rakentaa yhteen suuntaan linkitetyn listan $[1, 2, 3]$ tähän tapaan:

```
a.arvo = 1
a.seuraava = b
b.arvo = 2
b.seuraava = c
c.arvo = 3
c.seuraava = null
```

Tässä toteutuksessa jokaisessa solmuoliossa on kaksi kenttää: **arvo** on solmussa oleva arvo ja **seuraava** viittaa listan seuraavaan solmuun. Listan loppumisen ilmaisee viimeisessä solmussa oleva arvo **null**. Voisimme käydä läpi listan solmut näin:

```
s = a
while s != null
    print(s.arvo)
    s = s.seuraava
```

4.2.1 Listan operaatiot

Käytännössä järkevä tapa toteuttaa linkitetty lista on tehdä listasta kaksisuuntainen, jolloin tiedämme jokaisessa solmussa seuraavan ja edellisen solmun ja voimme kulkea listaa molempiin suuntiin.

Linkitetyn listan etuna on, että voimme lisätä ja poistaa alkioita $O(1)$ -ajassa kaikissa listan kohdissa. Kun haluamme lisätä listalle alkion, luomme

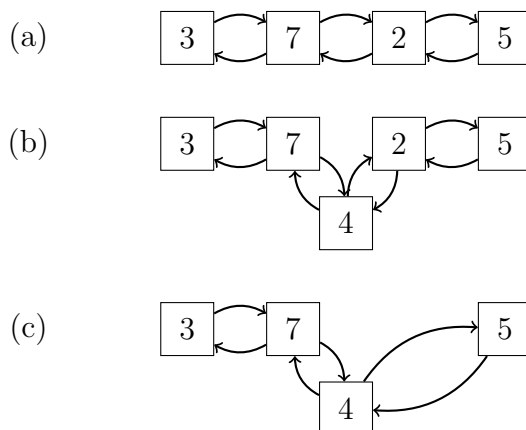


Figure 4.7: (a) Alkuperäinen lista $[3, 7, 2, 5]$. (b) Listan keskelle lisätään alkio 4. (c) Listasta poistetaan alkio 2.

ensin uuden solmun ja muutamme sitten sen vieressä olevien solmujen viittauksia niin, että ne viittaavat uuteen solmuun. Vastaavasti kun haluamme poistaa alkion, muutamme viittauksia niin, että solmu ohitetaan.

Kuva 4.7 näyttää esimerkin linkitetyn listan käsittelystä. Listan sisältönä on aluksi $[3, 7, 2, 5]$. Sitten lisäämme listan keskelle alkion 4, jolloin luomme ensin uuden solmun alkion 4 ja muutamme sitten viittauksia alkion 7 ja 2 välillä niin, että alkio 4 tulee niiden väliin. Lopuksi poistamme listasta alkion 2, jolloin yhdistämme alkion 4 ja 5 suoraan toisiinsa.

Pääsemme listan ensimmäiseen ja viimeiseen solmuun tehokkaasti, koska muistissa on viittaukset niihin. Sen sijaan jos haluamme päästä johonkin muuhun listan kohtaan, matka täytyy aloittaa listan alusta tai lopusta ja kulkea askel kerrallaan viittauksia seuraten. Niinpä listan keskellä olevaan kohtaan pääseminen vie aikaa $O(n)$. Joudumme liikkumaan solmuihin linkkejä pitkin, koska solmut voivat olla eri puolilla muistia eikä ole keinoa tietää suoraan, mihin mikäkin solmu on tallennettu.

4.2.2 Listojen vertailua

Taulukko 4.1 esittää yhteenvedon taulukkolistan ja linkitetyn listan ominaisuuksista. Kummassakin toteutuksessa on yksi operaatio, joka ei ole tehokas. Taulukkolistassa pääsemme tehokkaasti mihin tahansa listan kohtaan, mutta on hidasta muokata listaa keskeltä. Linkitettyssä listassa voimme helposti muokata listaa mistä tahansa, mutta keskelle pääseminen on hidasta.

Huomaa, että keskelle pääsemisen hitaus rajoittaa melko paljon linkitetyn listan käyttämistä. Vaikka pystymme sinänsä muokkaamaan listaa mistä tahansa kohdasta tehokkaasti, meidän tulee ensin *päästä* kyseiseen kohtaan. Jos

operaatio	taulukkolista	linkitetty lista
pääsy listan alkuun	$O(1)$	$O(1)$
pääsy listan loppuun	$O(1)$	$O(1)$
pääsy listan keskelle	$O(1)$	$O(n)$
lisäys/poisto listan alussa	$O(1)$	$O(1)$
lisäys/poisto listan lopussa	$O(1)$	$O(1)$
lisäys/poisto listan keskellä	$O(n)$	$O(1)$

Table 4.1: Taulukkolistan ja linkitetyn listan aikavaativuuksia.

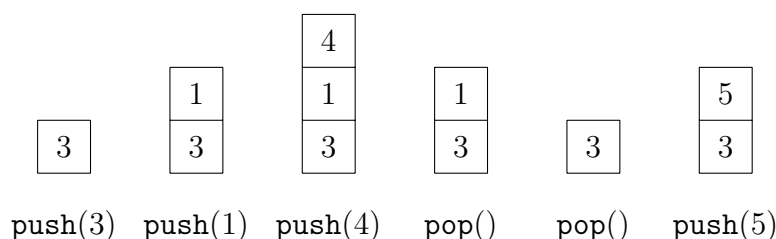


Figure 4.8: Esimerkki pinon käsittelystä: lisäämme tyhjään pinoon alkiot 3, 1 ja 4, poistamme kaksi ylintä alkioita ja lisäämme lopuksi alkion 5.

jostain syystä on etukäteen tiedossa viittaus listan keskelle, voimme muokata kyseistä kohtaa tehokkaasti, mutta muuten tulee ensin kulkea haluttuun kohtaan, missä kuluu aikaa $O(n)$.

4.3 Pino ja jono

Listan avulla voimme toteuttaa myös kaksi erikoistunutta tietorakennetta, pinon ja jonon, jotka sisältävät vain osan listan ominaisuuksista eli niiden operaatiot ovat listaa rajoittuneempia.

Pino (stack) on tietorakenne, jonka operaatiot ovat alkion lisääminen pinon päälle (**push**), ylimmän alkion poistaminen (**pop**) sekä ylimmän alkion hakeminen. Esimerkiksi kuvassa 4.8 lisäämme ensin tyhjään pinoon kolme alkioita, poistamme sitten kaksi alkioita ja lisäämme vielä yhden alkion.

Jono (queue) on tietorakenne, jossa voimme lisätä alkioita jonon loppuun (**enqueue**), poistaa alkioita jonon alusta (**dequeue**) ja hakea alkioita molemmista päistä. Esimerkiksi kuvassa 4.9 lisäämme ensin tyhjään jonoon kolme alkioita, poistamme sitten yhden alkion ja lisäämme vielä yhden alkion.

Pystymme toteuttamaan sekä pinon että jonon helposti listana niin, että niiden operaatiot toimivat ajassa $O(1)$. Mutta mitä järkeä on luoda uusia tietorakenteita, jotka ovat *huonompia* kuin lista? Listassa voimme käsitellä mitä tahansa

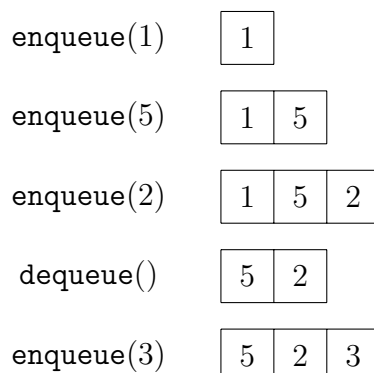


Figure 4.9: Esimerkki jonon käsittelystä: lisäämme tyhjään jonon alkiot 1, 5 ja 2, poistamme yhden alkion ja lisäämme vielä alkion 3.

alkioita, mutta pinossa ja jonossa emme pääse käsiksi keskellä oleviin alkioihin. Selitys on siinä, että pino ja jono ovat hyödyllisiä *käsitteitä* algoritmien suunnittelussa. Voimme usein ajatella algoritmissa tarvittavaa tietorakennetta pinona tai jonona ja toteuttaa sen sitten listana.

Tarkastellaan esimerkkinä ongelmaa, jossa annettuna on n merkin pituinen *sulkulauseke*, joka muodostuu kaarisulkeista (ja) sekä hakasulkeista [ja]. Haluamme selvittää, onko lauseke *oikein muodostettu* eli onko jokaiselle aloittavalle sululle vastaava lopettava pari. Esimerkiksi lauseke `[()]()` on oikein muodostettu, kun taas lauseke `[()()]` ei ole. Voimme ratkaista ongelman $O(n)$ -ajassa pinon avulla käymällä läpi lausekkeen merkit vasemmalta oikealle. Kun vastaan tulee aloittava sulku (tai [, lisäämme sen pinoon. Kun taas vastaan tulee lopettava sulku) tai], tutkimme, mikä on pinossa ylimpänä oleva merkki. Jos merkki on vastaava aloittava sulku, poistamme sen pinosta, ja muuten toteamme, että lauseke on virheellinen. Jos lausekkeen läpikäynnin aikana ei esiinny virheitä ja pino on lopuksi tyhjä, lauseke on oikein muodostettu.

4.4 Ohjelmointikielten toteutukset

4.4.1 Java

Javan `ArrayList` on taulukkolista, jossa alkion lisääminen ja poistaminen on tehokasta, kun se tapahtuu listan lopussa. Metodi `add` lisää tehokkaasti alkion listan loppuun:

```
ArrayList<Integer> lista = new ArrayList<>();  
lista.add(1);  
lista.add(2);  
lista.add(3);  
// tulos: [1,2,3]
```

Metodi `remove` poistaa alkion annetusta listan kohdasta. Metodi toimii tehokkaasti, kun poistokohta on listan lopussa. Esimerkiksi seuraava koodi poistaa listan viimeisen alkion:

```
lista.remove(lista.size()-1);
```

Koska lista on tallennettu taulukkona, pääsemme myös tehokkaasti käsiksi missä tahansa kohdassa olevaan alkioon metodeilla `get` ja `set`:

```
// hae alkio kohdasta 3  
int x = lista.get(3);  
// muuta kohdan 5 alkioiksi 8  
lista.set(5,8);
```

`ArrayDeque` on taulukkolista, joka sallii tehokkaat lisäykset ja poistot sekä listan alussa että lopussa. Metodit `addFirst` ja `addLast` lisäävät alkioita, ja metodit `removeFirst` ja `removeLast` poistavat alkioita.

```
ArrayDeque<Integer> lista = new ArrayDeque<>();  
lista.addLast(1);  
lista.addLast(2);  
lista.addLast(3);  
lista.removeFirst();  
lista.addFirst(4);  
// tulos: [4,2,3]
```

Lisäksi voimme hakea listan ensimmäisen ja viimeisen alkion metodeilla `getFirst` ja `getLast`:

```
int eka = lista.getFirst();  
int vika = lista.getLast();
```

Rajoituksena on kuitenkin, että yleisiä metodeita `get` ja `set` ei ole tarjolla eli emme pääse tehokkaasti käsiksi listan keskellä oleviin alkioihin.

`LinkedList` on kaksisuuntainen linkitetty lista, jota voimme käsitellä samaan tapaan kuin `ArrayDeque`-listaa:

```
LinkedList<Integer> lista = new LinkedList<>();
lista.addLast(1);
lista.addLast(2);
lista.addLast(3);
lista.removeFirst();
lista.addFirst(4);
// tulos: [4,2,3]
```

`LinkedList` tarjoaa myös metodit `get` ja `set`, joiden avulla pääsee käsiksi tiettyssä kohdassa listalla olevaan alkioon. Nämä metodit vievät kuitenkin aikaa $O(n)$, koska joudumme kulkemaan ensin oikeaan kohtaan listan alusta tai lopusta.

Voimme käsitellä listaa myös *iteraattorilla* (*iterator*), joka osoittaa tiettyyn listan kohtaan. Esimerkiksi seuraava koodi luo iteraattorin, joka osoittaa listan alkuun, siirtää iteraattoria kaksi askelta eteenpäin ja lisää alkion iteraattorin kohdalle eli listan toisen ja kolmannen alkion väliin.

```
ListIterator<Integer> x = lista.listIterator();
x.next();
x.next();
x.add(5);
```

Iteraattorin avulla voimme muokata tehokkaasti linkitettyä listaa siitä kohdasta, jossa iteraattori on tällä hetkellä.

4.4.2 Python

Pythonin tavallinen lista on taulukkolista, jota voi käsitellä tehokkaasti listan lopusta. Metodi `append` lisää alkion listan loppuun:

```
lista = []
lista.append(1)
lista.append(2)
lista.append(3)
# tulos: [1,2,3]
```

Metodi `pop` puolestaan poistaa listan viimeisen alkion:

```
lista.pop()
```

Pääsemme käsiksi listan alkioihin tehokkaasti `[]`-syntaksin avulla:


```
x = lista[3]
lista[5] = 8
```

Pythonissa on myös lista `deque`, joka on toteutettu linkitettyinä listana. Lisäykset ja poistot ovat tehokkaita sekä listan alussa että lopussa. Saamme tietorakenteen käyttöön näin:

```
from collections import deque
```

Tavallisen listan tavoin `deque` tarjoaa metodit `append` ja `pop`, mutta lisäksi siinä on metodit `appendleft` ja `popleft`, joiden avulla voi lisätä ja poistaa alkioita listan alusta:

```
lista = deque()
lista.append(1)
lista.append(2)
lista.append(3)
lista.popleft()
lista.appendleft(4)
# tulos: [4,2,3]
```

Listan alkuun ja loppuun pääsee käsiksi näin:

```
eka = lista[0]
vika = lista[-1]
```

Myös listan keskellä olevia alkioita voi käsitellä `[]`-syntaksilla, mutta tämä vie aikaa $O(n)$, koska kyseessä on linkitetty lista.

4.5 Miten valita lista?

Voimme käyttää usein algoritmin toteutukseen joko taulukkolistaa tai linkitettyä listaa ja saada aikaan aikavaativuuden kannalta yhtä tehokkaan algoritmin. Esimerkiksi jos algoritmi lisää listan loppuun alkioita, sekä taulukkolistassa että linkitettyssä listassa lisäämisen aikavaativuus on $O(1)$. Mutta kumpi lista on parempi valinta käytännössä?

Useimmissa tapauksissa parempi valinta on taulukkolista, koska nykyaikaiset tietokoneet suosivat taulukkolistan käyttämistä linkitetyn listan sijaan. Kuvassa 4.10 näkyy, miten taulukkolista ja linkitetty lista asettuvat tietokoneen muistissa. Taulukkolistan alkiot ovat peräkkäin, kun taas linkitetyn listan alkiot voivat olla eri puolilla muistia sekalaisessa järjestyksessä. Nykyaikaisen

	3	7	2	5			

	3					2	
				7			
		5					

Figure 4.10: Taulukkolista ja linkitetty lista tietokoneen muistissa.

prosessorin välimuistit ja seuraavaksi suoritettavien komentojen ennustus on toteutettu niin, että ne ovat parhaimmillaan silloin, kun tieto on tallennettu muistissa peräkkäin – eli juuri kuten taulukkolistassa. Tämä näkyy käytännössä siinä, että taulukkolistan käsittely on selvästi tehokkaampaa kuin linkitetyn listan käsittely.

Vaikka linkitetty lista ei ole usein käytännössä hyvä tietorakenne, linkitetyn rakenteen idea on hyödyllinen ja tarvitsemme sitä myöhemmin monimutkaisemmissa tietorakenteissa.

Chapter 5

Hajautustaulu

Hajautustaulu (*hash table*) on tietorakenne, joka tarjoaa tehokkaat operaatiot alkion lisäämiseen, hakemiseen ja poistamiseen. Voimme toteuttaa hajautustaulun avulla *joukon*, jossa on kokoelma alkioita, sekä *hakemiston*, joka sisältää avain-arvo-pareja.

Yksi tapa ajatella hajautusta on, että yleistämme taulukon ideaa: taulukossa indeksit ovat aina 0, 1, 2, jne., mutta hajautustaulussa voimme käyttää indeksien sijasta vaikkapa merkkijonoja. Tämä onnistuu käyttämällä hajautusfunktioita, joka muuttaa muun tyyppisen alkion indeksiksi, jotta löydämme sille paikan taulukosta.

5.1 Hajautustaulun toteutus

Hajautustaulun perustana on *hajautusfunktio* (*hash function*) $f(x)$, joka antaa *hajautusarvon* (*hash value*) mille tahansa alkiolle x . Hajautusarvo on kokonaisluku väliltä $0, 1, \dots, N - 1$, missä N on hajautustaulun koko. Hajautusarvo määrää, mihin hajautustaulun kohtaan alkio tallennetaan.

Tarkastellaan esimerkkinä hajautusfunktioita $f(x) = x \bmod N$, missä x on kokonaisluku. Tämä hajautusfunktio antaa hajautusarvoksi luvun x jakojäännöksen N :llä. Esimerkiksi kun $N = 10$ ja tallennamme hajautustauluun luvun 14, saamme sen hajautusarvoksi $f(14) = 4$. Tämä tarkoittaa, että luvun paikka hajautustaulussa on kohdassa 4. Vastaavasti $f(21) = 1$, eli luvun 21 paikka hajautustaulussa on kohdassa 1. Kun $N = 10$, luvun hajautusarvo on tässä tapauksessa siis luvun viimeinen numero.

Yleensä hajautustaulun koko N on selvästi pienempi kuin mahdollisten alkioden määrä ja hajautuksessa voi tapahtua *törmäys* (*collision*) eli kahdelle alkiolle tulee sama hajautusarvo. Esimerkiksi $f(14) = f(24) = 4$ eli luvut 14 ja 24 kuuluvat samaan paikkaan hajautustaulussa. Tämän vuoksi

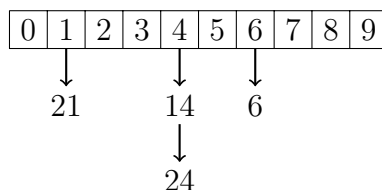


Figure 5.1: Ketjuhajautusta käyttävä hajautustaulu, joka vastaa joukkoa $\{6, 14, 21, 24\}$. Hajautusfunktiona on $f(x) = x \bmod N$.

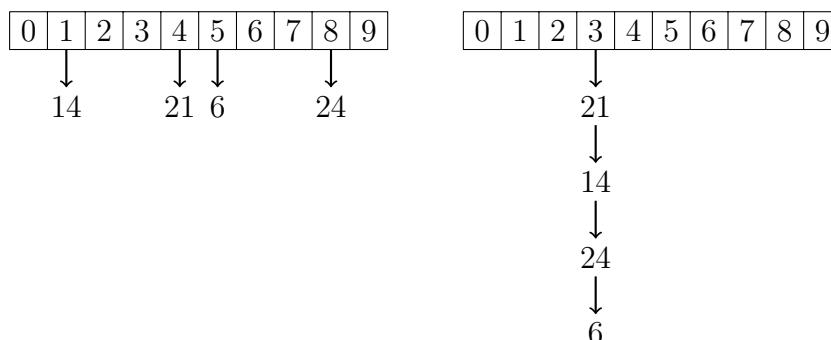


Figure 5.2: Kaksi hajautustaulua joukolle $\{6, 14, 21, 24\}$. Vasen tilanne on paras mahdollinen, oikea tilanne taas huonoin mahdollinen.

hajautustaulun toteutuksessa täytyy päättää, miten törmäykset käsitellään. Kaksi tavallista tapaa tähän ovat ketjuhajautus sekä avoin hajautus.

5.1.1 Ketjuhajautus

Ketjuhajautus (*chaining*) on hajautustaulun toteutustapa, jossa jokainen hajautustaulun kohta sisältää listan alkioista, joilla on kyseinen hajautusarvo. Tällöin mahdolliset alkioiden törmäykset eivät haittaa, koska listoissa voi olla mikä tahansa määrä alkioita.

Kuvassa 5.1 on esimerkkinä ketjuhajautusta käyttävä hajautustaulu, johon on tallennettu joukko $\{6, 14, 21, 24\}$ käyttäen hajautusfunktiota $f(x) = x \bmod N$. Kohdassa 1 on alkio 21, kohdassa 4 on alkio 14 ja 24 ja kohdassa 6 on alkio 6. Muissa kohdissa olevat listat ovat vielä tyhjiä.

Kun haluamme tarkastaa, onko joukossa alkioita x , laskemme ensin sen hajautusarvon $f(x)$. Tämän jälkeen käymme läpi kaikki kohdan $f(x)$ listassa olevat alkioita ja tarkastamme, onko jokin niistä alkio x . Vastaavasti kun haluamme lisätä alkion x joukkoon tai poistaa alkion x joukosta, teemme muutoksen kohdassa $f(x)$ olevaan listaan.

Ketjuhajautuksessa hajautustaulun operaatioiden aikavaativuus on $O(m)$,

0	1	2	3	4	5	6	7	8	9
	21			14	24	6			

Figure 5.3: Avointa hajautusta käyttävä hajautustaulu, joka vastaa joukkoa $\{6, 14, 21, 24\}$. Hajautusfunktiona on $f(x) = x \bmod N$.

missä m on listan alkioden määrä. Hajautustaulu toimii siis tehokkaasti, jos siinä olevat listat ovat lyhyitä. Tämä toteutuu, jos käytetty hajautusfunktio sijoittaa alkioita tasaisesti hajautustaulun eri puolille. Kuvassa 5.2 on kaksi ääriesimerkkiä hajautuksen onnistumisesta erilaisilla hajautusfunktioilla. Vasemmassa hajautustaulussa jokainen alkio on eri listassa, mikä on paras mahdollinen tilanne. Oikeassa hajautustaulussa kaikki alkiot ovat puolestaan samassa listassa, mikä on huonoin mahdollinen tilanne.

5.1.2 Avoim hajautus

Toinen tapa toteuttaa hajautustaulu on *avoin hajautus* (*open addressing*), jossa alkiot tallennetaan suoraan hajautustauluun listojen sijasta. Kuitenkin jos alkio kuuluisi paikkaan, jossa on jo valmiina jokin toinen alkio, sille valitaan toinen paikka jonkin säännön perusteella.

Avoimessa hajautuksessa alkion paikkaa hajautustaulussa etsitään *kokeilufunktiolla* (*probe function*) $h(x, i)$, jossa x on alkio ja i ilmaisee, montako kertaa sille on yritetty etsiä paikkaa. Tarkastelemme seuraavaksi yksinkertaista kokeilufunktiota

$$h(x, i) = (f(x) + i) \bmod N,$$

mikä tarkoittaa, että alkion x paikkaa etsitään sen hajautusarvon $f(x)$ antamasta kohdasta lähtien kulkemalla taulukossa oikealle, kunnes vapaa paikka löytyy. Tämän menetelmän nimi on *lineaarinen kokeilu* (*linear probing*).

Kuvassa 5.3 näkyy joukon $\{6, 14, 21, 24\}$ sijoittuminen hajautustauluun käyttäen hajautusfunktiota $f(x) = x \bmod N$ ja lineaarista kokeilua. Tässä alkio 14 on lisätty ennen alkioita 21 ja 24, minkä vuoksi alkio 14 on sen hajautusarvon mukaisessa kohdassa 4 mutta alkio 24 on kohdassa 5, koska kohta 4 oli jo varattu. Kuvassa 5.4 mukaan lisätään vielä alkio 44, jolla on myös hajautusarvo 4. Koska kohdissa 4, 5 ja 6 on jo alkio, alkion 44 kohdaksi tulee 7. Alkio voi sijoittua siis kauaskin hajautusarvon määräämästä kohdasta, jos hajautustaulussa on täyttä.

Kun haluamme löytää alkion x hajautustaulusta avointa hajautusta käyttäen, tutkimme hajautustaulun kohtia kokeilufunktion mukaisesti, kunnes joko löydämme alkion tai tulemme tyhjään kohtaan, jolloin voimme päätellä, että hajautustaulussa ei ole alkioita. Esimerkiksi kun haluamme löytää kuvan

0	1	2	3	4	5	6	7	8	9
	21			14	24	6	44		

Figure 5.4: Hajautustauluun lisätään vielä alkio 44, jonka hajautusarvo on 4. Sille löytyy paikka vasta kohdasta 7.

0	1	2	3	4	5	6	7	8	9
	21			14	24	X	44		

Figure 5.5: Hajautustaulusta on poistettu alkio 6, jonka tilalla on merkki X. Tämän ansiosta alkion 44 haku ei pysähdy kohtaan 6.

5.4 hajautustaulusta alkion 44, aloitamme kohdasta $f(44) = 4$ ja kuljemme eteenpäin kohtaan 7 asti.

Alkion poistaminen aiheuttaa hieman ongelmia avoimessa hajautuksessa, koska jos vain merkitsemme alkion kohdan tyhjäksi, tämä voi häiritä tulevia hakuja. Esimerkiksi jos poistaisimme kuvan 5.4 hajautustaulusta alkion 6 merkitsemällä kohdan 6 tyhjäksi, emme enää löytäisi alkiota 44, koska haku pysähtyisi tyhjään kohtaan 6. Kuitenkin voimme kiertää ongelman käyttämällä jotain erikoismerkkiä poistetulle alkiolle. Kuvassa 5.5 poistetun alkion merkinä on X, joka tarkoittaa, että kyseinen kohta on tyhjä mutta siinä on ollut alkio. Niinpä alkion 44 haku ei pääty tähän kohtaan, vaikka siihen voi myös sijoittaa uuden alkion myöhemmin.

Avoimen hajautuksen tehokkuus riippuu käytetystä hajautusfunktioista ja kokeilufunktioista. Hajautustaulun operaatiot vievät aikaa $O(m)$, missä m on kokeiltavien paikkojen määrä, kun haluamme löytää paikan uudelle alkiolle tai etsiä alkion taulusta. Käytännössä lineaarisessa kokeilussa voi tulla ongelmaksi, että alkiot kasautuvat tiettyihin hajautustaulun osiin. Kehittyneemmissä kokeilufunktioissa alkion paikan etsimisessä tehdään vaihtelevan kokoisia hyppyjä hajautustaulussa.

5.1.3 Hajautusfunktion valinta

Hajautusfunktio $f(x)$ määrittää, mihin kohtaan hajautustaulua alkio x sijoitetaan. Sen täytyy antaa jokaiselle mahdolliselle alkiolle hajautusarvo eli kokonaisluku väliltä $0, 1, \dots, N - 1$, missä N on hajautustaulun koko. Kun tämä ehto täyttyy, hajautusfunktion voi muilta osin suunnitella periaatteessa miten tahansa. Mutta miten hajautusfunktio kannattaa valita?

Jos hajautettavat alkiot ovat kokonaislukuja, suoraviivainen hajautusfunktio on esimerkissä käyttämämme $f(x) = x \bmod N$, joka muuttaa luvun välille $0 \dots N - 1$ jakojäännöksen avulla. Tämä on helposti toteutettava ja

yleensä hyvin toimiva hajautusfunktio. Entä jos alkiot ovat jotain muuta tyyppiä kuin kokonaislukuja? Tällöin voimme ensin keksiä jonkin järkevän keinon muuttaa alkion kokonaisluvuksi, minkä jälkeen voimme jälleen ottaa jakojäännöksen N :llä.

Tarkastellaan tilannetta, jossa haluamme hajauttaa merkkijonoja eli täytyy löytää tapa muuttaa merkkijono kokonaisluvuksi. Oletamme, että merkkijonossa on k merkkiä, joiden merkkikoodit ovat $c_0, c_1, \dots, c_{k-1}$. Esimerkiksi jos merkkijono on `testi`, merkkikoodit¹ ovat $c_0 = 116$, $c_1 = 101$, $c_2 = 115$, $c_3 = 116$ ja $c_4 = 105$. Yksi tapa muuttaa merkkijono kokonaisluvuksi on laskea merkkikoodien summa

$$c_0 + c_1 + \dots + c_{k-1},$$

jolloin merkkijonoa `testi` vastaa kokonaisluku

$$116 + 101 + 115 + 116 + 105 = 553.$$

Tämä on sinänsä järkevä tapa laskea hajautusarvo, mutta siinä on yksi ongelma: kaksi merkkijonoa saavat aina saman hajautusarvon, jos niissä on samat merkit eri järjestyksessä. Pystymme parantamaan hajautusarvon laskentaa lisäämällä summaan *kertoimet* käyttäen kaavaa

$$A^{k-1}c_0 + A^{k-2}c_1 + \dots + A^0c_{k-1},$$

missä A on vakio. Esimerkiksi jos $A = 7$, merkkijonoa `testi` vastaa kokonaisluku

$$7^4 \cdot 116 + 7^3 \cdot 101 + 7^2 \cdot 115 + 7^1 \cdot 116 + 7^0 \cdot 105 = 319711.$$

Tämän menetelmän nimi on *polynominen hajautus* (*polynomial hashing*), ja se on yksi käytännössä hyvä merkkijonon hajautustapa.

5.1.4 Miten hyvin hajautus toimii?

Hajautustaulun operaatiot vievät aikaa $O(m)$, missä m on ketjuhajautuksessa listan pituus ja avoimessa hajautuksessa kokeilujonon pituus. Mutta kuinka suuri m on? Tähän vaikuttavat alkioden määrä n , hajautustaulun koko N , hajautusfunktio f sekä avoimessa hajautuksessa kokeilufunktio h .

Jos kaikki sujuu hyvin, hajautusfunktio jakaa alkioita tasaisesti hajautustaulun eri puolille ja listojen tai kokeilujonojen pituus on luokkaa n/N . Niinpä jos valitsemme hajautustaulun koon niin, että N on samaa luokkaa kuin

¹Käytämme tässä merkkien ASCII-koodia. Javassa merkin `c` koodin saa selville kirjoittamalla `(int)c`, ja Pythonissa voi käyttää funktiota `ord(c)`.

n , operaatiot toimivat tehokkaasti ajassa $O(1)$. Kuitenkin on myös mahdollista, että hajautus epäonnistuu ja alkiot jakautuvat hajautustauluun epätasaisesti. Pahimmassa tapauksessa kaikki alkiot saavat saman hajautusarvon ja ne kaikki tallennetaan samaan listaan tai kokeilujonoon, jolloin operaatiot vievät aikaa $O(n)$.

Hajautuksessa vaikeutena on, ettemme tiedä etukäteen, mitä alkioita hajautustauluun tallennetaan. Tarkastellaan tilannetta, jossa hajautettavat alkiot ovat kokonaislukuja ja käytössä on hajautusfunktio $f(x) = x \bmod N$. Tämä on hyvä hajautusfunktio olettaen, että eri jakojäännöksiä esiintyy tasaisesti aineistossa. Tämä oletus ei kuitenkaan välttämättä päde: esimerkiksi voi olla, että jostain syystä jokainen hajautettava alkio on parillinen. Nyt jos myös N on parillinen, jokainen hajautusarvo on parillinen ja vain puolet hajautustaulun kohdista on käytössä. Parempi tapa voi olla valita N niin, että se on *alkuluku*. Tällöin on vähemmän todennäköistä, että aineistossa mahdollisesti olevat säännöllisyydet aiheuttaisivat törmäyksiä.

Emme voi kuitenkaan koskaan olla etukäteen varmoja, että hajautus toimii hyvin. Riippumatta hajautustavasta *ilkeä vastustaja* voi antaa joukon alkioita, jotka kaikki saavat saman hajautusarvon. Kaikeksi onneksi hajautus toimii yleensä aina *käytännössä* hyvin ja voimme ajatella, että hajautustaulun operaatiot ovat $O(1)$ -aikaisia, kunhan hajautustaulun koko on riittävän suuri ja hajautus on toteutettu järkevästi. Vaikka on mahdollista, että hajautus epäonnistuu, tämän riski on pieni eikä meidän tarvitse yleensä murehtia siitä käytännössä.

5.1.5 Hajautustaulu hakemistona

Tähän mennessä olemme tallentaneet hajautustauluun alkioden joukon, mutta voimme tallentaa myös avain-arvo-pareja, joissa avaimen hajautusarvo määrittää, mihin hajautustaulun kohtaan pari sijoitetaan. Saamme näin aikaan *hakemiston*, jota voi ajatella taulukon yleistykseenä. Taulukossa avaimet ovat kokonaisluvut 0, 1, 2, jne., mutta hakemistossa ne voivat olla mitä tahansa arvoja kuten merkkijonoja.

Kuvassa 5.6 on esimerkkinä hajautustauluun tallennettu hakemisto, joka vastaa seuraavaa ”taulukkoa”:

<pre>taulu["abc"] = 5 taulu["xyz"] = 1 taulu["aaa"] = 8</pre>

Tässä tapauksessa hakemiston avaimet ovat merkkijonoja ja arvot ovat kokonaislukuja. Hajautustaulun ansiosta voimme käsitellä hakemistoa taulukon

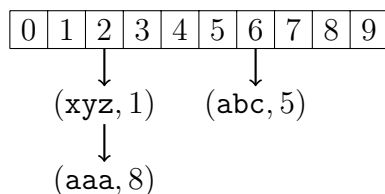


Figure 5.6: Hakemiston tallentaminen hajautustauluun.

tavoin niin, että operaatiot vievät aikaa $O(1)$.

Voimme myös toteuttaa hakemiston, jonka avaimet ovat kokonaislukuja. Tällaisessa tietorakenteessa on järkeä, jos avaimet ovat niin suuria, että emme voi käyttää hajautustaulun sijasta tavallista taulukkoa. Kuitenkin jos avaimet ovat pieniä kokonaislukuja, taulukko on parempi valinta, koska sen vakio-ertoimet ovat paljon hajautustaulua pienemmät.

5.2 Ohjelmointikielten toteutukset

5.2.1 Java

Javan tietorakenne `HashSet` on hajautusta käyttävä joukko. Esimerkiksi seuraava koodi luo joukon, johon tallennetaan kokonaislukuja:

```

HashSet<Integer> joukko = new HashSet<>();
joukko.add(3);
joukko.add(5);
joukko.add(8);
System.out.println(joukko.contains(5)); // true
joukko.remove(5);
System.out.println(joukko.contains(5)); // false

```

Tietorakenne `HashMap` on hajautukseen perustuva hakemisto, joka sisältää avain-arvo-pareja. Esimerkiksi seuraava koodi luo hakemiston, jossa avaimet ovat merkkijonoja ja arvot ovat kokonaislukuja:

```

HashMap<String,Integer> hakemisto = new HashMap<>();
hakemisto.put("apina",1);
hakemisto.put("banaani",2);
hakemisto.put("cembalo",3);
System.out.println(hakemisto.get("apina")); // 1

```

Java käyttää hajautuksessa ketjuhajautusta ja olettaa, että luokissa on metodi `hashCode`, jonka avulla olio kertoo pyydettyä hajautusarvonsa.

Metodin tulee palauttaa jokin kokonaisluku, jonka perusteella lasketaan paikka hajautustaulussa. Seuraava koodi testaa hajautusta:

```
System.out.println(((Integer)123).hashCode()); // 123
System.out.println("apina".hashCode()); // 93022541
```

Tästä näkee, että kokonaisluvun hajautusarvo on suoraan kyseinen luku. Huomaa, että meidän täytyy muuttaa luku oliotyyppiseksi (`Integer`), jotta voimme kutsua `hashCode`-metodia. Merkkijonon `apina` hajautusarvo on puolestaan 93022541. On tunnettua, että Java käyttää merkkijonon hajautusarvon laskemiseen polynomista hajautusta vakiolla $A = 31$, joten voimme laskea Javan hajautusarvon myös itse seuraavasti:

$$31^4 \cdot 97 + 31^3 \cdot 112 + 31^2 \cdot 105 + 31^1 \cdot 110 + 31^0 \cdot 97 = 93022541$$

5.2.2 Python

Pythonin tietorakenne `set` on hajautusta käyttävä joukko. Seuraava koodi luo joukon ja lisää sinne kokonaislukuja:

```
joukko = set()
joukko.add(3)
joukko.add(5)
joukko.add(8)
print(5 in joukko) # True
joukko.remove(5)
print(5 in joukko) # False
```

Tietorakenne `dict` (merkitään lyhyesti `{}`) on puolestaan hajautukseen perustuva hakemisto. Voimme käyttää hakemistoa vaikkapa näin:

```
hakemisto = {}
hakemisto["apina"] = 1
hakemisto["banaani"] = 2
hakemisto["cembalo"] = 3
print(hakemisto["apina"]) # 1
```

Pythonissa hajautus on toteutettu avoimella hajautuksella, ja funktio `hash` antaa olion hajautusarvon. Voimme testata funktion toimintaa Python-tulkissa näin:

```
>>> hash(123)
123
>>> hash("apina")
-3191961394091913473
```

Pythonissa merkkijonon hajautusarvo lasketaan menetelmällä, jonka toiminta vaihtelee satunnaisesti ohjelman eri suorituserroilla. Niinpä jos käynnistämme Python-tulkin uudestaan, saamme eri tuloksen:

```
>>> hash("apina")
-5370222958536247804
```

Tämän satunnaisuuden etuna on, että mahdollinen ilkeä vastustaja ei voi etsiä kiinteää joukkoa merkkijonoja, jotka tuottavat saman hajautusarvon Pythonissa, koska hajautustapa muuttuu joka suorituskerralla.

5.3 Hajautustaulu algoritmeissa

Hajautustaulun ansiosta voimme käyttää algoritmeissamme joukkoja ja hakemistoja, joiden operaatiot toimivat tehokkaasti. Voimme alkajaisiksi ratkoa mukavammin ajassa $O(n)$ sellaisia ongelmia, jotka olemme ratkoneet aiemmin järjestämisen avulla ajassa $O(n \log n)$.

Aloitamme ongelmasta, jossa haluamme selvittää, montako eri alkioita taulukko sisältää. Ratkaisimme ongelman aiemmin järjestämällä taulukon ja tutkimalla sen jälkeen vierekkäisiä alkioita. Nyt kun käytössämme on hajautustaulu, voimme vain lisätä kaikki alkiot joukkoon ja hakea lopuksi joukon koon. Näin saamme aikaan seuraavan algoritmin:

```
alkiot = []
for i = 0 to n-1
    alkiot.add(taulu[i])
print(alkiot.size())
```

Tässä `alkiot` on hajautustaulua käyttävä joukko, minkä ansiosta algoritmi toimii ajassa $O(n)$.

Tarkastellaan sitten ongelmaa, jossa haluamme selvittää taulukon yleisimmän alkion. Ratkaisimme tämänkin ongelman aiemmin järjestämällä taulukon, mutta hajautustaulun avulla voimme lähestyä ongelmaa toisella tavalla luomalla hakemiston, jonka avaimet ovat taulukon alkioita ja arvot niiden esiintymiskertoja. Nyt voimme vain käydä läpi taulukon sisällön ja pitää kirjaa, montako kertaa mikäkin alkio esiintyy taulukossa:

```
laskuri = []
suurin = 0
for i = 0 to n-1
    laskuri[taulu[i]] += 1
    if laskuri[taulu[i]] > suurin
        suurin = laskuri[taulu[i]]
        yleisin = taulu[i]
print(yleisin)
```

Tässä hakemisto `laskuri` on toteutettu hajautustaulun avulla, jolloin avaimet voivat olla mitä tahansa lukuja ja operaatiot toimivat ajassa $O(1)$. Tuloksena on algoritmi, jonka aikavaativuus on $O(n)$.

Kuten nämä esimerkit osoittavat, hajautustaulu *helpottaa* algoritmien luomista, koska ongelmia ei tarvitse pukea järjestämisen muotoon vaan voimme käsitellä niitä suoremmin. Mutta toisaalta olemme ratkoneet vain uudestaan ongelmia, jotka ovat hoituneet mainiosti myös järjestämisen avulla. Antaisiko hajautustaulu meille jotain todellisia uusia mahdollisuuksia algoritmien suunnittelussa?

Hajautustaulu osoittaa todelliset kyntensä silloin, kun haluamme pitää yllä aidosti *dynaamista* tietorakennetta eli haluamme vuorotellen muuttaa tietorakennetta ja hakea sieltä tietoa. Tällöin emme voisi enää toteuttaa algoritmia, joka järjestää koko aineiston kerran alussa. Esimerkki tällaisesta tilanteesta on, että käytössämme on funktio `hae_luku`, joka antaa lukuja yksi kerrallaan. Jokaisen luvun jälkeen meidän tulee ilmoittaa, montako eri lukua olemme saaneet tähän mennessä, ennen kuin voimme pyytää funktiolta seuraavan luvun. Voimme ratkaista ongelman seuraavasti ajassa $O(n)$ hajautustaulun avulla:

```
alkiot = []
for i = 1 to n
    luku = hae_luku()
    alkiot.add(luku)
print(alkiot.size())
```

Tällaisesta algoritmista käytetään joskus nimeä *online-algoritmi*. Tämä tarkoittaa, että algoritmille annetaan syötettä alkio kerrallaan ja algoritmi pystyy ilmoittamaan senhetkisen vastauksen joka alkion käsittelyn jälkeen. Vastaavasti *offline-algoritmi* tarvitsee käyttöönsä heti koko syötteen, jotta se voi käsitellä syötettä kokonaisuutena, kuten järjestää sen. Monissa ongelmissa online-algoritmi on vaikeampi saada aikaan kuin offline-algoritmi.

Chapter 6

Binäärihakupuu

Binäärihakupuu (*binary search tree*) on tietorakenne, joka pitää yllä alkioiden joukkoa, samaan tapaan kuin hajautustaulu. Binäärihakupuu eroaa hajautustaulusta kuitenkin siinä, että se säilyttää alkioita *järjestyksessä*. Tämän ansiosta voimme esimerkiksi etsiä tehokkaasti joukon pienimmän tai suurimman alkion, mikä ei ole mahdollista hajautustaulussa.

Aloitamme luvun tutustumalla binääripuiden teoriaan, minkä jälkeen perehdymme binäärihakupuun toimintaan. Käsitlemme binäärihakupuun tehokkaasta toteutuksesta esimerkkinä AVL-puun, joka on yksinkertainen tasapainoinen binäärihakupuu. Tasapainotuksen ansiosta saamme toteutettua binäärihakupuun niin, että sen operaatiot ovat tehokkaita.

6.1 Taustaa binääripuista

Binäärihakupuun taustalla on yleisempi tietorakenne *binääripuu* (*binary tree*). Ennen kuin tutustumme binäärihakupuuhun, onkin hyvä selvittää ensin, mikä on binääripuu ja mitä ominaisuuksia siihen liittyy.

Binääripuu muodostuu n solmusta. Puussa ylimpänä on solmu, jota kutsutaan *juureksi* (*root*). Jokaisella solmulla voi olla vasen ja oikea *lapsi* (*child*), ja kaikilla solmuilla juurta lukuun ottamatta on yksikäsitteinen *vanhempi* (*parent*). Puun *lehtiä* (*leaf*) ovat solmut, joilla ei ole lapsia.

Binääripuun rakenne on rekursiivinen: jokainen solmu toimii juurena *alipuulle* (*subtree*), joka on myös binääripuu. Solmun x alipuu sisältää solmun x sekä kaikki solmut, joihin pääsee laskeutumalla alaspäin solmusta x . Voimme myös ajatella asiaa niin, että jokaisen binääripuun solmun vasen ja oikea lapsi on toinen (mahdollisesti tyhjä) binääripuu.

Kuvassa 6.1 on esimerkki binääripuusta, jossa on 8 solmua. Solmu 1 on puun juuri, ja solmut 4, 5, 7 ja 8 ovat puun lehtiä. Solmun 3 vasen lapsi on

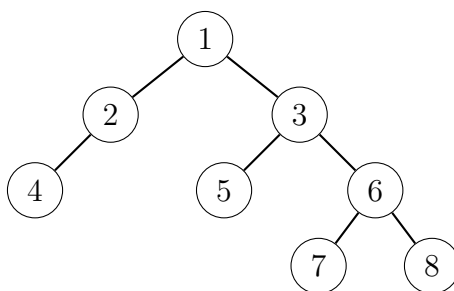


Figure 6.1: Binääripuu, jossa on 8 solmua. Puun juuri on solmu 1, ja puun lehtiä ovat solmut 4, 5, 7 ja 8.

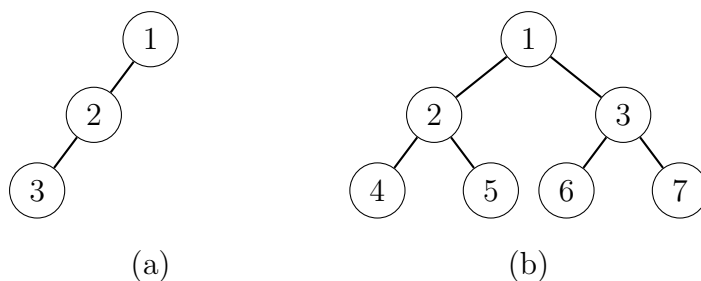


Figure 6.2: (a) Vähiten solmuja sisältävä korkeuden 2 binääripuu. (b) Eniten solmuja sisältävä korkeuden 2 binääripuu.

solmu 5, oikea lapsi on solmu 6 ja vanhempi on solmu 1. Solmun 3 alipuu sisältää solmut 3, 5, 6, 7 ja 8.

Binääripuun juuren *syvyys* (*depth*) on 0 ja jokaisen muun solmun syvyys on yhtä suurempi kuin sen vanhemman syvyys. Binääripuun *korkeus* (*height*) on puolestaan suurin puun solmussa esiintyvä syvyys eli toisin sanoen suurin askelten määrä juuresta alaspäin lehteen. Esimerkiksi kuvan 6.1 puun korkeus on 3, koska solmujen 7 ja 8 syvyys on 3.

Jos binääripuun korkeus on h , siinä on vähintään $h+1$ solmua, jolloin puu on pelkkä solmujen lista, ja enintään $2^{h+1} - 1$ solmua, jolloin kaikilla tasoilla on kaikki mahdolliset solmut. Kuva 6.2 näyttää esimerkit näistä tapauksista, kun puun korkeus on 2.

6.1.1 Binääripuun käsittely

Voimme toteuttaa binääripuun linkitettyinä rakenteena niin, että jokainen puun solmu on olio, jossa on viittaus vasempaan ja oikeaan lapseen sekä mahdollisesti muita kenttiä, kuten solmuun liittyvä arvo. Jos solmulla ei ole vasenta tai oikeaa lasta, viittauksena on `null`.

Rekursio on luonteva tapa toteuttaa monia binääripuun käsittelyyn liittyviä operaatioita. Esimerkiksi seuraava funktio laskee, montako solmua sille annetussa puussa on:

```
function laske_solmut(solmu)
  if solmu == null
    return 0
  return 1 + laske_solmut(solmu.vasen) +
    laske_solmut(solmu.oikea)
```

Funktiolle annetaan parametrina solmu, joka vastaa puun juurta. Jos puu on tyhjä, siinä ei ole yhtään solmua. Muuten puussa on juurisolmu sekä vasemman ja oikean alipuun solmut. Pystymme laskemaan alipuiden solmut rekursiivisesti kutsumalla samaa funktiota uudestaan.

Seuraava funktio puolestaan selvittää, mikä on puun korkeus. Huomaa, että jos puu on tyhjä, tulkintana on, että sen korkeus on -1 .

```
function korkeus(solmu)
  if solmu == null
    return -1
  return 1 + max(korkeus(solmu.vasen), korkeus(solmu.oikea))
```

6.1.2 Läpikäyntijärjestykset

Voimme käydä läpi binääripuun solmut rekursiivisesti juuresta alkaen. Solmujen läpikäyntiin on kolme tavallista järjestystä:

- *esijärjestys (pre-order)*: käsittelemme ensin juuren, sitten vasemman alipuun ja lopuksi oikean alipuun
- *sisäjärjestys (in-order)*: käsittelemme ensin vasemman alipuun, sitten juuren ja lopuksi oikean alipuun
- *jälkijärjestys (post-order)*: käsittelemme ensin vasemman alipuun, sitten oikean alipuun ja lopuksi juuren

Esimerkiksi kuvan 6.1 puussa esijärjestys on $[1, 2, 4, 3, 5, 6, 7, 8]$, sisäjärjestys on $[4, 2, 1, 5, 3, 7, 6, 8]$ ja jälkijärjestys on $[4, 2, 5, 7, 8, 6, 3, 1]$.

Voimme käydä binääripuun solmut läpi kaikissa yllä mainituissa järjestyksissä rekursion avulla. Esimerkiksi seuraava proseduuuri tulostaa puun solmut sisäjärjestyksessä, kun sille annetaan parametrina puun juuri:

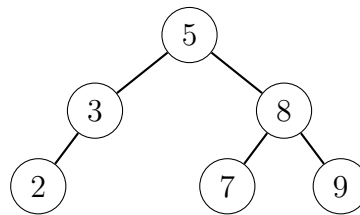


Figure 6.3: Joukkoa $\{2, 3, 5, 7, 8, 9\}$ vastaava binäärihakupuu.

```

procedure lapikaynti(solmu)
  if solmu == null
    return
  lapikaynti(solmu.vasen)
  print(solmu.arvo)
  lapikaynti(solmu.oikea)

```

6.2 Binäärihakupuun toiminta

Binäärihakupuu on binääripuu, jonka kukin solmu vastaa yhtä joukon alkia. Solmut on järjestetty niin, että jokaisessa solmussa kaikki vasemman alipuun solmut ovat arvoltaan pienempiä ja vastaavasti kaikki oikean alipuun solmut ovat arvoltaan suurempia. Tämän ansiosta voimme löytää kätevästi halutun alkion puusta aloittamalla haun puun juuresta.

Kuvassa 6.3 on esimerkkinä joukkoa $\{2, 3, 5, 7, 8, 9\}$ vastaava binäärihakupuu, jonka juurena on alkio 5. Vasemmassa alipuussa on kaikki alkia 5 pienemmät alkio, eli se vastaa joukkoa $\{2, 3\}$. Oikeassa alipuussa taas on kaikki alkia 5 suuremmat alkio, eli se vastaa joukkoa $\{7, 8, 9\}$. Huomaa, että tämä on yksi monista tavoista muodostaa binäärihakupuu kyseiselle joukolle ja voisimme valita myös minkä tahansa muun alkion puun juureksi.

6.2.1 Operaatioiden toteutus

Seuraavaksi käymme läpi, kuinka voimme toteuttaa binäärihakupuun avulla operaatioita joukon alkioden käsittelemiseen. Osoittautuu, että voimme toteuttaa kaikki operaatiot ajassa $O(h)$, missä h on puun korkeus.

Alkion etsiminen

Kun haluamme etsiä joukosta alkia x , lähdemme liikkeelle puun juuresta ja kuljemme alaspäin puussa. Kun olemme solmussa, jossa on alkio a , vaih-

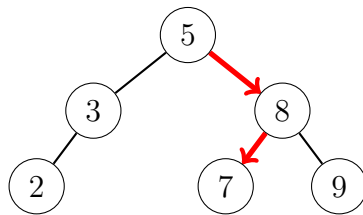


Figure 6.4: Alkion 7 etsiminen joukosta $\{2, 3, 5, 7, 8, 9\}$ juuresta alkaen.

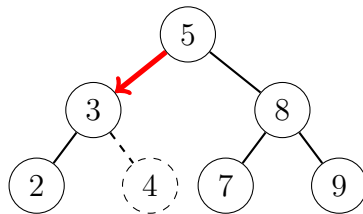


Figure 6.5: Alkion 4 lisääminen joukkoon $\{2, 3, 5, 7, 8, 9\}$.

toehtoja on kolme. Jos $a = x$, olemme löytäneet halutun alkion, jos $a > x$, jatkamme hakua solmun vasempaan lapseen, ja jos $a < x$, jatkamme hakua solmun oikeaan lapseen. Jos kuitenkin solmulla ei ole lasta, johon meidän tulisi edetä, toteamme, ettei joukossa ole alkiota x .

Kuva 6.4 näyttää, kuinka löydämme alkion 7 joukosta $\{2, 3, 5, 7, 8, 9\}$. Juurena on alkio 5, joten alkion 7 täytyy olla juuren oikeassa alipuussa. Tämän alipuun juurena on alkio 8, joten nyt taas tiedämme, että alkion 7 täytyy olla vasemmassa alipuussa, josta se löytyykin.

Alkion lisääminen

Kun haluamme lisätä joukkoon alkion x , jota ei vielä ole joukossa, kuljemme ensin puussa aivan kuin etsisimme alkiota x . Sitten kun olemme päässeet solmuun, jolla ei ole lasta, johon meidän tulisi edetä, luomme uuden solmun alkiolle x ja lisäämme sen tähän kohtaan lapseksi.

Kuva 6.5 näyttää, kuinka lisäämme alkion 4 joukkoon $\{2, 3, 5, 7, 8, 9\}$. Kun haemme puusta alkiota 4, päädyimme solmuun, jossa on alkio 3 ja jolla ei ole oikeaa lasta. Niinpä luomme alkiolle 4 uuden solmun, jonka asetamme alkion 3 solmun oikeaksi lapseksi.

Pienin alkio / suurin alkio

Kun haluamme löytää joukon pienimmän alkion, lähdemme liikkeelle juuresta ja etenemme joka askeleella solmun vasempaan lapseen. Kun solmulla ei ole enää vasenta lasta, olemme löytäneet joukon pienimmän alkion.

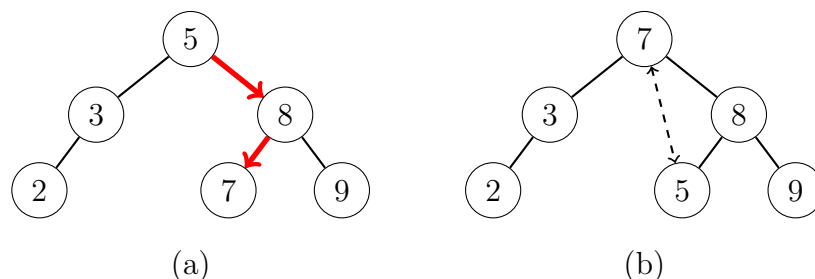


Figure 6.6: Alkion 5 poistaminen joukosta $\{2, 3, 5, 7, 8, 9\}$. (a) Koska alkiolla 5 on kaksi lasta, etsimme seuraavan suuremman alkion 7. (b) Vaihdamme keskenään alkiot 5 ja 7, minkä jälkeen voimme poistaa helposti alkion 5.

Vastaavalla tavalla löydämme joukon suurimman alkion etenemällä koko ajan oikeaan lapseen juuresta.

Seuraava suurempi alkio / edellinen pienempi alkio

Kun haluamme löytää joukon pienimmän alkion, joka on suurempi kuin x , lähdemme liikkeelle puun juuresta. Kun olemme solmussa, jossa on alkio a , etenemme vasempaan lapseen, jos $a > x$, ja oikeaan lapseen, jos $a \leq x$. Jatkamme näin, kunnes emme voi edetä alemmas. Haluttu alkio on pienin alkiota x suurempi alkio kaikista alkiosta, joiden kautta kuljimme.

Kun haluamme vastaavasti löytää joukon suurimman alkion, joka on pienempi kuin x , menetellemme käänteisesti edelliseen nähden.

Alkion poistaminen

Kun haluamme poistaa joukosta alkion x , etsimme ensin alkiota x vastaavan solmun tavalliseen tapaan. Jos solmulla ei ole lapsia tai vain yksi lapsi, on helppoa poistaa solmu puusta ja säilyttää puun rakenne muuten ennallaan. Jos kuitenkin solmulla on kaksi lasta, tilanne on hankalampi. Tällöin etsimme alkion y , joka on pienin x :ää suurempi alkio, ja vaihdamme keskenään alkiot x ja y puussa. Tämän jälkeen on helppoa poistaa solmu, jossa on nyt alkio x , koska sillä ei voi olla kahta lasta (jos solmulla olisi vasen lapsi, y ei olisi pienin x :ää suurempi alkio).

Kuva 6.6 näyttää, kuinka poistamme joukosta $\{2, 3, 5, 7, 8, 9\}$ alkion 5. Alkio on puun juuresta ja solmulla on kaksi lasta, joten meidän tulee etsiä ensin pienin alkiota 5 suurempi alkio, joka on 7. Vaihdamme sitten keskenään arvot 5 ja 7, minkä jälkeen on helppoa poistaa alkio 5.

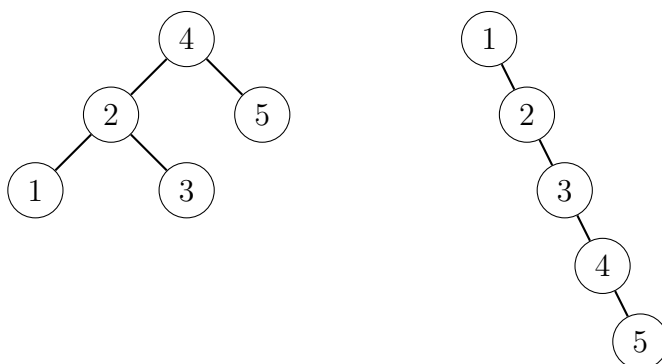


Figure 6.7: Kaksi binäärihakupuuta joukolle $\{1, 2, 3, 4, 5\}$. Vasemman puun korkeus on 2 ja oikean puun korkeus on 4.

6.2.2 Operaatioiden tehokkuus

Binäärihakupuun operaatiot vievät aikaa $O(h)$, missä h on puun korkeus, joten operaatioiden tehokkuus riippuu puun korkeudesta. Operaatioiden tehokkuuteen vaikuttaa siis, miten olemme rakentaneet puun. Esimerkiksi kuvassa 6.7 on kaksi mahdollista binäärihakupuuta joukolle $\{1, 2, 3, 4, 5\}$. Vasemman puun korkeus on 2, kun taas oikean puun korkeus on 4.

Jotta binäärihakupuun toimisi tehokkaasti, haluamme, että puun korkeus ei kasva liian suureksi. Tarkemmin ottaen tavoitteena on, että solmut ovat jakautuneet tasaisesti puun eri puolille ja puun korkeus on $O(\log n)$ eli puu on *tasapainoinen* (*balanced*). Jos onnistumme tässä, kaikki puun operaatiot toimivat tehokkaasti ajassa $O(\log n)$. Saavutamme tavoitteen lisäämällä puuhun ehtoja, jotka rajoittavat sen korkeutta sopivasti.

Binäärihakupuun tasapainottamiseen tunnetaan monia menetelmiä. Tutustumme seuraavaksi AVL-puuhun, joka on varhaisin tunnettu tasapainoinen binäärihakupuun. AVL-puu on yksinkertaisempi kuin monet myöhemmin kehitetyt rakenteet, minkä vuoksi se sopii hyvin esittelemään puiden tasapainotuksen ideoita. Ohjelmointikielten standardikirjastoissa käytetään kuitenkin yleensä muita rakenteita kuten punamustaa puuta.

6.3 AVL-puu

AVL-puu (*AVL tree*) on tasapainoinen binäärihakupuun, jonka korkeus on aina $O(\log n)$, minkä ansiosta puun operaatiot toimivat tehokkaasti ajassa $O(\log n)$. AVL-puussa jokaiseen solmuun liittyy *tasapainoehto*, joka takaa, että puu on tasapainoinen. Puuta päivittäessä täytyy pitää huolta siitä, että tasapainoehto säilyy voimassa kaikissa solmuissa.

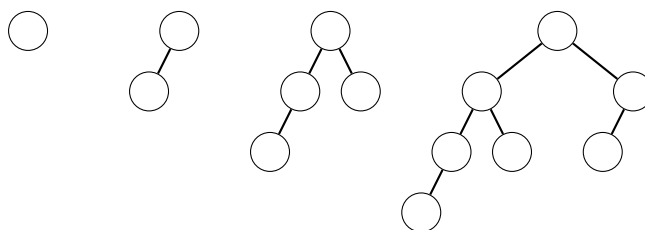


Figure 6.8: Vähiten solmuja sisältävät AVL-puut korkeuksille 0, 1, 2 ja 3.

6.3.1 Tasapainoehto

AVL-puun tasapainoehtona on, että *jokaisessa solmussa vasemman ja oikean lapsen alipuiden korkeusero on enintään 1*.

Esimerkiksi kuvan 6.7 vasen puu on AVL-puu, kun taas oikea puu ei ole. Oikea puu ei ole AVL-puu, koska esimerkiksi solmussa 1 vasemman lapsen alipuun korkeus on -1 mutta oikean lapsen alipuun korkeus on 3. Korkeuksien erona on siis 4, vaikka ero saisi olla enintään 1.

Kutsumme AVL-puun tasapainoehtoa *AVL-ehdoksi*. Osoittautuu, että jos binäärihakupuu täyttää AVL-ehdon, sen korkeus on $O(\log n)$. Eli jos pystymme toteuttamaan puun operaatiot niin, että AVL-ehto säilyy, saamme aikaan binäärihakupuun, jonka operaatiot toimivat ajassa $O(\log n)$.

Miksi sitten AVL-ehto takaa, että binäärihakupuun korkeus on $O(\log n)$? Voimme lähestyä asiaa pahimman tapauksen kautta: kun tiedämme, että AVL-puussa on n solmua, mikä on sen *suurin mahdollinen* korkeus? Voimme selvittää tämän laskemalla ensin käänteisesti, mikä on *pienin mahdollinen* solmujen määrä AVL-puussa, jonka korkeus on h .

Merkitään $f(h)$:lla korkeutta h olevan AVL-puun pienintä mahdollista solmujen määrää. Kuvan 6.8 mukaisesti funktion ensimmäiset arvot ovat $f(0) = 1$, $f(1) = 2$, $f(2) = 4$ ja $f(3) = 7$. Yleisemmin

$$f(h) = 1 + f(h-1) + f(h-2),$$

kun $h \geq 2$, koska jos haluamme rakentaa AVL-puun korkeutta h , jossa on mahdollisimman vähän solmuja, meidän kannattaa laittaa juuren lapsiksi AVL-puut korkeutta $h-1$ ja $h-2$ niin, että kummassakin alipuussa on mahdollisimman vähän solmuja. Funktiolle pätee

$$f(h) \geq 2f(h-2),$$

eli funktion arvo ainakin kaksinkertaistuu kahden askeleen välein. Voimme ilmaista tämän alarajan

$$f(h) \geq 2^{h/2},$$

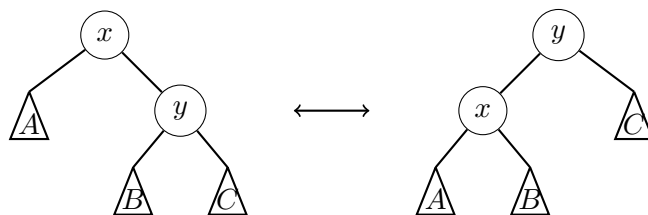


Figure 6.9: Kierrot, joiden avulla korjaamme AVL-puuta.

jonka voimme taas muuttaa ylärajaksi

$$h \leq 2 \log f(h).$$

Tarkastellaan sitten puuta, jossa on n solmua ja jonka korkeus on h . Korkeudelle täytyy päteä $f(h) \leq n$, koska korkeutta h olevassa puussa on vähintään $f(h)$ solmua. Niinpä saamme ylärajan

$$h \leq 2 \log n,$$

mikä tarkoittaa samaa kuin $h = O(\log n)$.

6.3.2 Kiertojen toteuttaminen

Voimme toteuttaa AVL-puun operaatiot muuten samaan tapaan kuin yleisessä binäärihakupuussa, mutta alkion lisäämisen ja poistamisen jälkeen täytyy varmistaa, että AVL-ehto on edelleen voimassa. Tämä onnistuu tekemällä sopivia *kiertoja* (*rotation*), jotka muuttavat puun rakennetta. Jotta voimme toteuttaa kierrot, pidämme jokaisessa solmussa tietoa siitä, mikä on solmusta alkavan alipuun korkeus.

Osoittautuu, että voimme korjata puun rakenteen kaikissa tilanteissa käyttäen kahta kiertotyyppiä, jotka on esitetty kuvassa 6.9. Kierrämme solmuja x ja y , joihin liittyvät alipuut A , B ja C . Voimme tehdä kierron joko vasemmalta oikealle, jolloin solmu y nousee ylöspäin, tai oikealta vasemmalle, jolloin solmu x nousee ylöspäin.

Kun lisäämme AVL-puuhun solmun, jonkin solmun AVL-ehto voi rikkoutua. Tämä ilmenee niin, että jossain solmussa lasten alipuiden korkeudet ovat ennen lisäämistä h ja $h + 1$ ja lisäämisen jälkeen h ja $h + 2$. Kuva 6.10 näyttää esimerkin tällaisesta tilanteesta. Vasemmassa puussa solmun 3 lasten korkeudet ovat 0 ja 1, joten AVL-ehto on kunnossa. Oikeassa puussa olemme lisänneet solmun 9, minkä seurauksena solmun 3 lasten korkeudet ovat 0 ja 2 eikä AVL-ehto enää päde.

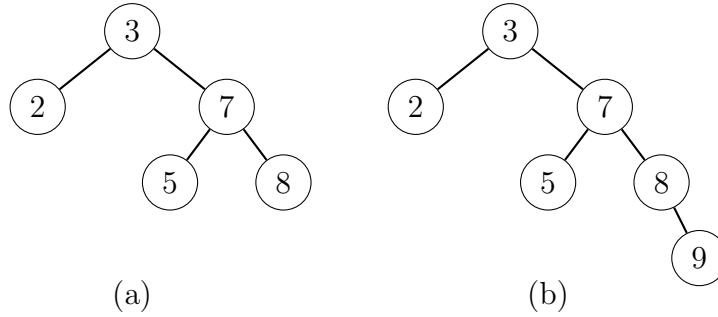


Figure 6.10: (a) Jokaisessa puun solmussa pätee AVL-ehto. (b) AVL-ehto menee rikki solmussa 3, kun lisäämme puuhun solmun 9.

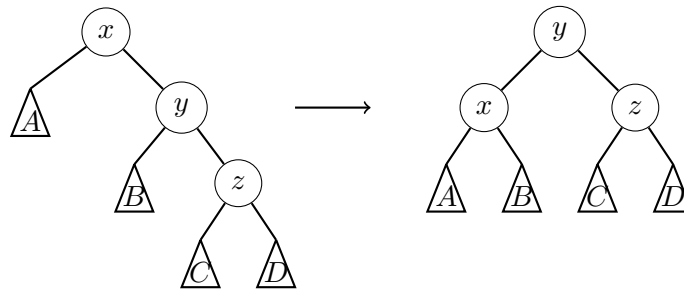


Figure 6.11: Tapaus 1: nostamme solmua y ylöspäin.

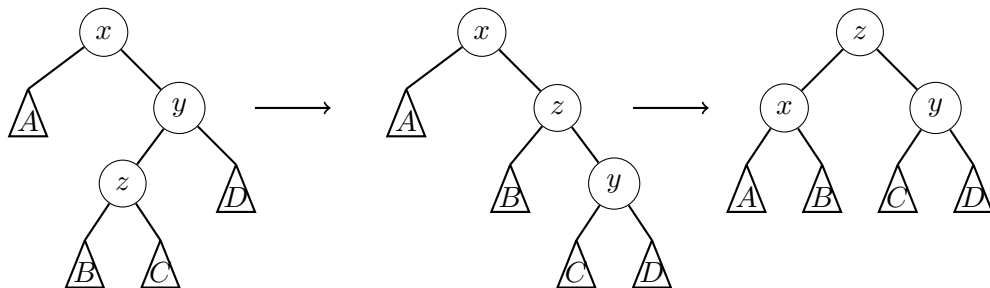


Figure 6.12: Tapaus 2: nostamme solmua z kahdesti ylöspäin.

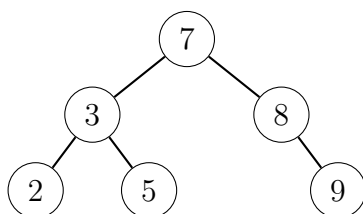


Figure 6.13: Kierrämme solmun 7 puun juureksi, jolloin AVL-ehto pätee taas.

Solmun lisäämisen jälkeen kuljemme puussa ylöspäin lisätystä solmusta juureen ja päivitämme solmujen korkeudet. Jos jokin solmu ei täytä AVL-ehtoa, korjaamme asian tekemällä yhden tai kaksi kiertoa. Oletetaan, että x on alimpana puussa oleva solmu, jossa AVL-ehto ei päde, y on x :n lapsi, jonka alipuussa on lisätty solmu, ja z on puolestaan y :n lapsi, jonka alipuussa on lisätty solmu. Tapauksia on kaksi: jos y ja z ovat samanpuoleisia lapsia, kierrämme solmua y kerran ylöspäin (kuva 6.11), ja muuten kierrämme solmua z kahdesti ylöspäin (kuva 6.12). Tämän korjauksen jälkeen AVL-ehto on jälleen voimassa kaikissa puun solmuissa, eli riittää aina korjata ehto alimmassa solmussa, jossa se ei ole voimassa.

Kuvassa 6.10(b) AVL-ehto ei ole voimassa solmussa 3, koska vasemman alipuun korkeus on 0 ja oikean alipuun korkeus on 2. Tässä tapauksessa $x = 3$, $y = 7$ ja $z = 8$. Koska y on x :n oikea lapsi ja z on y :n oikea lapsi, meidän riittää tehdä yksi kierto, joka nostaa solmua 7 ylöspäin puun juureksi. Tuloksena on kuvan 6.13 mukainen puu, jossa AVL-ehto on jälleen voimassa, koska solmun 7 kummankin alipuun korkeus on nyt 1.

Kun poistamme puusta solmun, menettelemme melko samalla tavalla kuin lisäämisessä. Nyt on mahdollista, että ennen poistoa jossakin solmussa lasten alipuiden korkeudet ovat h ja $h - 1$ ja poiston jälkeen h ja $h - 2$. Poiston jälkeen nousemme puussa ylöspäin poistetun solmun vanhemmasta alkaen, ja jos vastaan tulee solmu x , jossa AVL-ehto ei päde, korjaamme asian. Tällä kertaa valitsemme solmut y ja z niin, että y on x :n lapsi, jonka korkeus on suurin, ja samoin z on y :n lapsi, jonka korkeus on suurin. Jos y :n kummankin lapsen korkeus on sama, valitsemme z :n niin, että se on samanpuoleinen lapsi kuin y . Sitten korjaamme AVL-ehdon kierroilla kuten solmun lisäämisessä. Toisin kuin lisäämisessä, saatamme joutua korjaamaan ehdon useassa solmussa, koska ehdon korjaaminen yhdessä solmussa voi rikkoa sen jossain ylemmässä solmussa. Kun lopulta saavumme juureen, AVL-ehto pätee jälleen kaikissa solmuissa.

Koska AVL-puun korkeus on $O(\log n)$ ja jokainen kierto tapahtuu vakioajassa, pystymme korjaamaan tasapainon sekä lisäämisen että poistamisen

jälkeen ajassa $O(\log n)$. Lisäämisen jälkeen kuljemme puuta ylöspäin $O(\log n)$ askelta ja teemme enintään kaksi kiertoa. Poistamisen jälkeen taas kuljemme puuta ylöspäin $O(\log n)$ askelta ja teemme enintään $O(\log n)$ kiertoa.

6.4 Ohjelmointikielten toteutukset

6.4.1 Java

Javan tietorakenteet `TreeSet` ja `TreeMap` toteuttavat tasapainoisen binäärihakupuun. Ne perustuvat punamustaan puuhun, joka on AVL-puun tapainen puurakenne mutta monimutkaisempi. Nämä rakenteet tarjoavat samat metodit kuin `HashSet` ja `HashMap`, mutta lisäksi tehokkaita metodeita, jotka liittyvät alkioden järjestykseen puussa.

Seuraava koodi luo `TreeSet`-rakenteen ja lisää siihen lukuja:

```
TreeSet<Integer> luvut= new TreeSet<>();
luvut.add(4);
luvut.add(1);
luvut.add(8);
luvut.add(7);
```

Koska joukko on järjestyksessä, pystymme etsimään tehokkaasti pienimmän ja suurimman alkion metodeilla `first` ja `last`:

```
int pienin = joukko.first(); // 1
int suurin = joukko.last();  // 8
```

Pystymme myös etsimään tehokkaasti seuraavan tiettyä alkiota suuremman tai pienemmän alkion metodeilla `higher` ja `lower`:

```
int suurempi = joukko.higher(5); // 7
int pienempi = joukko.lower(5);  // 4
```

Seuraava koodi luo sanakirjan `TreeMap`-rakenteen avulla:

```
TreeMap<String,String> sanakirja = new TreeMap<>();
sanakirja.put("apina","monkey");
sanakirja.put("banaani","banana");
sanakirja.put("cembalo","harpsichord");
```

Tämä sanakirja muodostuu avain-arvo-pareista, joissa avaimet ovat suomen kielen sanoja ja arvot ovat englannin kielen sanoja. Sanakirjan sisältö on

järjestetty avainten perusteella. Esimerkiksi voimme selvittää, mikä on aakkosjärjestyksessä ensimmäinen ja viimeinen avain:

```
String eka = sanakirja.firstKey(); // apina  
String vika = sanakirja.lastKey(); // cembalo
```

Samoin voimme selvittää lähinnä tiettyä avainta olevat avaimet:

```
String seuraava = sanakirja.higherKey("biisoni"); // cembalo  
String edellinen = sanakirja.lowerKey("biisoni"); // banaan
```

6.4.2 Python

Pythonin standardikirjastossa *ei* ole tasapainoisen binäärihakupuun toteutusta. Tämä on kielen suunnittelussa tehty päätös, joka liittyy ihanteeseen, että kielessä on yksi selkeä tapa toteuttaa tietty asia. Koska Pythonissa on hajautustaulua käyttävät tietorakenteet `set` ja `dict`, binäärihakupuulle ei ole nähty tarvetta.

Miten selviämme Pythonissa ilman binäärihakupuuta? Mahdollisia ratkaisuja ovat:

- Useissa tilanteissa hajautustaulu riittää: binäärihakupuuta tarvitaan vain, jos haluamme etsiä alkioita järjestyksen perusteella.
- Vaikka järjestykselle olisi tarvetta, voimme käyttää usein binäärihakupuun sijasta listan järjestämistä tai seuraavassa luvussa käsiteltävää binääriokeoa, jonka toteutus on Pythonin standardikirjastossa.
- Voimme tarvittaessa käyttää myös jotain standardikirjaston ulkopuolista binäärihakupuun toteutusta.

Itse asiassa yleinen suuntaus suosituissa ohjelmointikielissä vaikuttaa olevan, että standardikirjastossa on tarjolla hajautustaulu mutta ei binäärihakupuuta. Näin on myös JavaScript-kielessä, jossa tietorakenteet `Set` ja `Map` käyttävät hajautustaulua. Vaikuttaa siltä, että käytännön ohjelmoinnissa ei ole usein tarvetta säilyttää joukon alkioita järjestyksessä.

Miksi sitten kahdesta vaihtoehdoisesta tietorakenteesta (hajautustaulu ja binäärihakupuuta) on valittu hajautustaulu, jossa on vähemmän ominaisuuksia? Syynä on, että hajautustaulun toteutus on paljon yksinkertaisempi, minkä seurauksena se toimii yleensä tehokkaammin. Ero näkyy myös aikavaativuuksissa: hajautustaulun operaatiot vievät keskimäärin aikaa $O(1)$, kun taas binäärihakupuussa aikaa kuluu $O(\log n)$.

6.5 Tehokkuusvertailu

Monissa ongelmissa on kaksi mahdollista lähestymistapaa: voimme käyttää joko joukkorakenteita tai taulukon järjestämistä. Vaikka molemmat tavat johtavat tehokkaaseen ratkaisuun, vakiokertoimissa voi olla merkittäviä eroja, jotka vaikuttavat käytännön tehokkuuteen.

Tarkastelemme seuraavaksi ongelmaa, jossa meille on annettu n lukua sisältävä taulukko, ja haluamme selvittää, montako eri lukua taulukossa on. Ratkaisemme ongelman Javalla kolmella eri tavalla ja tutkimme sitten ratkaisujen tehokkuutta.

Ratkaisu 1: TreeSet

Ensimmäinen tapa ratkaista tehtävä on luoda `TreeSet`, johon lisäämme taulukon luvut. Koska jokainen luku voi esiintyä joukossa vain kerran, joukon koko ilmaisee meille, montako eri lukua taulukossa on. Tämä ratkaisu vie aikaa $O(n \log n)$, koska jokainen `add`-operaatio vie aikaa $O(\log n)$.

Ratkaisu 2: HashSet

Emme tarvitse `TreeSet`-rakenteen alkioiden järjestystä, joten saamme toisen ratkaisun käyttämällä sen sijaan `HashSet`-rakennetta. Koodi säilyy muuten täysin samanlaisena. Tämä ratkaisu vie aikaa $O(n)$ hajautuksen ansiosta.

Ratkaisu 3: järjestäminen

Kolmas tapa ratkaista tehtävä on käyttää järjestämistä: kopioimme ensin luvut uuteen taulukkoon, järjestämme tämän taulukon ja tutkimme sitten, monessako kohdassa järjestetyssä taulukossa luku vaihtuu. Tämä ratkaisu vie aikaa $O(n \log n)$, koska taulukon järjestäminen vie aikaa $O(n \log n)$.

Vertailun tulokset

Taulukko 6.1 esittää tehokkuusvertailun tulokset. Jokaisessa testissä taulukossa on satunnaisia lukuja väliltä $1 \dots 10^9$.

Osoittautuu, että ratkaisujen välillä on merkittäviä tehokkuuseroja. Ensinnäkin `HashSet`-ratkaisu on noin kolme kertaa nopeampi kuin `TreeSet`-ratkaisu. Tämä onkin odotettavaa, koska hajautustaulun operaatiot vievät aikaa $O(1)$, kun taas binäärihakupuun operaatiot vievät aikaa $O(\log n)$. Selvästi nopein ratkaisu on kuitenkin kolmas järjestämistä käyttävä ratkaisu, joka on noin kymmenen kertaa `TreeSet`-ratkaisua nopeampi.

taulukon koko n	TreeSet	HashSet	järjestäminen
10^6	0.74 s	0.25 s	0.09 s
$2 \cdot 10^6$	1.60 s	0.45 s	0.19 s
$4 \cdot 10^6$	5.60 s	1.56 s	0.52 s
$8 \cdot 10^6$	12.19 s	4.50 s	0.97 s

Table 6.1: Algoritmien suoritusaikojen vertailu.

Miten on mahdollista, että sekä **TreeSet**-ratkaisun että järjestämisratkaisun aikavaativuus on $O(n \log n)$, mutta järjestämisratkaisu on kymmenen kertaa nopeampi? Tämä johtuu siitä, että taulukon järjestäminen on hyvin kevyt operaatio ja se tehdään vain kerran. Kun sitten käytetään **TreeSet**-rakennetta, sen taustalla joudutaan ylläpitämään tasapainoista binäärihakupuuta. Tällöin alkioiden lisäykset ovat raskaita operaatioita, koska jokaiselle uudelle alkiolle täytyy etsiä paikka puusta ja mahdollisesti korjata lisäyksen jälkeen puun rakennetta kiertojen avulla.

Vaikka hajautustaulu ja binäärihakupuu ovat käteviä, niitä ei siis kannata käyttää turhaan. Jos haluamme ratkaista ongelman todella tehokkaasti, kannattaa miettiä, voisimmeko käyttää tavalla tai toisella järjestämistä näiden tietorakenteiden sijaan.

Chapter 7

Keko

Keko (*heap*) on tietorakenne, jonka operaatiot ovat alkion lisääminen sekä pienimmän tai suurimman alkion etsiminen ja poistaminen. Vaikka voisimme toteuttaa nämä operaatiot myös binäärihakupuun avulla, keon etuna on, että saamme aikaan yleistä joukkorakennetta *kevyemmän* rakenteen, kun rajoitumme tilanteeseen, jossa käytössämme on vain nämä operaatiot.

Tutustumme tässä luvussa binäärikeko-rakenteeseen, joka on tavallisimmin käytetty kekorakenne. Binäärikeko toteutetaan binääripuuna, ja se mahdollistaa alkion etsimisen ajassa $O(1)$ sekä alkioden lisäykset ja poistot ajassa $O(\log n)$. Pystymme toteuttamaan binäärikeon tehokkaasti taulukkona, koska sen toiminnot ovat yleistä binääripuuta rajatumpia.

7.1 Binäärikeko

Binäärikeko (*binary heap*) on binääripuu, jonka kaikki tasot alinta tasoa lukuun ottamatta ovat täynnä solmuja. Alimman tason solmut on puolestaan sijoitettu mahdollisimman vasemmalle ylempien solmujen lapsiksi.

Keon luonnissa täytyy päättää, onko se *minimikeko* vai *maksimikeko*. Minimikeossa voimme etsiä ja poistaa pienimmän alkion, kun taas maksimikeossa voimme etsiä ja poistaa suurimman alkion. Keon toiminta perustuu siihen, että jokainen keon solmu täyttää *kekoehdon*. Minimikeossa ehtona on, että jokaisen solmun arvo on pienempi tai yhtä suuri kuin sen kummankin lapsen arvo. Maksimikeossa puolestaan ehtona on, että jokaisen solmun arvo on suurempi tai yhtä suuri kuin sen kummankin lapsen arvo. Kekoehdon ansiosta minimikeon juuressa on keon pienin alkio ja maksimikeon juuressa on keon suurin alkio.

Kuvassa 7.1 on minimikeko, johon on tallennettu kymmenen alkioita. Keon kolme ensimmäistä tasoa ovat täynnä ja neljännellä tasolla kolme en-

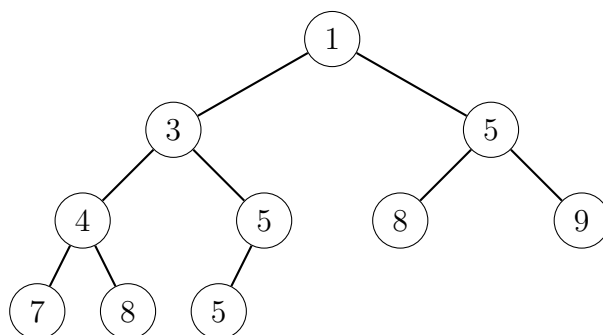


Figure 7.1: Minimikeko, joka sisältää alkioit $[1, 3, 4, 5, 5, 5, 7, 8, 8, 9]$.

simmäistä kohtaa on käytetty. Keon juurena on joukon pienin alkio 1, ja kaikki solmut täyttävät kekoehdon. Huomaa, että sama alkio voi esiintyä monta kertaa keossa, kuten tässä keossa alkioit 5 ja 8.

7.1.1 Keon tallentaminen

Tallennamme binäärikeon *taulukkona*, joka sisältää keon solmujen arvot järjestyksessä ylhäältä alaspäin ja vasemmalta oikealle. Tämä tehokas tallennustapa on mahdollinen, koska keon kaikki tasot ovat täynnä solmuja. Tavallinen tapa on tallentaa keko taulukkoon niin, että indeksointi alkaa 1:stä. Esimerkiksi kuvan 7.1 keko vastaa seuraavaa taulukkoa:

1	2	3	4	5	6	7	8	9	10
1	3	5	4	5	8	9	7	8	5

Taulukkototeutuksen etuna on, että voimme laskea helposti, missä kohdissa keon alkioit ovat taulukossa. Ensinnäkin keon juuri eli pienin tai suurin alkio on aina kohdassa 1. Lisäksi jos tiedämme, että tietty solmu on kohdassa k , niin solmun vasen lapsi on kohdassa $2k$, solmun oikea lapsi on kohdassa $2k + 1$ ja solmun vanhempi on kohdassa $\lfloor k/2 \rfloor$. Esimerkissämme solmu 3 on taulukossa kohdassa 2, joten sen vasen lapsi on kohdassa 4, oikea lapsi on kohdassa 5 ja vanhempi on kohdassa 1.

Käytännössä haluamme yleensä, että pystymme lisäämään keoon uusia alkioita, jolloin saattaa olla tarpeen suurentaa taulukkoa. Voimme toteuttaa tämän samalla tavalla kuin taulukkolistassa, jolloin taulukon suurentaminen ei hidasta keon operaatioita.

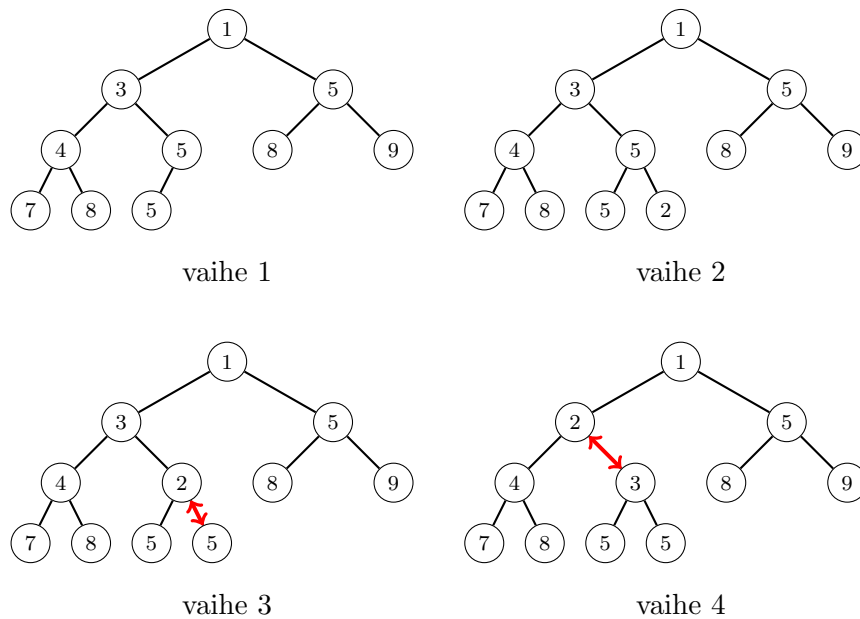


Figure 7.2: Lisäämme alkion 2 kekon ja nostamme sitä ylöspäin, kunnes kekoehto tulee jälleen voimaan.

7.1.2 Operaatioiden toteutus

On helppoa etsiä minimikeon pienin alkio tai maksimikeon suurin alkio $O(1)$ -ajassa, koska tämä alkio on aina keon juuressa. Seuraavaksi näemme, kuinka voimme toteuttaa alkion lisäämisen sekä pienimmän tai suurimman alkion poistamisen $O(\log n)$ -ajassa.

Alkion lisääminen

Kun lisäämme uuden alkion kekon, lisäämme sen ensin seuraavaan vapaana olevaan paikkaan puussa. Jos alimmalla tasolla on tilaa, lisäämme sen sinne mahdollisimman vasemmalle, ja muuten aloitamme uuden tason, jossa on toistaiseksi vain lisättävä solmu. Alkion lisäämisen jälkeen täytyy varmistaa, että kekoehto säilyy edelleen voimassa. Tämä tapahtuu siirtämällä alkioita ylöspäin keossa, kunnes kekoehto tulee voimaan.

Kuva 7.2 näyttää, mitä tapahtuu, kun lisäämme alkion 2 esimerkikekoomme. Lisäämme alkion ensimmäiseen vapaaseen kohtaan keon alimmalla tasolla. Koska alkio 2 on pienempi kuin sen vanhempi 5, vaihdamme nämä alkiot keskenään. Tämän jälkeen alkio 2 on pienempi kuin sen vanhempi 3, joten vaihdamme myös nämä alkiot keskenään. Nyt kekoehto on voimassa eikä meidän tarvitse enää tehdä muutoksia kekon.

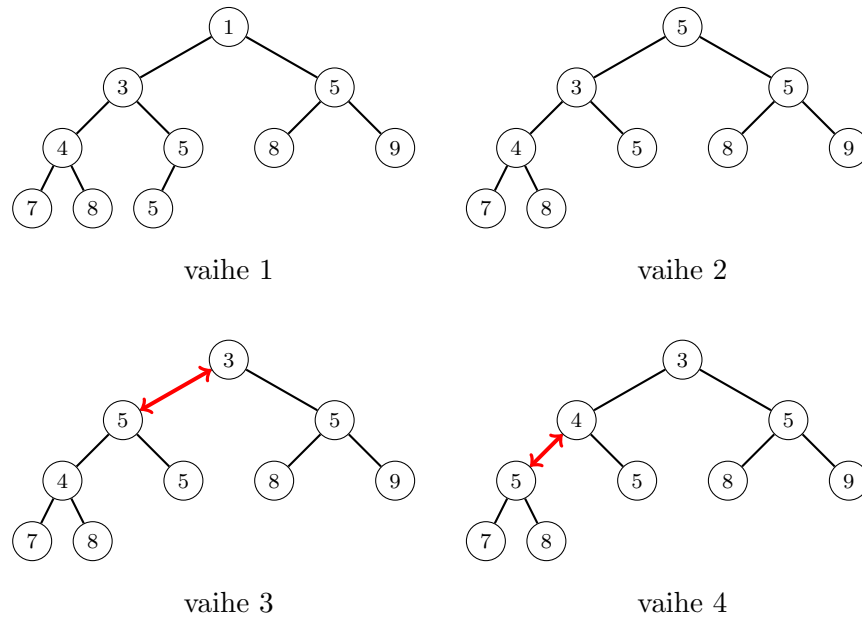


Figure 7.3: Poistamme keon juuresta olevan alkion korvaamalla sen viimeisellä alkiolla ja laskettamalla sitä alaspäin puussa.

Alkion lisääminen kekkoon vie aikaa $O(\log n)$, koska keossa on $O(\log n)$ tasoa ja kuljemme aina ylöspäin keon pohjalta huippua kohden, kunnes olemme löytäneet alkiolle sopivan paikan keosta.

Alkion poistaminen

Kun haluamme poistaa keon juuresta olevan alkion, siirrämme ensin keon viimeisen alkion keon juureksi ja poistamme sille kuuluneen solmun. Tämän jälkeen lasketamme juureen nostettua alkiota alaspäin keossa, kunnes kekoehto on jälleen voimassa kaikkialla. Koska solmulla voi olla kaksi lasta, voi olla kaksi vaihtoehtoa, kumman lapsista nostamme ylemmäs. Jos keko on minimikeko, valitsemme lapsen, jossa on pienempi arvo, ja jos keko on maksimikeko, valitsemme vastaavasti lapsen, jossa on suurempi arvo.

Kuva 7.3 näyttää, kuinka poistamme esimerkikekostamme pienimmän alkion eli juuresta olevan alkion 1. Aluksi korvaamme alkion 1 keon viimeisellä alkiolla 5 ja poistamme keosta alkiolle 5 kuuluneen solmun. Tämän jälkeen vaihdamme keskenään alkion 5 ja sen vasemman lapsen alkion 3, ja sitten vielä alkion 5 ja sen vasemman lapsen alkion 4. Tämän jälkeen kekoehto on voimassa ja olemme onnistuneet poistamaan pienimmän alkion keosta.

Alkion poistaminen keosta vie aikaa $O(\log n)$, koska keossa on $O(\log n)$

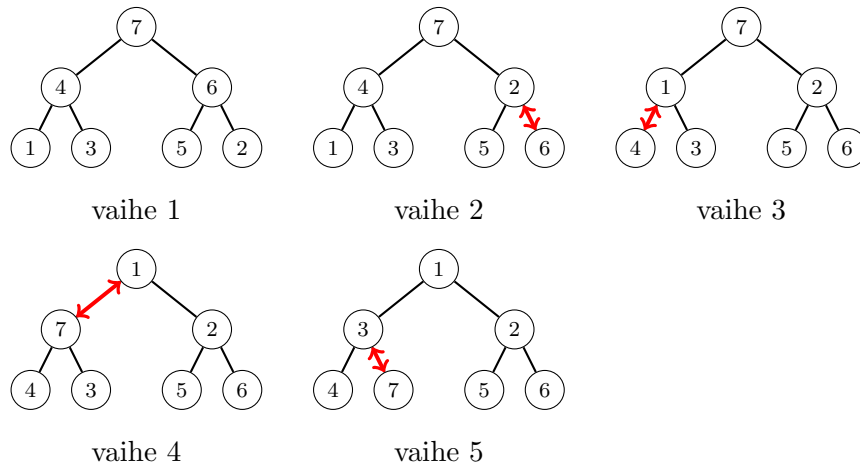


Figure 7.4: Muutamme taulukon keoksi korjaamalla kekoehdon alipuissa.

tasoa ja kuljemme polkua alaspäin keon huipulta pohjaa kohden.

7.2 Lisää keosta

Koska keko on tallennettu taulukkona, voimme tulkita minkä tahansa taulukon kekona, kunhan vain kekoehto on voimassa taulukon kaikissa kohdissa. Käymme seuraavaksi läpi menetelmän, jonka avulla voimme *muuttaa* taulukon keoksi $O(n)$ -ajassa. Tämän jälkeen tutustumme kekojärjestämiseen, joka on $O(n \log n)$ -aikainen järjestämisalgoritmi.

7.2.1 Taulukosta keoksi

Oletetaan, että meillä on n alkiota sisältävä taulukko ja haluamme muuttaa sen keoksi. Suoraviivainen tapa on luoda tyhjä keko ja lisätä jokainen taulukon alkio siihen erikseen $O(\log n)$ -ajassa. Tällä tavalla saamme rakennettua keon $O(n \log n)$ -ajassa. Osoittautuu kuitenkin, että pystymme myös muuttamaan taulukon *suoraan* keoksi tehokkaammin ajassa $O(n)$.

Ideana on järjestää alkuperäisen taulukon alkioita uudestaan niin, että kekoehto tulee voimaan taulukon jokaiseen kohtaan – jolloin taulukko on muuttunut keoksi. Käymme läpi taulukon alkiot lopusta alkuun ja varmistamme jokaisessa kohdassa, että kekoehto on voimassa kyseisestä kohdasta alka- vassa alipuussa. Jos kekoehto ei ole voimassa, korjaamme sen laskettamalla kyseisen kohdan alkiota alaspäin keossa. Kun lopulta pääsemme taulukon alkuun, kekoehto on voimassa koko taulukossa.

Kuva 7.4 näyttää esimerkin, jossa muutamme taulukon $[7, 4, 6, 1, 3, 5, 2]$

minimikeoksi. Kun tulkitsemme taulukon kekona, kekoehto on aluksi rikki monessa taulukon kohdassa. Ensin korjaamme kekoehdon tason 2 alipuissa vaihtamalla keskenään alkiot 2 ja 6 ja sitten alkiot 1 ja 4. Tämän jälkeen korjaamme kekoehdon tason 1 alipuussa eli koko keossa laskettamalla alkion 7 keon huipulta pohjalle. Nyt kekoehto on voimassa kaikkialla taulukossa, joten olemme onnistuneet muuttamaan taulukon keoksi.

Miksi tämä vie aikaa vain $O(n)$? Oletetaan, että keossa on h tasoa ja kaikki tasot ovat täynnä solmuja eli keossa on $n = 2^h - 1$ solmua. Laskemme jokaiselle tasolle, montako alkioita laskeutuu enintään jostakin tämän tason solmusta alaspäin. Ensinnäkin tasolta 1 tasolle 2 laskeutuu enintään 1 alkio – juuressa oleva alkio. Vastaavasti tasolta 2 tasolle 3 laskeutuu enintään $1 + 2$ alkioita ja tasolta 3 tasolle 4 laskeutuu enintään $1 + 2 + 4$ alkioita. Yleisemmin tasolta k tasolle $k+1$ laskeutuu enintään $1 + 2 + \dots + 2^{k-1} = 2^k - 1$ alkioita. Koska tasoja on h ja alimmalta tasolta ei voi laskeutua alaspäin, kokonaistyömäärä on enintään

$$(2^1 - 1) + (2^2 - 1) + \dots + (2^{h-1} - 1) = 2^h - h - 1 \leq n,$$

joten aikaa kuluu vain $O(n)$.

7.2.2 Kekojärjestäminen

Kekojärjestäminen (*heap sort*) on järjestämisalgoritmi, jonka toiminta perustuu kekoon. Ideana on muuttaa järjestettävä taulukko ensin keoksi ja sen jälkeen poistaa alkiot keosta yksi kerrallaan järjestyksessä. Kekojärjestäminen vie aikaa $O(n \log n)$, koska taulukon muuttaminen keoksi vie aikaa $O(n)$ ja n alkion poistaminen keosta vie aikaa $O(n \log n)$.

Kuva 7.5 näyttää esimerkin kekojärjestämisestä, kun järjestämme taulukon $[5, 2, 3, 1, 7, 4, 6]$ pienimmästä suurimpaan. Muutamme ensin taulukon maksimikeoksi, jolloin taulukosta tulee $[7, 5, 6, 1, 2, 4, 3]$. Tämän jälkeen poistamme yksi kerrallaan keon juuressa olevan alkion vaihtamalla sen keon viimeisen alkion kanssa. Tämän seurauksena keosta poistuneet alkiot (merkitty katkoviivoilla) muodostavat lopulta järjestetyn taulukon.

Olemme siis saaneet aikaan kolmannen $O(n \log n)$ -aikaisen järjestämisalgoritmin lomitusjärjestämisen ja pikajärjestämisen rinnalle. Kekojärjestämisen etuna on, että sen tilavaativuus on vain $O(1)$, koska keon operaatiot käyttävät vain yksittäisiä muuttujia. Kekojärjestäminen ei ole käytännössä yhtä tehokas algoritmi kuin lomitusjärjestäminen tai pikajärjestäminen, minkä vuoksi se ei ole saavuttanut samanlaista asemaa järjestämisalgoritmien joukossa. Siinä on kuitenkin yksi kiinnostava ominaisuus: jos haluamme selvittää vain taulukon k pienintä tai suurinta alkioita, tämä onnistuu ajassa $O(n + k \log n)$, koska

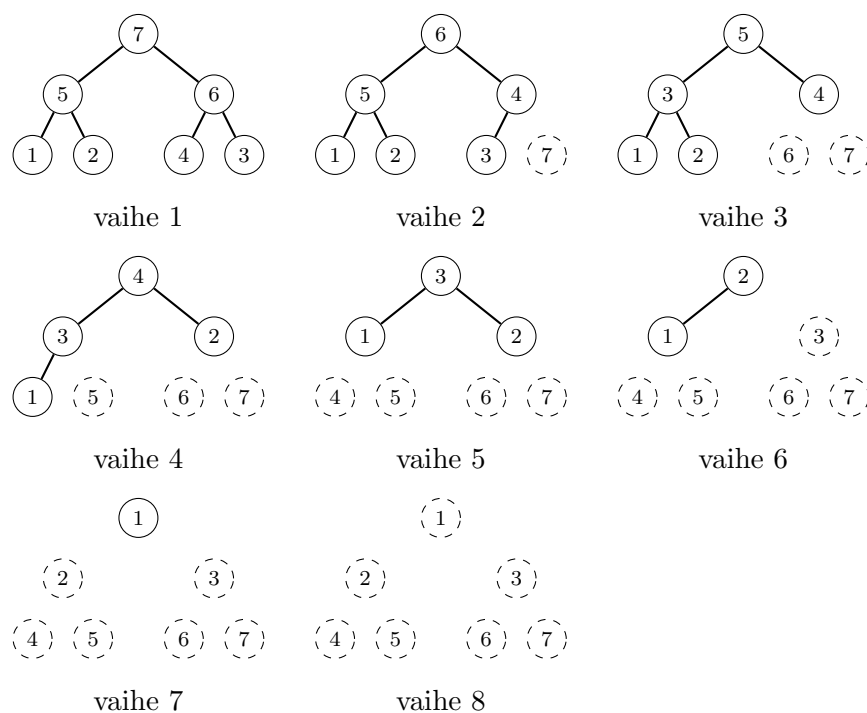


Figure 7.5: Esimerkki kekojärjestämisestä.

keon luonti vie aikaa $O(n)$ ja sen jälkeen riittää poistaa keosta k kertaa pienin tai suurin alkio ajassa $O(\log n)$.

7.3 Ohjelmointikielten toteutukset

7.3.1 Java

Javassa kekorakenteesta käytetään nimeä *prioriteettijono* (*priority queue*). Tietorakenne `PriorityQueue` toteuttaa binäärikeon, joka on oletuksena minimikeko. Metodi `peek` hakee pienimmän alkion, ja metodi `poll` hakee ja poistaa pienimmän alkion.

```
PriorityQueue<Integer> jono = new PriorityQueue<>();
jono.add(5);
jono.add(3);
jono.add(8);
jono.add(7);
System.out.println(jono.peek()); // 3
System.out.println(jono.poll()); // 3
System.out.println(jono.poll()); // 5
```

Jos haluamme luoda prioriteettijonon, joka onkin maksimikeko, voimme tehdä tämän seuraavasti:

```
PriorityQueue<Integer> jono =
    new PriorityQueue<>(Collections.reverseOrder());
```

7.3.2 Python

Pythonin standardikirjaston moduulissa `heapq` on funktioita, joiden avulla listaa voi käsitellä binäärikekona. Toteutuksessa keko on minimikeko ja tavallisesta käytännöstä poiketen keko on 0-indeksoitu eli keon pienin alkio on aina listan kohdassa 0.

Seuraava koodi näyttää, miten voimme käsitellä kekoa:

```
from heapq import heappush, heappop

keko = []
heappush(keko, 5)
heappush(keko, 3)
heappush(keko, 8)
heappush(keko, 7)
print(keko[0]) # 3
heappop(keko)
print(keko[0]) # 5
```

Lisäksi saatavilla on funktio `heapify`, joka muuttaa olemassa olevan listan minimikeoksi lineaarisessa ajassa:

taulukon koko n	PriorityQueue	TreeSet
10^6	0.29 s	0.78 s
$2 \cdot 10^6$	0.71 s	1.50 s
$4 \cdot 10^6$	1.56 s	3.72 s
$8 \cdot 10^6$	3.68 s	9.43 s

Table 7.1: Algoritmien suoritusaikojen vertailu.

```

from heapq import heapify, heappop

keko = [5,3,8,7]
heapify(keko)
print(keko[0]) # 3
heappop(keko)
print(keko[0]) # 5

```

7.4 Tehokkuusvertailu

Mitä hyötyä keosta oikeastaan on? Meillähän on olemassa jo binäärihakupuun, jonka avulla voimme toteuttaa kaikki keon operaatiot ja *enemmänkin*. Keossa voimme hakea ja poistaa vain pienimmän tai suurimman alkion, mutta binäärihakupuussa voimme käsitellä myös muita alkioita.

Keon etuna on, että siinä on tehokkaan taulukkototeutuksen ansiosta pienemmät *vakiokertoimet* kuin binäärihakupuussa. Jos meille riittää, että voimme hakea ja poistaa vain pienimmän tai suurimman alkion, voi siis olla hyvä ratkaisu käyttää kekoa binäärihakupuun sijasta. Mutta kuinka suuria erot ovat käytännössä?

Tästä antaa kuvaa testi, jossa vertailemme keskenään Javan tietorakenteita `PriorityQueue` ja `TreeSet`. Testissä aineistona on taulukko, jossa on satunnaisessa järjestyksessä luvut $1, 2, \dots, n$. Lisäämme ensin taulukon $n/2$ ensimmäistä lukua joukkoon. Tämän jälkeen käymme läpi loput $n/2$ lukua ja jokaisen luvun kohdalla lisäämme sen joukkoon ja poistamme joukon pienimmän luvun.

Taulukko 7.1 näyttää testin tulokset. Tämän testin perusteella näyttää siltä, että keon käyttämisestä on todellista hyötyä, koska `PriorityQueue` toimii 2–3 kertaa nopeammin kuin `TreeSet`.

Chapter 8

Peruuttava haku

Peruuttava haku (*backtracking*) on menetelmä, jonka avulla voimme käydä järjestelmällisesti kaikki yhdistelmät, jotka voidaan muodostaa annetuista aineksista. Peruuttava haku on raa'an voiman algoritmi, jonka toteutus on yleensä suoraviivainen, ja menetelmä on käyttökelpoinen silloin, kun yhdistelmien määrä on niin pieni, että ehdimme käydä kaikki läpi.

Tässä luvussa tutustumme ensin peruuttavan haun algoritmeihin, jotka käyvät läpi lukujen yhdistelmiä. Tämän jälkeen näemme, miten peruuttavaa hakua voi käyttää kahdessa vaikeammassa ongelmassa ja miten hakua voi tehostaa. Lopuksi toteutamme pelin tekoälyn minimax-algoritmillä, joka perustuu peruuttavaan hakuun.

8.1 Silmukoista rekursioon

Oletetaan, että haluamme käydä läpi kaikki n luvun yhdistelmät, joissa jokainen luku on kokonaisluku väliltä $1 \dots m$. Tällaisia yhdistelmiä on yhteensä m^n , koska kohtia on n ja joka kohdassa luvun voi valita m tavalla. Esimerkiksi jos $n = 3$ ja $m = 4$, yhdistelmät ovat $[1, 1, 1]$, $[1, 1, 2]$, $[1, 1, 3]$, $[1, 1, 4]$, $[1, 2, 1]$, $[1, 2, 2]$, $[1, 2, 3]$, $[1, 2, 4]$, jne.

Jos lukujen määrä n on etukäteen tiedossa, voimme luoda n sisäkkäistä silmukkaa, joista jokainen käy m lukua läpi. Esimerkiksi seuraava koodi käy läpi kaikki yhdistelmät tapauksessa $n = 3$:

```
for a = 1 to m
  for b = 1 to m
    for c = 1 to m
      print(a,b,c)
```

Tämä on sinänsä mainio ratkaisu, mutta siinä on yksi ongelma: luku-

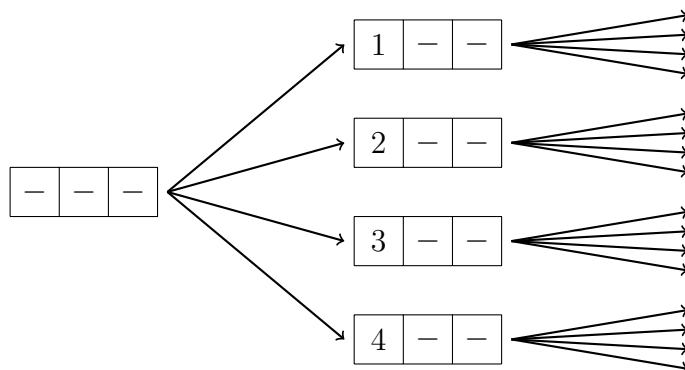


Figure 8.1: Yhdistelmien muodostaminen alkaa ($n = 3$ ja $m = 4$).

jen määrä n vaikuttaa silmukoiden määrään. Jos haluaisimme muuttaa n :n arvoa, meidän täytyisi muuttaa koodin silmukoiden määrää, mikä ei ole hyvä asia. Peruuttavan haun avulla voimme kuitenkin toteuttaa ratkaisun rekursiivisesti niin, että sama koodi toimii kaikille n :n arvoille.

8.1.1 Haun toteuttaminen

Seuraava rekursiivinen proseduur `haku` muodostaa yhdistelmiä peruuttavan haun avulla. Parametri k tarkoittaa kohtaa, johon seuraava luku asetetaan. Jos $k = n$, jokin yhdistelmä on valmistunut, jolloin se tulostetaan. Muuten `haku` käy läpi kaikki tavat sijoittaa kohtaan k luku $1 \dots m$ ja jatkaa rekursiivisesti kohtaan $k + 1$. `Haku` lähtee käyntiin kutsulla `haku(0)`, ja `luvut` on n -kokoinen taulukko, johon yhdistelmä muodostetaan.

```

procedure haku(k)
  if k == n
    print(luvut)
  else
    for i = 1 to m
      luvut[k] = i
      haku(k+1)

```

Kuva 8.1 näyttää, miten `haku` lähtee liikkeelle tapauksessa $n = 3$ ja $m = 4$. Merkki ”-” tarkoittaa lukua, jota ei ole vielä valittu. Haun ensimmäinen taso valitsee yhdistelmän ensimmäisen luvun kohtaan 0. Tämän valintaan on neljä vaihtoehtoa, koska mahdolliset luvut ovat $1 \dots 4$, joten `haku` haarautuu neljään osaan. Tämän jälkeen `haku` jatkaa rekursiivisesti eteenpäin ja valitsee muihin kohtiin tulevat luvut.

Voimme arvioida algoritmin tehokkuutta laskemalla, montako kertaa prose-

duuria **haku** kutsutaan yhteensä haun aikana. Proseduuria kutsutaan kerran parametrilla 0, m kertaa parametrilla 1, m^2 kertaa parametrilla 2, jne., joten kutsujen määrä on yhteensä

$$1 + m + m^2 + \dots + m^n = \frac{m^{n+1} - 1}{m - 1} = O(m^n).$$

Tästä näkee, että kutsujen yhteismäärä on samaa luokkaa kuin viimeisen tason kutsujen määrä. Viimeisellä tasolla tehdäänkin enemmän kutsuja kuin kaikilla muilla tasoilla yhteensä.

8.1.2 Osajoukkojen läpikäynti

Tarkastellaan sitten tilannetta, jossa haluamme käydä läpi kaikki n alkion joukon *osajoukot*. Osajoukkoja on yhteensä 2^n , koska jokainen alkio joko kuuluu tai ei kuulu osajoukkoon. Esimerkiksi joukon $\{2, 3, 5, 9\}$ osajoukkoja ovat $\{2, 5\}$ ja $\{3, 5, 9\}$.

Osoittautuu, että voimme muodostaa osajoukot käymällä läpi kaikki n luvun yhdistelmät, joissa jokainen luku on 0 tai 1. Ideana on, että jokainen yhdistelmän luku kertoo, kuuluuko tietty alkio osajoukkoon, Alkio kuuluu osajoukkoon tarkalleen silloin, kun sen kohdalla on luku 1. Esimerkiksi kun joukkona on $\{2, 3, 5, 9\}$, yhdistelmä $[1, 0, 1, 0]$ vastaa osajoukkoa $\{2, 5\}$ ja yhdistelmä $[0, 1, 1, 1]$ vastaa osajoukkoa $\{3, 5, 9\}$.

Seuraava koodi näyttää, miten voimme käydä osajoukot läpi peruuttavan haun avulla. Proseduri **haku** valitsee, otetaanko kohdassa k oleva alkio mukaan osajoukkoon vai ei, ja merkitsee tämän tiedon taulukkoon **valinta**. Kuten ennenkin, **haku** lähtee käyntiin kutsulla **haku(0)**.

```
procedure haku(k)
  if k == n
    // käsittele osajoukko
  else
    for i = 0 to 1
      valinta[k] = i
      haku(k+1)
```

8.1.3 Permutaatioiden läpikäynti

Peruuttavan haun avulla voimme myös käydä läpi joukon *permutaatiot* eli erilaiset järjestykset. Kun joukossa on n alkioita, siitä voidaan muodostaa kaikkiaan $n!$ permutaatiota. Esimerkiksi joukon $\{1, 2, 3, 4\}$ permutaatioita ovat $\{2, 4, 1, 3\}$ ja $\{4, 3, 1, 2\}$.

Tässä tilanteessa haluamme käydä läpi n luvun yhdistelmiä, joissa jokainen luku on väliltä $1 \dots n$ ja lisäksi mikään luku ei toistu. Saamme tämän aikaan lisäämällä hakuun uuden taulukon `mukana`, joka kertoo, onko tietty luku jo mukana. Joka vaiheessa haku valitsee yhdistelmään vain sellaisia lukuja, joita ei ole valittu siihen aiemmin.

```

procedure haku(k)
  if k == n
    print(luvut)
  else
    for i = 1 to n
      if not mukana[i]
        mukana[i] = true
        luvut[k] = i
        haku(k+1)
        mukana[i] = false

```

8.2 Esimerkkejä

Käymme seuraavaksi läpi kaksi vaativampaa esimerkkiä peruuttavan haun soveltamisesta. Ratkaisemme ensin shakkiin liittyvän ongelman, ja tämän jälkeen etsimme parhaan tavan työtehtävien jakamiseen. Molemmissa soveluksissa näemme myös, miten peruuttavaa hakua voi tehostaa.

8.2.1 Kuningatarongelma

Tehtävämme on laskea, monellako tavalla $n \times n$ -shakkilaudalle voidaan asettaa n kuningatarta niin, etteivät mitkään kaksi kuningatarta uhkaa toisiaan. Shakissa kuningattaret voivat uhata toisiaan vaaka-, pysty- tai vinosuuntaisesti. Esimerkiksi tapauksessa $n = 4$ mahdollisia sijoitustapoja on kaksi, jotka on esitetty kuvassa 8.2.

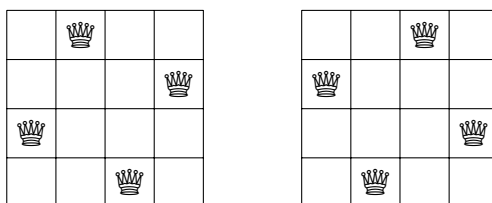


Figure 8.2: Kuningatarongelman ratkaisut tapauksessa $n = 4$.

Voimme ratkaista tehtävän toteuttamalla algoritmin, joka käy laudan läpi

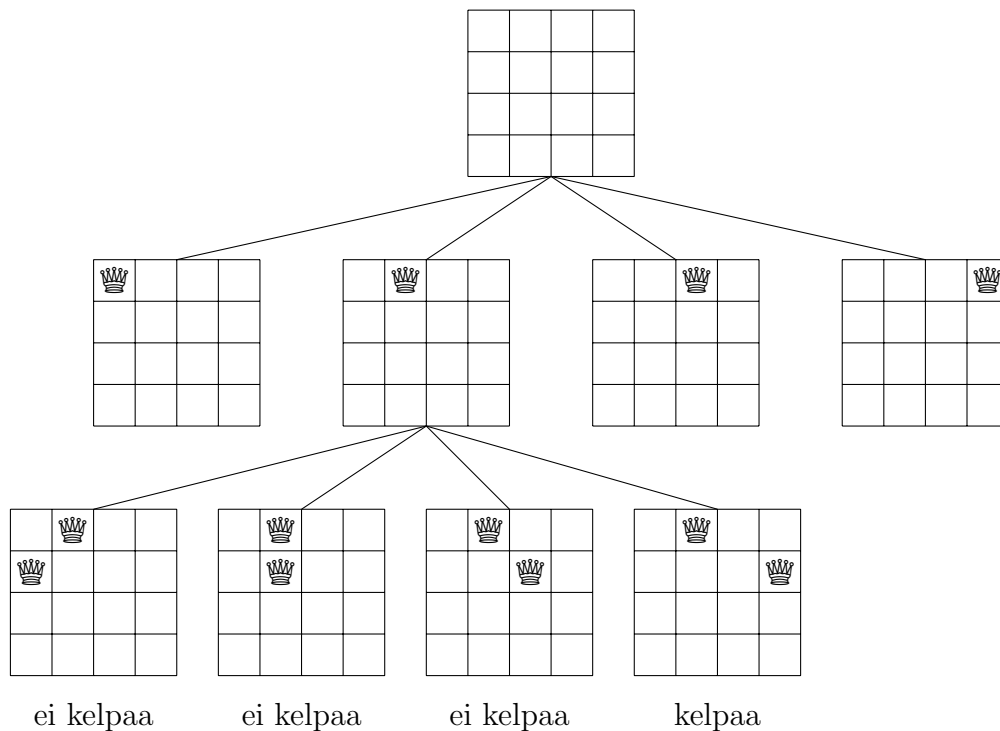


Figure 8.3: Peruuttavan haun toiminta kuningatarongelmassa.

ylhäältä alaspäin ja asettaa yhden kuningattaren jokaiselle riville. Kuva 8.3 esittää haun toimintaa tapauksessa $n = 4$. Ensimmäisen rivin kuningatar voidaan asettaa mihin tahansa sarakkeeseen, mutta seuraavilla riveillä aiemmat valinnat rajoittavat hakua. Kuvassa näkyy toisen kuningattaren sijoittaminen, kun ensimmäinen kuningatar on toisessa sarakkeessa. Tällöin ainoa vaihtoehto on, että toinen kuningatar on viimeisessä sarakkeessa, koska kaikissa muissa tapauksissa kuningattaret uhkaisivat toisiaan.

Seuraava proseduuuri **haku** esittää peruuttavan haun algoritmin, joka laskee kuningatarongelman ratkaisut:

```

procedure haku(y)
  if y == n
    laskuri += 1
  else
    for x = 0 to n-1
      if voi_sijoittaa(y,x)
        kohta[y] = x
        haku(y+1)

```

Oletamme, että laudan rivit ja sarakkeet on numeroitu $0 \dots n-1$. Parametri y kertoo, mille riville seuraava kuningatar tulee sijoittaa, ja haku lähtee käyntiin kutsulla `haku(0)`. Jos rivinä on n , kaikki kuningattaret on jo sijoitettu, joten yksi ratkaisu on löytynyt. Muuten suoritetaan silmukka, joka käy läpi mahdolliset sarakkeet muuttujan x avulla. Jos kuningatar voidaan sijoittaa sarakkeeseen x eli se ei uhkaa mitään aiemmin sijoitettua kuningatarta, merkitään taulukkoon `kohta`, että kuningatar y on sarakkeessa x , ja haku jatkuu eteenpäin rekursiivisesti.

Lisäksi täytyy toteuttaa funktio `voi_sijoittaa`, joka tutkii, voidaanko uusi kuningatar sijoittaa rivin y sarakkeeseen x . Tämä voidaan selvittää taulukon `kohta` avulla näin:

```
function voi_sijoittaa(y,x)
  for i = 0 to y-1
    if kohta[i] == x
      return false
    if abs(i-y) == abs(kohta[i]-x)
      return false
  return true
```

Funktio käy läpi kaikki aiemmin sijoitetut kuningattaret. Jos aiemmin sijoitettu kuningatar olisi samassa sarakkeessa (ensimmäinen ehto) tai samalla vinorivillä (toinen ehto) kuin uusi kuningatar, tällaista sijoitusta ei voida tehdä ja funktio palauttaa `false`. Jos taas mikään aiempi kuningatar ei uhkaa uutta kuningatarta, funktio palauttaa `true`. Huomaa jälkimmäisessä ehdossa kätevä tapa tarkastaa itseisarvon (funktio `abs`) avulla, ovatko kuningattaret samalla vinorivillä. Kuningattaret ovat samalla vinorivillä tarkalleen silloin, kun niiden vaaka- ja pystysuuntaiset erot ovat samat.

Nyt meillä on valmis algoritmi, jonka avulla voimme käydä läpi kuningatarongelman ratkaisuja. Taulukko 8.1 näyttää ratkaisujen määrät tapauksissa $n = 1 \dots 10$. Algoritmi selvittää nämä tapaukset salamannopeasti, mutta suuremmilla n :n arvoilla algoritmi alkaa viedä paljon aikaa. Syynä tähän on, että kuningatarten sijoitustapojen määrä kasvaa *eksponentiaalisesti* eli räjähdysmäisen nopeasti. Esimerkiksi tapauksessa $n = 20$ erilaisia ratkaisuja on jo yli 39 miljardia.

Voimme kuitenkin koettaa nopeuttaa algoritmia parantamalla sen toteutusta. Yksi helppo tehostus on hyödyntää *symmetriaa*. Jokaista kuningatarongelman ratkaisua vastaa toinen ratkaisu, joka saadaan peilaamalla ratkaisu vaakasuuntaisesti. Esimerkiksi kuvassa 8.2 ratkaisut voidaan muuttaa toisikseen peilaamalla. Tämän havainnon ansiosta voimme puolittaa algoritmin suoritusajan lisäämällä vaatimuksen, että ensimmäinen kuningatar asetetaan laudan vasempaan puoliskoon, ja kertomalla lopuksi vastauksen kahdella.

laudan koko n	ratkaisujen määrä
1	1
2	0
3	0
4	2
5	10
6	4
7	40
8	92
9	352
10	724

Table 8.1: Kuningatarongelman ratkaisujen määriä.

Jos laudan koko on pariton, täytyy vielä käsitellä erikseen tapaus, jossa ensimmäinen kuningatar sijoitetaan keskisarakkeeseen.

Toinen mahdollinen tehostus olisi toteuttaa funktio `voi_sijoittaa` paremmin. Tällä hetkellä se käy läpi kaikki aiemmin sijoitetut kuningattaret ja vie aikaa $O(n)$, mutta funktio on mahdollista toteuttaa myös ajassa $O(1)$ ottamalla käyttöön uusia aputaulukoita, joissa on tietoa, mitkä sarakkeet ja vinorivit ovat uhattuina. Tämä tehostus on kuitenkin vaikeampi toteuttaa kuin symmetristen ratkaisujen karsinta.

Kuningatarongelma on kuitenkin pohjimmiltaan vaikea ongelma, eikä sen ratkaisuun tunneta mitään oleellisesti raakaa voimaa parempaa tapaa. Tällä hetkellä suurin tapaus, jonka ratkaisu tunnetaan, on $n = 27$. Tämän tapauksen käsittely vei aikaa noin vuoden laskentaklusterilla, jossa oli suuri määrä rinnakkain laskevia suorittimia¹.

8.2.2 Työtehtävien jakaminen

Tarkastellaan tilannetta, jossa on n työtehtävää ja n työntekijää. Tehtävät tulee jakaa työntekijöille niin, että jokainen työntekijä suorittaa tarkalleen yhden työtehtävän. Jokaisesta työtehtävästä tiedetään, paljonko sen suorittaminen maksaa kullakin työntekijällä, ja tavoitteena on etsiä ratkaisu, jossa kokonaishinta on pienin mahdollinen.

Kuvassa 8.4 on esimerkki tilanteesta, jossa $n = 3$. Tässä optimaalinen tapa jakaa työtehtävät on, että B suorittaa ensimmäisen työtehtävän, C suorittaa toisen työtehtävän ja A suorittaa kolmannen työtehtävän. Tämän

¹27-Queens Puzzle: Massively Parallel Enumeration and Solution Counting. <https://github.com/preusser/q27>

Työntekijä A	4	8	5
Työntekijä B	1	1	3
Työntekijä C	4	2	6

Figure 8.4: Optimaalinen tapa jakaa työtehtävät.

ratkaisun kustannus on $1 + 2 + 5 = 8$.

Oletamme, että työtehtävät ja työntekijät on numeroitu $0 \dots n - 1$ ja voimme lukea taulukosta `hinta[a][b]`, paljonko työtehtävän a suorittaminen maksaa työntekijällä b . Toteutamme peruuttavan haun algoritmin, joka käy läpi työtehtävät järjestyksessä ja valitsee jokaiselle työntekijän.

Seuraava proseduuuri `haku` saa kaksi parametria: k on seuraavaksi käsiteltävä työtehtävä ja h on tähän mennessä muodostunut hinta. Haku lähtee käyntiin kutsulla `haku(0, 0)`. Taulukko `mukana` pitää kirjaa, mitkä työntekijät on jo valittu, ja muuttujassa p on yhteishinta parhaassa tähän mennessä löytyneessä ratkaisussa. Ennen hakua muuttujan p arvona on ∞ , koska mitään ratkaisua ei ole vielä olemassa.

```

procedure haku(k,h)
  if k == n
    p = min(p,h)
  else
    for i = 0 to n-1
      if not mukana[i]
        mukana[i] = true
        haku(k+1,h+hinta[k][i])
        mukana[i] = false

```

Tämä on toimiva peruuttavan haun algoritmi, ja haun päätteeksi muuttujassa p on parhaan ratkaisun yhteishinta. Algoritmi on kuitenkin hidas, koska se käy aina läpi kaikki $n!$ mahdollista ratkaisua. Koska haluamme vain löytää parhaan ratkaisun emmekä käydä läpi kaikkia ratkaisuja, pystymme tehostamaan algoritmia lisäämään siihen ehdon, joka lopettaa ratkaisun muodostamisen, jos siitä ei voi tulla aiempaa parempi.

Testaamme seuraavaksi algoritmia tapauksella, jossa $n = 20$ ja jokainen taulukossa `hinta` oleva arvo on satunnainen kokonaisluku välillä $1 \dots 100$. Tässä tapauksessa edellä kuvattu algoritmi kävisi läpi

$$20! = 2432902008176640000$$

eri ratkaisua, mikä veisi aikaa *satoja vuosia*. Jotta voimme ratkaista tapauk-

sen, meidän täytyy parantaa algoritmia niin, että se ei käy läpi kaikkia ratkaisuja mutta löytää kuitenkin parhaan ratkaisun. Tässä avuksi tulee tekniikka, josta käytetään usein nimeä *branch and bound*. Siinä ideana on tehostaa peruuttavaa hakua vähentämällä tutkittavien ratkaisujen määrää sopivien ylä- ja alarajojen avulla.

Keskeinen havainto on, että voimme rajoittaa hakua muuttujan p avulla. Tässä muuttujassa on joka hetkellä tähän mennessä parhaan löydetyn ratkaisun yhteishinta, joten tämä on *yläraja* sille, kuinka suuri parhaan ratkaisun yhteishinta voi olla. Toisaalta muuttujassa h on muodosteilla olevan ratkaisun hinta tässä vaiheessa, joka on *alaraja* yhteishinnalle. Jos $h \geq p$, muodosteilla olevasta ratkaisusta ei voi tulla aiempaa parempaa. Voimmekin lisätä algoritmin alkuun seuraavan tarkastuksen:

```
procedure haku(k,h)
  if h >= p
    return
  ...
```

Tämän ansiosta ratkaisun muodostaminen päättyy heti, jos sen hinta on yhtä suuri tai suurempi kuin parhaan tiedossa olevan ratkaisun hinta. Tämän tehostuksen avulla saamme ratkaistua muodostamamme tapauksen testikoneella 152 sekunnissa. Tämä on hieno saavutus, koska alkuperäinen algoritmi ei kyennyt ratkaisemaan tapausta lainkaan.

Voimme kuitenkin tehostaa algoritmia vielä lisää laskemalla tarkemman arvion alarajalle. Muodosteilla olevan ratkaisun hinta on varmasti ainakin h , mutta voimme lisäksi arvioida, paljonko myöhemmin valittavat työntekijät lisäävät kustannusta:

```
procedure haku(k,h)
  if h+arvio(k) >= p
    return
  ...
```

Tässä funktion *arvio* tulee antaa jokin arvio, paljonko ainakin maksaa suorittaa jäljellä olevat työtehtävät $k \dots n - 1$. Yksi kätevä tapa saada arvio on käydä läpi jäljellä olevat työtehtävät ja valita jokaisen tekijäksi *halvin* työntekijä välittämättä siitä, onko kyseinen työntekijä mahdollisesti valittu aiemmin. Tämä antaa alarajan sille, paljonko kustannuksia on ainakin vielä tiedossa. Tämän tehostuksen jälkeen tapauksen $n = 20$ käsittely vie testikoneella vain 7 sekuntia. Paremman alarajan ansiosta saimme siis algoritmin vielä noin 20 kertaa nopeammaksi verrattuna edelliseen versioon.

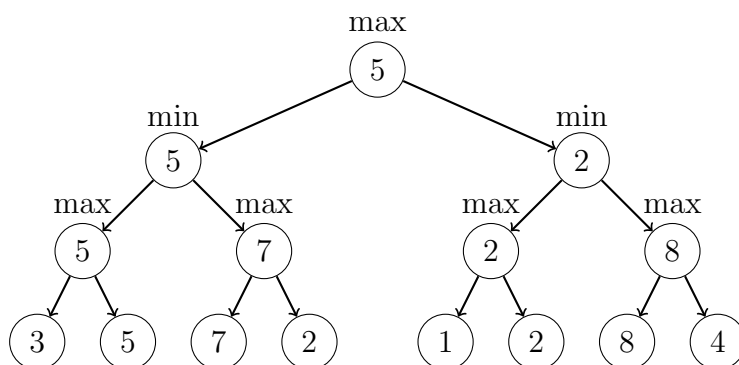


Figure 8.5: Esimerkki minimax-algoritmin toiminnasta.

8.3 Pelin tekoäly

Peruuttavan haun avulla voi myös toteuttaa tekoälyn kahden pelaajan peliin, kuten ristinolla tai shakkiin. Tällaisen tekoälyn täytyy pystyä päättämään, miten toimia annetussa pelin tilanteessa. Ideana on luoda haku, joka käy läpi järjestelmällisesti mahdollisia siirtoja, joita pelaajat voivat tehdä.

8.3.1 Minimax-algoritmi

Minimax-algoritmi on peruuttavaan hakuun perustuva algoritmi, joka valitsee tekoälyn siirron pelissä. Se käy läpi pelinkulkuja tiettyyn syvyyteen asti, arvioi pelitilanteet ja valitsee siirron, joka takaa mahdollisimman hyvältä vaikuttavan tuloksen. Algoritmin nimi tulee siitä, että se vuorotellen *minimoi* ja *maksimoi* tulosta puun kerroksissa.

Kuvassa 8.5 on esimerkki algoritmin toiminnasta, kun hakusyvyys on kolme tasoa ja joka tilanteessa on kaksi mahdollista siirtoa. Puun juurella oleva solmu vastaa nykyistä pelitilannetta, jossa tekoälyn täytyy päättää seuraava siirto, ja jokainen askel puussa alaspäin tarkoittaa yhtä siirtoa. Tässä tekoäly tutkii, mitä kaikkea voi tapahtua, kun se siirtää ensin itse, sitten vastustaja siirtää, ja sitten se siirtää uudestaan. Puun alimmalla tasolla on jokaisen kolmen siirron päässä olevan pelitilanteen *hyvyys* eli tekoälyn arvio, kuinka hyvä kyseinen tilanne on sen itsensä kannalta.

Tekoäly valitsee aina siirron, joka on mahdollisimman hyvä sen itsensä kannalta. Toisaalta tekoäly olettaa, että vastustaja valitsee siirron, joka on mahdollisimman huono tekoälyn kannalta. Niinpä joka toisella tasolla solmuissa on maksimi lasten arvoista ja joka toisella tasolla minimi. Tässä esimerkissä tekoäly tekee siirron, joka takaa, että kolmen siirron päästä pelitilanteen hyvyys on ainakin 5 riippumatta vastustajan toimista.

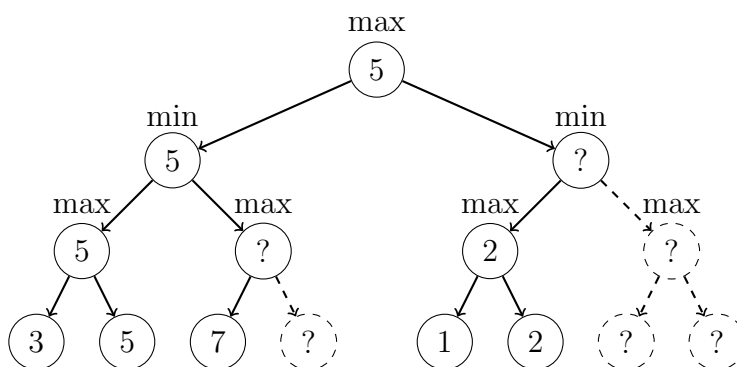


Figure 8.6: Alfa-beeta-karsinnan vaikutus.

Tekoälyn pelitaito riippuu kahdesta asiasta: (1) montako tasoa alemmas se tutkii puuta ja (2) miten hyvin se osaa arvioida pohjalla olevia pelitilanteita. Mitä syvemmälle haku jatkuu, sitä paremmin tekoäly pelaa, koska se saa enemmän tietoa pelinkuluista, mutta sitä kauemmin vie, ennen kuin tekoäly tekee päätöksen. Pelitilanteen arviointi puolestaan vaatii tietoa pelattavasta pelistä. Esimerkiksi shakissa pelitilannetta voi arvioida tutkimalla, mitä nappuloita itsellä ja vastustajalla on jäljellä.

8.3.2 Alfa-beeta-karsinta

Minimax-algoritmin toimintaa on mahdollista tehostaa alfa-beeta-karsinnalla, joka jättää puun osia tutkimatta, jos on selvää, että ne eivät voi vaikuttaa tekoälyn valintaan. Tämä muistuttaa aiemmin peruuttavan haun yhteydessä hyödyntämäämme branch and bound -tekniikkaa.

Kuva 8.6 näyttää, kuinka alfa-beeta-karsinta onnistuu tehostamaan hakua esimerkkitilanteessamme. Katkoviivoilla esitetyt puun osat ovat sellaisia, joita ei ole tarpeen tutkia, koska ne eivät voisi vaikuttaa ylempänä puussa olevaan osittain laskettuun minimiin tai maksimiin.

Kun tekoäly saapuu vasemmassa haarassa solmuun, jonka arvona on 7, sen ei enää tarvitse tutkia muita vasemman haaran solmuja. Tämä johtuu siitä, että tekoäly tietää jo tässä vaiheessa, että vasemman haaran minimi on enintään 5. Niinpä sen ei tarvitse laskea maksimia luvusta 7 ja jostain toisesta luvusta, koska tämä ei voisi muuttaa minimiä.

Kun sitten oikeassa haarassa tekoäly on laskenut toiseksi alimmalla tasolla olevaan solmuun maksimin 2, sen ei enää tarvitse tutkia muita oikean haaran solmuja. Tämä johtuu siitä, että juuren maksimi on ainakin 5 ja oikeassa haarassa valittaisiin minimi luvusta 2 ja jostain toisesta luvusta, mikä ei voisi kasvattaa juuressa olevaa maksimia.

Chapter 9

Dynaaminen ohjelmointi

Dynaaminen ohjelmointi (*dynamic programming*) on tekniikka, jonka avulla voi monessa tilanteessa laskea ongelman ratkaisujen yhteismäärän tai löytää ratkaisun, joka on jollain tavalla optimaalinen. Ideana on muotoilla ongelma rekursiivisesti niin, että ongelman ratkaisu voidaan muodostaa pienemmistä osaongelmista. Tämän jälkeen saamme aikaan tehokkaan algoritmin, kun käsittelemme jokaisen osaongelman vain kerran.

Tässä luvussa tutustumme ensin dynaamisen ohjelmoinnin perusteisiin toteuttamalla tehokkaan algoritmin, joka laskee palikkayhdistelmien määrän. Tämän jälkeen käymme läpi kokoelman muita esimerkkejä, jotka esittelevät lisää dynaamisen ohjelmoinnin mahdollisuuksia.

9.1 Perustekniikat

Aloitamme dynaamiseen ohjelmointiin tutustumisen tehtävästä, jossa haluamme rakentaa palikoista tornin, jonka korkeus on n . Kunkin palikan korkeus on 1, 2 tai 3, ja jokaista palikkatyyppiä on saatavilla rajattomasti. Monellako tavalla voimme rakentaa tornin?

Esimerkiksi kun tornin korkeus on $n = 4$, voimme rakentaa sen 7 tavalla

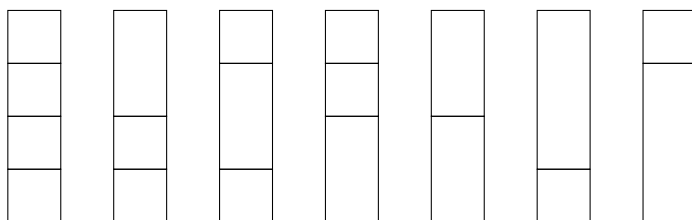


Figure 9.1: Voimme rakentaa korkeuden 4 tornin 7 tavalla palikoista, joiden korkeudet ovat 1, 2 ja 3.

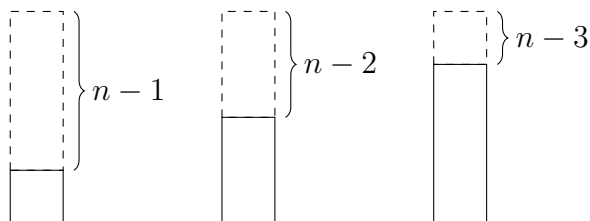


Figure 9.2: Rekursiivinen idea: kun alamme rakentaa tornia, voimme laittaa pohjalle korkeuden 1, 2 tai 3 palikan.

kuvan 9.1 mukaisesti. Jos n on pieni, voimme laskea tornien määrän helposti käymällä läpi kaikki tavat, mutta tornien määrä kasvaa nopeasti emmekä voi käyttää raakaa voimaa suuremmilla n :n arvoilla. Seuraavaksi ratkaisemmekin ongelman tehokkaasti dynaamisella ohjelmoinnilla.

9.1.1 Rekursiivinen esitys

Jotta voimme käyttää dynaamista ohjelmointia, ongelma täytyy pystyä esittämään *rekursiivisesti* niin, että saamme laskettua ongelman ratkaisun käyttäen osaongelmina pienempiä vastaavia ongelmia. Tässä tehtävässä luonteva rekursiivinen funktio on $\text{tornit}(n)$: monellako tavalla voimme rakentaa tornin, jonka korkeus on n ? Esimerkiksi $\text{tornit}(4) = 7$, koska voimme rakentaa korkeuden 4 tornin 7 tavalla.

Funktion pienten arvojen laskeminen on helppoa. Ensinnäkin $\text{tornit}(0) = 1$, koska on tarkalleen yksi tapa rakentaa tyhjä torni: siinä ei ole mitään palikoita. Sitten $\text{tornit}(1) = 1$, koska ainoa tapa rakentaa korkeuden 1 torni on valita palikka, jonka korkeus on 1, ja $\text{tornit}(2) = 2$, koska voimme rakentaa korkeuden 2 tornin valitsemalla joko kaksi palikkaa, jonka kummankin korkeus on 1, tai yhden palikan, jonka korkeus on 2.

Kuinka voisimme sitten laskea funktion arvon *yleisessä* tapauksessa, kun tornin korkeus on n ? Tässä voimme miettiä, kuinka tornin rakentaminen alkaa. Mahdollisuuksia on kolme: voimme laittaa ensin palikan, jonka korkeus on 1, 2 tai 3. Jos aloitamme korkeuden 1 palikalla, sen päälle täytyy rakentaa korkeuden $n - 1$ torni. Vastaavasti jos aloitamme korkeuden 2 tai 3 palikalla, sen päälle täytyy rakentaa torni, jonka korkeus on $n - 2$ tai $n - 3$. Kuva 9.2 havainnollistaa tämän idean. Niinpä voimme laskea tornien määrän rekursiivisesti kaavalla

$$\text{tornit}(n) = \text{tornit}(n - 1) + \text{tornit}(n - 2) + \text{tornit}(n - 3),$$

kun $n \geq 3$. Esimerkiksi voimme laskea

$$\text{tornit}(3) = \text{tornit}(2) + \text{tornit}(1) + \text{tornit}(0) = 4$$

korkeus n	<code>tornit(n)</code>
0	1
1	1
2	2
3	4
4	7
5	13
6	24
7	44
8	81
9	149

Table 9.1: Tornien määrät, kun korkeus n on $0, 1, \dots, 9$.

ja

$$\text{tornit}(4) = \text{tornit}(3) + \text{tornit}(2) + \text{tornit}(1) = 7,$$

jolloin olemme saaneet laskettua esimerkkitapaustamme vastaavasti, että voimme rakentaa korkeuden 4 tornin 7 tavalla.

Taulukko 9.1 näyttää funktion `tornit( $n$ )` arvot, kun $n = 0, 1, \dots, 9$. Kuten taulukosta voi huomata, funktion arvo kasvaa nopeasti: se lähes kaksinkertaistuu joka askeleella. Kun n on suuri, onkin valtavasti mahdollisuuksia tornin rakentamiseen.

9.1.2 Tehokas toteutus

Nyt kun olemme saaneet aikaan rekursiivisen funktion, voimme toteuttaa sen ohjelmoimalla seuraavasti:

```
function tornit(n)
  if (n == 0) return 1
  if (n == 1) return 1
  if (n == 2) return 2
  return tornit(n-1)+tornit(n-2)+tornit(n-3)
```

Tämä on toimiva ratkaisu, mutta siinä on yksi ongelma: funktion arvon laskeminen vie kauan aikaa, jos n on vähänkin suurempi. Käytännössä laskenta alkaa hidastua parametrin $n = 30$ tienoilla. Esimerkiksi arvon `tornit(40)` laskeminen vie aikaa noin minuutin ja arvon `tornit(50)` vie aikaa niin kauan, että emme jaksakaan odottaa laskennan valmistumista.

Syynä laskennan hitauteen on, että funktiota `tornit` kutsutaan uudestaan ja uudestaan samoilla parametreilla ja tornien määrä lasketaan lop-

pujen lopuksi summana luvuista 1 ja 2 pohjatapauksista. Niinpä kun tornien määrä on suuri, laskenta on tuomittu viemään kauan aikaa. Voimme kuitenkin tehostaa laskentaa toteuttamalla sen hieman toisella tavalla.

Tässä astuu kuvaan dynaamisen ohjelmoinnin keskeinen idea *taulukointi* (*memoization*): laskemme funktion arvon kullekin parametrille vain kerran ja tallennamme tulokset taulukkoon myöhempää käyttöä varten. Tätä varten luomme taulukon `tornit`, jossa kohtaan `tornit[i]` tallennetaan funktion arvo `tornit(i)`. Kun haluamme laskea korkeuden n tornien määrän, täytämme taulukon kohdat $0, 1, \dots, n$. Seuraava koodi toteuttaa laskennan:

```
tornit[0] = 1
tornit[1] = 1
tornit[2] = 2
for i = 3 to n
    tornit[i] = tornit[i-1]+tornit[i-2]+tornit[i-3]
```

Koodin suorituksen jälkeen taulukon arvo `tornit[n]` kertoo, monellako tavalla voimme rakentaa korkeuden n tornin.

Tämän toteutuksen etuna on, että se on huomattavasti nopeampi kuin rekursiivinen funktio. Koska koodissa on vain yksi `for`-silmukka, se vie aikaa vain $O(n)$, eli voimme käsitellä tehokkaasti myös suuria n :n arvoja. Esimerkiksi voimme nyt laskea salamannopeasti, että

$$\text{tornit}(50) = 10562230626642$$

eli on yli 10562 miljardia tapaa rakentaa korkeuden 50 torni.

9.2 Esimerkkejä

Olemme nyt tutustuneet dynaamisen ohjelmoinnin perusideaan, mutta tämä on vasta alkua sille, mitä kaikkea tekniikan avulla pystyy tekemään. Seuraavaksi käymme läpi kokoelman tehtäviä, jotka esittelevät lisää dynaamisen ohjelmoinnin mahdollisuuksia.

9.2.1 Pisin nouseva alijono

Ensimmäinen tehtävä on selvittää, kuinka pitkä on n alkioita sisältävän taulukon *pin nouseva alijono* (*longest increasing subsequence*) eli mahdollisimman pitkä vasemmalta oikealle etenevä jono alkioita, jossa seuraava alkio on aina edellistä suurempi. Kuvassa 9.3 on esimerkki taulukosta, jonka pisin nouseva alijono $[2, 5, 7, 8]$ on pituudeltaan 4.

0	1	2	3	4	5	6	7
6	2	5	1	7	4	8	3

Figure 9.3: Taulukon pisin nouseva alijono on $[2, 5, 7, 8]$.

Voimme lähestyä tehtävää laskemalla jokaiselle taulukon kohdalle $k = 0, 1, \dots, n - 1$ arvon $\text{pisin}(k)$: kuinka pitkä on pisin nouseva alijono, joka päättyy kohtaan k . Kun olemme laskeneet kaikki nämä arvot, suurin arvoista kertoo meille, kuinka pitkä on pisin nouseva alijono koko taulukossa. Esimerkiksi kuvan 9.3 taulukossa $\text{pisin}(6) = 4$, koska kohtaan 6 päättyvä pisin nouseva alijono on pituudeltaan 4.

Millainen on sitten pisin kohtaan k päättyvä alijono? Yksi mahdollisuus on, että alijonossa on vain kohdan k alkio, jolloin $\text{pisin}(k) = 1$. Muussa tapauksessa alijonossa on ensin kohtaan x päättyvä pisin nouseva alijono, missä $x < k$, ja sitten vielä kohdan k alkio. Tämä edellyttää, että kohdan x alkio on pienempi kuin kohdan k alkio. Tuloksena olevan alijonon pituus on $\text{pisin}(x) + 1$. Tämä antaa mahdollisuuden dynaamiseen ohjelmointiin: kun haluamme laskea arvon $\text{pisin}(k)$, käymme läpi kaikki mahdolliset tavat valita kohta x ja valitsemme niistä parhaan vaihtoehdon.

Seuraava koodi laskee jokaiselle $k = 0, 1, \dots, n - 1$ pisimmän kohtaan k päättyvän alijonon pituuden yllä kuvattua ideaa käyttäen. Koodi olettaa, että taulukon sisältö on taulukossa `taulu`, ja se muodostaa taulukon `pin`, jossa on pisimpien alijonojen pituudet.

```

for k = 0 to n-1
  pin[k] = 1
  for x = 0 to k-1
    if taulu[x] < taulu[k] and pin[x]+1 > pin[k]
      pin[k] = pin[x]+1

```

Koodin suorituksen jälkeen pisimmän nousevan alijonon pituus on siis suurin taulukon `pin` arvoista. Tämä algoritmi vie aikaa $O(n^2)$, koska käymme jokaisessa kohdassa k läpi kaikki taulukon edelliset kohdat.

Mitä jos haluaisimme selvittää pisimmän nousevan alijonon pituuden lisäksi, mistä alkioista se muodostuu? Tämä onnistuu laajentamalla hie-
man koodia. Rakennamme taulukon `aiempi`, joka kertoo jokaisessa kohdassa, missä on tähän kohtaan päättyvän pisimmän alijonon edellinen alkio. Voimme muodostaa taulukon seuraavasti:

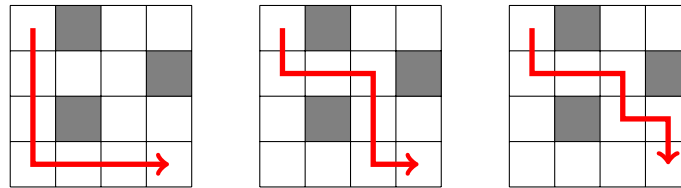


Figure 9.4: Mahdolliset reitit vasemmasta yläkulmasta oikeaan alakulmaan.

```

for k = 0 to n-1
  pisin[k] = 1
  aiempi[k] = -1
  for x = 0 to k-1
    if taulu[x] < taulu[k] and pisin[x]+1 > pisin[k]
      pisin[k] = pisin[x]+1
      aiempi[k] = x

```

Tämän jälkeen jokaisessa kohdassa k arvo $\text{aiempi}[k]$ kertoo pisimmän alijonon edellisen alkion kohdan. Kuitenkin jos alijonon pituus on 1, taulukossa on arvo -1 . Voimme nyt selvittää kohtaan k päättyvän alijonon alkioit käänteisesti seuraavasti:

```

while k != -1
  print(taulu[k])
  k = aiempi[k]

```

Voimme käyttää vastaavaa tekniikkaa dynaamisessa ohjelmoinnissa aina, kun haluamme selvittää, mistä aineksista paras ratkaisu muodostuu.

9.2.2 Reitti ruudukossa

Olemme $n \times n$ -ruudukon vasemmassa yläkulmassa ja haluamme päästä oikeaan alakulmaan. Jokaisella vuorolla voimme siirtyä askeleen alaspäin tai oikealle. Kuitenkin joissakin ruuduissa on este, emmekä voi kulkea sellaisen ruudun kautta. Montako mahdollista reittiä on olemassa? Esimerkiksi kuvassa 9.4 on 4×4 -ruudukko, jossa on kolme mahdollista reittiä ruudukon vasemmasta yläkulmasta oikeaan alakulmaan.

Tässä tehtävässä osaongelmat ovat *kaksiulotteisia*, koska olemme reitin joka vaiheessa tietyn rivin tietyssä sarakkeessa. Niinpä määrittelemme rekursiivisen funktion, jolla on kaksi parametria: $\text{reitit}(y, x)$ kertoo, montako reittiä on vasemmasta yläkulmasta ruutuun (y, x) . Numeroimme rivit ja sarakkeet $1, 2, \dots, n$ ja haluamme laskea arvon $\text{reitit}(n, n)$, joka on reittien

määrä vasemmasta yläkulmasta oikeaan alakulmaan.

Hyödyllinen havainto on, että jokaisessa ruudussa on kaksi mahdollisuutta, kuinka reitti voi tulla ruutuun, koska voimme tulla joko ylhäältä tai vasemmalta. Kun haluamme laskea reittien määrää, laskemmekin yhteen ylhäältä ja vasemmalta tulevat reitit. Rajoituksena jos ruudussa on este, siihen tulevien reittien määrä on aina nolla. Tämän perusteella saamme aikaan seuraavan dynaamisen ohjelmoinnin algoritmin:

```
for y = 1 to n
  for x = 1 to n
    if este[y][x]
      reitit[y][x] = 0
    else if y == 1 and x == 1
      reitit[y][x] = 1
    else
      reitit[y][x] = reitit[y-1][x] + reitit[y][x-1]
```

Koodi laskee jokaiseen ruutuun reittien määrän vasemmasta yläkulmasta kyseiseen ruutuun. Jos ruudussa on este, reittien määrä on 0, koska mitään reittiä ei ole olemassa. Jos ruutu on vasen yläkulma (eli $y = 1$ ja $x = 1$), reittien määrä on 1, koska reitti alkaa siitä ruudusta. Muuten reittien määrä saadaan laskemalla yhteen ylhäältä ja vasemmalta tulevat reitit. Tuloksena on algoritmi, joka vie aikaa $O(n^2)$.

Huomaa, että tässä käytämme taulukon kohtia $1, 2, \dots, n$ ja oletamme, että kohdissa 0 on arvona nolla. Tämä on kätevää, koska meidän ei tarvitse tehdä erikoistapauksia ruudukon yläreunaa ja vasenta reunaa varten.

9.2.3 Repunpakkaus

Termi *repunpakkaus* (*knapsack*) viittaa ongelmaan, jossa halutaan selvittää, millaisia yhdistelmiä voidaan muodostaa tavaroista, joilla on tietyt painot. Ongelmasta on monia muunnelmia, joiden yhdistävänä tekijänä on, että ne voi ratkaista tehokkaasti dynaamisella ohjelmoinnilla.

Seuraavaksi keskitymme ongelmaan, jossa meillä on n tavaraa, joilla on painot $p_1, p_2, \dots, p_n$. Haluamme selvittää kaikki mahdolliset yhteispainot, jotka voimme muodostaa valitsemalla jonkin osajoukon tavaroista. Esimerkiksi jos tavaroiden painot ovat $[1, 3, 3, 4]$, niin mahdolliset yhteispainot ovat

$$[0, 1, 3, 4, 5, 6, 7, 8, 10, 11].$$

Esimerkiksi yhteispaino 6 on listalla, koska saamme sen painoista $3 + 3 = 6$, ja yhteispaino 8 on listalla, koska saamme sen painoista $1 + 3 + 4 = 8$.

	0	1	2	3	4	5	6	7	8	9	10	11
$k = 0$	✓											
$k = 1$	✓	✓										
$k = 2$	✓	✓		✓	✓							
$k = 3$	✓	✓		✓	✓		✓	✓				
$k = 4$	✓	✓		✓	✓	✓	✓	✓	✓		✓	✓

Figure 9.5: Mahdolliset yhteispainot, kun painot ovat $[1, 3, 3, 4]$.

On helppoa laskea, mikä on suurin mahdollinen tavaroiden yhteispaino, koska saamme sen valitsemalla kaikki tavarat. Suurin yhteispaino on siis

$$s = p_1 + p_2 + \dots + p_n.$$

Tämä on meille hyödyllinen yläraja, koska tiedämme nyt, että tavaroiden yhteispaino on aina jokin luku välillä $0 \dots s$.

Voimme lähestyä tehtävää dynaamisella ohjelmoinnilla luomalla funktion `painot(k)`: mitkä kaikki yhteispainot voimme muodostaa, jos käytössämme on tavarat $1, 2, \dots, k$? Oletamme, että funktio palauttaa *taulukon*, jossa on $s + 1$ alkiota: jokaiselle yhteispainolle $0, 1, \dots, s$ tieto siitä, voimmeko muodostaa sen painoista $p_1, p_2, \dots, p_k$. Tapaus `painot(0)` on helppo, koska kun ei ole mitään tavaroita, ainoa mahdollinen yhteispaino on 0. Tämän jälkeen pystymme laskemaan tapauksen `painot(k)` ottamalla lähtökohdaksi tapauksen `painot(k - 1)` ja selvittämällä, mitä uusia yhteispainoja voimme muodostaa, kun saamme käyttää myös painoa p_k .

Kuva 9.5 näyttää dynaamisen ohjelmoinnin taulukoiden sisällön esimerkissämme, jossa painot ovat $[1, 3, 3, 4]$. Ensimmäisellä rivillä $k = 0$, joten ainoa yhteispaino on 0. Toisella rivillä $k = 1$, joten saamme käyttää painoa $p_1 = 1$ ja voimme muodostaa yhteispainot 0 ja 1. Kolmannella rivillä $k = 2$, jolloin saamme käyttöömmme painon $p_2 = 3$ ja voimme muodostaa yhteispainot 0, 1, 3 ja 4. Viimeisellä rivillä käytössämme ovat kaikki painot, joten se vastaa ongelman ratkaisua.

Voimme toteuttaa dynaamisen ohjelmoinnin kätevästi niin, että koodissa on vain yksi boolean-tila `painot`, jossa on $s + 1$ alkiota. Taulukko kertoo laskennan jokaisessa vaiheessa, mitkä yhteispainot ovat mahdollisia sillä hetkellä. Aluksi taulukon kohdassa 0 on arvo `true` ja kaikissa muissa kohdissa on arvo `false`. Tämän jälkeen päivitämme taulukkoa lisäämällä mukaan painoja yksi kerrallaan.

```

painot[0] = true
for i = 1 to n
  for j = s to 0
    if painot[j]
      painot[j+p[i]] = true

```

Tärkeä yksityiskohta algoritmissa on, että käymme jokaisen painon kohdalla taulukon läpi *lopusta alkuun*. Syynä tähän on, että tällä tavalla saamme laskettua oikealla tavalla, mitä uusia yhteispainoja voidaan muodostaa, kun saamme käyttää uutta painoa kerran. Laskennan jälkeen voimme tulostaa kaikki mahdolliset yhteispainot näin:

```

for i = 0 to s
  if painot[i]
    print(i)

```

Tuloksena olevan algoritmin aikavaativuus on $O(ns)$. Algoritmin tehokkuus riippuu siis paitsi tavaroiden määrästä, myös niiden painoista. Jotta algoritmi on käyttökelpoinen, painojen summan s täytyy olla niin pieni, että voimme varata niin suuren taulukon.

9.2.4 Binomikertoimet

Binomikerroin $\binom{n}{k}$ ilmaisee, monellako tavalla voimme muodostaa n alkion joukosta k alkion osajoukon. Esimerkiksi $\binom{5}{3} = 10$, koska voimme muodostaa joukosta $\{1, 2, 3, 4, 5\}$ seuraavat 3 alkion osajoukot:

- $\{1, 2, 3\}$ • $\{1, 3, 5\}$ • $\{2, 4, 5\}$
- $\{1, 2, 4\}$ • $\{1, 4, 5\}$ • $\{3, 4, 5\}$
- $\{1, 2, 5\}$ • $\{2, 3, 4\}$
- $\{1, 3, 4\}$ • $\{2, 3, 5\}$

Binomikertoimien laskemiseen on monia tapoja. Dynaamisen ohjelmoinnin kannalta kiinnostava tapa on rekursiivinen kaava

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}.$$

Voimme perustella kaavan tarkastelemalla k alkion osajoukon muodostamista joukosta $\{1, 2, \dots, n\}$. Jos otamme osajoukkoon mukaan alkion n , tämän

jälkeen tulee muodostaa vielä $k - 1$ alkion osajoukko joukosta $\{1, 2, \dots, n - 1\}$. Jos taas emme ota osajoukkoon mukaan alkia n , tämän jälkeen tulee muodostaa k alkion osajoukko joukosta $\{1, 2, \dots, n - 1\}$.

Lisäksi pohjatapauksina

$$\binom{n}{0} = 1,$$

koska voimme muodostaa tyhjän osajoukon yhdellä tavalla, ja

$$\binom{n}{k} = 0, \text{ jos } k > n,$$

koska n alkista ei voi muodostaa osajoukkoa, jossa on yli n alkia.

Tämä rekursiivinen kaava tarjoaa meille tavan laskea tehokkaasti binomikertoimia dynaamisen ohjelmoinnin avulla. Voimme toteuttaa laskennan seuraavasti:

```
binom[0][0] = 1
for i = 1 to n
  binom[i][0] = 1
  for j = 1 to k
    binom[i][j] = binom[i-1][j-1] + binom[i-1][j]
```

Koodin suorituksen jälkeen taulukon kohdassa `binom[n][k]` on binomikerroin $\binom{n}{k}$. Algoritmi vie aikaa $O(nk)$, joten sitä voi käyttää melko suurten binomikertoimien laskemiseen.

Chapter 10

Verkkojen perusteet

Voimme ratkaista monia algoritmiikan ongelmia esittämällä tilanteen *verkkona* (*graph*) ja käyttämällä sitten sopivaa verkkoalgoritmia. Tyypillinen esimerkki verkosta on tieverkosto, joka muodostuu kaupungeista ja niiden välisistä teistä. Tällaisessa verkossa ongelmana voi olla selvittää vaikkapa, kuinka voimme matkustaa kaupungista a kaupunkiin b .

Tässä luvussa aloitamme verkkoihin tutustumisen käymällä läpi verkkojen käsitteitä sekä tapoja esittää verkkoja ohjelmoinnissa. Tämän jälkeen näemme, miten voimme tutkia verkkojen rakennetta ja ominaisuuksia syvyyshaun ja leveyshaun avulla. Seuraavissa kirjan luvuissa jatkamme verkkojen käsittelyä ja opimme lisää verkkoalgoritmeja.

10.1 Verkkojen käsitteitä

Verkko muodostuu *solmuista* (*node* tai *vertex*) ja niiden välisistä *kaarista* (*edge*). Kuvassa 10.1 on verkko, jossa on viisi solmua ja seitsemän kaarta. Merkitsemme verkon solmujen määrää kirjaimella n ja kaarten määrää kirjaimella m . Lisäksi numeroimme verkon solmut kokonaisluvuin $1, 2, \dots, n$.

Kaksi solmua ovat *vierekkäin* (*adjacent*), jos niiden välillä on kaari. Solmun *naapureja* (*neighbor*) ovat kaikki solmut, joihin se on yhteydessä kaarella, ja solmun *aste* (*degree*) on sen naapureiden määrä. Kuvassa 10.1 solmun 2

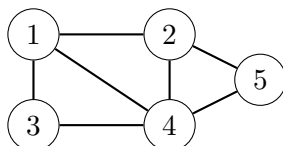


Figure 10.1: Verkko, jossa on viisi solmua ja seitsemän kaarta.

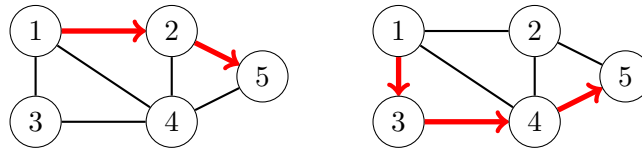
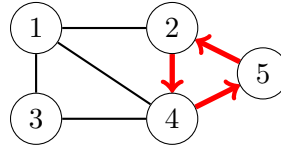


Figure 10.2: Kaksi polkua solmusta 1 solmuun 5.

Figure 10.3: Verkossa on sykli $2 \rightarrow 4 \rightarrow 5 \rightarrow 2$.

naapurit ovat 1, 4 ja 5, joten solmun aste on 3.

Polku ja sykli

Verkossa oleva *polku* (*path*) on kaaria pitkin kulkeva reitti lähtösolmusta kohdesolmuun. Kuva 10.2 näyttää kaksi mahdollista polkua solmusta 1 solmuun 5. Polut ovat $1 \rightarrow 2 \rightarrow 5$ ja $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$.

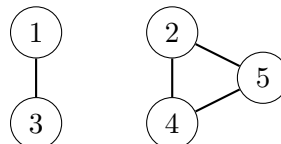
Polku on *sykli* (*cycle*), jos sen alku- ja loppusolmu on sama, siinä on ainakin yksi kaari eikä se kulje kahta kertaa saman solmun tai kaaren kautta. Kuvassa 10.3 on esimerkkinä sykli $2 \rightarrow 4 \rightarrow 5 \rightarrow 2$. Jos verkossa ei ole yhtään sykliä, verkko on *sykkitön* (*acyclic*).

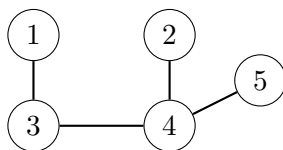
Yhtenäisyys ja komponentit

Verkko on *yhtenäinen* (*connected*), jos minkä tahansa kahden solmun välillä on polku. Kuvan 10.1 verkko on yhtenäinen, mutta kuvan 10.4 verkko ei ole yhtenäinen, koska esimerkiksi solmujen 1 ja 2 välillä ei ole polkua.

Verkko voidaan esittää aina kokoelmana yhtenäisiä *komponentteja* (*component*). Kuvassa 10.4 yhtenäiset komponentit ovat $\{1, 3\}$ ja $\{2, 4, 5\}$.

Verkko on *puu* (*tree*), jos se on sekä yhtenäinen että sykkitön. Puussa kaarten määrä on aina yhden pienempi kuin solmujen määrä, ja jokaisen kah-

Figure 10.4: Verkon yhtenäiset komponentit ovat $\{1, 3\}$ ja $\{2, 4, 5\}$.

Figure 10.5: *Puu eli yhtenäinen, syklitön verkko.*

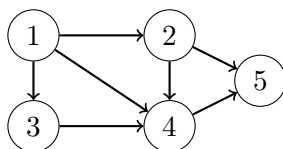
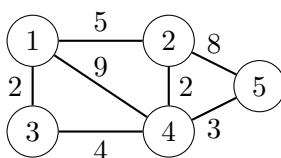
den solmun välillä on yksikäsitteinen polku. Kuvassa 10.5 on esimerkkinä puu, jossa on viisi solmua ja neljä kaarta.

Suunnattu verkko

Kun verkko on *suunnattu* (*directed*), jokaisella kaarella on tietty suunta, jonka mukaisesti kaarta pitkin tulee kulkea. Suunnat rajoittavat siis verkossa liikkumista. Kuvassa 10.6 on esimerkkinä suunnattu verkko, jossa voimme kulkea solmusta 1 solmuun 5 polkua $1 \rightarrow 2 \rightarrow 5$, mutta emme voi kulkea mitenkään solmusta 5 solmuun 1.

Painotettu verkko

Kun verkko on *painotettu* (*weighted*), jokaiseen kaareen liittyy jokin paino, joka kuvaa tyypillisesti kaaren pituutta. Kun kuljemme polkua painotetussa verkossa, polun pituus on kaarten painojen summa. Kuvassa 10.7 on esimerkkinä painotettu verkko, jossa polun $1 \rightarrow 2 \rightarrow 5$ pituus on $5 + 8 = 13$ ja polun $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ pituus on $2 + 4 + 3 = 9$.

Figure 10.6: *Suunnattu verkko.*Figure 10.7: *Painotettu verkko.*

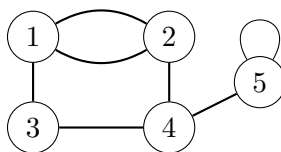


Figure 10.8: Verkko, joka ei ole yksinkertainen.

Yksinkertainen verkko

Usein oletuksena on, että verkko on *yksinkertainen* (*simple*), jolloin siinä ei ole kahta samanlaista kaarta ja jokainen kaari yhdistää kaksi eri solmua. Kaikki tähän mennessä kirjassa esitetyt verkot ovat olleet yksinkertaisia.

Kuvassa 10.8 on esimerkkinä verkko, joka ei ole yksinkertainen. Tähän on kaksi syytä: solmujen 1 ja 2 välillä on kaksi kaarta ja lisäksi solmusta 5 on kaari itseensä.

10.2 Verkot ohjelmoinnissa

Verkon esittämiseen ohjelmoinnissa on monia mahdollisuuksia. Sopivan esitystavan valintaan vaikuttaa, miten haluamme käsitellä verkkoa algoritmista, koska jokaisessa esitystavassa on omat etunsa. Seuraavaksi käymme läpi kolme tavallista esitystapaa.

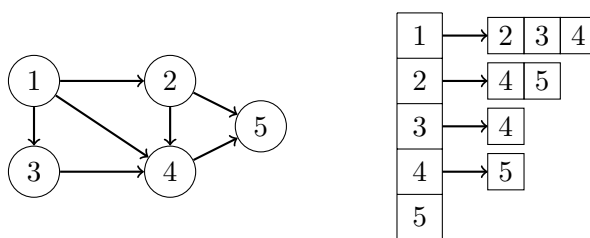


Figure 10.9: Verkon vieruslistaesitys.

10.2.1 Vieruslistaesitys

Tavallisin tapa esittää verkko on luoda kullekin solmulle *vieruslista* (*adjacency list*), joka kertoo, mihin solmuihin voimme siirtyä solmusta kaaria pitkin. Kuvassa 10.9 on esimerkkinä verkko ja sitä vastaava vieruslistaesitys. Voimme luoda vieruslistat näin ohjelmoinnissa:


```
verkko[1].add(2)
verkko[1].add(3)
verkko[1].add(4)
verkko[2].add(4)
verkko[2].add(5)
verkko[3].add(4)
verkko[4].add(5)
```

Vieruslistaesitys on monessa tilanteessa hyvä tapa tallentaa verkko, koska haluamme usein selvittää, mihin solmuihin pääsemme siirtymään tietystä solmusta kaaria pitkin. Esimerkiksi seuraava koodi käy läpi solmut, joihin voimme siirtyä solmusta *a* kaarella:

```
for b in verkko[a]
    // käsittele solmu b
```

Suuntaamaton verkko voidaan esittää vieruslistoina niin, että jokainen kaari tallennetaan molempiin suuntiin. Jos taas verkko on painotettu, voimme tallentaa jokaisesta kaaresta sekä kohdesolmun että painon.

10.2.2 Kaarilistaesitys

Toinen tapa tallentaa verkko on luoda *kaarilista* (*edge list*), joka sisältää kaikki verkon kaaret. Voisimme luoda esimerkkiverkon kaarilistan näin:

```
kaaret.add((1,2))
kaaret.add((1,3))
kaaret.add((1,4))
kaaret.add((2,4))
kaaret.add((2,5))
kaaret.add((3,4))
kaaret.add((4,5))
```

Tässä tapauksessa jokainen listan alkio on pari, jossa on kaaren alku- ja loppusolmu. Vastaavasti voisimme tallentaa myös kaaret kolmikkoina, joissa on lisäksi mukana kaaren paino.

Kaarilista on hyvä esitystapa algoritmeissa, joissa tulee pystyä käymään helposti läpi kaikki verkon kaaret eikä ole tarvetta selvittää tietystä solmusta lähteviä kaaria.

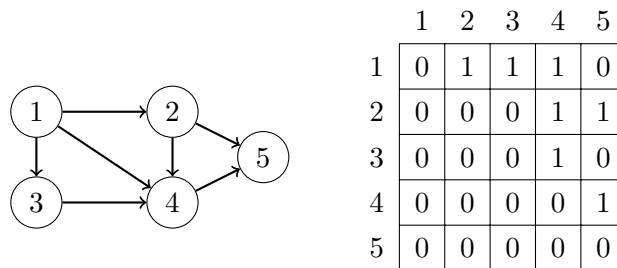


Figure 10.10: Verkon vierusmatriisiesitys.

10.2.3 Vierusmatriisiesitys

Vierusmatriisi (*adjacency matrix*) on kaksiulotteinen rakenne, joka kertoo jokaisesta verkon kaaresta, esiintyykö se verkossa. Matriisin rivin a sarakkeessa b oleva arvo ilmaisee, onko verkossa kaarta solmusta a solmuun b . Kuvassa 10.10 on esimerkki verkon vierusmatriisiesityksestä. Tässä tapauksessa matriisin jokainen arvo on 0 (ei kaarta) tai 1 (kaari).

Ohjelmoinnissa voimme tallentaa vierusmatriisin kaksiulotteisena taulukkona tai listana. Tässä on esimerkkiverkkoa vastaava vierusmatriisi:

```

verkko[1][2] = 1
verkko[1][3] = 1
verkko[1][4] = 1
verkko[2][4] = 1
verkko[2][5] = 1
verkko[3][4] = 1
verkko[4][5] = 1

```

Jos verkko on painotettu, kaarten painot voidaan vastaavasti merkitä vierusmatriisiin. Vierusmatriisin etuna on, että siitä voi tarkastaa helposti, onko tietty kaari verkossa. Esitystapa kuluttaa kuitenkin paljon muistia, eikä sitä voi käyttää, jos verkon solmujen määrä on suuri.

10.3 Verkon läpikäynti

Tutustumme seuraavaksi kahteen keskeiseen verkkoalgoritmiin, jotka käyvät läpi verkossa olevia solmuja ja kaaria. Ensin käsittelemme syvyyshakua, joka on yleiskäyttöinen algoritmi verkon läpikäyntiin, ja sen jälkeen leveyshakua, jonka avulla voimme löytää lyhimpiä polkuja verkossa.

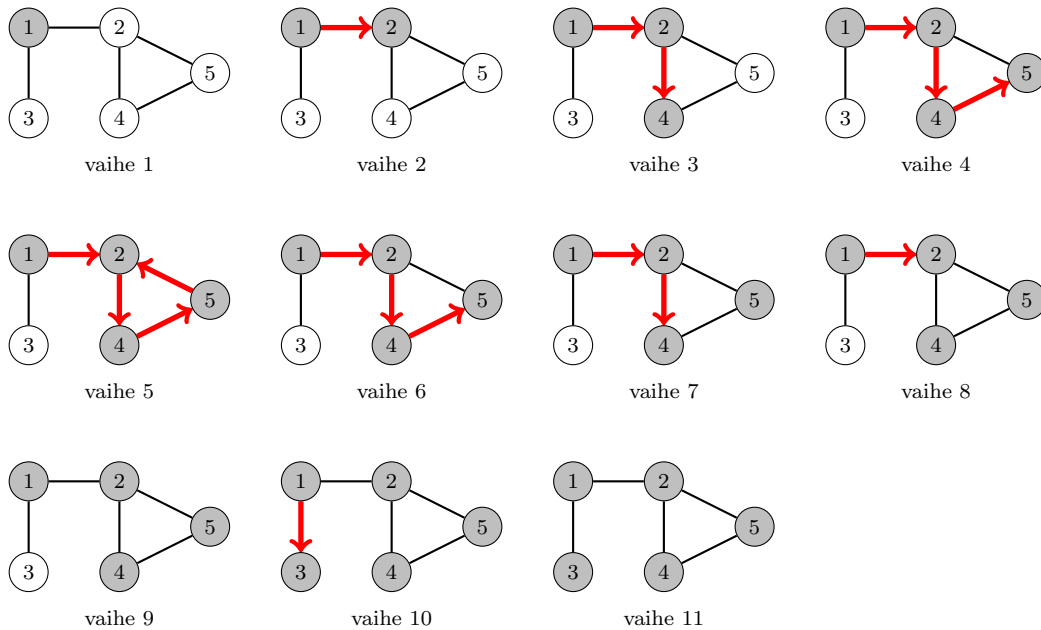


Figure 10.11: Esimerkki syvyysshaun toiminnasta.

10.3.1 Syvyyshaku

Syvyyshaku (*depth-first search* eli *DFS*) on verkkojen käsittelyn yleistyökalu, jonka avulla voimme selvittää monia asioita verkon rakenteesta. Kun alamme tutkia verkkoa syvyysshaulla, tulee päättää ensin, mistä solmusta haku lähtee liikkeelle. Haku etenee vuorollaan kaikkiin solmuihin, jotka ovat saavutettavissa lähtösolmusta kulkemalla kaaria pitkin.

Syvyyshaku pitää yllä jokaisesta verkon solmusta tietoa, onko solmussa jo vierailtu. Kun haku saapuu solmuun, jossa se ei ole vierailut aiemmin, se merkitsee solmun vierailuksi ja alkaa käydä läpi solmusta lähteviä kaaria. Jokaisen kaaren kohdalla haku etenee verkon niihin osiin, joihin pääsee kaaren kautta. Lopulta kun haku on käynyt läpi kaikki kaaret, se perääntyy taaksepäin samaa reittiä kuin tuli solmuun.

Kuvassa 10.11 on esimerkki syvyysshaun toiminnasta. Jokaisessa vaiheessa harmaat solmut ovat solmuja, joissa haku on jo vierailut. Tässä esimerkissä haku lähtee liikkeelle solmusta 1, josta pääsee kaarella solmuihin 2 ja 3. Haku etenee ensin solmuun 2, josta pääsee edelleen solmuihin 4 ja 5. Tämän jälkeen haku ei enää löydä uusia solmuja tästä verkon osasta, joten se perääntyy takaisin kulkemaansa reittiä solmuun 1. Lopuksi haku käy vielä solmussa 3, josta ei pääse muihin uusiin solmuihin. Nyt haku on käynyt läpi kaikki solmut, joihin pääsee solmusta 1.

Syvyyshaku on mukavaa toteuttaa rekursiivisesti seuraavaan tapaan:

```
procedure haku(solmu)
  if vierailtu[solmu]
    return
  vierailtu[solmu] = true
  for naapuri in verkko[solmu]
    haku(naapuri)
```

Haku käynnistyy, kun kutsumme proseduuria `haku` parametrina haun lähtösolmu. Jokaisessa kutsussa proseduuuri tarkistaa ensin, onko se jo käynyt parametrina annetussa solmussa, ja päättyy heti tässä tilanteessa. Muuten proseduuuri merkitsee, että solmussa on nyt käyty, ja etenee rekursiivisesti kaikkiin solmun naapureihin. Haku vie aikaa $O(n+m)$, koska jokainen solmu ja kaari käsitellään enintään kerran.

Mihin voisimme sitten käyttää syvyyshakua? Seuraavassa on joitakin esimerkkejä syvyysshaun käyttökohteista:

Polun etsiminen

Syvyysshaun avulla voimme etsiä verkosta polun solmusta a solmuun b , jos tällainen polku on olemassa. Tämä tapahtuu aloittamalla haku solmusta a ja pysähtymällä, kun vastaan tulee solmu b . Jos polkuja on useita, syvyyshaku löytää jonkin niistä riippuen solmujen käsittelyjärjestyksestä.

Yhtenäisyys ja komponentit

Suuntaamaton verkko on yhtenäinen, jos kaikki solmut ovat yhteydessä toisiinsa. Voimme tarkastaa verkon yhtenäisyyden aloittamalla syvyysshaun jostakin solmusta ja tutkimalla, saavuttaako haku kaikki verkon solmut. Lisäksi voimme löytää verkon yhtenäiset komponentit käymällä läpi solmut ja aloittamalla uuden syvyysshaun aina, kun vastaan tulee vierailematon solmu. Jokainen syvyyshaku muodostaa yhden komponentin.

Syklin etsiminen

Jos suuntaamaton verkko sisältää syklin, huomaamme tämän syvyysshaun aikana siitä, että tulemme toista kautta johonkin solmuun, jossa olemme käyneet jo aiemmin. Niinpä löydämme syvyysshaun avulla jonkin verkossa olevan syklin, jos sellainen on olemassa.

Toinen tapa tutkia syklin olemassaolo on laskea jokaisesta verkon komponentista solmujen ja kaarten määrä. Jos komponentissa on x solmua ja

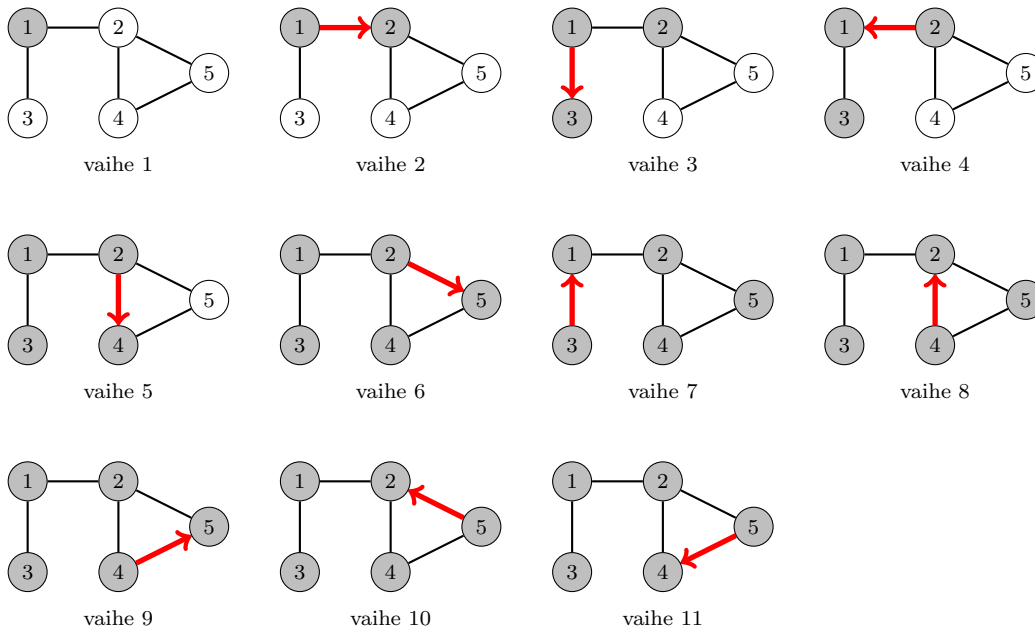


Figure 10.12: Esimerkki leveyshaun toiminnasta.

siinä ei ole syklä, siinä tulee olla tarkalleen $x - 1$ kaarta eli komponentin tulee olla puu. Jos kaaria on enemmän, komponentissa on varmasti sykli.

10.3.2 Leveyshaku

Leveyshaku (*breadth-first search* eli *BFS*) käy syvyysshaun tavoin läpi kaikki verkon solmut, joihin pääsee kaaria pitkin annetusta lähtösolmusta. Erona on kuitenkin, missä järjestyksessä solmut käydään läpi. Leveyshaku käy solmuja läpi *kerroksittain* niin, että se käsittelee solmut siinä järjestyksessä kuin ne ovat tulleet ensimmäistä kertaa vastaan haun aikana.

Vaikka voisimme käyttää leveyshakua yleisenä algoritmina verkon läpikäyntiin syvyysshaun tavoin, käytämme sitä tavallisesti silloin, kun olemme kiinnostuneita verkon *lyhimmistä poluista*. Leveyshaun avulla pystymme nimittäin määrittämään lyhimmän polun pituuden eli *etäisyyden* lähtösolmusta kuhunkin haun aikana kohtaamaamme solmuun. Tässä oletamme, että polun pituus tarkoittaa sen kaarten määrää eli lyhin polku on mahdollisimman vähän kaaria sisältävä polku.

Kuvassa 10.12 on esimerkki leveyshaun toiminnasta, kun aloitamme haun solmusta 1 lähtien. Käsittelemme ensin solmun 1, josta pääsemme uusiin solmuihin 2 ja 3. Tämä tarkoittaa, että lyhimmat polut solmuihin 2 ja 3 ovat $1 \rightarrow 2$ ja $1 \rightarrow 3$ eli etäisyys näihin solmuihin on 1. Sitten käsittelemme

solmun 2, josta pääsemme uusiin solmuihin 4 ja 5. Tämä tarkoittaa, että lyhimmät polut solmuihin 4 ja 5 ovat $1 \rightarrow 2 \rightarrow 4$ ja $1 \rightarrow 2 \rightarrow 5$ eli etäisyys näihin solmuihin on 2. Lopuksi käsittelemme vielä solmut 3, 4 ja 5, joista emme kuitenkaan pääse enää uusiin solmuihin.

Tavallinen tapa toteuttaa leveyshaku on käyttää *jonoa*, jossa on käsiteltäviä odottavia solmuja. Jonon ansiosta pystymme käymään läpi solmut siinä järjestyksessä kuin olemme löytäneet ne leveyshaun aikana. Oletamme, että jonossa on metodi `enqueue`, joka lisää alkion jonon loppuun, sekä metodi `dequeue`, joka hakee ja poistaa jonon ensimmäisen alkion. Seuraava koodi suorittaa leveyshaun lähtösolmusta `alku` alkaen:

```
jono.enqueue(alku)
vierailtu[alku] = true
etaisyys[alku] = 0
while not jono.empty()
    solmu = jono.dequeue()
    for naapuri in verkko[solmu]
        if vierailtu[naapuri]
            continue
        jono.enqueue(naapuri)
        vierailtu[naapuri] = true
        etaisyys[naapuri] = etaisyys[solmu]+1
```

Lisäämme ensin jonoon lähtösolmun ja merkitsemme, että olemme vierailleet siinä ja että etäisyys siihen on 0. Tämän jälkeen alamme käsitellä solmuja siinä järjestyksessä kuin ne ovat jonossa. Käsittelemme solmun käymällä läpi sen naapurit. Jos emme ole aiemmin käyneet naapurissa, lisäämme sen jonoon ja päivitämme taulukoita. Haku vie aikaa $O(n + m)$, koska käsittelemme jokaisen solmun ja kaaren enintään kerran.

10.3.3 Esimerkki: Labyrintti

Olemme labyrintissa ja haluamme päästä ruudusta *A* ruutuun *B*. Joka vuorolla voimme siirtyä yhden askeleen ylöspäin, alaspäin, vasemmalle tai oikealle. Miten löydämme reitin? Esimerkiksi kuvassa 10.13 voimme kulkea ruudusta *A* ruutuun *B* reittiä, joka muodostuu 9 askeleesta.

Voimme esittää ongelman verkkona niin, että jokainen lattiaruutu on yksi verkon solmuista ja kahden solmun välillä on kaari, jos vastaavat ruudut ovat vierekkäin labyrintissa. Kuva 10.14 näyttää esimerkkilabyrinttimme verkkona. Tätä esitystapaa käyttäen ruudusta *A* on reitti ruutuun *B* tarkalleen silloin, kun vastaavat verkon solmut kuuluvat samaan yhtenäiseen komponenttiin. Voimme käytännössä etsiä reitin syvyyshaulla tai leveyshaulla.

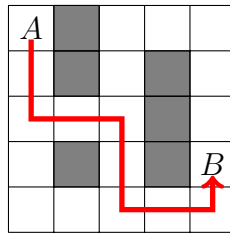


Figure 10.13: Reitti labyrintissa ruudusta A ruutuun B .

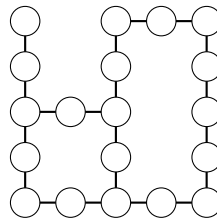


Figure 10.14: Labyrintin esittäminen verkkona.

Tässä tapauksessa labyrinttia ei tarvitse erikseen muuttaa verkoksi, vaan voimme toteuttaa haut *implisiittiseen* verkkoon eli teemme haun labyrinttiin sen omassa esitysmuodossa. Käytännössä labyrintti on kätevää tallentaa kaksiulotteisena taulukkona, joka kertoo, mitkä ruudut ovat seinäruutuja.

Esimerkiksi seuraava koodi toteuttaa syvyysshaun labyrintissa:

```

procedure haku(y,x)
  if y < 0 or x < 0 or y >= n or x >= n
    return
  if seinä[y][x] or vierailtu[y][x]
    return
  vierailtu[y][x] = true
  haku(y+1,x)
  haku(y-1,x)
  haku(y,x+1)
  haku(y,x-1)

```

Syvyysshaku löytää *jonkin* reitin ruudusta A ruutuun B , mutta ei välttämättä lyhintä reittiä. Jos haluamme löytää lyhimmän reitin, meidän tulee käyttää vastaavasti leveyshakua.

Chapter 11

Lyhimmät polut

Monissa verkkoihin liittyvissä ongelmissa on kysymys siitä, että haluamme löytää *lyhimmän polun* (*shortest path*) verkon solmusta toiseen. Esimerkiksi voimme haluta selvittää, mikä on nopein reitti kahden katuosoitteen välillä tai mikä on halvin tapa lentää kaupungista toiseen. Näissä ja muissa sovelluksissa on tärkeää, että löydämme lyhimmän polun tehokkaasti.

Olemme käyttäneet aiemmin leveyshakua lyhimpien polkujen etsimiseen. Tämä onkin hyvä ratkaisu, kun haluamme löytää polut, joiden kaarten määrä on pienin. Tässä luvussa keskitymme kuitenkin vaikeampaan tilanteeseen, jossa verkko on *painotettu* ja haluamme löytää polut, joissa painojen summa on pienin. Tällöin emme voi enää käyttää leveyshakua vaan tarvitsemme kehittyneempiä menetelmiä.

Lyhimpien polkujen etsimiseen painotetussa verkossa on monia algoritmeja, joilla on erilaisia ominaisuuksia. Tässä luvussa käymme läpi ensin Bellmanin ja Fordin algoritmin ja Dijkstran algoritmin, jotka etsivät lyhimmät polut annetusta lähtösolmusta kaikkiin verkon solmuihin. Tämän jälkeen tutustumme Floydin ja Warshallin algoritmiin, joka etsii samanaikaisesti lyhimmät polut kaikkien verkon solmujen välillä.

11.1 Lyhimmät polut lähtösolmusta

Tavallisin tilanne käytännön verkko-ongelmissa on, että haluamme löytää lyhimmän polun solmusta a solmuun b . Yksittäisen lyhimmän polun etsiminen vaatii usein kuitenkin, että etsimme sitä ennen muitakin lyhimpiä polkuja. Niinpä keskitymme alusta asti yleisempään ongelmaan, jossa olemme valinneet jonkin solmun lähtösolmuksi ja haluamme määrittää *jokaiselle* verkon solmulle, kuinka pitkä on lyhin polku lähtösolmusta solmuun eli mikä on solmun etäisyys lähtösolmusta.

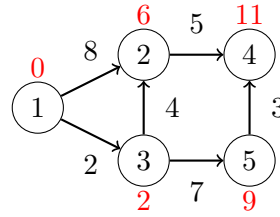


Figure 11.1: Lyhimpien polkujen pituudet solmusta 1 alkaen.

Kuvassa 11.1 on esimerkkinä verkko, jossa lähtösolmuna on solmu 1 ja jokaisen solmun viereen on merkitty sen etäisyys. Esimerkiksi solmun 5 etäisyys on 9, koska lyhin polku solmusta 1 solmuun 5 on $1 \rightarrow 3 \rightarrow 5$, jonka pituus on $2 + 7 = 9$. Käytämme tätä verkkoa esimerkkinä, kun tutustumme seuraavaksi kahteen algoritmiin lyhimpien polkujen etsimiseen.

11.1.1 Bellmanin ja Fordin algoritmi

Bellmanin ja Fordin algoritmi etsii lyhimmat polut annetusta lähtösolmusta kaikkiin verkon solmuihin. Algoritmi muodostaa taulukon, joka kertoo jokaiselle verkon solmulle sen etäisyyden lähtösolmusta. Algoritmi toimii missä tahansa verkossa, kunhan verkossa ei ole negatiivista sykliä eli sykliä, jonka painojen summa on negatiivinen.

Bellmanin ja Fordin algoritmi pitää yllä *arvioita* solmujen etäisyyksistä niin, että aluksi etäisyys lähtösolmuun on 0 ja etäisyys kaikkiin muihin solmuihin on ääretön. Tämän jälkeen algoritmi alkaa parantaa etäisyyksiä etsimällä verkosta kaaria, joiden kautta kulkeminen lyhentää polkuja. Jokaisella askeleella algoritmi etsii kaaren $a \rightarrow b$, jonka avulla pääsemme solmuun b aiempaa lyhempää polkua solmun a kautta. Algoritmi merkitsee solmun b uudeksi etäisyysarvioksi tämän polun pituuden. Kun mitään etäisyysarviota ei voi enää parantaa, algoritmi päättyy ja kaikki etäisyydet vastaavat todellisia lyhimpien polkujen pituuksia.

Kuva 11.2 näyttää esimerkin Bellmanin ja Fordin algoritmin toiminnasta, kun lähtösolmuna on solmu 1. Jokaisen solmun vieressä on ilmoitettu sen etäisyysarvio: aluksi etäisyys solmuun 1 on 0 ja etäisyys kaikkiin muihin solmuihin on ääretön. Jokainen etäisyyden muutos näkyy kuvassa omana vaiheenaan. Ensin parannamme etäisyyttä solmuun 2 kulkemalla kaarta $1 \rightarrow 2$, jolloin etäisyydeksi tulee 8. Sitten parannamme etäisyyttä solmuun 3 kulkemalla kaarta $1 \rightarrow 3$, jolloin solmun uudeksi etäisyydeksi tulee 2. Jatkamme samalla tavalla, kunnes emme voi enää parantaa mitään etäisyyttä ja kaikki etäisyydet vastaavat lyhimpien polkujen pituuksia.

Bellmanin ja Fordin algoritmi on mukavaa toteuttaa käyttäen verkon

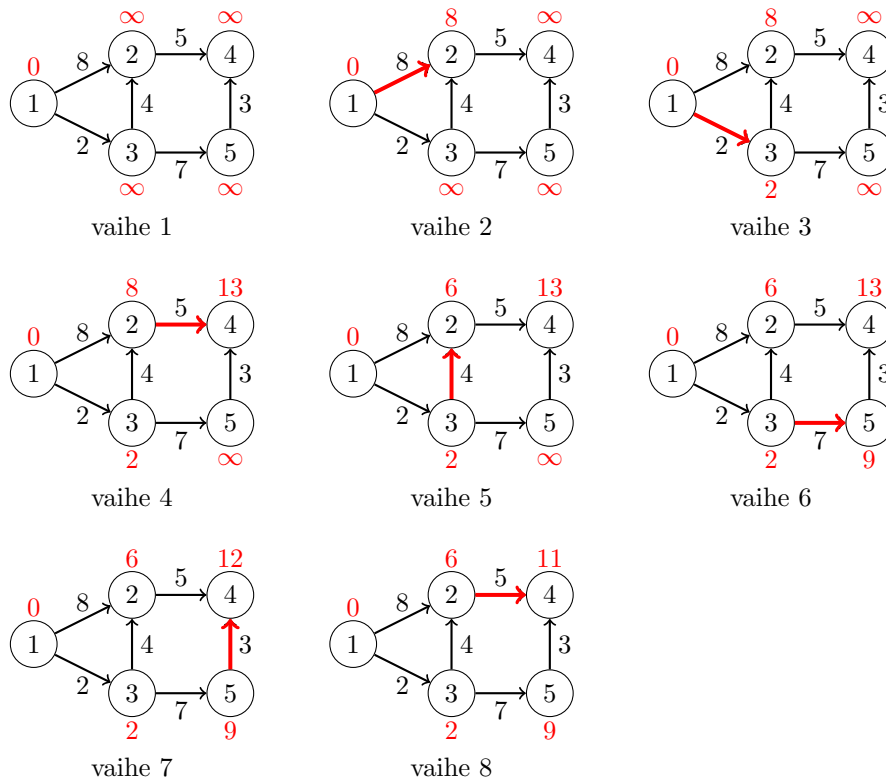


Figure 11.2: Esimerkki Bellmanin ja Fordin algoritmin toiminnasta.

kaarilistaesitystä, jossa jokaisesta kaaresta on tallennettu alku- ja loppu-solmu sekä paino. Toteutamme algoritmin niin, että se muodostuu *kierroksista*, joista jokainen käy läpi kaikki verkon kaaret ja koettaa parantaa etäisyysarvioita niiden avulla. Voimme toteuttaa algoritmin näin:

```
while true
    muutos = false
    for kaari in kaaret
        nyky = etaisyys[kaari.loppu]
        uusi = etaisyys[kaari.alku]+kaari.paino
        if uusi < nyky
            etaisyys[kaari.loppu] = uusi
            muutos = true
    if not muutos
        break
```

Algoritmi käy jokaisella kierroksella läpi verkon kaaret ja tutkii kunkin kaaren kohdalla, mikä on nykyinen etäisyys kaaren kohdesolmuun sekä mikä on

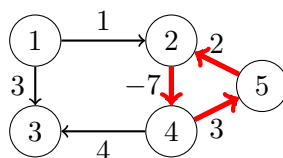


Figure 11.3: Negatiivinen sykli $2 \rightarrow 4 \rightarrow 5 \rightarrow 2$, jonka avulla voimme lyhentää polkuja loputtomasti.

uusi etäisyys, jos kuljemmekin solmuun kaaren kautta. Jos uusi etäisyys on pienempi, päivitämme sen solmun etäisyysarvioksi. Muuttujassa `muutos` on tieto siitä, onko jokin etäisyys muuttunut kierroksen aikana, ja algoritmi päättyy, kun mikään etäisyys ei muuttunut.

Algoritmin analyysi

Olemme nyt kuvailleet ja toteuttaneet Bellmanin ja Fordin algoritmin, mutta miten voimme olla varmoja, että se löytää lyhimät polut, ja miten nopeasti se toimii? Jotta voimme vastata näihin kysymyksiin, tarvitsemme kaksi havaintoa koskien verkon lyhimpiä polkuja.

Ensimmäinen havainto on, että jos $s_1 \rightarrow s_2 \rightarrow \dots \rightarrow s_k$ on lyhin polku solmusta s_1 solmuun s_k , niin myös $s_1 \rightarrow s_2$ on lyhin polku solmusta s_1 solmuun s_2 , $s_1 \rightarrow s_2 \rightarrow s_3$ on lyhin polku solmusta s_1 solmuun s_3 , jne., eli jokainen lyhimmän polun alkuosa on myös lyhin polku vastaavaan solmuun. Jos näin ei olisi, voisimme parantaa lyhintä polkua solmusta s_1 solmuun s_k parantamalla jotain polun alkuosaa, mikä aiheuttaisi ristiriidan.

Toinen havainto on, että n solmun verkossa jokainen lyhin polku voi sisältää enintään $n - 1$ kaarta, kun oletamme, että verkossa ei ole negatiivista sykliä. Jos polkuun kuuluisi n kaarta tai enemmän, jokin solmu esiintyisi polulla monta kertaa. Tämä ei ole kuitenkaan mahdollista, koska ei olisi järkeä kulkea monta kertaa saman solmun kautta, kun haluamme saada aikaan lyhimmän polun.

Tarkastellaan nyt, mitä tapahtuu algoritmin kierroksissa. Ensimmäisen kierroksen jälkeen olemme löytäneet lyhimät polut, joissa on enintään yksi kaari. Toisen kierroksen jälkeen olemme löytäneet lyhimät polut, joissa on enintään kaksi kaarta. Sama jatkuu, kunnes $n - 1$ kierroksen jälkeen olemme löytäneet lyhimät polut, joissa on enintään $n - 1$ kaarta. Koska missään lyhimässä polussa ei voi olla enempää kaaria, algoritmi päättyy ja olemme löytäneet kaikki lyhimät polut. Algoritmin suorittaa siis enintään $n - 1$ kierrosta, joista jokainen käy läpi kaikki verkon kaaret ajassa $O(m)$. Niinpä algoritmi löytää kaikki lyhimät polut ajassa $O(nm)$.

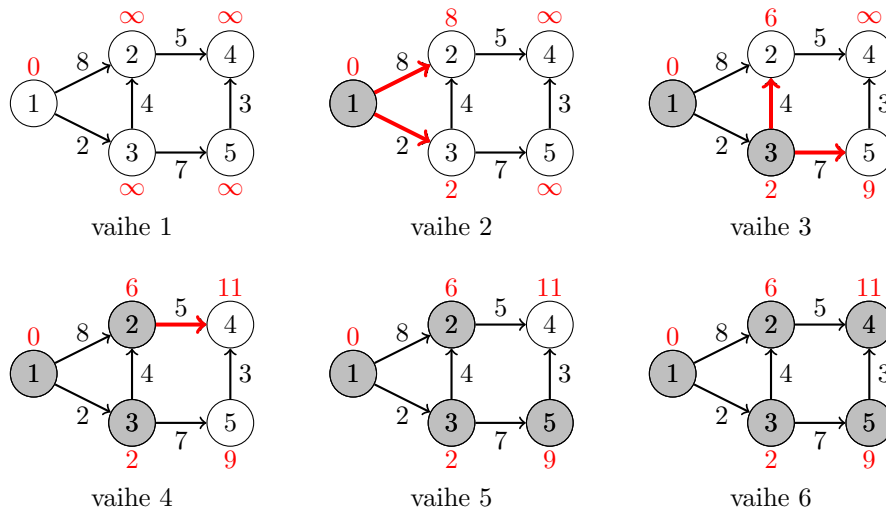


Figure 11.4: Esimerkki Dijkstran algoritmin toiminnasta.

Mitä tapahtuu sitten, jos verkossa on negatiivinen sykli? Esimerkiksi kuvan 11.3 verkossa on negatiivinen sykli $2 \rightarrow 4 \rightarrow 5 \rightarrow 2$, jonka paino on -2 . Tässä tilanteessa Bellmanin ja Fordin algoritmi jatkuu ikuisesti, koska syklin kautta kulkevia polkuja voi lyhentää loputtomasti. Oikeastaan ongelma on siinä, että lyhin polku ei ole mielekäs käsite, jos polun osana on negatiivinen sykli. Voimme kuitenkin havaita verkossa olevan negatiivisen syklin Bellmanin ja Fordin algoritmin avulla: verkossa on negatiivinen sykli tarkalleen silloin, kun jotain etäisyyttä voi parantaa vielä $n - 1$ kierroksen jälkeen.

11.1.2 Dijkstran algoritmi

Dijkstran algoritmi on Bellmanin ja Fordin algoritmin tehostettu versio, jonka toiminta perustuu oletukseen, että verkossa ei ole negatiivisen painoisia kaaria. Bellmanin ja Fordin algoritmin tapaan Dijkstran algoritmi pitää yllä arvioita etäisyyksistä lähtösolmusta muihin solmuihin. Erona on kuitenkin tapa, miten Dijkstran algoritmi parantaa etäisyyksiä.

Dijkstran algoritmissa verkon solmut kuuluvat kahteen luokkaan: käsittelemättömiin ja käsiteltyihin. Aluksi kaikki solmut ovat käsittelemättömiä. Algoritmi etsii joka askeleella käsittelemättömän solmun, jonka etäisyysarvio on pienin. Sitten algoritmi käy läpi kaikki solmusta lähtevät kaaret ja koettaa parantaa etäisyyksiä niiden avulla. Tämän jälkeen solmu on käsitelty eikä sen etäisyys enää muutu, eli aina kun olemme käsitelleet solmun, olemme saaneet selville sen lopullisen etäisyyden.

Kuva 11.4 näyttää esimerkin Dijkstran algoritmin toiminnasta. Solmun harmaa väri tarkoittaa, että se on käsitelty. Aluksi valitsemme käsittelyyn solmun 1, koska sen etäisyys 0 on pienin. Sitten jäljellä ovat solmut 2, 3, 4 ja 5, joista valitsemme käsittelyyn solmun 3, jonka etäisyys 2 on pienin. Tämän jälkeen valitsemme käsittelyyn solmun 2, jonka etäisyys on 6. Sama jatkuu, kunnes olemme käsitelleet kaikki verkon solmut.

Dijkstran algoritmi etsii joka askeleella verkosta käsittelemättömän solmun, jonka etäisyysarvio on pienin. Koska haluamme saada algoritmista tehokkaan, solmut täytyy pystyä löytämään nopeasti. Tavallinen tapa toteuttaa Dijkstran algoritmi on käyttää *kekoa*, jonka avulla pienimmän etäisyyden solmu löytyy logaritmisessa ajassa. Tallennamme kekoon pareja, joissa on solmun etäisyys ja tunnus ja jotka järjestetään etäisyyden mukaan pienimmästä suurimpaan. Aluksi keossa on vain lähtösolmua vastaava solmu, jonka etäisyys on 0. Tämän jälkeen haemme joka askeleella keosta solmun, jonka etäisyys on pienin. Jos solmu on jo käsitelty, emme tee mitään. Muuten käymme läpi kaikki solmusta lähtevät kaaret ja tarkastamme, voimmeko parantaa etäisyyksiä niiden avulla. Aina kun voimme parantaa etäisyyttä, lisäämme uuden etäisyyden kekoon.

Dijkstran algoritmi on mukavaa toteuttaa käyttäen verkon vieruslistaesitystä. Voimme toteuttaa algoritmin seuraavasti:

```
keko.push((0,alku))
while not keko.empty()
    solmu = keko.pop().tunnus
    if kasitelty[solmu]
        continue
    kasitelty[solmu] = true
    for kaari in verkko[solmu]
        nyky = etaisyys[kaari.loppu]
        uusi = etaisyys[solmu]+kaari.paino
        if uusi < nyky
            etaisyys[kaari.loppu] = uusi
            keko.push((uusi,kaari.loppu))
```

Tässä merkintä `keko.pop().tunnus` tarkoittaa, että keosta poistetaan ylin alkio, josta erotetaan parin toinen osa eli solmun tunnus.

Huomaa, että keossa voi olla samaan aikaan *useita* etäisyyksiä samalle solmulle, koska lisäämme kekoon uuden solmun aina etäisyyden parantuessa. Käsittelemme kuitenkin jokaisen solmun vain kerran, koska aina kun olemme hakeneet uuden solmun keosta käsittelyä varten, varmistamme ensin, että emme ole käsitelleet sitä aiemmin.

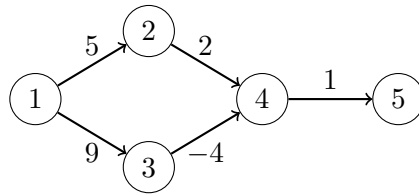


Figure 11.5: Dijkstran algoritmi ei toimi oikein negatiivisen kaaren takia.

Algoritmin analyysi

Dijkstran algoritmi on ahne algoritmi, koska se etsii joka vaiheessa vielä käsittelemättömän solmun, jonka etäisyys on pienin, minkä jälkeen kyseisen solmun etäisyys ei enää muutu. Miten voimme olla varmoja, että olemme löytäneet tässä vaiheessa oikean etäisyyden?

Voimme ajatella asiaa siltä kannalta, että jos etäisyyttä olisi mahdollista parantaa, niin verkossa olisi oltava jokin toinen vielä käsittelemätön solmu, jonka kautta voisimme muodostaa lyhemmän polun. Kuitenkin tiedämme, että kaikkien muiden tarjolla olevien solmujen etäisyydet ovat suurempia tai yhtä suuria eivätkä etäisyydet voi lyhentyä, koska verkossa ei ole negatiivisia kaaria. Tästä syystä voimme turvallisesti valita käsittelyyn pienimmän etäisyyden solmun ja kiinnittää sen etäisyyden.

Dijkstran algoritmi toimii siis oikein, jos verkossa ei ole negatiivisia kaaria, mutta kuinka nopeasti algoritmi toimii? Ensinnäkin algoritmi käy läpi verkon solmut ja kaaret, missä kuluu aikaa $O(n + m)$. Lisäksi algoritmista on joukko kekoon liittyviä operaatioita, jotka vaikuttavat tehokkuuteen. Pahimmassa tapauksessa lisäämme jokaisen kaaren kohdalla kekoon uuden alkion, eli lisäykset kekoon vievät aikaa $O(m \log m)$. Toisaalta poistamme kaikki alkiot aikanaan keosta, mihin menee myös aikaa $O(m \log m)$. Algoritmin kokonaisaikaavaativuus on siis $O(n + m \log m)$.

Voimme vielä hieman siistiä aikaavaativuutta, kun oletamme, ettei verkossa ole kahta kaarta, joiden alkuperä- ja loppusolmu ovat samat. Tällöin $m \leq n^2$, joten $\log m \leq \log(n^2) = 2 \log n$ ja saamme algoritmin aikaavaativuudeksi $O(n + m \log n)$.

Entä mitä tapahtuu, jos verkossa kuitenkin on negatiivinen kaari? Tällöin Dijkstran algoritmi ei toimi välttämättä oikein. Kuva 11.5 näyttää esimerkin tällaisesta tilanteesta. Algoritmi seuraa ahneesti ylemmää polkua ja toteaa, että pienin etäisyys solmusta 1 solmuun 5 on 8. Kuitenkin parempi tapa olisi kulkea alemmaa polkua, jolloin polun pituus on vain 6.

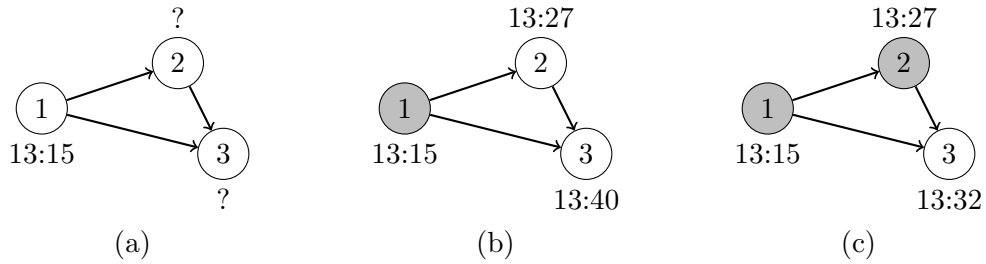


Figure 11.6: Nopeimman reitin etsiminen paikasta 1 paikkaan 3. (a) Matka alkaa kello 13:15. (b) Käymme läpi paikasta 2 lähtevät yhteydet. (c) Käymme läpi paikasta 3 lähtevät yhteydet.

11.1.3 Esimerkki: Reittiopas

Nyt meillä on tarvittavat tiedot, joiden avulla voimme luoda *reittioppaan* eli järjestelmän, jonka avulla voi etsiä eri kulkuvälineitä käyttäviä reittejä kaupungissa. Voimme mallintaa kaupungin verkkona, jonka solmut ovat kaupungin paikkoja ja kaaret ovat mahdollisia yhteyksiä paikkojen välillä. Reittioppaan tulisi ilmoittaa *nopein* reitti paikasta a paikkaan b .

Reittioppaan toteuttamiseen liittyy yksi lisähaaste: kulkuvälineillä on tietyt aikataulut, jotka rajoittavat niiden käyttöä. Esimerkiksi bussilinjan lähtöjä saattaa olla 10 minuutin välein. Meidän tulee siis ottaa huomioon reitin etsimisessä, mihin aikaan saavumme mihinkin paikkaan. Voimme toteuttaa tämän tallentamalla verkon kaaret muodossa ”matka alkaa ajanhetkenä x ja päättyy ajanhetkenä y ”.

Koska kaikissa matkoissa kuluu positiivinen määrä aikaa, voimme etsiä reittejä Dijkstran algoritmin avulla. Toteutamme algoritmin niin, että määritämme jokaiseen paikkaan *varhaisimman* ajan, jolloin voimme päästä kyseiseen paikkaan. Alussa tiedämme, että olemme lähtöpaikassa matkamme alkuhetkenä. Sitten kun otamme käsittelyyn uuden paikan, käymme läpi siitä lähteviä yhteyksiä, joihin ehdimme optimaalista aikataulua noudattaen. Saamme tehostettua hakua merkittävästi ottamalla huomioon jokaisesta linjasta vain ensimmäisen lähdön, johon ehdimme. Tämä on perusteltua, koska ei voi missään tilanteessa olla hyvä idea odottaa myöhempään lähtöön.

Kuva 11.6 näyttää esimerkkitalanteen, jossa haluamme kulkea paikasta 1 paikkaan 3 ja lähdemme matkaan kello 13:15. Paikasta 1 lähtee 10 minuutin välein yhteys paikkaan 2 (kesto 7 minuuttia) sekä 30 minuutin välein yhteys paikkaan 3 (kesto 10 minuuttia). Niinpä pääsemme paikkaan 2 kello 13:27 ja paikkaan 3 kello 13:40. Sitten paikasta 2 lähtee 5 minuutin välein yhteys paikkaan 3 (kesto 2 minuuttia). Tämän avulla pääsemme paikkaan 3 kello 13:32, eli nopein reitti kulkee paikan 2 kautta.

Käytännössä hakua voisi vielä tehostaa monella tavalla. Esimerkiksi kun löydämme jonkin reitin kohdepaikkaan, voimme siitä lähtien hylätä kaikki paikat, joihin ehdimme myöhemmin kuin kohdepaikkaan.

11.2 Kaikki lyhimvät polut

Tarkastellaan seuraavaksi ongelmaa, jossa haluamme etsiä lyhimvät polut verkon *kaikista* solmuista *kaikkiin* solmuihin. Yksi tapa ratkaista tehtävä olisi suorittaa Bellmanin ja Fordin tai Dijkstran algoritmi jokaisesta verkon solmusta alkaen. Voimme kuitenkin ratkaista tehtävän suoremmin etsimällä kaikki polut *samanaikaisesti* Floydin ja Warshallin algoritmilla.

11.2.1 Floydin ja Warshallin algoritmi

Floydin ja Warshallin algoritmi muodostaa $n \times n$ -kokoisen *etäisyysmatriisin* (*distance matrix*), jossa rivin a sarakkeessa b on lyhimmän polun pituus solmusta a solmuun b . Algoritmi alustaa ensin matriisin niin, että siihen on merkitty vain etäisyydet, jotka toteutuvat kulkemalla yksittäistä kaarta, ja kaikissa muissa matriisin kohdissa etäisyys on ääretön. Sitten algoritmi suorittaa n kierrosta, jotka on numeroitu $1, 2, \dots, n$. Kierroksella k algoritmi etsii polkuja, joissa on välisolmuna solmu k sekä mahdollisesti muita välisolmuja joukosta $1, 2, \dots, k-1$. Jos tällainen polku parantaa etäisyyttä, algoritmi päivittää uuden etäisyyden matriisiin. Lopulta jokainen solmu on ollut välisolmuna poluilla, jolloin on saatu selville kaikki lyhimvät polut.

Kuva 11.7 näyttää esimerkin Floydin ja Warshallin algoritmin toiminnasta. Kierroksella 1 etsimme polkuja, joissa solmu 1 on välisolmuna. Tällaisia polkuja ei ole, koska solmuun 1 ei pääse mistään solmusta, joten matriisi ei muutu. Kierroksella 2 huomaamme, että voimme kulkea solmun 2 kautta solmusta 1 solmuun 4, jolloin saamme etäisyyden 8. Samoin voimme kulkea solmun 2 kautta solmusta 3 solmuun 4, jolloin saamme etäisyyden 5. Jatkamme vastaavasti, kunnes kierroksen 4 jälkeen olemme saaneet selville kaikki etäisyydet ja etäisyysmatriisi on lopullinen.

Floydin ja Warshallin algoritmin mukavana puolena on, että se on helppo toteuttaa: riittää luoda kolme sisäkkäistä for-silmukkaa, jotka suorittavat matriisin päivitykset. Seuraavassa koodissa pääsilmukan muuttuja k kertoo, mikä kierros on kyseessä eli mikä solmu on välisolmu. Kaksi muuta silmukkaa käyvät läpi kaikki solmuparit (i, j) ja koettavat parantaa niiden etäisyyksiä kulkemalla solmun k kautta.

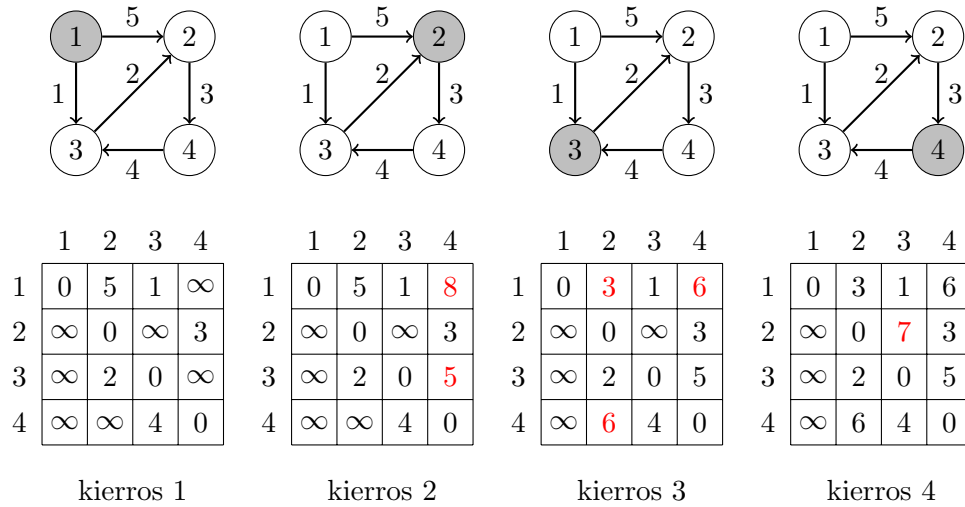


Figure 11.7: Esimerkki Floydin ja Warshallin algoritmin toiminnasta.

```

for k = 1 to n
  for i = 1 to n
    for j = 1 to n
      etaisyys[i][j] = min(etaisyys[i][j],
                           etaisyys[i][k] + etaisyys[k][j])

```

Algoritmin aikavaativuus on $O(n^3)$, koska se muodostuu kolmesta sisäkkäisestä for-silmukasta.

Algoritmin analyysi

Miksi Floydin ja Warshallin algoritmi toimii? Voimme ymmärtää algoritmia tarkastelemalla sen toimintaa ”käänteisesti” rekursiivisesti. Kun verkossa on lyhin polku solmusta a solmuun b , millainen tämä polku voi olla?

Yksi mahdollisuus on, että polku on vain kaari solmusta a solmuun b . Tällöin sen pituus on merkitty etäisyysmatriisiin algoritmin alussa. Muussa tapauksessa polussa on yksi tai useampi välisolmu. Oletetaan, että x on välisolmu, jonka tunnus on suurin. Saamme nyt kaksi osatehtävää: meidän tulee ensin kulkea solmusta a solmuun x ja sen jälkeen solmusta x solmuun b niin, että kummallakin polulla jokaisen välisolmun tunnus on pienempi kuin x . Voimme käsitellä nämä osatehtävät rekursiivisesti.

Floydin ja Warshallin algoritmi muodostaa joka vaiheessa polkuja, joissa voi olla välisolmuina solmuja $1, 2, \dots, i$. Kun haluamme muodostaa lyhimmän polun solmusta a solmuun b , meillä on kaksi vaihtoehtoa: Jos solmu i on

algoritmi	aikavaativuus	erityistä
leveyshaku	$O(n + m)$	ei salli painoja kaarissa
Bellmanin ja Fordin algoritmi	$O(nm)$	
Dijkstran algoritmi	$O(n + m \log n)$	ei salli negatiivisia kaaria
Floydin ja Warshallin algoritmi	$O(n^3)$	etsii kaikki polut

Table 11.1: Algoritmit lyhimpien polkujen etsimiseen.

välisolmuna, yhdistämme lyhimmän polun solmusta a solmuun i ja solmusta i solmuun b . Jos taas solmu i ei ole välisolmuna, olemme käsitelleet polun jo aiemmin. Algoritmin päätteeksi välisolmuina voi olla solmuja $1, 2, \dots, n$, eli mikä tahansa verkon solmu voi olla välisolmu.

11.2.2 Algoritmien vertailua

Olemme nyt käyneet läpi monia algoritmeja lyhimpien polkujen etsintään ja voimme alkaa muodostaa yleiskuvaa aiheesta. Taulukko 11.1 näyttää yhteenvedon algoritmien tehokkuudesta ja ominaisuuksista.

Käytännössä leveyshaku ja Dijkstran algoritmi ovat yleisimmin tarvittavat algoritmit: jos kaarilla ei ole painoja, käytämme leveyshakua, ja muuten Dijkstran algoritmia. Dijkstran algoritmin rajoituksena on, että verkossa ei saa olla negatiivisia kaaria, mutta tällä rajoituksella ei ole yleensä merkitystä käytännön ongelmissa, koska kaarten painot eivät useimmiten voi olla negatiivisia. Esimerkiksi tien pituus tai lennon hinta ei voi yleensä olla negatiivinen. Jos kuitenkin verkossa voi olla negatiivisia kaaria, voimme turvautua Bellmanin ja Fordin algoritmiin.

Miten sitten Floydin ja Warshallin algoritmi vertautuu muihin algoritmeihin? Tämä riippuu siitä, onko verkko *harva* (*sparse*) vai *tiheä* (*dense*). Harvassa verkossa on vähän kaaria ja $m \approx n$, kun taas tiheässä verkossa on paljon kaaria ja $m \approx n^2$. Floydin ja Warshallin algoritmi on parhaimmillaan silloin, kun verkko on tiheä, koska sen aikavaativuus ei riipu kaarten määrästä. Esimerkiksi jos etsimme kaikki lyhimmät polut suorittamalla n kertaa Dijkstran algoritmin, harvassa verkossa aikaa kuluu $O(n^2 \log n)$, mutta tiheässä verkossa aikaa kuluu $O(n^3 \log n)$. Siis harvassa verkossa aikavaativuus on parempi kuin Floydin ja Warshallin algoritmissa, mutta tiheässä verkossa se on huonompi. Toisaalta Floydin ja Warshallin algoritmin vakiokertoimet ovat hyvin pienet sen yksinkertaisen rakenteen ansiosta, minkä ansiosta algoritmi voi toimia käytännössä yllättävänkin nopeasti.

Chapter 12

Suunnatut syklittömät verkot

Verkkoalgoritmien suunnittelussa aiheuttavat usein vaikeuksia verkossa olevat syklit, ja monen ongelman ratkaiseminen on vaikeaa nimenomaan sen takia, että täytyy ottaa huomioon, mitä tapahtuu sykleissä. Tässä luvussa katsomme, miten asiat muuttuvat, kun voimmekin olettaa, että käsiteltävänä on suunnattu verkko, jossa *ei ole* syklejä.

Kun verkko on suunnattu ja syklitön, voimme muodostaa sille aina *topologisen järjestyksen*, joka antaa luontevan järjestyksen käsitellä verkon solmut niiden riippuvuuksien mukaisesti. Tämän ansiosta voimme myös hyödyntää dynaamista ohjelmointia verkon polkujen käsittelyssä. Itse asiassa tulemme huomaamaan, että *mikä tahansa* dynaamisen ohjelmoinnin algoritmi voidaan nähdä suunnatun syklittömän verkon käsittelynä.

Entä jos verkossa kuitenkin on syklejä? Osoittautuu, että voimme silti esittää sen syvärakenteen syklittömänä verkkona muodostamalla verkon *vahvasti yhtenäiset komponentit*. Tämän ansiosta voimme tietyissä tilanteissa käsitellä verkkoa mukavasti syklittömän verkon tavoin, vaikka se ei olisikaan alun perin syklitön.

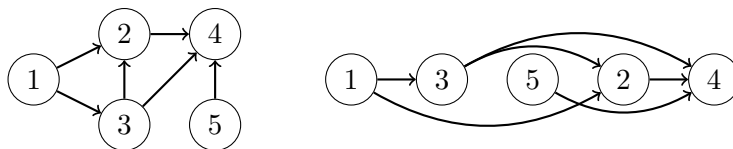


Figure 12.1: Verkko ja yksi sen topologinen järjestys $[1, 3, 5, 2, 4]$.

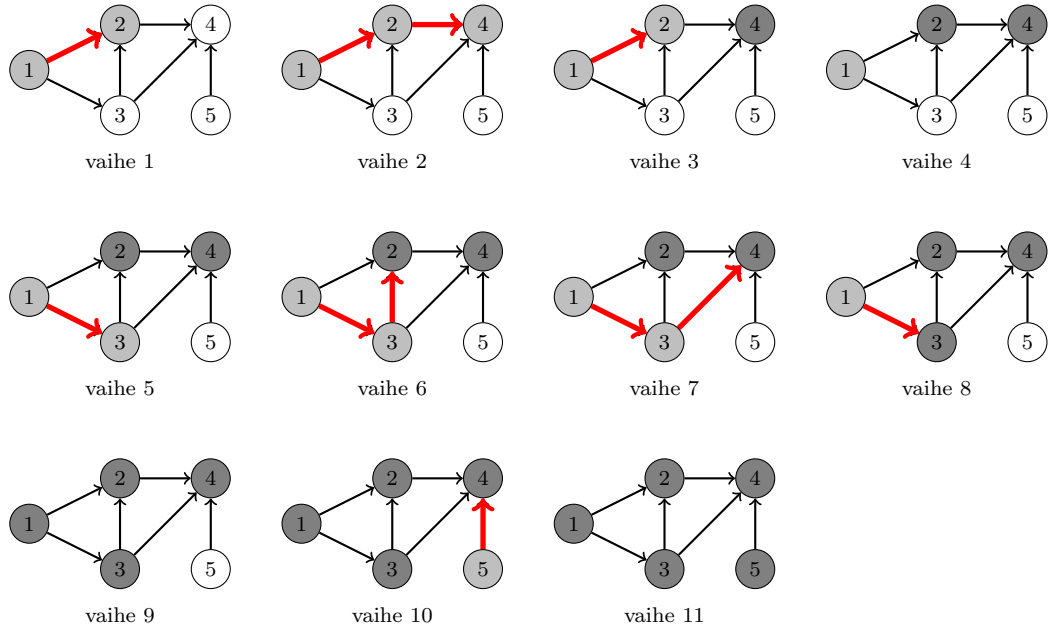


Figure 12.2: Esimerkki topologisen järjestyksen muodostamisesta.

12.1 Topologinen järjestys

Topologinen järjestys (*topological sort*) on suunnatun verkon solmujen järjestys, jossa pätee, että jos solmusta a on kaari solmuun b , niin solmu a on ennen solmua b järjestyksessä. Topologinen järjestys voidaan esittää listana, joka ilmaisee solmujen järjestyksen. Kuvassa 12.1 on esimerkkinä verkko ja yksi sen topologinen järjestys $[1, 3, 5, 2, 4]$.

Osoittautuu, että voimme muodostaa suunnatulle verkolle topologisen järjestyksen tarkalleen silloin, kun verkko on syklitön (*directed acyclic graph* eli *DAG*). Tutustumme seuraavaksi tehokkaaseen algoritmiin, joka muodostaa topologisen järjestyksen tai toteaa, ettei järjestystä voi muodostaa verkossa olevan syklin takia.

12.1.1 Järjestyksen muodostaminen

Voimme muodostaa topologisen järjestyksen suorittamalla joukon syvyyshakuja, joissa jokaisella solmulla on kolme mahdollista tilaa:

- tila 0 (valkoinen): solmussa ei ole käyty
- tila 1 (harmaa): solmun käsittely on kesken
- tila 2 (musta): solmun käsittely on valmis

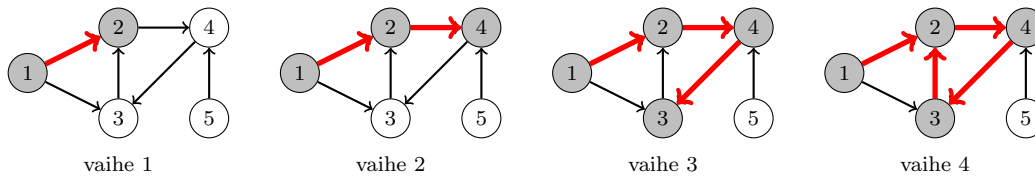


Figure 12.3: Topologista järjestystä ei voi muodostaa syklin takia.

Algoritmin alussa jokainen solmu on valkoinen. Käymme läpi kaikki verkon solmut ja aloitamme aina syvyysshaun solmusta, jos se on valkoinen. Aina kun saavumme uuteen solmuun, sen väri muuttuu valkoisesta harmaaksi. Sitten kun olemme käsitelleet kaikki solmusta lähtevät kaaret, solmun väri muuttuu harmaasta mustaksi ja lisäämme solmun listalle. Tämä lista käänteisessä järjestyksessä on verkon topologinen järjestys. Kuitenkin jos saavumme jossain vaiheessa toista kautta harmaaseen solmuun, verkossa on sykli eikä topologista järjestystä ole olemassa.

Kuva 12.2 näyttää, kuinka algoritmi muodostaa topologisen järjestyksen esimerkiverkossa. Tässä tapauksessa suoritamme kaksi syvyyshakua, joista ensimmäinen alkaa solmusta 1 ja toinen alkaa solmusta 5. Algoritmin tuloksena on lista $[4, 2, 3, 1, 5]$, joten käänteinen lista $[5, 1, 3, 2, 4]$ on verkon topologinen järjestys. Huomaa, että tämä on eri järjestys kuin kuvassa 12.1 – topologinen järjestys ei ole yksikäsitteinen ja voimme yleensä muodostaa järjestyksen monella tavalla. Kuva 12.3 näyttää puolestaan tilanteen, jossa topologista järjestystä ei voida muodostaa verkossa olevan syklin takia. Tässä verkossa on sykli $2 \rightarrow 4 \rightarrow 3 \rightarrow 2$, jonka olemassaolon huomaamme siitä, että tulemme uudestaan harmaaseen solmuun 2.

Algoritmin analyysi

Miksi algoritmi toimii? Tarkastellaan ensin tilannetta, jossa verkossa on sykli. Jos algoritmi saapuu uudestaan harmaaseen solmuun, on selvää, että verkossa on sykli, koska algoritmi on onnistunut pääsemään harmaasta solmusta itseensä kulkemalla jotain polkua verkossa. Toisaalta jos verkossa on sykli, algoritmi tulee jossain vaiheessa ensimmäistä kertaa johonkin syklin solmuun x . Tämän jälkeen se käy läpi solmusta lähtevät kaaret ja aikanaan käy varmasti läpi kaikki syklin solmut ja saapuu uudestaan solmuun x . Niinpä algoritmi tunnistaa kaikissa tilanteissa oikein verkossa olevan syklin.

Jos sitten verkossa ei ole sykliä, algoritmi lisää jokaisen solmun listalle sen jälkeen, kun se on käsitellyt kaikki solmusta lähtevät kaaret. Jos siis verkossa on kaari $a \rightarrow b$, solmu b lisätään listalle ennen solmua a . Lopuksi lista käännetään, jolloin solmu a tulee ennen solmua b . Tämän ansiosta

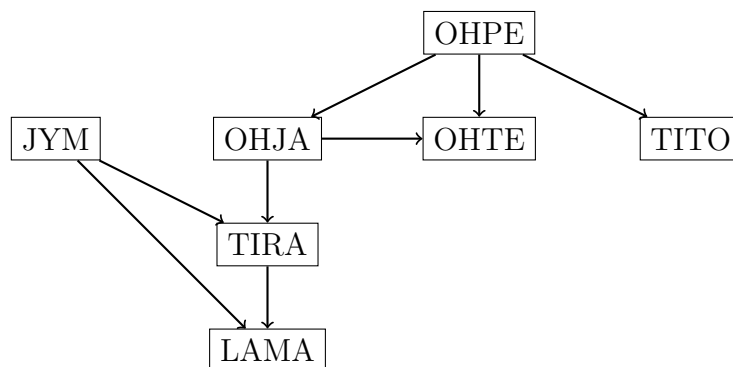


Figure 12.4: Kurssien esitietovaatimukset verkkona.

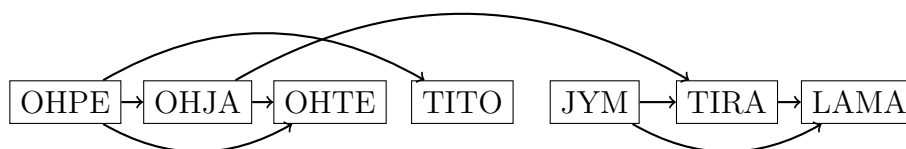


Figure 12.5: Topologinen järjestys antaa kurssien suoritusjärjestyksen.

jokaiselle kaarelle $a \rightarrow b$ pätee, että solmu a tulee järjestykseen ennen solmua b , eli algoritmi muodostaa kelvollisen topologisen järjestyksen.

Algoritmin aikavaativuus on $O(n + m)$, koska se käy kaikki verkon solmut ja kaaret läpi syvyysshaun avulla.

12.1.2 Esimerkki: Kurssivalinnat

Yliopiston kurssit ja niiden esitietovaatimukset voidaan esittää suunnattuna verkkona, jonka solmut ovat kursseja ja kaaret kuvaavat, missä järjestyksessä kurssit tulisi suorittaa.

Kuvassa 12.4 on esimerkkinä joitakin tietojenkäsittelytieteen kandiohjelman kursseja. Tällaisen verkon topologinen järjestys antaa yhden tavan suorittaa kurssit esitietovaatimusten mukaisesti. Kuvassa 12.5 näkyy esimerkkinä topologinen järjestys, joka vastaa suoritusjärjestyksestä OHPE, OHJA, OHTE, TITO, JYM, TIRA, LAMA.

On selvää, että kurssien ja esitietovaatimusten muodostaman verkon tulee olla sykliton, jotta kurssit voi suorittaa halutulla tavalla. Jos verkossa on sykli, topologista järjestystä ei ole olemassa eikä ole mitään mahdollisuutta suorittaa kursseja esitietovaatimusten mukaisesti.

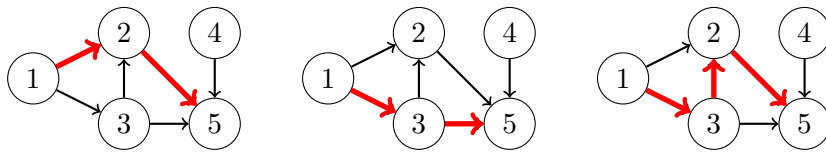


Figure 12.6: Mahdolliset polut solmusta 1 solmuun 5.

12.2 Dynaaminen ohjelmointi

Kun tiedämme, että suunnattu verkko on syklitön, voimme ratkaista helposti monia verkon polkuihin liittyviä ongelmia *dynaamisen ohjelmoinnin* avulla. Tämä on mahdollista, koska topologinen järjestys antaa selkeän järjestyksen, jossa voimme käsitellä solmut.

12.2.1 Polkujen laskeminen

Tarkastellaan esimerkkinä ongelmaa, jossa haluamme laskea, montako polkua suunnatussa syklittömässä verkossa on solmusta 1 solmuun n . Esimerkiksi kuvan 12.6 verkossa solmusta 1 solmuun 5 on kolme mahdollista polkua: $1 \rightarrow 2 \rightarrow 5$, $1 \rightarrow 3 \rightarrow 5$ ja $1 \rightarrow 3 \rightarrow 2 \rightarrow 5$.

Polkujen määrän laskeminen on vaikea ongelma yleisessä verkossa, jossa voi olla syklejä. Itse asiassa tehtävä ei ole edes mielekäs sellaisenaan: jos verkossa on sykli, voimme kiertää sykliä miten monta kertaa tahansa ja tuottaa aina vain uusia polkuja, joten polkuja tulee äärettömästi. Nyt kuitenkin oletamme, että verkko on syklitön, jolloin polkujen määrä on rajoitettu ja voimme laskea sen tehokkaasti dynaamisella ohjelmoinnilla.

Jotta voimme käyttää dynaamista ohjelmointia, meidän täytyy määritellä ongelma rekursiivisesti. Sopiva funktio on $\text{polut}(x)$, joka antaa polkujen määrän solmusta 1 solmuun x . Tätä funktiota käyttäen $\text{polut}(n)$ kertoo, montako polkua on solmusta 1 solmuun n . Esimerkiksi kuvan 12.6 tilanteessa funktion arvot ovat seuraavat:

$$\begin{aligned}\text{polut}(1) &= 1 \\ \text{polut}(2) &= 2 \\ \text{polut}(3) &= 1 \\ \text{polut}(4) &= 0 \\ \text{polut}(5) &= 3\end{aligned}$$

Nyt meidän täytyy enää löytää tapa laskea funktion arvoja. Pohjatapauksessa olemme solmussa 1, jolloin on aina yksi tyhjä polku:

$$\text{polut}(1) = 1$$

Entä sitten, kun olemme jossain muussa solmussa x ? Tällöin käymme läpi kaikki solmut, joista pääsemme solmuun x kaarella, ja laskemme yhteen näihin solmuihin tulevien polkujen määrät. Kun oletamme, että solmuun x on kaari solmuista $u_1, u_2, \dots, u_k$, saamme seuraavan rekursiivisen kaavan:

$$\text{polut}(x) = \text{polut}(u_1) + \text{polut}(u_2) + \dots + \text{polut}(u_k)$$

Esimerkiksi kuvan 12.6 verkossa solmuun 5 on kaari solmuista 2, 3 ja 4, joten

$$\text{polut}(5) = \text{polut}(2) + \text{polut}(3) + \text{polut}(4) = 2 + 1 + 0 = 3.$$

Koska tiedämme, että verkko on sykliton, voimme laskea funktion arvoja tehokkaasti dynaamisella ohjelmoinnilla. Oleellista on, että emme voi joutua koskaan silmukkaan laskiessamme arvoja. Käytännössä laskemme arvot josakin solmujen topologisessa järjestyksessä.

12.2.2 Ongelmat verkkoina

Itse asiassa voimme esittää *minkä tahansa* dynaamisen ohjelmoinnin algoritmin suunnatun syklittömän verkon käsittelynä. Ideana on, että muodostamme verkon, jossa jokainen solmu on yksi osaongelma ja kaaret ilmaisevat, miten osaongelmat liittyvät toisiinsa.

Tarkastellaan esimerkkinä luvusta 9 tuttua tehtävää, jossa haluamme laskea, monellako tavalla voimme muodostaa korkeuden n tornin, kun voimme käyttää palikoita, joiden korkeudet ovat 1, 2 ja 3. Voimme esittää tämän tehtävän verkkona niin, että solmut ovat tornien korkeuksia ja kaaret kertovat, kuinka voimme rakentaa tornia palikoista. Jokaisesta solmusta x on kaari solmuihin $x + 1$, $x + 2$ ja $x + 3$, ja polkujen määrä solmusta 0 solmuun n on yhtä suuri kuin tornin rakentamistapojen määrä. Esimerkiksi kuva 12.7 näyttää verkon, joka vastaa tapausta $n = 4$. Solmusta 0 solmuun 4 on yhteensä 7 polkua, eli voimme rakentaa korkeuden 4 tornin 7 tavalla.

Olemme saaneet siis uuden tavan luonnehtia dynaamista ohjelmointia: *voimme käyttää dynaamista ohjelmointia, jos pystymme esittämään ongelman suunnattuna syklittömänä verkkona.*

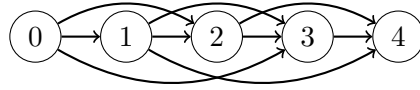


Figure 12.7: Tornitehtävän tapaus $n = 4$ esitettynä verkkona.

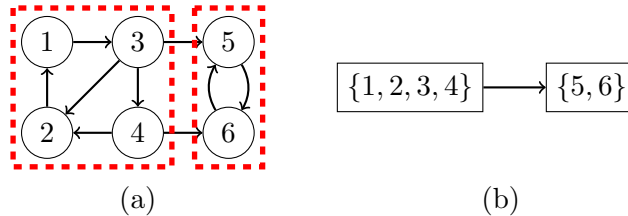


Figure 12.8: (a) Verkon vahvasti yhtenäiset komponentit. (b) Komponenttiverkko, joka kuvaa verkon syvärakenteen.

12.3 Vahvasti yhtenäisyys

Jos suunnatussa verkossa on sykli, emme voi muodostaa sille topologista järjestystä emmekä käyttää dynaamista ohjelmointia. Mikä neuvoksi, jos kuitenkin haluaisimme tehdä näin?

Joskus voimme selviytyä tilanteesta käsittelemällä verkon vahvasti yhtenäisiä komponentteja. Suunnattu verkko on *vahvasti yhtenäinen* (*strongly connected*), jos mistä tahansa solmusta on polku mihin tahansa solmuun. Voimme esittää suunnatun verkon aina yhtenä tai useampana vahvasti yhtenäisenä komponenttina, joista muodostuu syklitön *komponenttiverkko*. Tämä verkko esittää alkuperäisen verkon syvärakenteen.

Kuvassa 12.8 on esimerkkinä verkko, joka muodostuu kahdesta vahvasti yhtenäisestä komponentista. Ensimmäinen komponentti on $\{1, 2, 3, 4\}$ ja toinen komponentti on $\{5, 6\}$. Komponenteista muodostuu syklitön komponenttiverkko, jossa on kaari solmusta $\{1, 2, 3, 4\}$ solmuun $\{5, 6\}$. Tämä tarkoittaa, että voimme liikkua miten tahansa joukon $\{1, 2, 3, 4\}$ solmuissa sekä joukon $\{5, 6\}$ solmuissa. Lisäksi pääsemme joukosta $\{1, 2, 3, 4\}$ joukkoon $\{5, 6\}$, mutta emme pääse takaisin joukosta $\{5, 6\}$ joukkoon $\{1, 2, 3, 4\}$.

12.3.1 Kosarajun algoritmi

Kosarajun algoritmi on tehokas algoritmi, joka muodostaa suunnatun verkon vahvasti yhtenäiset komponentit. Algoritmissa on kaksi vaihetta, joista kumpikin käy läpi verkon solmut syvyysshaulla. Ensimmäinen vaihe muistuttaa topologisen järjestyksen etsimistä ja tuottaa listan solmuista. Toinen vaihe muo-

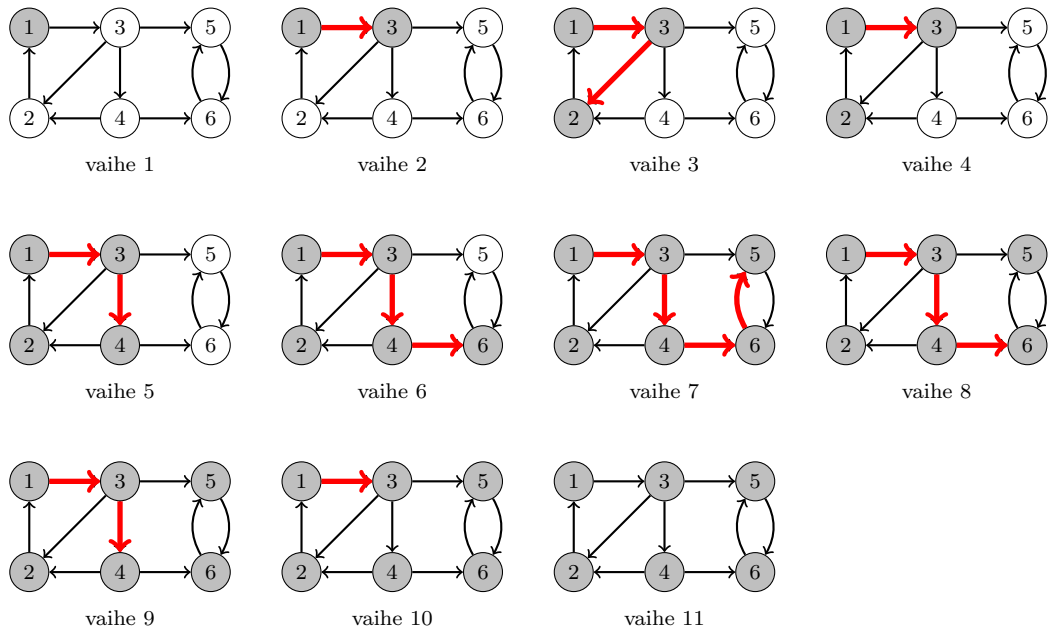


Figure 12.9: Kosarajun algoritmin ensimmäinen vaihe.

dostaa vahvasti yhtenäiset komponentit tämän listan perusteella.

Algoritmin ensimmäisessä vaiheessa käymme läpi verkon solmut ja aloitamme uuden syvyyshaun aina, jos emme ole vielä käyneet solmussa. Syvyyshaun aikana lisäämme solmun listalle, kun olemme käyneet läpi kaikki solmusta lähtevät kaaret. Toimimme siis kuten topologisen järjestyksen muodostamisessa, mutta emme välitä, jos tulemme toista reittiä solmuun, jota ei ole vielä käsitelty loppuun.

Algoritmin toisen vaiheen alussa käännämme jokaisen verkon kaaren suunnan. Tämän jälkeen käymme läpi käänteisessä järjestyksessä listalla olevat solmut. Aina kun vuoroon tulee solmu, jossa emme ole vielä käyneet, aloitamme siitä solmusta syvyyshaun, joka muodostaa uuden vahvasti yhtenäisen komponentin. Lisäämme komponenttiin kaikki solmut, joihin pääsemme syvyyshaun aikana ja jotka eivät vielä kuulu mihinkään komponenttiin.

Tarkastellaan seuraavaksi, kuinka Kosarajun algoritmi toimii esimerkkiverkossa. Kuva 12.9 näyttää algoritmin ensimmäisen vaiheen, joka muodostaa solmuista listan $[2, 5, 6, 4, 3, 1]$. Kuva 12.10 näyttää algoritmin toisen vaiheen, jossa käännämme ensin verkon kaaret ja käymme sitten läpi solmut järjestyksessä $[1, 3, 4, 6, 5, 2]$. Vahvasti yhtenäiset komponentit syntyvät solmuista 1 ja 6 alkaen. Kaarten kääntämisen ansiosta solmusta 1 alkava vahvasti yhtenäinen komponentti ei ”vuoda” solmujen 5 ja 6 alueelle.

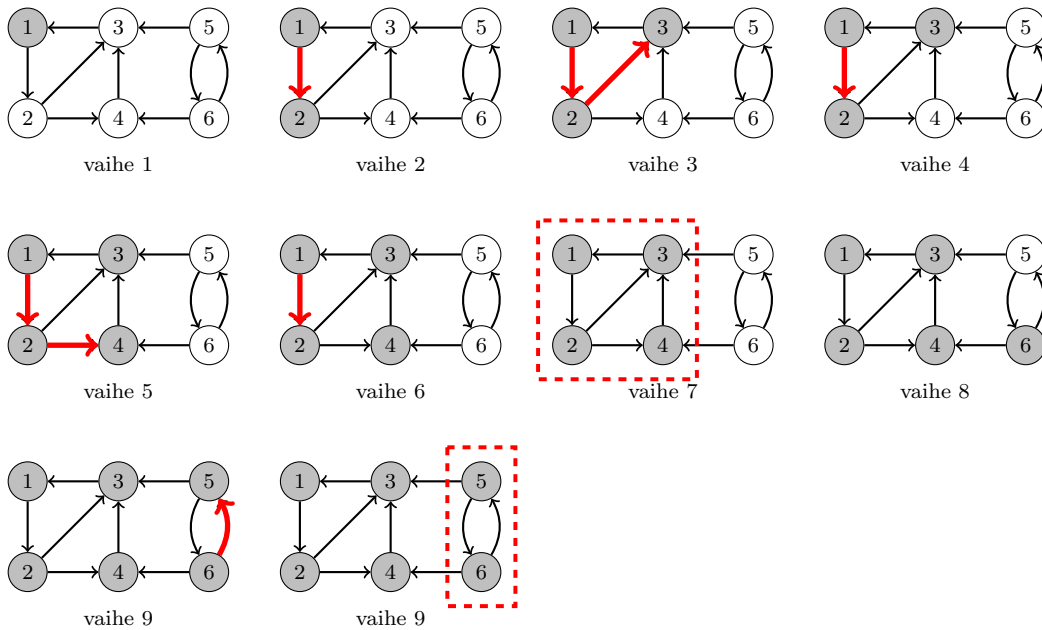


Figure 12.10: Kosarajun algoritmin toinen vaihe.

Algoritmin analyysi

Kosarajun algoritmin kriittinen osa on sen toinen vaihe, jossa algoritmi muodostaa vahvasti yhtenäiset komponentit. On selvää, että jokainen syvyysbushaku löytää vahvasti yhtenäiseen komponenttiin kuuluvat solmut, mutta miksi komponenttiin ei voi tulla lisäksi ylimääräisiä solmuja?

Voimme tarkastella asiaa muodostettavan komponenttiverkon näkökulmasta. Jos meillä on komponentti A , josta pääsee kaarella komponenttiin B , algoritmin ensimmäisessä vaiheessa jokin A :n solmu lisätään listalle kaikkien B :n solmujen jälkeen. Kun sitten käymme läpi listan käänteisessä järjestyksessä, jokin A :n solmu tulee vastaan ennen kaikkia B :n solmuja. Niinpä alamme rakentaa ensin komponenttia A emmekä mene komponentin B puolelle, koska verkon kaaret on käännetty. Sitten kun myöhemmin muodostamme komponentin B , emme mene käännettyä kaarta komponenttiin A , koska komponentti A on jo muodostettu.

Algoritmin aikavaativuus on $O(n + m)$, koska se käy kahdesti verkon solmut ja kaaret läpi syvyysbushaun avulla.

12.3.2 Esimerkki: Luolapeli

Olemme pelissä luolastossa, joka muodostuu n luolasta ja joukosta käytäviä niiden välillä. Jokainen käytävä on yksisuuntainen. Jokaisessa luolassa on yksi

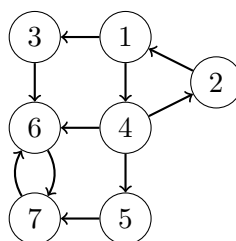


Figure 12.11: Luolasto, jossa on 7 luolaa ja 10 käytävää. Haluamme kulkea luolasta 1 luolaan 7 keräten mahdollisimman paljon aarteita.

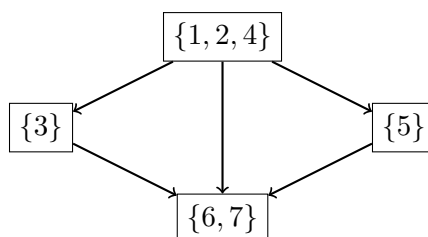


Figure 12.12: Luolaston vahvasti yhtenäiset komponentit.

aarre, jonka voimme ottaa mukaamme, jos kuljemme luolan kautta. Peli alkaa luolasta 1 ja päättyy luolaan n . Montako aarretta voimme saada, jos valitsemme parhaan mahdollisen reitin? On sallittua kulkea saman luolan kautta monta kertaa tarvittaessa.

Voimme mallintaa tilanteen verkkona, jonka solmut ovat luolia ja kaaret ovat käytäviä. Haluamme löytää reitin solmusta 1 solmuun n niin, että kuljemme mahdollisimman monen solmun kautta. Esimerkiksi kuva 12.11 näyttää verkkona luolaston, joka muodostuu seitsemästä luolasta ja kymmenestä käytävästä. Yksi optimaalinen reitti luolasta 1 luolaan 7 on

$$1 \rightarrow 4 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 6 \rightarrow 7,$$

jota seuraten saamme kerättyä kaikki aarteet paitsi luolassa 5 olevan aarteen. Ei ole olemassa reittiä, jota noudattamalla saisimme haltuumme kaikki luolaston aarteet.

Voimme ratkaista ongelman tehokkaasti määrittämällä ensin verkon vahvasti yhtenäiset komponentit. Tämän jälkeen riittää löytää solmun 1 komponentista solmun n komponenttiin kulkeva polku, jossa komponenttien kokojen summa on suurin mahdollinen. Koska verkko on sykliton, tämä onnistuu dynaamisella ohjelmoinnilla.

Kuva 12.12 näyttää vahvasti yhtenäiset komponentit esimerkkiverkossa. Tästä esityksestä näemme suoraan, että optimaalisia reittejä on olennaisesti kaksi: voimme kulkea joko luolan 3 tai luolan 5 kautta.

Chapter 13

Komponentit ja virittävät puut

Tähän mennessä olemme tarkastelleet verkkoja, joiden rakenne säilyy samana koko algoritmin ajan. Mitä tapahtuu sitten, jos verkkoon tulee *muutoksia*, kuten lisäämme verkkoon uusia kaaria?

Tutustumme tässä luvussa union-find-rakenteeseen, joka on hyödyllinen työkalu verkkojen käsittelyssä. Rakenteen avulla voimme pitää kirjaa verkon yhtenäisistä komponenteista ja päivittää rakennetta tehokkaasti, kun lisäämme verkkoon kaaria. Voimme esimerkiksi tarkkailla, montako yhtenäistä komponenttia verkossa on milläkin hetkellä.

Käsitlemme myös pienimmän virittävän puun ongelmaa, jossa haluamme kytkeä verkon solmut toisiinsa kaaria käyttäen niin, että kaarten yhteispaino on pienin. Voimme ratkaista ongelman tehokkaasti Kruskalin algoritmilla, joka perustuu union-find-rakenteeseen, tai Primin algoritmilla, joka muistuttaa Dijkstran algoritmia.

13.1 Union-find-rakenne

Union-find-rakenne on tietorakenne, joka pitää yllä kokoelmaa alkioiden joukkoja ja tarjoaa seuraavat tehokkaat operaatiot:

- tarkasta, ovatko kaksi alkioita samassa joukossa
- yhdistä kaksi joukkoa samaksi joukoksi

Oletamme, että alkiot ovat $1, 2, \dots, n$, ja jokainen alkio kuuluu tarkalleen yhteen joukkoon. Esimerkiksi kun $n = 8$, joukot voisivat olla $A = \{1, 4\}$, $B = \{2, 5, 6\}$ ja $C = \{3, 7, 8\}$. Tässä esimerkiksi alkiot 1 ja 2 ovat eri joukoissa. Kun yhdistämme sitten joukot A ja B , niistä syntyy joukko $\{1, 2, 4, 5, 6\}$. Tästä lähtien alkiot 1 ja 2 ovatkin samassa joukossa.

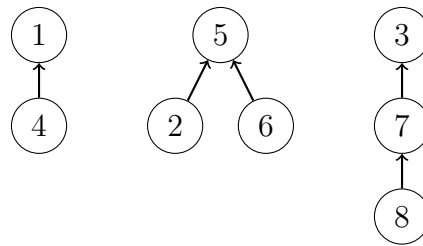


Figure 13.1: Union-find-rakenne joukoille $\{1, 4\}$, $\{2, 5, 6\}$ ja $\{3, 7, 8\}$.

13.1.1 Rakenteen toteutus

Toteutamme union-find-rakenteen niin, että jokaisessa joukossa yksi alkioista on joukon *edustaja*. Kutakin joukkoa vastaa puu, jonka juurena on joukon edustaja ja muut alkiot viittaavat edustajaan yhden tai useamman kaaren kautta. Kun haluamme tarkastaa, ovatko kaksi alkioita samassa joukossa, selvitämme alkioden edustajat ja vertaamme edustajia toisiinsa.

Kuvassa 13.1 on esimerkkinä union-find-rakenne, joka vastaa joukkoja $A = \{1, 4\}$, $B = \{2, 5, 6\}$ ja $C = \{3, 7, 8\}$. Tässä tapauksessa joukkojen edustajat ovat 1, 5 ja 3. Esimerkiksi jos haluamme tarkastaa, ovatko alkiot 2 ja 6 samassa joukossa, selvitämme ensin alkioden edustajat kulkemalla polkuja $2 \rightarrow 5$ ja $6 \rightarrow 5$. Kummankin alkion edustaja on 5, joten alkiot ovat samassa joukossa.

Jotta saamme toteutettua union-find-rakenteen, pidämme yllä jokaiselle alkioille x arvoa `vanhempi[x]`, joka kertoo seuraavan alkion ylempänä puussa. Kuitenkin jos x on joukon edustaja, `vanhempi[x] = x`. Esimerkiksi kuvassa 13.1 `vanhempi[2] = 5` ja `vanhempi[5] = 5`. Tämän ansiosta pystymme selvittämään alkion x edustajan seuraavasti:

```
function edustaja(x)
    while x != vanhempi[x]
        x = vanhempi[x]
    return x
```

Tämän jälkeen voimme tarkastaa seuraavalla funktiolla, ovatko alkiot a ja b samassa joukossa. Alkiot ovat samassa joukossa täsmälleen silloin, kun niillä on sama edustaja.

```
function sama(a,b)
    return edustaja(a) == edustaja(b)
```

Haluamme toteuttaa vielä operaation, jolla voimme yhdistää kaksi joukkoa toisiinsa. Tämän operaation toteutus ratkaisee, kuinka tehokas tietorakenne

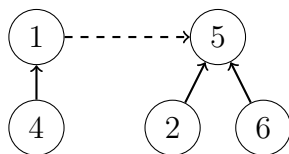


Figure 13.2: Tehokas yhdistäminen. Alkion 1 joukon koko on 2 ja alkion 5 joukon koko on 3, joten yhdistämme alkion 1 alkioon 5.

on. Alkion edustajan etsiminen vie aikaa $O(k)$, missä k on polun pituus, joten yhdistäiset tulisi toteuttaa niin, että puussa on vain lyhyitä polkuja. Saavutamme tämän tavoitteen yhdistämällä kaksi joukkoa aina niin, että *pienemmän* joukon edustaja alkaa osoittaa *suuremman* joukon edustajaan. Jos joukot ovat yhtä suuria, voimme toteuttaa yhdistämisen kummin päin vain.

Kuva 13.2 näyttää, mitä tapahtuu, kun yhdistämme joukot $A = \{1, 4\}$ ja $B = \{2, 5, 6\}$. Joukon A edustaja on 1 ja siinä on kaksi alkia, kun taas joukon B edustaja on 5 ja siinä on kolme alkia. Koska joukko A on pienempi, asetamme joukon A edustajan osoittamaan joukon B edustajaan. Tämän jälkeen kaikki alkiot kuuluvat samaan joukkoon ja alkio 5 on tästä lähtien koko joukon edustaja.

Nyt olemme valmiita toteuttamaan operaation, joka yhdistää toisiinsa joukot, joissa on alkiot a ja b . Oletamme, että alkiot ovat eri joukoissa ennen yhdistämistä. Jotta voimme toteuttaa yhdistämisen tehokkaasti, täytyy myös pitää kirjaa kunkin joukon koosta. Seuraavassa toteutuksessa `koko[x]` kertoo, montako alkia alkion x edustama joukko sisältää. Aluksi `koko[x] = 1` jokaiselle alkion x , koska jokainen alkio on omassa joukossa.

```

procedure yhdistä(a,b)
  a = edustaja(a)
  b = edustaja(b)
  if koko[a] < koko[b]
    swap(a,b)
  vanhempi[b] = a
  koko[a] += koko[b]
  
```

Kun toteutamme yhdistämiset näin, jokainen puussa esiintyvä polku sisältää vain $O(\log n)$ alkia. Tämä johtuu siitä, että aina kun kuljemme polkua askeleen ylöspäin alkion a alkioon b , `koko[b]  $\geq 2 \cdot$  koko[a]` eli edustajaa vastaavan joukon koko ainakin *kaksinkertaistuu*. Koska joukossa on enintään n alkia, kuljemme siis yhteensä enintään $O(\log n)$ askelta. Niinpä kaikki union-find-rakenteen operaatiot toimivat ajassa $O(\log n)$.

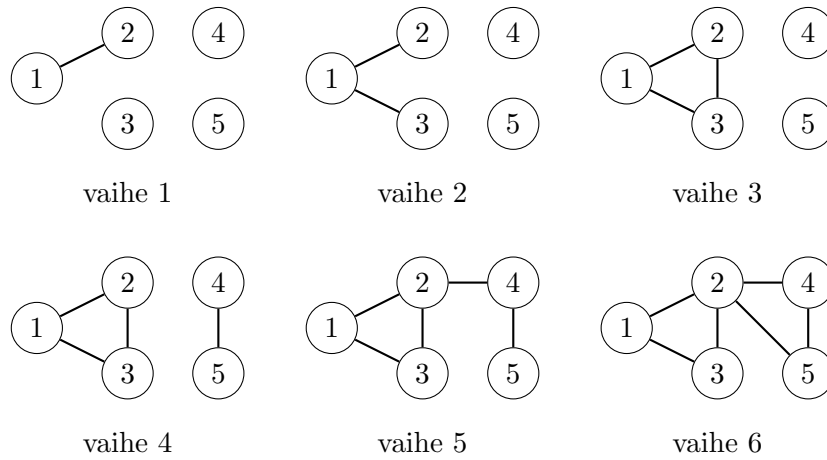


Figure 13.3: Esimerkki kaupunkien yhdistämisestä teillä. Vaiheen 5 jälkeen kaikki kaupungit ovat yhteydessä toisiinsa.

13.1.2 Esimerkki: Kaupungit

Bittimaassa on n kaupunkia, joiden välillä ei ole vielä yhtään tietä. Sitten teitä aletaan rakentaa yksi kerrallaan, yhteensä m tietä. Jokainen tie yhdistää kaksi kaupunkia toisiinsa. Minkä tien rakentamisen jälkeen kaikki kaupungit ovat ensimmäistä kertaa yhteydessä toisiinsa?

Kuva 13.3 näyttää esimerkkitapauksen, jossa $n = 5$, $m = 6$ ja tiet rakennetaan järjestyksessä $(1, 2)$, $(1, 3)$, $(2, 3)$, $(4, 5)$, $(2, 4)$ ja $(2, 5)$. Kaikki kaupungit ovat yhteydessä toisiinsa vaiheen 5 jälkeen.

Ratkaisu 1: Union-find-rakenne

Pidämme yllä verkon komponentteja union-find-rakenteen avulla. Aluksi jokainen solmu on omassa komponentissaan eli joukot ovat $\{1\}, \{2\}, \dots, \{n\}$. Sitten jokaisen kaaren kohdalla tarkastamme, ovatko sen päätesolmut eri joukoissa, ja jos ovat, yhdistämme joukot. Kun lopulta kaikki solmut ovat samassa joukossa, verkko on tullut yhtenäiseksi.

Tuloksena oleva algoritmi vie aikaa $O(n + m \log n)$, koska luomme ensin n komponenttia ajassa $O(n)$ ja käsittelemme tämän jälkeen m kaarta. Jokaisen kaaren kohdalla suoritamme enintään kaksi operaatiota union-find-rakenteessa ajassa $O(\log n)$.

Ratkaisu 2: Binäärihaku

Toinen tapa ratkaista tehtävä on hyödyntää *binäärihakua*. Jos on olemassa arvaus, että kaikki kaupungit ovat yhteydessä x lisäyksen jälkeen, voimme

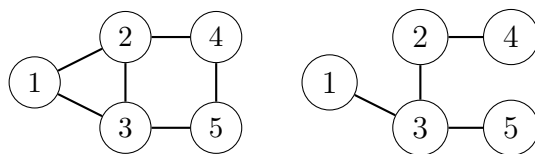
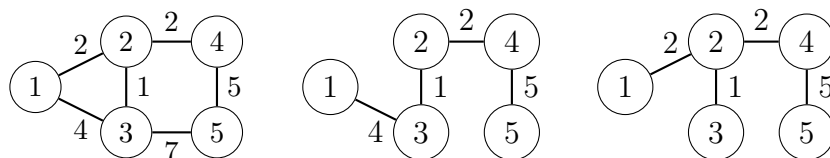


Figure 13.4: Verkko ja yksi sen virittävistä puista.

Figure 13.5: Painotettu verkko ja kaksi virittävää puuta, joiden painot ovat $4 + 1 + 2 + 5 = 12$ ja $2 + 1 + 2 + 5 = 10$.

tarkastaa helposti, pitääkö arvaus paikkansa: lisäämme ensin x ensimmäistä tietä tyhjään verkkoon ja tarkastamme sitten, onko verkko yhtenäinen. Tämä vie aikaa $O(n + m)$ käyttäen syvyyshakua.

Jos verkko on yhtenäinen ensimmäistä kertaa vaiheessa k , verkko ei ole yhtenäinen vaiheissa $1, 2, \dots, k-1$ ja on yhtenäinen vaiheissa $k, k+1, \dots, m$, koska kaarten lisääminen ei voi poistaa verkon yhtenäisyyttä. Tämän ansiosta voimme etsiä arvon k binäärihaun avulla. Binäärihaku suorittaa $O(\log m)$ askelta ja ratkaisu vie aikaa $O((n + m) \log m)$.

13.2 Pienin virittävä puu

Verkon *virittävä puu* (*spanning tree*) on kokoelma verkon kaaria, jotka kytkevät kaikki verkon solmut toisiinsa. Kuten puut yleensäkin, virittävä puu on yhtenäinen ja sykliton eli jokaisen kahden solmun välillä on yksikäsitteinen polku. Kuvassa 13.4 on esimerkkinä verkko ja yksi sen virittävistä puista.

Jos verkko on painotettu, kiinnostava ongelma on etsiä verkon *pienin virittävä puu* (*minimum spanning tree*). Tämä on virittävä puu, jonka kaarten painojen summa on mahdollisimman pieni. Esimerkiksi kuvassa 13.5 on painotettu verkko ja kaksi sen virittävää puuta, joiden painot ovat 12 ja 10. Näistä jälkimmäinen on verkon pienin virittävä puu.

13.2.1 Kruskalin algoritmi

Kruskalin algoritmi muodostaa verkon pienimmän virittävän puun aloittamalla tyhjästä verkosta, jossa on vain verkon solmut, ja lisäämällä siihen

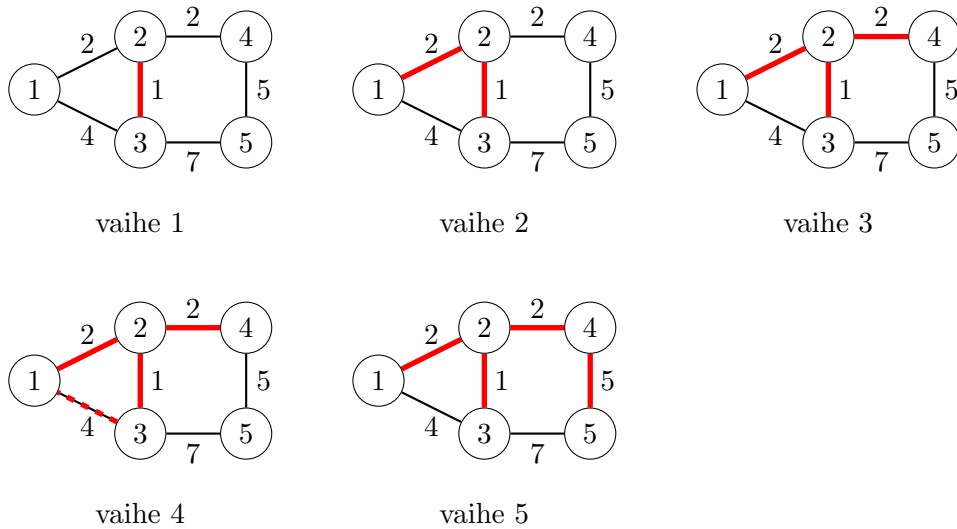


Figure 13.6: Esimerkki Kruskalin algoritmin toiminnasta.

kaaria. Algoritmi käy läpi tarjolla olevat kaaret järjestyksessä niiden painon mukaan kevyimmästä raskaimpaan. Jokaisen kaaren kohdalla algoritmi ottaa kaaren mukaan, jos se yhdistää kaksi eri komponenttia. Kun kaikki komponentit on yhdistetty, pienin virittävä puu on valmis.

Kuva 13.6 näyttää, kuinka Kruskalin algoritmi löytää pienimmän virittävän puun esimerkiverkossamme. Verkon kaaret järjestyksessä kevyimmästä raskaimpaan ovat:

kaari	paino
(2, 3)	1
(1, 2)	2
(2, 4)	2
(1, 3)	4
(4, 5)	5
(3, 5)	7

Algoritmi käsittelee ensin kaaren (2, 3). Solmut 2 ja 3 ovat eri komponenteissa, joten kaari otetaan mukaan puuhun. Tämän jälkeen algoritmi käsittelee kaaret (1, 2) ja (2, 4), jotka valitaan myös puuhun. Seuraavaksi vuorossa on kaari (1, 3), mutta tämä kaari ei tule puuhun, koska solmut 1 ja 3 ovat jo samassa komponentissa. Lopuksi algoritmi ottaa mukaan kaaren (4, 5), jolloin pienin virittävä puu on valmis.

Voimme toteuttaa Kruskalin algoritmin tehokkaasti käyttäen union-find-rakennetta. Algoritmin alussa järjestämme kaaret painojärjestykseen, missä kuluu aikaa $O(m \log m)$. Kun oletamme, että jokaisen solmuparin välillä on

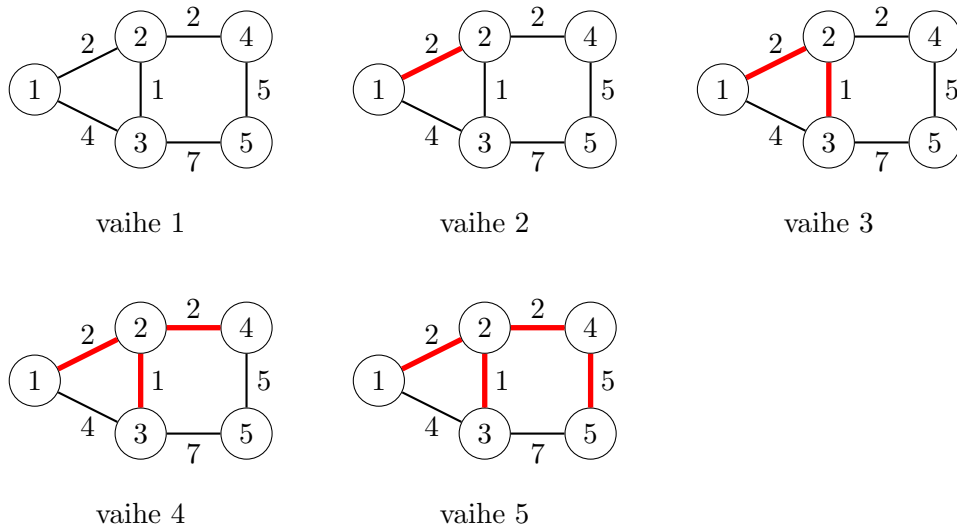


Figure 13.7: Esimerkki Primin algoritmin toiminnasta.

enintään yksi kaari, tämä sievenee muotoon $O(m \log n)$. Tämän jälkeen luomme kullekin solmulle komponentin, käymme kaaret läpi ja jokaisen kaaren kohdalla otamme kaaren mukaan, jos se yhdistää kaksi eri komponenttia. Tässä kuluu aikaa $O(n + m \log n)$, kun käytämme union-find-rakennetta. Algoritmi vie siis yhteensä aikaa $O(n + m \log n)$.

13.2.2 Primin algoritmi

Primin algoritmi tarjoaa toisen lähestymistavan pienimmän virittävän puun muodostamiseen. Algoritmi aloittaa puun muodostamisen tilanteesta, jossa puussa on vain yksi solmu. Tämän jälkeen se etsii joka vaiheessa kevyimmän kaaren, jonka toinen päätesolmu kuuluu puuhun ja toinen päätesolmu on vielä puun ulkopuolella, ja lisää puuhun tämän kaaren. Kun kaikki solmut on lisätty puuhun, pienin virittävä puu on valmis.

Kuva 13.7 näyttää esimerkin Primin algoritmin toiminnasta. Voimme aloittaa puun rakentamisen mistä tahansa solmusta, ja tässä esimerkissä aloitamme solmusta 1. Solmuun 1 on yhteydessä kaksi kaarta (1, 2) ja (1, 3), joista valitsemme kaaren (1, 2), koska se on kevyempi. Seuraavaksi tarjolla ovat kaaret (1, 3), (2, 3) ja (2, 4), joista valitsemme kaaren (2, 3). Tämän jälkeen lisäämme puuhun vastaavalla tavalla kaaret (2, 4), (4, 5), minkä jälkeen pienin virittävä puu on valmis.

Primin algoritmi muistuttaa paljon Dijkstran algoritmia. Erona on, että Dijkstran algoritmista valitsemme seuraavaksi solmun, jonka etäisyys *alkusolmuun* on pienin, mutta Primin algoritmista valitsemme solmun, jonka

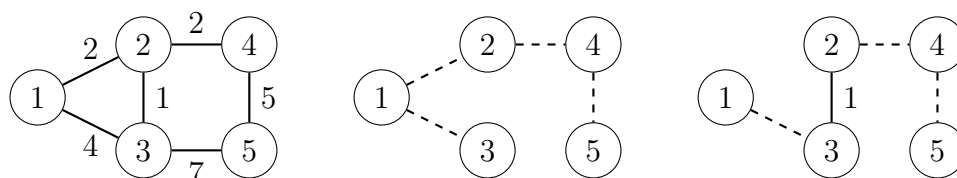


Figure 13.8: Kruskalin algoritmi: Pienin virittävä puu sisältää varmasti kaaren $(2, 3)$, koska muuten saisimme paremman ratkaisun sen avulla.

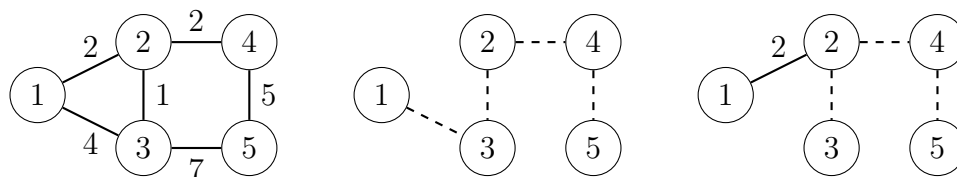


Figure 13.9: Primin algoritmi: Pienin virittävä puu sisältää varmasti kaaren $(1, 2)$, koska muuten saisimme paremman ratkaisun sen avulla.

etäisyys johonkin solmuun puussa on pienin. Voimme myös toteuttaa Primin algoritmin tehokkaasti samaan tapaan kuin Dijkstran algoritmin keon avulla, jolloin algoritmi vie aikaa $O(n + m \log n)$. Aikavaativuus on siis sama kuin Kruskalin algoritmilla, ja on käytännössä makuasia, kumman algoritmin valitsemme.

13.2.3 Miksi algoritmit toimivat?

Kruskalin ja Primin algoritmit ovat ahneita algoritmeja: ne lisäävät joka askeleella kevyimmän mahdollisen kaaren puuhun. Miksi algoritmit tuottavat pienimmän virittävän puun joka tilanteessa?

Voimme ajatella asiaa näin: Jos on kaksi solmua a ja b , jotka ovat eri komponenteissa, ne on yhdistettävä jotenkin samaan komponenttiin algoritmin aikana. Jos kevyin saatavilla oleva kaari on solmujen a ja b välillä, se kannattaa valita, koska ilman tätä kaarta komponentit tulisi yhdistää myöhemmin käyttäen raskaampaa kaarta.

Tarkastellaan ensin Kruskalin algoritmia. Mitä tapahtuu, jos emme valitse kevyintä kaarta algoritmin alussa? Kuvassa 13.8 näkyy kuvitteellinen tilanne, jossa katkoviivoilla esitetty pienin virittävä puu ei sisällä kevyintä kaarta $(2, 3)$, jonka paino on 1. Ei ole kuitenkaan mahdollista, että tämä olisi todellisuudessa pienin virittävä puu, koska voisimme vaihtaa jonkin puun kaaren kaareen $(2, 3)$, jolloin puun paino pienenee. Voimme siis huoletta valita kevyimmän kaaren puuhun Kruskalin algoritmin alussa. Samasta syystä voimme

tämän jälkeen valita seuraavaksi kevyimmän kaaren, jne.

Primin algoritmissa voimme käyttää melko samanlaista päättelyä. Oletetaan, että algoritmi lähtee liikkeelle solmusta 1, ja tarkastellaan ensimmäisen kaaren valintaa. Kuvassa 13.9 näkyy kuvitteellinen pienin virittävä puu, jossa emme ole valinneet alussa kevyintä kaarta $(1, 2)$. Saamme kuitenkin aikaan paremman ratkaisun, kun tutkimme, minkä kaaren kautta solmu 1 on yhteydessä solmuun 2, ja korvaamme tämän kaaren kaarella $(1, 2)$. Niinpä Primin algoritmin alussa on optimaalista valita puuhun kevyin kaari. Voimme soveltaa vastaavaa päättelyä algoritmin joka vaiheessa.

Huomaa, että Kruskalin ja Primin algoritmit toimivat myös silloin, kun verkossa on negatiivisia kaaria, koska ainoastaan kaarten painojärjestys merkitsee. Tämän ansiosta voimme etsiä algoritmien avulla myös verkon *suurimman virittävän puun* (*maximum spanning tree*) eli virittävän puun, jossa kaarten painojen summa on suurin. Tämä onnistuu muuttamalla ensin jokaisen verkossa olevan kaaren paino käänteiseksi ja etsimällä sitten pienimmän virittävän puun. Toinen tapa on vain toteuttaa Kruskalin tai Primin algoritmi niin, että joka vaiheessa valitaan painavin mahdollinen kaari.

Chapter 14

Maksimivirtaus

Tässä luvussa tarkastelemme ongelmaa, jossa haluamme välittää mahdollisimman paljon *virtausta* (*flow*) verkon solmusta toiseen, kun jokaisella kaarella on tietty kapasiteetti, jota emme saa ylittää. Voimme esimerkiksi haluta siirtää tietokoneverkossa tietoa mahdollisimman tehokkaasti koneesta toiseen, kun tiedämme verkon rakenteen ja yhteyksien nopeudet.

Tutustumme aluksi Fordin ja Fulkersonin algoritmiin, jonka avulla voimme sekä selvittää maksimivirtauksen että ymmärtää paremmin, mistä ongelmassa on kysymys. Tämän jälkeen tarkastelemme joitakin verkko-ongelmia, jotka pystymme ratkaisemaan *palauttamalla* ne maksimivirtaukseen.

14.1 Maksimivirtauksen laskeminen

Käsitlemme suunnattua verkkoa, jossa on kaksi erityistä solmua: *lähtösolmu* (*source*) ja *kohdesolmu* (*sink*). Haluamme muodostaa verkkoon mahdollisimman suuren virtauksen eli *maksimivirtauksen* (*maximum flow*) lähtösolmusta kohdesolmuun niin, että jokaiseen välisolmuun tuleva virtaus on yhtä suuri kuin solmusta lähtevä virtaus. Kullakin verkon kaarella on *kapasiteetti* (*capacity*), jota virtauksen määrä kaarta pitkin ei saa ylittää.

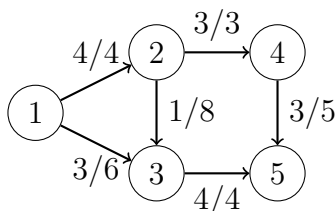


Figure 14.1: Maksimivirtaus solmusta 1 solmuun 5.

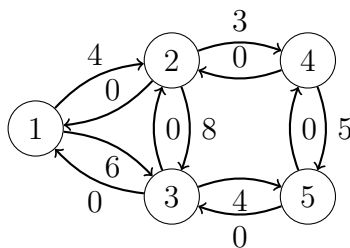


Figure 14.2: Verkon esitysmuoto Fordin ja Fulkersonin algoritmissa.

Kuvassa 14.1 näkyy esimerkkinä verkon maksimivirtaus, kun lähtösolmu on 1 ja kohdesolmu 5. Tässä tapauksessa maksimivirtauksen suuruus on 7. Jokaisessa kaaressa merkintä v/k tarkoittaa, että kaaren kautta kulkee virtausta v ja kaaren kapasiteetti on k . Solmusta 1 lähtevä virtauksen määrä on $4+3=7$, solmuun 5 saapuva virtauksen määrä on $3+4=7$ ja kaikissa muissa solmuissa saapuva virtaus on yhtä suuri kuin lähtevä virtaus.

14.1.1 Fordin ja Fulkersonin algoritmi

Fordin ja Fulkersonin algoritmi on tavallisin menetelmä verkon maksimivirtauksen etsimiseen, ja tutustumme seuraavaksi tämän algoritmin toimintaan. Algoritmi muodostaa lähtösolmusta kohdesolmuun polkuja, jotka kasvattavat virtausta pikkuhiljaa. Kun mitään polkua ei voi enää muodostaa, algoritmi on saanut valmiiksi maksimivirtauksen.

Jotta voimme käyttää algoritmia, esitämme verkon erityisessä muodossa, jossa jokaista alkuperäisen verkon kaarta vastaa *kaksi* kaarta: alkuperäinen kaari, jonka painona on kaaren kapasiteetti, sekä sille käänteinen kaari, jonka painona on 0. Käänteisten kaarten avulla pystymme tarvittaessa *peruuttamaan* virtausta algoritmin aikana. Kuva 14.2 näyttää, kuinka esitämme esimerkkiverkon algoritmissa¹.

Algoritmi muodostaa joka vaiheessa *täydennyspolun* (*augmenting path*), joka on mikä tahansa polku lähtösolmusta kohdesolmuun, jossa jokaisen kaaren paino on positiivinen. Polun muodostamisen jälkeen virtauksen suuruus lähtösolmusta kohdesolmuun kasvaa p :llä, missä p on pienin kaaren paino polulla. Lisäksi jokaisen polulla olevan kaaren paino vähenee p :llä ja jokaisen niille käänteisen kaaren paino kasvaa p :llä. Etsimme tällä tavalla uusia polkuja, kunnes mitään täydennyspolkua ei voi enää muodostaa.

¹Kirjallisuudessa tällaisesta verkosta käytetään usein termiä *jäännösverkko* (*residual graph*), kuitenkin sillä erotuksella, että mukana ovat kulloinkin vain kaaret, joiden paino on positiivinen. Kuitenkin käytännössä algoritmin toteutus on mukavampi, kun verkon rakenne on aina sama ja mukana ovat myös 0-painoiset kaaret.

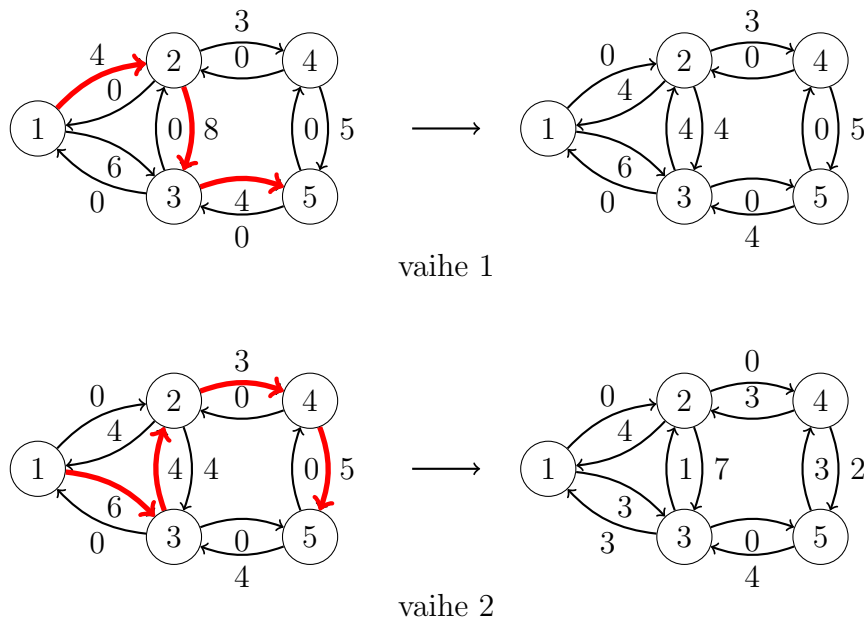


Figure 14.3: Esimerkki Fordin ja Fulkersonin algoritmin toiminnasta.

Kuva 14.3 näyttää, kuinka Fordin ja Fulkersonin algoritmi muodostaa maksimivirtauksen esimerkkiverkossa. Algoritmi muodostaa ensin polun $1 \rightarrow 2 \rightarrow 3 \rightarrow 5$, jossa pienin paino on 4. Tämän seurauksena virtaus kasvaa 4:llä, polulla olevien kaarten paino vähenee 4:llä ja käänteisten kaarten paino kasvaa 4:llä. Tämän jälkeen algoritmi muodostaa polun $1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 5$, jolloin virtaus kasvaa 3:lla. Huomaa, että tämä polku peruuttaa kaarta $2 \rightarrow 3$ menevää virtausta, koska se kulkee käänteisen kaaren $3 \rightarrow 2$ kautta. Tämän jälkeen algoritmi ei enää pysty muodostamaan mitään täydennyspolkua, joten verkon maksimivirtauksen suuruus on $4 + 3 = 7$.

Kun olemme saaneet maksimivirtauksen muodostettua, voimme selvittää jokaisessa alkuperäisessä kaaressa kulkevan virtauksen tutkimalla, miten kaaren paino on muuttunut algoritmin aikana. Kaarta pitkin kulkeva virtaus on yhtä suuri kuin kaaren painon vähennys algoritmin aikana. Esimerkiksi kuvassa 14.3 kaaren $4 \rightarrow 5$ paino on alussa 5 ja algoritmin suorituksen jälkeen 2, joten kaarta pitkin kulkevan virtauksen määrä on $5 - 2 = 3$.

14.1.2 Yhteys minimileikkaukseen

Fordin ja Fulkersonin algoritmin toimintaidea on sinänsä järkevä, koska täydennyspolut lähtösolmusta kohdesolmuun kasvattavat virtausta, mutta ei ole silti päältä päin selvää, miksi algoritmi löytää varmasti *suurimman* mahdol-

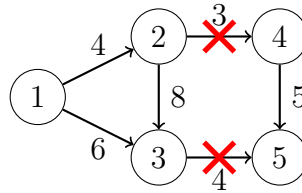


Figure 14.4: Minimileikkaus, jossa poistamme kaaret $2 \rightarrow 4$ ja $3 \rightarrow 5$.

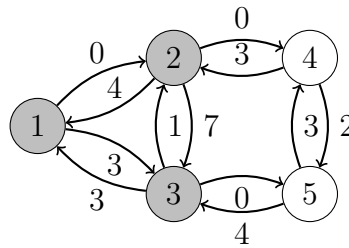


Figure 14.5: Solmut 1, 2 ja 3 ovat saavutettavissa lähtösolmusta.

lisen virtauksen. Jotta voimme ymmärtää paremmin algoritmin toimintaa, tarkastelemme seuraavaksi toista verkko-ongelmaa, joka antaa meille uuden näkökulman maksimivirtaukseen.

Lähtökohtamme on edelleen suunnattu verkko, jossa on lähtösolmu ja kohdesolmu. Sanomme, että joukko kaaria muodostaa *leikkauksen* (*cut*), jos niiden poistaminen verkosta estää kulkemisen lähtösolmusta kohdesolmuun. *Minimileikkaus* (*minimum cut*) on puolestaan leikkaus, jossa kaarten yhteispaino on mahdollisimman pieni. Kuvassa 14.4 on minimileikkaus, jossa poistamme kaaret $2 \rightarrow 4$ ja $3 \rightarrow 5$ ja jonka paino on $3 + 4 = 7$.

Osoittautuu, että verkon maksimivirtauksen suuruus on aina sama kuin minimileikkauksen paino, ja tämä yhteys auttaa perustelemaan, miksi Fordin ja Fulkersonin algoritmi toimii. Ensinnäkin voimme havaita, että *minkä tahansa* verkon leikkauksen paino on vähintään yhtä suuri kuin maksimivirtauksen määrä. Tämä johtuu siitä, että virtauksen täytyy ylittää leikkaukseen kuuluva kaaria, jotta se pääsee lähtösolmusta kohdesolmuun. Esimerkiksi kuvassa 14.4 virtaus voi päästä solmusta 1 solmuun 5 joko kulkemalla kaarta $2 \rightarrow 4$ tai kaarta $3 \rightarrow 5$. Niinpä virtauksen suuruus ei voi ylittää näiden kaarten painojen summaa.

Toisaalta Fordin ja Fulkersonin algoritmi muodostaa sivutuotteenaan verkon leikkauksen, jonka paino on sama kuin verkossa olevan virtauksen suuruus. Löydämme leikkauksen etsimällä ensin kaikki solmut, joihin pääsemme lähtösolmusta positiivisia kaaria pitkin algoritmin lopputilanteessa. Esimerkkiverkosamme nämä solmut ovat 1, 2 ja 3 kuvan 14.5 mukaisesti. Kun valitsemme

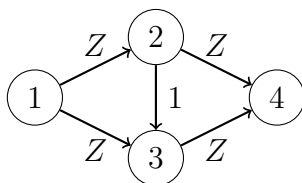


Figure 14.6: Verkko, joka voi aiheuttaa ongelmia algoritmillemme.

sitten alkuperäisen verkon kaaret, jotka johtavat näiden solmujen ulkopuolelle ja joiden kapasiteetti on käytetty kokonaan, saamme aikaan verkon leikkauksen. Esimerkissämme nämä kaaret ovat $2 \rightarrow 4$ ja $3 \rightarrow 5$.

Koska olemme löytäneet virtauksen, jonka suuruus on sama kuin leikkauksen paino, ja toisaalta virtauksen suuruus ei voi ylittää minkään leikkauksen painoa, olemme siis löytäneet maksimivirtauksen ja minimileikkauksen, joten Fordin ja Fulkersonin algoritmi toimii oikein.

14.1.3 Polkujen valitseminen

Voimme muodostaa täydennyspolkuja Fordin ja Fulkersonin algoritmin aikana miten tahansa, mutta polkujen valintatapa vaikuttaa algoritmin tehokkuuteen. Riippumatta polkujen valintatavasta jokainen polku kasvattaa virtausta ainakin *yhellä* yksiköllä. Niinpä joudumme etsimään enintään f täydennyspolkua, kun verkon maksimivirtauksen suuruus on f . Jos muodostamme polut syvyyshaulla, jokaisen polun muodostaminen vie aikaa $O(m)$, joten saamme algoritmin ajankäytölle ylärajan $O(fm)$.

Voiko todella käydä niin, että jokainen polku parantaa virtausta vain yhdellä? Tämä on mahdollista, ja kuva 14.6 tarjoaa esimerkin asiasta. Tässä Z on jokin suuri vakio, joka on kaaren kapasiteetti. Jos muodostamme polut syvyyshaulla ja valitsemme haussa aina solmun, jonka tunnus on pienin, muodostamme vuorotellen polkuja $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ ja $1 \rightarrow 3 \rightarrow 2 \rightarrow 4$ niin, että lisäämme ja peruutamme edestakaisin kaarta $2 \rightarrow 3$ kulkevaa virtausta. Tämän vuoksi joudumme muodostamaan $2Z$ polkua, ennen kuin olemme saaneet selville verkon maksimivirtauksen.

Voimme kuitenkin estää tämän ilmiön määrittelemällä tarkemmin, miten muodostamme täydennyspolkuja algoritmin aikana. *Edmondsin ja Karpin algoritmi* on Fordin ja Fulkersonin algoritmin versio, jossa muodostamme polut *leveyshaulla*. Tämä tarkoittaa, että valitsemme aina polun, jossa on mahdollisimman vähän kaaria. Leveyshakua käyttäen muodostamme kuvan 14.6 verkossa vain kaksi täydennyspolkua $1 \rightarrow 2 \rightarrow 4$ ja $1 \rightarrow 3 \rightarrow 4$, jotka antavat suoraan maksimivirtauksen.

Osoittautuu, että Edmondsin ja Karpin algoritmin täytyy muodostaa

aina vain $O(nm)$ täydennyspolkua, joten algoritmi vie aikaa $O(nm^2)$ riippumatta virtauksen suuruudesta. Voimme perustella tämän aikavaativuuden kahden havainnon avulla:

Havainto 1: Kun x on mikä tahansa verkon solmu, etäisyys lyhintä polkua pitkin lähtösolmusta solmuun x ei voi koskaan pienentyä täydennyspolun muodostamisen jälkeen. Voimme perustella tämän tekemällä vastaoletuksen, että täydennyspolku on juuri pienentänyt etäisyyttä solmuun x . Tarkastellaan lyhintä polkua lähtösolmusta solmuun x ja ensimmäistä tällä polulla olevaa kaarta $a \rightarrow b$, jossa etäisyys solmuun a ei ole pienentynyt mutta etäisyys solmuun b on pienentynyt täydennyspolun seurauksena. Tämä tarkoittaa, että ennen täydennyspolun muodostamista verkossa ei ollut positiivista kaarta $a \rightarrow b$, vaan tämä kaari on syntynyt siitä, että täydennyspolku on kulkenut positiivista kaarta $b \rightarrow a$ ja virtaukset ovat muuttuneet. Tämä ei kuitenkaan ole mahdollista, koska tällöin a :n etäisyys olisi ollut suurempi kuin b :n etäisyys ennen täydennyspolun muodostamista eikä kaari $a \rightarrow b$ voisi olla osana uutta lyhintä polkua.

Havainto 2: Jokaiseen täydennyspolkuun liittyy jokin kaari $a \rightarrow b$, jonka kapasiteetti käytetään kokonaan, ja jokainen kaari voi esiintyä enintään $n/2$ kertaa tässä roolissa. Syynä on, että kun kaaren $a \rightarrow b$ kapasiteetti käytetään kokonaan, algoritmin täytyy muodostaa täydennyspolku, joka kulkee vastakkaiseen suuntaan kaarta $b \rightarrow a$, ennen kuin kaaren $a \rightarrow b$ kapasiteettia voi käyttää uudestaan. Koska etäisyys lähtösolmusta solmuun ei voi pienentyä, etäisyys lähtösolmusta kaaren alkusolmuun a kasvaa ainakin kahdella aina, kun täydennyspolku kulkee toiseen suuntaan kaarta $b \rightarrow a$.

Näiden havaintojen perusteella Edmondsin ja Karpin algoritmi muodostaa enintään $nm/2$ täydennyspolkua, koska voidaan valita enintään m tavalla, minkä kaaren kapasiteetti käytetään kokonaan, ja enintään $n/2$ tavalla, mikä on kaaren alkusolmun etäisyys.

14.2 Maksimivirtauksen sovelluksia

Maksimivirtauksen etsiminen on tärkeä ongelma, koska pystymme *palauttamaan* monia verkko-ongelmia maksimivirtaukseen. Tämä tarkoittaa, että muutamme toisen ongelman jotenkin sellaiseen muotoon, että se vastaa maksimivirtausta. Tutustumme seuraavaksi joihinkin tällaisiin ongelmiin.

14.2.1 Erilliset polut

Tehtävämme on muodostaa mahdollisimman monta *erillistä* polkua verkon lähtösolmusta kohdesolmuun. Erillisyyys tarkoittaa, että jokainen verkon

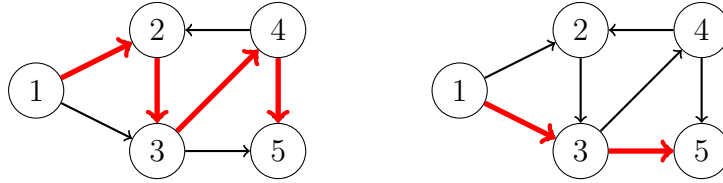


Figure 14.7: Kaksi erillistä polkua solmusta 1 solmuun 5.

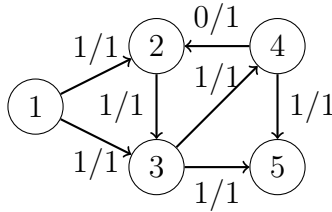


Figure 14.8: Erilliset polut tulkittuna maksimivirtauksena.

kaari saa esiintyä enintään yhdellä polulla. Saamme kuitenkin halutessamme kulkea saman solmun kautta useita kertoja. Esimerkiksi kuvassa 14.7 voimme muodostaa kaksi erillistä polkua solmusta 1 solmuun 5, mutta ei ole mahdollista muodostaa kolmea erillistä polkua.

Voimme ratkaista ongelman tulkitsemalla erilliset polut maksimivirtauksena. Ideana on etsiä maksimivirtaus lähtösolmusta kohdesolmuun olettaen, että jokaisen kaaren kapasiteetti on 1. Tämä maksimivirtaus on yhtä suuri kuin suurin erillisten polkujen määrä. Kuva 14.8 näyttää maksimivirtauksen esimerkiverkossa.

Miksi maksimivirtauksen suuruus on sama kuin erillisten polkujen määrä? Ensinnäkin erilliset polut muodostavat yhdessä virtauksen, joten maksimivirtauksen suuruus ei voi alittaa erillisten polkujen määrää. Toisaalta jos verkossa on virtaus, jonka suuruus on k , voimme muodostaa k erillistä polkua valitsemalla kaaria ahneesti lähtösolmusta alkaen, joten maksimivirtauksen suuruus ei voi myöskään ylittää erillisten polkujen määrää.

14.2.2 Maksimiparitus

Verkon *paritus* (*matching*) on joukko kaaria, joille pätee, että jokainen solmu on enintään yhden kaaren päätepisteenä. *Maksimiparitus* (*maximum matching*) on puolestaan paritus, jossa on mahdollisimman paljon kaaria. Keskitymme tapaukseen, jossa verkko on *kaksijakoinen* (*bipartite*) eli voimme jakaa verkon solmut vasempaan ja oikeaan ryhmään niin, että jokainen kaari kulkee ryhmien välillä.

Kuvassa 14.9 on esimerkkinä kaksijakoinen verkko, jonka maksimiparitus

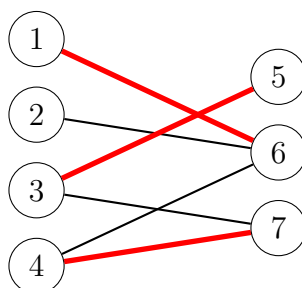


Figure 14.9: Kaksijakoisen verkon maksimiparitus.

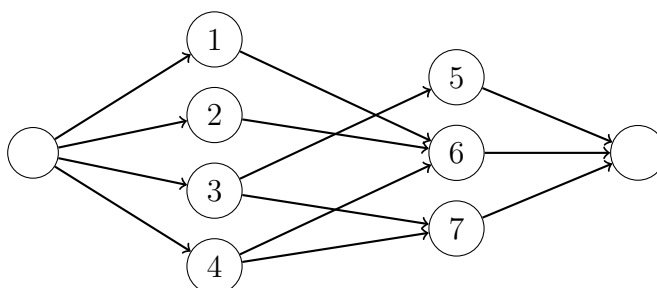


Figure 14.10: Maksimiparitus tulkittuna maksimivirtauksena.

on 3. Tässä vasen ryhmä on $\{1, 2, 3, 4\}$, oikea ryhmä on $\{5, 6, 7\}$ ja maksimiparitus muodostuu kaarista $(1, 6)$, $(3, 5)$ ja $(4, 7)$.

Voimme tulkita maksimiparituksen maksimivirtauksena lisäämällä verkkoon kaksi uutta solmua: lähtösolmun ja kohdesolmun. Lähtösolmusta pääsee kaarella jokaiseen vasemman ryhmän solmuun, ja jokaisesta oikean ryhmän solmusta pääsee kaarella kohdesolmuun. Lisäksi suuntaamme alkuperäiset kaaret niin, että ne kulkevat vasemmasta ryhmästä oikeaan ryhmään. Kuva 14.10 näyttää tuloksena olevan verkon esimerkissämme. Maksimivirtaus tässä verkossa vastaa alkuperäisen verkon maksimiparitusta, kun valitsemme paritukseen kaikki kaaret, joiden kautta kulkee virtausta.

14.2.3 Pienin polkupeite

Polkupeite (path cover) on joukko verkon polkuja, jotka kattavat yhdessä kaikki verkon solmut. Oletamme, että verkko on suunnattu ja sykliton, ja haluamme muodostaa mahdollisimman pienen polkupeitteen niin, että jokainen solmu esiintyy tarkalleen yhdessä polussa. Kuvassa 14.11 on esimerkkinä verkko ja sen pienin polkupeite, joka muodostuu kahdesta polusta.

Voimme ratkaista pienimmän polkupeitteen etsimisen ongelman maksimivirtauksen avulla muodostamalla verkon, jossa jokaista alkuperäistä sol-

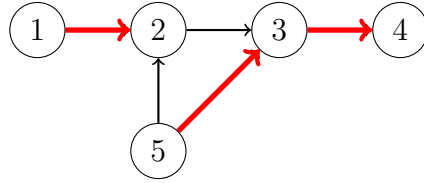


Figure 14.11: Polkupeite, joka muodostuu poluista $1 \rightarrow 2$ ja $5 \rightarrow 3 \rightarrow 4$.

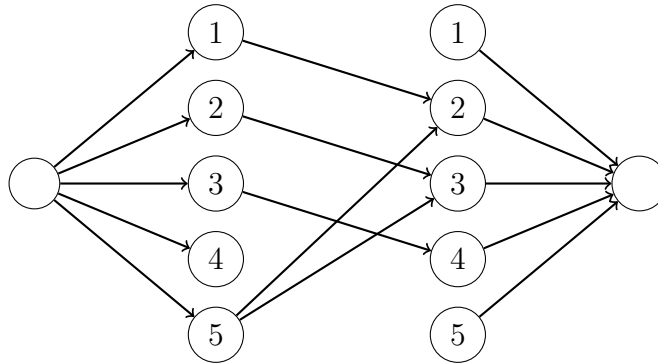


Figure 14.12: Polkupeitteen etsiminen maksimivirtauksen avulla.

mua vastaa kaksi solmua: vasen ja oikea solmu. Vasemmasta solmusta on kaari oikeaan solmuun, jos alkuperäisessä verkossa on vastaava kaari. Lisäämme vielä verkkoon lähtösolmun ja kohdesolmun niin, että lähtösolmusta pääsee kaikkiin vasempiin solmuihin ja kaikista oikeista solmuista pääsee kohdesolmuun. Tämän verkon maksimivirtaus antaa meille alkuperäisen verkon pienimmän polkupeitteen.

Kuva 14.12 näyttää tuloksena olevan verkon esimerkissä. Ideana on, että maksimivirtaus etsii, mitkä kaaret kuuluvat polkuihin: jos kaari solmusta a solmuun b kuuluu virtaukseen, niin vastaavasti polkupeitteessä on polku, jossa on kaari $a \rightarrow b$. Koska virtauksessa voi olla valittuna vain yksi kaari, joka alkaa tietyistä solmista tai päättyy tiettyyn solmuun, tuloksena on varmasti joukko polkuja. Toisaalta mitä enemmän kaaria saamme laitettua polkuihin, sitä pienempi on polkujen määrä.

Chapter 15

NP-ongelmat

Olemme tässä kirjassa tutustuneet moniin algoritmeihin, jotka toimivat tehokkaasti. Kuitenkin on myös suuri määrä ongelmia, joiden ratkaisemiseen ei tällä hetkellä tunneta mitään tehokasta algoritmia. Jos vastaamme tulee tällainen ongelma, hyvät neuvot ovat kalliit.

Vaikeiden ongelmien yhteydessä esiintyy usein kirjainyhdistelmä NP. Eri-tyisen tunnettu on kysymys P vs. NP, jonka ratkaisijalle on luvattu miljoonan dollarin potti. Hankalalta tuntuvasta ongelmasta saatetaan arvella, että se on NP-täydellinen tai NP-vaikea. Nyt on aika selvittää, mitä nämä käsitteet oikeastaan tarkoittavat.

15.1 Vaativuusluokat

Laskennallisia ongelmia luokitellaan *vaativuusluokkiin* (*complexity class*), joista jokaisessa on joukko ongelmia, joiden ratkaisemisen vaikeudessa on jotain yhteistä. Tunnetuimmat vaativuusluokat ovat P ja NP.

Keskitymme tässä luvussa *päätösongelmiin* (*decision problem*), joissa algoritmin tulee antaa aina vastaus ”kyllä” tai ”ei”. Esimerkiksi ongelma ”onko verkossa polkua solmusta a solmuun b ?” on päätösongelma. Tulemme huomaamaan, että voimme muotoilla monenlaisia ongelmia päätösongelmina eikä tämä rajoita juurikaan, mitä ongelmia voimme tarkastella.

15.1.1 Luokka P

Luokka P sisältää päätösongelmat, joiden ratkaisemiseen on olemassa *polynominen* algoritmi eli algoritmi, jonka aikavaativuus on enintään $O(n^k)$, missä k on vakio. Lähes kaikki tässä kirjassa esitetyt algoritmit ovat olleet polynomisia. Tuttuja polynomisia aikavaativuuksia ovat esimerkiksi $O(1)$,

$O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$ ja $O(n^3)$.

Esimerkiksi ongelma ”onko verkossa polkua solmusta a solmuun b ?” kuuluu luokkaan P, koska voimme ratkaista sen monellakin tavalla polynomisessa ajassa. Voimme vaikkapa aloittaa syvyysshaun solmusta a ja tarkastaa, pääsemekö solmuun b . Tuloksena on algoritmi, joka toimii lineaarisessa ajassa, joten ongelma kuuluu luokkaan P.

Luokan P tarkoituksena on kuvata ongelmia, jotka voimme ratkaista jos-sain mielessä *tehokkaasti*. Tässä tehokkuuden määritelmä on varsin karkea: pidämme algoritmia tehokkaana, jos sillä on mikä tahansa polynominen aikavaativuus. Onko $O(n^{100})$ -aikainen algoritmi siis tehokas? Ei, mutta käytännössä vakio k on yleensä pieni ja polynominen aikavaativuus on osoitautunut toimivaksi tehokkuuden mittariksi.

15.1.2 Luokka NP

Luokka NP sisältää päätösongelmat, joissa jokaisessa ”kyllä”-tapauksessa on olemassa *todiste*, jonka avulla voimme *tarkastaa* polynomisessa ajassa, että vastaus todellakin on ”kyllä”. Todiste on merkkijono, jonka koko on polynominen suhteessa syötteeseen, ja se antaa meille lisätietoa siitä, minkä takia ”kyllä”-vastaus pitää paikkansa syötteelle.

Esimerkki luokkaan NP kuuluvasta ongelmasta on ”onko verkossa polkua solmusta a solmuun b , joka kulkee tasan kerran jokaisen verkon solmun kautta?”. Tämän ongelman ratkaisemiseen ei tunneta polynomista algoritmia, mutta jokaisessa ”kyllä”-tapauksessa on olemassa todiste: halutunlainen polku solmusta a solmuun b . Voimme tarkastaa helposti polynomisessa ajassa, että todisteen kuvaamalla polulla on vaaditut ominaisuudet.

Jos vastaus syötteeseen on ”ei”, tähän ei tarvitse liittyä mitään todistetta. Usein olisikin hankalaa antaa todiste siitä, että jotain asiaa *ei* ole olemassa. Esimerkiksi jos etsimme verkosta tietynlaista polkua, on helppoa todistaa polun olemassaolo, koska voimme vain näyttää kyseisen polun, mutta ei ole vastaavaa keinoa todistaa, että polkua ei ole olemassa.

Huomaa, että kaikki luokan P ongelmat kuuluvat myös luokkaan NP. Tämä johtuu siitä, että luokan P ongelmissa voimme tarkastaa ”kyllä”-vastauksen *tyhjän* todisteen avulla: voimme saman tien ratkaista koko ongelman alusta alkaen polynomisessa ajassa.

15.1.3 P vs. NP

Äkkiseltään voisi kuvitella, että luokassa NP täytyy olla enemmän ongelmia kuin luokassa P. Luokassa NP meidän riittää vain tarkastaa ”kyllä”-vastauksen todiste, mikä tuntuu helpommalta kuin muodostaa ongelman ratkaisu tyhjästä (kuva



Figure 15.1: P vs. NP : Voisiko olla yhtä helppoa muodostaa ratkaisu tyhjästä kuin tarkastaa, onko annettu ratkaisu oikein?

15.1). Monet uskovatkin, että luokka NP on suurempi kuin luokka P , mutta kukaan ei ole onnistunut todistamaan asiaa.

Tietojenkäsittelytieteen merkittävä avoin kysymys on, päteekö $P = NP$ vai $P \neq NP$. Monet tutkijat ovat tarttuneet haasteeseen 70-luvulta lähtien, mutta tähän mennessä kaikki ovat epäonnistuneet. Ongelman ratkaisija saisi maineen ja kunnian lisäksi myös tuntuvan rahallisen korvauksen, koska Clay-instituutti on luvannut miljoonan dollarin palkinnon sille, joka todistaa, että $P = NP$ tai $P \neq NP$. Voi olla kuitenkin, että tämä on yksi *vaikeimmista* tavoista ansaita miljoona dollaria.

Jos pätee $P \neq NP$, kuten uskotaan, vaikeutena on keksiä keino todistaa, että jotakin luokan NP ongelmaa on mahdotonta ratkaista polynomisessa ajassa. Tämän todistaminen on vaikeaa, koska meidän pitää näyttää, että tehokasta algoritmia ei ole olemassa, vaikka laatisimme algoritmin miten tahansa. Vaikka moni on koettanut tuloksetta ratkoa tunnettuja NP -ongelmia, kysymys saat-
taa silti olla siitä, että tehokas algoritmi olisi olemassa mutta kukaan ei vain ole vielä löytänyt sitä.

15.1.4 Muita luokkia

P ja NP ovat tunnetuimmat vaativuusluokat, mutta niiden lisäksi on suuri määrä muitakin luokkia. Yksi tällainen luokka on $PSPACE$, joka sisältää ongelmat, joiden ratkaisuun riittää polynominen määrä *muistia*. $PSPACE$ sisältää kaikki luokan NP ongelmat, mutta siinä on myös ongelmia, joiden ei tiedetä kuuluvan luokkaan NP .

Luokat BPP ja ZPP puolestaan liittyvät satunnaisuuteen. Luokka BPP sisältää ongelmat, joiden ratkaisuun on polynomiaikainen algoritmi, joka antaa oikean vastauksen ainakin todennäköisyydellä $2/3$. Luokka ZPP taas sisältää ongelmat, joiden ratkaisuun on algoritmi, jonka suoritusajan *odotusarvo* on polynominen. Sekä BPP että ZPP sisältävät luokan P , jonka ongelmat voi ratkaista polynomisessa ajassa ilman satunnaisuuttakin.

Vaativuusluokkien suhteet toisiinsa tunnetaan yleensä ottaen huonosti. Vaikka kysymys P vs. NP on saanut eniten huomiota, myös vastaavat kysymykset esimerkiksi luokkien $PSPACE$, BPP ja ZPP kohdalla ovat avoimia

ongelmia. Kukaan ei tiedä, onko näissä luokissa loppujen lopuksi mitään ongelmaa, joka ei kuuluisi myös luokkaan P.

15.2 NP-täydellisyys

Sanomme, että ongelma on *NP-täydellinen* (*NP-complete*), jos se kuuluu luokkaan NP ja mikä tahansa luokan NP ongelma voidaan *palauttaa* siihen polynomisessa ajassa. NP-täydelliset ongelmat ovat luokan NP vaikeimpia ongelmia: jos voisimme ratkaista jonkin NP-täydellisen ongelman tehokkaasti, voisimme ratkaista minkä tahansa luokan NP ongelman tehokkaasti.

Kiinnostava ilmiö on, että lähes kaikki tunnetut luokan NP ongelmat joko kuuluvat myös luokkaan P tai ovat NP-täydellisiä. Nykyään tunnetaan tuhansia erilaisia NP-täydellisiä ongelmia. Jos keksisimme mihin tahansa niistä polynomisessa ajassa toimivan ratkaisun, olisimme samalla onnistuneet todistamaan, että $P = NP$.

15.2.1 SAT-ongelma

Ensimmäinen löydetty NP-täydellinen ongelma oli SAT-ongelma, jossa annettuna on konjunkttiivisessa normaalimuodossa oleva looginen kaava ja haluamme selvittää, voimmeko valita muuttujien arvot niin, että kaava on tosi. Konjunkttiivinen normaalimuoto tarkoittaa, että kaava koostuu lausekkeista, jotka on yhdistetty ja-operaatiolla ($\wedge$), ja jokainen lauseke muodostuu muuttujista ja niiden negaatioista, jotka on yhdistetty tai-operaatiolla ($\vee$).

Esimerkiksi kaava

$$(\neg x_1 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$$

on mahdollista saada todeksi, koska voimme esimerkiksi asettaa muuttujat x_1 ja x_2 epätosi ja muuttujan x_3 todeksi. Vastaavasti kaava

$$(x_1 \vee x_2) \wedge (\neg x_1 \vee x_2) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_2)$$

ei ole tosi, vaikka valitsisimme muuttujien arvot miten tahansa.

Kun haluamme osoittaa, että SAT on NP-täydellinen ongelma, meidän täytyy näyttää, että se kuuluu luokkaan NP ja mikä tahansa luokan NP ongelma voidaan palauttaa siihen. Luokkaan NP kuulumisen on helppoa nähdä: ”kyllä”-tapauksessa todiste on kullekin muuttujalle valittu arvo. Huomattavasti vaikeampaa on osoittaa, että *jokainen* luokan NP ongelma voidaan palauttaa SAT-ongelmaan polynomisessa ajassa.

Tässä kirjassa emme käsittele todistusta yksityiskohtaisesti, mutta voimme kuitenkin kuvailla sen perusideaa. Tarkastellaan tiettyä luokan NP ongelmaa, joka meidän täytyy pystyä palauttamaan SAT-ongelmaan. Koska ongelma voi olla mikä tahansa luokkaan NP kuuluva, tiedämme siitä vain, että on olemassa algoritmi, joka tarkastaa polynomisessa ajassa ”kyllä”-tapauksen todisteen. Tämä vastaa sitä, että on olemassa *epädeterministinen* Turingin kone, joka rakentaa ja tarkastaa tällaisen todisteen polynomisessa ajassa. Nyt kun haluamme tarkastaa annetusta syötteestä, onko vastaus siihen ”kyllä”, voimme muodostaa konjunkttiivisessa normaalimuodossa olevan loogisen kaavan, joka luonnehtii Turingin koneen laskentaa, kun koneelle annetaan kyseinen syöte. Voimme muodostaa kaavan niin, että sen voi saada todeksi tarkalleen silloin, kun vastaus syötteeseen on ”kyllä”. Niinpä olemme onnistuneet palauttamaan alkuperäisen ongelman SAT-ongelmaan.

15.2.2 Ongelmien palautukset

Kun tiedämme, että SAT-ongelma on NP-täydellinen, voimme osoittaa muita ongelmia NP-täydellisiksi palautusten avulla. Ideana on, että jos ongelma A on NP-täydellinen ja voimme palauttaa sen polynomisessa ajassa ongelmaksi B , myös ongelma B on NP-täydellinen.

Palautus $\text{SAT} \rightarrow 3\text{SAT}$

Aloitamme osoittamalla, että 3SAT-ongelma on NP-täydellinen. 3SAT-ongelma on SAT-ongelman erikoistapaus, jossa jokaisessa $\wedge$ -merkeillä yhdistetyssä lausekkeessa on tarkalleen kolme muuttujaa. Esimerkiksi kaava

$$(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$$

on kelvollinen 3SAT-ongelman syöte. Jotta saamme palautettua SAT-ongelman 3SAT-ongelmaan, meidän on näytettävä, että voimme muuttaa polynomisessa ajassa minkä tahansa SAT-ongelman syötteen 3SAT-ongelman syötteeksi, jonka totuusarvo on sama.

Ideana on muokata jokaista SAT-ongelman syötteen lauseketta niin, että tuloksena on yksi tai useampia kolmen muuttujan lausekkeita. Merkitään k :lla lausekkeen muuttujien määrää. Jos $k = 1$ tai $k = 2$, toistamme viimeistä muuttujaa uudestaan, jotta saamme lausekkeeseen kolme muuttujaa. Esimerkiksi jos lauseke on (x_1) , muutamme sen muotoon $(x_1 \vee x_1 \vee x_1)$, ja jos lauseke on $(x_1 \vee x_2)$, muutamme sen muotoon $(x_1 \vee x_2 \vee x_2)$. Jos $k = 3$, meidän ei tarvitse tehdä mitään, koska lausekkeessa on valmiiksi kolme muuttujaa. Jos sitten $k > 3$, jaamme lausekkeen osiin, jotka ketjutetaan uusien

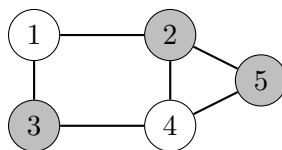


Figure 15.2: Solmut $\{2, 3, 5\}$ muodostavat solmupeitteen.

apumuuttujien avulla. Ketjun jokaisessa kohdassa vasemman lausekkeen viimeinen muuttuja on a_i ja oikean lausekkeen ensimmäinen muuttuja on $\neg a_i$. Tämä takaa, että ainakin yksi alkuperäinen muuttuja saa oikean arvon. Esimerkiksi jos lauseke on $(x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5)$, muutamme sen kolmeksi lausekkeeksi $(x_1 \vee x_2 \vee a_1)$, $(\neg a_1 \vee x_3 \vee a_2)$ ja $(\neg a_2 \vee x_4 \vee x_5)$.

Tämä palautus osoittaa, että 3SAT on NP-täydellinen ongelma, eli SAT-ongelman oleellinen vaikeus syntyy jo siitä, että lausekkeissa voi olla kolme muuttujaa¹. Palautuksen hyötynä on myös se, että myöhemmissä todistuksissa meidän on helpompaa käsitellä kolmen muuttujan lausekkeitä kuin vaihtelevan pituisia lausekkeitä.

Palautus 3SAT $\rightarrow$ solmupeite

Seuraavaksi osoitamme, että on NP-täydellinen ongelma tarkastaa, onko verkossa *solmupeitettä* (*vertex cover*), jossa on k solmua. Solmupeite on verkon solmujen osajoukko, joka on valittu niin, että jokaisessa kaaressa ainakin toinen päätesolmu kuuluu solmupeitteeseen. Esimerkiksi kuvassa 15.2 on verkko ja sen solmupeite, johon kuuluu kolme solmua.

Kun haluamme palauttaa 3SAT-ongelman solmupeiteongelmaan, meidän täytyy näyttää, että voimme tulkitä minkä tahansa 3SAT-ongelman tapauksen verkon solmupeitteen etsimisenä. Meidän tulee keksiä systemaattinen tapa muuttaa looginen kaava verkoksi, jonka k solmun solmupeite vastaa sitä, että kaava on totta.

Oletamme, että kaavassa esiintyy n muuttujaa $x_1, x_2, \dots, x_n$ ja siinä on m lauseketta. Muodostamme verkon, jossa on ensinnäkin n solmuparia, jotka vastaavat kaavan muuttujia. Kussakin solmuparissa on muuttujat x_i ja $\neg x_i$, joiden välillä on kaari. Lisäksi verkossa on m kolmen solmun ryhmää, jotka vastaavat lausekkeitä. Jokaisessa ryhmässä kaikki solmut ovat yhteydessä toisiinsa, minkä lisäksi kukin solmu on yhteydessä sitä vastaavaan solmuun pareissa. Esimerkiksi kaavaa

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_3 \vee x_4),$$

¹Entä ongelma 2SAT, jossa jokaisessa lausekkeessa on kaksi muuttujaa? Tämä *ei* ole NP-täydellinen ongelma, vaan kuuluu luokkaan P.

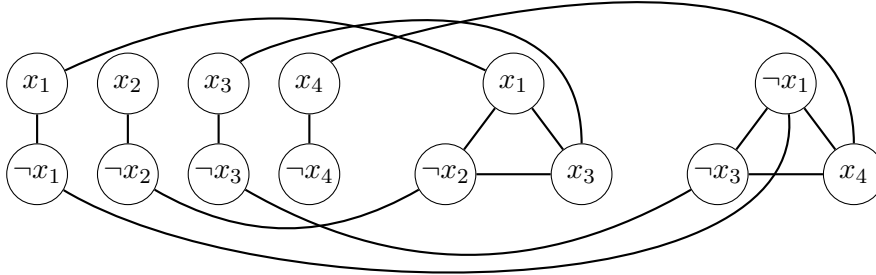


Figure 15.3: Kaava $(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_3 \vee x_4)$ verkkona.

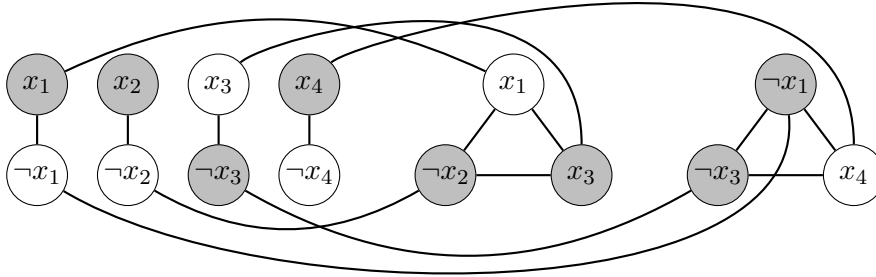


Figure 15.4: Ratkaisu, jossa x_1 , x_2 ja x_4 ovat tosia ja x_3 on epätosi.

vastaa kuvan 15.3 mukainen verkko.

Osoittautuu, että voimme saada kaavan todeksi tarkalleen silloin, kun verkossa on solmupeite, jossa on enintään $k = n + 2m$ solmua. Tällaiseen peitteeseen kuuluu toinen solmu jokaisesta parista x_i ja $\neg x_i$. Tämä määrittää, miten muuttujien arvot asetetaan. Lisäksi peitteeseen kuuluu kaksi solmua jokaisesta kolmen solmun ryhmästä. Koska ryhmässä on yksi solmu, joka ei kuulu peitteeseen, kyseisen solmun täytyy olla yhteydessä kaarella peitteeseen kuuluvaan solmuun. Tämä varmistaa, että jokaisessa lausekkeessa ainakin yksi kolmesta muuttujasta on asetettu oikein. Kuva 15.4 näyttää esimerkin solmupeitteestä, joka saa kaavan todeksi. Tämä vastaa ratkaisua, jossa x_1 , x_2 ja x_4 ovat tosia ja x_3 on epätosi.

Olemme siis onnistuneet palauttamaan 3SAT-ongelman solmupeiteongelmaksi niin, että verkon koko on polynominen suhteessa kaavan pituuteen, joten solmupeiteongelma on NP-täydellinen.

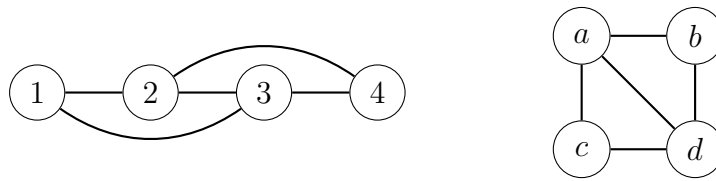


Figure 15.5: Kaksi verkkoa, jotka ovat isomorfiset.

15.2.3 Lisää ongelmia

Palautusten avulla on onnistuttu löytämään tuhansia NP-täydellisiä ongelmia. Esimerkiksi myös seuraavat ongelmat ovat NP-täydellisiä:

- onko verkossa k -kokoista *klikkiä* (*clique*) eli k solmun joukkoa, jossa jokaisen kahden solmun välillä on kaari?
- voimmeko värittää verkon solmut kolmella värillä niin, että jokaisen kaaren päätesolmut ovat eri värisiä?
- onko verkossa polkua, joka kulkee tasan kerran jokaisen verkon solmun kautta (eli *Hamiltonin polkua* (*Hamiltonian path*))?
- voiko annetuista n luvusta valita osajoukon, jonka summa on x ?
- onko olemassa k -merkkistä merkkijonoa, jonka alijonoja ovat kaikki annetut merkkijonot?

Entä millainen olisi luokan NP ongelma, joka ei kuulu luokkaan P eikä ole NP-täydellinen? Kukaan ei tiedä, onko tällaista ongelmaa olemassa, koska ei edes tiedetä, ovatko P ja NP eri luokat. Yksi ehdokas tällaiseksi ongelmaksi on kuitenkin ongelma, jossa haluamme tarkastaa, ovatko kaksi verkkoa *isomorfiset* eli onko verkkojen rakenne samanlainen, jos solmut asetetaan vastaamaan toisiaan sopivalla tavalla. Esimerkiksi kuvassa 15.5 olevat verkot ovat isomorfiset, koska voimme valita solmuille vastaavuudet $(1, c)$, $(2, a)$, $(3, d)$ ja $(4, b)$.

Verkkojen isomorfisuuden ongelma kuuluu luokkaan NP, koska on helppoa tarkastaa, onko verkoilla sama rakenne, jos tiedämme solmujen vastaavuudet. Ongelmaan ei kuitenkaan tunneta polynomiaikaista algoritmia, joten sen ei tiedetä kuuluvan luokkaan P. Toisaalta emme myöskään osaa tehdä mitään palautusta, joka osoittaisi ongelman olevan NP-täydellinen.

15.2.4 Optimointiongelmat

Käytännössä haluamme usein ratkaista päätösongelman sijasta *optimointiongelman* (*optimization problem*): haluamme etsiä pienimmän tai suurimman mahdollisen ratkaisun. Esimerkiksi emme halua tarkastaa, onko verkossa enintään k solmun solmupeitettä (pätösongelma), vaan haluamme etsiä *pienimmän* solmupeitteen (optimointiongelma). Osoittautuu kuitenkin, että päätösongelmat ja optimointiongelmat ovat loppujen lopuksi hyvin lähellä toisiaan.

Oletetaan, että meillä on keino tarkastaa tehokkaasti, onko verkossa k solmun solmupeitettä. Miten voimme menetellä, jos haluammekin etsiä pienimmän solmupeitteen? Ratkaisuna on käyttää *binäärihakua*: etsimme pienimmän arvon x , jolle pätee, että verkossa on x solmun solmupeite. Tämä tarkoittaa, että verkon pienin solmupeite sisältää x solmua. Koska käytämme binäärihakua, meidän riittää ratkaista päätösongelma vain logaritminen määrä kertoja, joten saamme ratkaistua optimointiongelman lähes yhtä tehokkaasti kuin päätösongelman.

Sanomme, että ongelma on *NP-vaikea* (*NP-hard*), jos voimme palauttaa kaikki luokan NP ongelmat siihen mutta ongelman ei tarvitse kuulua luokkaan NP. Jos ongelma on NP-vaikea, se on siis ainakin yhtä vaikea kuin luokan NP vaikeimmat ongelmat. Tyypillisiä NP-vaikeita ongelmia ovat NP-täydellisten päätösongelmien optimointiversiot, koska voimme palauttaa niihin NP-täydellisiä ongelmia mutta ne eivät kuulu luokkaan NP.

Käytännössä termejä ei käytetä aina näin täsmällisesti ja optimointiongelmaa saatetaan sanoa NP-täydelliseksi, vaikka se oikeastaan on NP-vaikea. Voimme myös vain puhua yleisesti NP-ongelmista, kun tarkoitamme NP-täydellisiä ja NP-vaikeita ongelmia.

15.3 Ongelmien ratkaiseminen

Jos saamme ratkaistavaksemme NP-vaikean ongelman, tilanne ei näytä hyvältä, koska edessämme on silloin ongelma, johon ei tunneta mitään tehokasta algoritmia. Emme voi toivoa, että osaisimme ratkaista tehokkaasti ongelmaa, jota kukaan muukaan ei ole osannut. Peli ei ole kuitenkaan välttämättä vielä menetetty, vaan voimme koettaa lähestyä ongelmaa monella tavalla.

Tarkastellaan seuraavaksi ongelmaa, jossa meille annetaan n kokonaislukua ja haluamme jakaa luvut kahteen ryhmään niin, että ryhmien summat ovat mahdollisimman lähellä toisiaan. Esimerkiksi jos $n = 5$ ja luvut ovat $[1, 2, 4, 5, 7]$, paras ratkaisu on muodostaa ryhmät $[1, 2, 7]$ ja $[4, 5]$, joiden summat ovat $1 + 2 + 7 = 10$ ja $4 + 5 = 9$. Tällöin summien ero on 1, eikä ole olemassa ratkaisua, jossa ero olisi 0.

Tämä ongelma on NP-vaikea, joten kukaan ei tiedä tehokasta algoritmia sen ratkaisemiseen. Seuraavaksi käymme läpi joukon tapoja, joiden avulla voimme kuitenkin yrittää ratkaista ongelmaa.

Raaka voima

Vaikka ongelma olisi NP-vaikea, voimme silti ratkaista sen pieniä tapauksia. Tässä tehtävässä voimme tehdä algoritmin, joka käy läpi kaikki tavat jakaa luvut kahteen ryhmään ja valitsee parhaan vaihtoehdon. Tämä vastaa sitä, että käymme läpi kaikki tavat valita toiseen ryhmään tulevat luvut eli kaikki lukujen osajoukot.

Koska n luvusta voi muodostaa 2^n osajoukkoa, tällainen algoritmi vie aikaa $O(2^n)$. Voimme toteuttaa algoritmin esimerkiksi rekursiolla samaan tapaan kuin teimme luvussa 8. Tuloksena oleva algoritmi on käyttökelpoinen, jos n on niin pieni, että ehdimme käydä kaikki osajoukot läpi. Käytännössä voimme ratkaista tehokkaasti tapauksia, joissa n on enintään noin 20.

Heuristiikat

Yksi mahdollisuus selviytyä vaikeasta ongelmasta on tyytyä optimaalisen ratkaisun sijasta johonkin *melko hyvään* ratkaisuun, joka ei välttämättä ole optimaalinen. Saamme tällaisen ratkaisun aikaan keksimällä jonkin *heuristiikan* (*heuristic*), joka muodostaa ratkaisun. Heuristiikka on jokin järkevä sääntö, jonka avulla algoritmi rakentaa ratkaisua eteenpäin askel kerrallaan.

Tässä tehtävässä yksinkertainen heuristiikka on käydä luvut läpi suurimmasta pienimpään ja sijoittaa luku aina ryhmään, jonka summa on sillä hetkellä pienempi. Jos kummankin ryhmän summa on yhtä suuri, sijoitamme luvun ensimmäiseen ryhmään. Tämä on järkevä heuristiikka, koska näin saamme monessa tapauksessa jaettua lukuja melko tasaisesti ryhmiin.

Esimerkiksi jos luvut ovat $[1, 2, 4, 5, 7]$, sijoitamme ensin luvun 7 ensimmäiseen ryhmään, sitten luvut 5 ja 4 toiseen ryhmään ja lopuksi luvut 2 ja 1 ensimmäiseen ryhmään. Saamme muodostettua näin ryhmät $[1, 2, 7]$ ja $[4, 5]$, joiden summien ero on 1 eli ratkaisu on optimaalinen.

Tällainen algoritmi ei kuitenkaan tuota aina optimaalista ratkaisua. Kun tunnemme algoritmin toiminnan, voimme keksiä tilanteita, joissa se antaa huonomman ratkaisun. Tällainen tapaus on esimerkiksi $[2, 2, 2, 3, 3]$. Heuristiikkaa käyttävä algoritmi tuottaa ryhmät $[2, 3]$ ja $[2, 2, 3]$, joiden summien ero on 2. Kuitenkin optimaalinen ratkaisu olisi muodostaa ryhmät $[2, 2, 2]$ ja $[3, 3]$, joiden summien ero on 0.

Satunnaisuus

Voimme hyödyntää ongelman ratkaisemisessa myös *satunnaisuutta*. Voimme tehdä algoritmin, joka tekee satunnaisia valintoja, ja lisäksi toistaa algoritmia monta kertaa ja valita parhaan ratkaisun.

Esimerkiksi voimme parantaa äskeistä heuristista algoritmia niin, että se sekoittaa alussa lukujen järjestyksen. Tämän jälkeen voimme toistaa algoritmia vaikkapa miljoona kertaa ja valita parhaan ratkaisun. Tämä voi parantaa merkittävästi algoritmin toimintaa, ja on vaikeampaa keksiä tapauksia, joissa algoritmi toimii huonosti.

Rajoittaminen

Joskus voimme saada aikaan tehokkaan algoritmin, kun rajoitamme sopivalla tavalla tehtävää. Esimerkiksi tässä tehtävässä on mahdollista luoda tehokas dynaamisen ohjelmoinnin ratkaisu, kunhan oletamme, että luvut ovat sopivan pieniä kokonaislukuja.

Ideana on tarkastella osaongelmia muotoa ”voimmeko valita a ensimmäisestä luvusta osajoukon, jonka lukujen summa on b ?” Jos luvut ovat pieniä, niin myös summat ovat pieniä, joten voimme ratkaista kaikki tällaiset osaongelmat rekursiivisesti. Tämän jälkeen riittää etsiä mahdollinen jako, jossa $a = n$ ja b on mahdollisimman lähellä lukujen summan puoliväliä.

Tällainen ratkaisu vie aikaa $O(ns)$, missä s on lukujen summa. Sanomme, että algoritmi on *pseudopolynominen*, koska sen aikavaativuus on polynominen mutta riippuu lukuarvojen suuruudesta. Yleisessä tapauksessa syötteenä annetut luvut voivat kuitenkin olla suuria kokonaislukuja, jolloin emme voi käyttää tällaista ratkaisua.

Appendix A

Matemaattinen tausta

Tämä liite käy läpi matematiikan merkintöjä ja käsitteitä, joita käytetään kirjassa algoritmien analysoinnissa.

Merkintöjä

Merkinnät $\lfloor x \rfloor$ ja $\lceil x \rceil$ tarkoittavat, että pyöristämme luvun x alaspäin ja ylöspäin kokonaisluvuksi. Esimerkiksi $\lfloor 5.23 \rfloor = 5$ ja $\lceil 5.23 \rceil = 6$.

Kertoma $n!$ lasketaan $1 \cdot 2 \cdot 3 \cdots n$. Esimerkiksi $5! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 = 120$.

Merkintä $a \bmod b$ tarkoittaa, mikä on jakojäännös, kun a jaetaan b :llä. Esimerkiksi $32 \bmod 5 = 2$, koska $32 = 6 \cdot 5 + 2$.

Summakaavat

Voimme laskea lukujen $1, 2, \dots, n$ summan kaavalla

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

Usein hyödyllinen on myös kaava

$$a^0 + a^1 + \cdots + a^n = \frac{a^{n+1} - 1}{a - 1},$$

joka pätee, kun $a \neq 1$. Tämän erikoistapaus on kaava

$$2^0 + 2^1 + \cdots + 2^n = 2^{n+1} - 1.$$

Logaritmi

Logaritmin määritelmän mukaan $\log_b n = x$ tarkalleen silloin kun $b^x = n$. Esimerkiksi $\log_2 32 = 5$, koska $2^5 = 32$.

Logaritmi $\log_b n$ kertoo, montako kertaa meidän tulee jakaa luku n luvulla b , ennen kuin pääsemme lukuun 1. Esimerkiksi $\log_2 32 = 5$, koska tarvitsemme 5 puolitusta:

$$32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$$

Tässä kirjassa oletamme, että logaritmin kantaluku on 2, jos ei ole toisin mainittu, eli voimme kirjoittaa lyhyesti $\log 32 = 5$.

Logaritmeille pätevät kaavat

$$\log_b(x \cdot y) = \log_b(x) + \log_b(y)$$

ja

$$\log_b(x/y) = \log_b(x) - \log_b(y).$$

Ylemmstä kaavasta seuraa myös

$$\log_b(x^k) = k \log_b(x).$$

Lisäksi voimme vaihtaa logaritmin kantalukua kaavalla

$$\log_u(x) = \frac{\log_b(x)}{\log_b(u)}.$$

Odotusarvo

Odotusarvo kuvaa, mikä on keskimääräinen tulos, jos satunnainen tapahtuma toistuu monta kertaa eli mitä tulosta voimme tavallaan odottaa.

Kun tapahtumalla on n mahdollista tulosta, odotusarvo lasketaan kaavalla

$$p_1 t_1 + p_2 t_2 + \cdots + p_n t_n,$$

missä p_i ja t_i ovat tapahtuman i todennäköisyys ja tulos. Esimerkiksi kun heitämme noppaa, tuloksen odotusarvo on

$$1/6 \cdot (1 + 2 + 3 + 4 + 5 + 6) = 7/2.$$

Index

- O -merkintä, 16
- O -notation, 8
- Ω -merkintä, 16
- Θ -merkintä, 16
- 3SAT-ongelma, 165

- alfa-beeta-karsinta, 95
- algorithms, 1
- alipuu, 59
- array, 4
- aste, 107
- AVL-ehto, 66
- AVL-puu, 65
- avoin hajautus, 51

- backtracking, 85
- Bellmanin ja Fordin algoritmi, 120
- binomikerroin, 105
- binäärihaku, 35
- binäärihakupuu, 59
- binäärikeko, 75
- binääripuu, 59
- BPP-luokka, 163
- branch and bound, 92

- conditional statement, 3
- constant factor, 8

- Dijkstran algoritmi, 123
- dynaaminen ohjelmointi, 97, 135

- Edmondsin ja Karpin algoritmi, 155
- erilliset polut, 156
- esijärjestys, 61
- etäisyys, 115, 119

- etäisyysmatriisi, 127

- Floydin ja Warshallin algoritmi, 127
- Fordin ja Fulkersonin algoritmi, 152
- function, 4

- hajautusarvo, 49
- hajautusfunktio, 49
- hajautustaulu, 49
- hakemisto, 54
- Hamiltonin polku, 168
- harva verkko, 129
- heuristiikka, 170
- hybridialgoritmi, 29

- implisiittinen verkko, 116
- index, 4
- input, 1
- inversio, 23
- isomorfiset verkot, 168
- iteraattori, 46

- jakojäännös, 173
- jono, 43
- juuri, 59
- jälkijärjestys, 61
- järjestäminen, 21
- jäännösverkko, 152

- kaari, 107
- kaarilista, 111
- kaksijakoinen verkko, 157
- keko, 75
- kekoehto, 75
- kekojärjestäminen, 80

- kertoma, 173
- ketjuhajautus, 50
- kierto, 67
- klikki, 168
- kokeilufunktio, 51
- komponentti, 108
- komponenttiverkko, 137
- Kosarajun algoritmi, 137
- Kruskalin algoritmi, 145
- kuutiollinen algoritmi, 12

- lapsi, 59
- laskemisjärjestäminen, 30
- lehti, 59
- leikkaus, 153
- leveyshaku, 115
- lineaarinen algoritmi, 11
- lineaarinen kokeilu, 51
- linkitetty lista, 40
- list, 4
- lista, 37
- lisäysjärjestäminen, 21
- logaritmi, 174
- logaritmisen algoritmi, 10
- lomitussjärjestäminen, 24
- loop, 3
- lyhin polku, 115, 119

- maksimikeko, 75
- maksimiparitus, 157
- maksimivirtaus, 151
- minimax-algoritmi, 94
- minimikeko, 75
- minimileikkaus, 153

- naapuri, 107
- neliöllinen algoritmi, 11
- NP-luokka, 162
- NP-ongelma, 161
- NP-täydellinen ongelma, 164
- NP-vaikea ongelma, 169

- odotusarvo, 174
- offline-algoritmi, 58
- online-algoritmi, 58
- optimointiongelma, 169

- P vs. NP, 162
- P-luokka, 161
- painotettu verkko, 109
- palautus, 156, 165
- paritus, 157
- pienin polkupeite, 158
- pienin virittävä puu, 145
- pikajärjestäminen, 26
- pino, 43
- pisin nouseva alijono, 100
- polku, 108
- polkujen laskeminen, 135
- polkupeite, 158
- polynominen algoritmi, 161
- polynominen hajautus, 53
- Primin algoritmi, 147
- printout, 1
- prioriteettijono, 75, 81
- procedure, 4
- pseudocode, 2
- pseudopolynominen algoritmi, 171
- PSPACE-luokka, 163
- puu, 108
- päätösongelma, 161

- recursion, 5
- repunpakkaus, 103

- SAT-ongelma, 164
- satunnainen algoritmi, 171
- sisäjärjestys, 61
- solmu, 107
- solmupeite, 166
- subroutine, 4
- summakaava, 173
- suunnattu sykliton verkko, 131
- suunnattu verkko, 109

suurin virittävä puu, 149
sykli, 108
syklin etsiminen, 114, 132
sykkitön verkko, 108
symmetria, 90
syvyyshaku, 113

tasapainoinen puu, 65
taulukkolista, 37
taulukointi, 100
tiheä verkko, 129
tilavaativuus, 17
time complexity, 7
todiste, 162
topologinen järjestys, 132
törmäys, 49

union-find-rakenne, 141

vaativuusluokka, 161
vahvasti yhtenäinen komponentti, 137
vahvasti yhtenäinen verkko, 137
vakaus, 28
vakioaikainen algoritmi, 10
vakiokerroin, 29
vanhempi, 59
variable, 3
verkko, 107
vieruslista, 110
vierusmatriisi, 112
virittävä puu, 145
virtaus, 151
väritys, 168

yhtenäinen komponentti, 108
yhtenäinen verkko, 108
yksinkertainen verkko, 110

ZPP-luokka, 163